

UNIT I

FUNDAMENTALS OF SECURITY

Computer security Concepts – Threats, Attacks, and Assets – Security Functional – Requirements fundamentals security design principals – Attack surfaces and Attack Trees. Computer security strategy – Number Theory: Prime Numbers and Factorization, Modular Arithmetic, GCD and Euclidean Algorithm, Chinese Remainder Theorem, Multiplication Modulo m and the Totient Functions, problems, Fermat and Euler Theorem. Primitive Roots and the structure of F^*_p , Number in other Bases, Fast computation of powers in Z/mZ , Multiplicative functions, Group Theory, Fields and Problems.

1. INTRODUCTION

Computer data often travels from one computer to another, leaving the safety of its protected physical surroundings. Once the data is out of hand, people with bad intention could modify or forge your data, either for amusement or for their own benefit.

Computer Security:

Computer security basically is the protection of computer systems and information from harm, theft, and unauthorized use. It is the process of preventing and detecting unauthorized use of your computer system.

1.1 What is Computer Security and its types?

One way to ascertain the similarities and differences among Computer Security is by asking what is being secured. For example,

- ✓ *Information security* is securing information from unauthorized access, modification & deletion
- ✓ *Application Security* is securing an application by building security features to

prevent from Cyber Threats such as SQL injection, DoS attacks, data breaches and etc.

- ✓ *Computer Security* means securing a standalone machine by keeping it updated and patched
- ✓ *Network Security* is by securing both the software and hardware technologies
- ✓ *Cybersecurity* is defined as protecting computer systems, which communicate over the computer networks

This definition introduces three key objectives that are at the heart of computer security:

Confidentiality: This term covers two related concepts:

Data confidentiality: Assures that private or confidential information is not made available or disclosed to unauthorized individuals.

Privacy: Assures that individuals control or influence what information related to them may be collected and stored and by whom and to whom that information may be disclosed.

Integrity: This term covers two related concepts:

Data integrity: Assures that information (both stored and in transmitted packets) and programs are changed only in a specified and authorized manner.

System integrity: Assures that a system performs its intended function in an unimpaired manner, free from deliberate or inadvertent unauthorized manipulation of the system.

Availability: Assures that systems work promptly and service is not denied to authorized users.

These three concepts form what is often referred to as the **CIA triad**. The three concepts embody the fundamental security objectives for both data and for information and computing services.

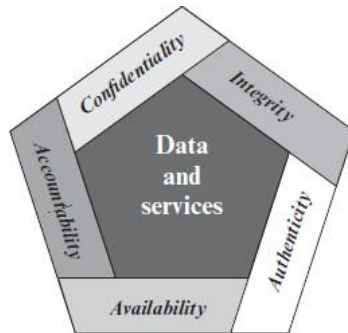


Fig 1.1 Essential Network and Computer Security Requirements

- **Confidentiality:** Preserving authorized restrictions on information access and disclosure, including means for protecting personal privacy and proprietary information. A loss of confidentiality is the unauthorized disclosure of information.
- **Integrity:** Guarding against improper information modification or destruction, including ensuring information nonrepudiation and authenticity. A loss of integrity is the unauthorized modification or destruction of information.
- **Availability:** Ensuring timely and reliable access to and use of information. A loss of availability is the disruption of access to or use of information or an information system.
- **Authenticity:** The property of being genuine and being able to be verified and trusted; confidence in the validity of a transmission, a message, or message originator. This means verifying that users are who they say they are and that each input arriving at the system came from a trusted source.
- **Accountability:** The security goal that generates the requirement for actions of an entity to be traced uniquely to that entity. This supports nonrepudiation, deterrence, fault isolation, intrusion detection and prevention, and afteraction recovery and legal action. Because truly secure systems are not yet an achievable goal, we must be able to trace a security breach to a responsible party. Systems

must keep records of their activities to permit later forensic analysis to trace security breaches or to aid in transaction disputes.

Computer Security Challenges:

1. Security is not simple it requires a lot of research and money
2. Potential attacks on the security features need to be considered.
3. Procedures used to provide particular services are often counter-intuitive.
4. It is necessary to decide where to use the various security mechanisms.
5. Requires constant monitoring.
6. Security mechanisms typically involve more than a particular algorithm or protocol.
7. Security is essentially a battle of wits between a perpetrator and the designer.
8. Little benefit from security investment is perceived until a security failure occurs.
9. Strong security is often viewed as an impediment to efficient and user-friendly operation.

1.2 Computer security threats

Computer security threats are possible dangers that can possibly hamper the normal functioning of your computer. In the present age, cyber threats are constantly increasing as the world is going digital.

Types of Threats:

A security threat is a threat that has the potential to harm computer systems and organizations. The cause could be physical, such as a computer containing sensitive information being stolen. It's also possible that the cause isn't physical, such as a viral attack.

1. Physical Threats: A physical danger to computer systems is a potential cause of an occurrence/event that could result in data loss or physical damage. It can be classified as:

- ✓ **Internal:** Short circuit, fire, non-stable supply of power, hardware failure due to excess humidity, etc. cause it.

- ✓ **External:** Disasters such as floods, earthquakes, landscapes, etc. cause it.
- ✓ **Human:** Destroying of infrastructure and/or hardware, thefts, disruption, and unintentional/intentional errors are among the threats.

2. Non-physical threats: A non-physical threat is a potential source of an incident that could result in:

- ✓ Hampering of the business operations that depend on computer systems.
- ✓ Sensitive – data or information loss
- ✓ Keeping track of other's computer system activities illegally.
- ✓ Hacking id & passwords of the users, etc.

The non-physical threads can be commonly caused by:

(i) Malware: Malware (“malicious software”) is a type of computer program that infiltrates and damages systems without the users’ knowledge. Malware tries to go unnoticed by either hiding or not letting the user know about its presence on the system. You may notice that your system is processing at a slower rate than usual.

(ii) Virus: It is a program that replicates itself and infects your computer’s files and programs, rendering them inoperable. It is a type of malware that spreads by inserting a copy of itself into and becoming part of another program. It spreads with the help of software or documents. They are embedded with software and documents and then transferred from one computer to another using the network, a disk, file sharing, or infected e-mail. They usually appear as an executable file.

(iii) Spyware: Spyware is a type of computer program that tracks, records, and reports a user’s activity (offline and online) without their permission for the purpose of profit or data theft. Spyware can be acquired from a variety of sources, including websites, instant chats, and emails. A user may also unwittingly obtain spyware by adopting a software program’s End User License Agreement.

Adware is a sort of spyware that is primarily utilized by advertising. When you go online, it keeps track of your web browsing patterns in order to compile data on the types of

websites you visit.

(iv) Worms: Computer worms are similar to viruses in that they replicate themselves and can inflict similar damage. Unlike viruses, which spread by infecting a host file, worms are freestanding programs that do not require a host program or human assistance to proliferate. Worms don't change programs; instead, they replicate themselves over and over. They just eat resources to make the system down.

(v) Trojan: A Trojan horse is malicious software that is disguised as a useful host program. When the host program is run, the Trojan performs a harmful/unwanted action. A Trojan horse, often known as a Trojan, is malicious malware or software that appears to be legal yet has the ability to take control of your computer. A Trojan is a computer program that is designed to disrupt, steal, or otherwise harm your data or network.

(vi) Denial Of Service Attacks: A Denial of Service attack is one in which an attacker tries to prohibit legitimate users from obtaining information or services. An attacker tries to make a system or network resource unavailable to its intended users in this attack. The web servers of large organizations such as banking, commerce, trading organizations, etc. are the victims.

(vii) Phishing: Phishing is a type of attack that is frequently used to obtain sensitive information from users, such as login credentials and credit card details. They deceive users into giving critical information, such as bank and credit card information, or access to personal accounts, by sending spam, malicious Web sites, email messages, and instant chats.

(viii) Key-Loggers: Keyloggers can monitor a user's computer activity in real-time. Keylogger is a program that runs in the background and records every keystroke made by a user, then sends the data to a hacker with the intent of stealing passwords and financial information.

1.3 Computer security attacks:

A useful means of classifying security attacks is in terms of *passive attacks* and *active attacks*

- ✓ A passive attack attempts to learn or make use of information from the system but

does not affect system resources.

- ✓ An active attack attempts to alter system resources or affect their operation

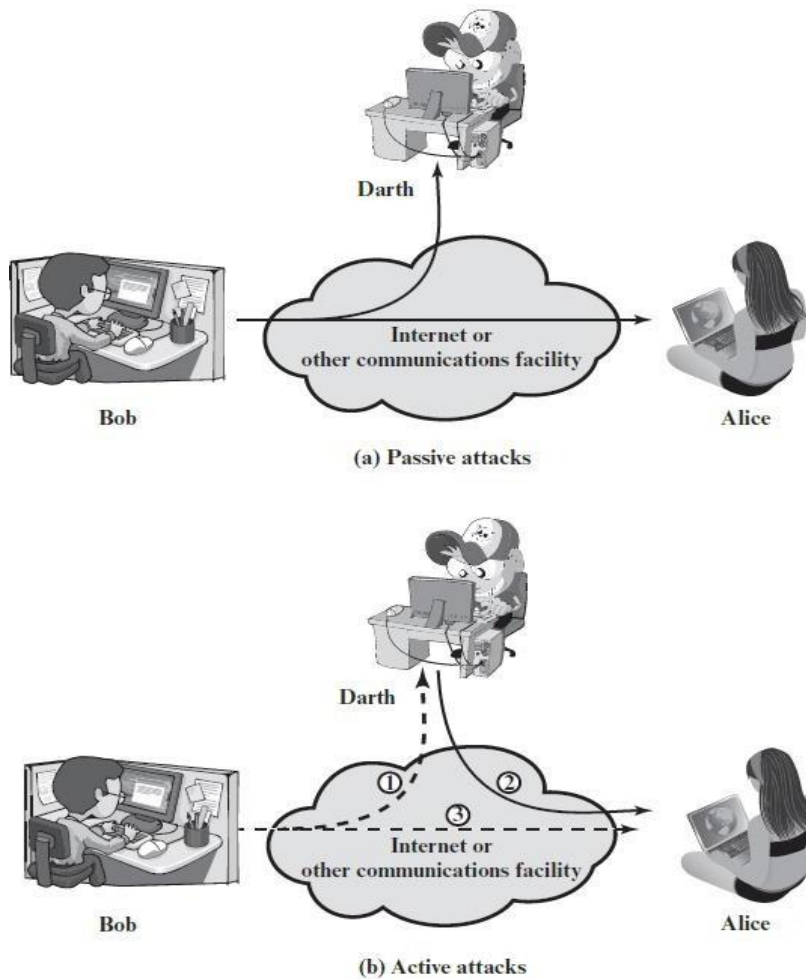


Fig 1.2 Security attacks

Passive attacks:

Passive attacks (Figure 1.2a) are in the nature of eavesdropping on, or monitoring of, transmissions. The goal of the opponent is to obtain information that is being transmitted. Two types of passive attacks are the release of message contents and traffic analysis. The **release of message contents** is easily understood. A telephone conversation, an electronic mail message, and a transferred file may contain sensitive or confidential information. We would like to prevent an opponent from learning the contents of these transmissions.

A second type of passive attack, **traffic analysis**, is subtler. Suppose that we had a way of masking the contents of messages or other information traffic so that opponents, even if they captured the message, could not extract the information from the message. The common technique for masking contents is encryption. If we had encryption protection in place, an opponent might still be able to observe the pattern of these messages. The opponent could determine the location and identity of communicating hosts and could observe the frequency and length of messages being exchanged. This information might be useful in guessing the nature of the communication that was taking place.

Active Attacks:

Active attacks (Figure 1.2b) involve some modification of the data stream or the creation of a false stream and can be subdivided into four categories: masquerade, replay, modification of messages, and denial of service.

A **masquerade** takes place when one entity pretends to be a different entity (path 2 of Figure 1.2b is active). A masquerade attack usually includes one of the other forms of active attack. For example, authentication sequences can be captured and replayed after a valid authentication sequence has taken place, thus enabling an authorized entity with few privileges to obtain extra privileges by impersonating an entity that has those privileges.

Replay involves the passive capture of a data unit and its subsequent retransmission to produce an unauthorized effect (paths 1, 2, and 3 active).

Modification of messages simply means that some portion of a legitimate message is altered, or that messages are delayed or reordered, to produce an unauthorized effect (paths 1 and 2 active). For example, a message meaning “Allow John Smith to read confidential file *accounts*” is modified to mean “Allow Fred Brown to read confidential file *accounts*.”

The **denial of service** prevents or inhibits the normal use or management of communications facilities (path 3 active). This attack may have a specific target; for example, an entity may suppress all messages directed to a particular destination (e.g., the security audit service). Another form of service denial is the disruption of an entire

network, either by disabling the network or by overloading it with messages so as to degrade performance. Active attacks present the opposite characteristics of passive attacks. Whereas passive attacks are difficult to detect, measures are available to prevent their success.

1.4 Assets:

An asset is property owned by a person or company, regarded as having value, and available to meet debts and commitments.

In cybersecurity, an asset is something valuable that a business owns or anything that can be directly used to create value. This is very similar to the accounting and finance idea of an asset.

The most common types of assets are software applications and laptops. But intellectual property and the data companies control are also considered assets.

From a cybersecurity view, there are two essential types of assets:

- Assets that must be configured to meet security outcomes
- Assets that could be impacted by a cyberattack or ransomware
- This means that assets that are managed in cybersecurity include:
- Laptops, computers, printers, network infrastructure - *these are often called "hardware assets"*.
- Finance software, word processing apps, design and marketing apps, industrial control software - *these are often called "software assets"*.
- Data about customers, your order book, supply chain information - *these are often called "data assets"*.
- Trade secrets, patents, R&D data - *these are often called "intellectual property assets"*.

2. SECURITY FUNCTIONAL REQUIREMENTS

- **Security Audit:** Perhaps you'll need to recognize, record, store, and analyze security-relevant activities. You'll need to identify what you want to make auditable, since often you can't leave all possible auditing capabilities enabled. Also, consider what to do when there's no room left for auditing - if you stop the

system, an attacker may intentionally do things to be logged and thus stop the system.

- **Communication/Non-repudiation:** This class is poorly named in the CC; officially it's called communication, but the real meaning is non-repudiation. Is it important that an originator cannot deny having sent a message, or that a recipient cannot deny having received it? There are limits to how well technology itself can support non-repudiation (e.g., a user might be able to give their private key away ahead of time if they wanted to be able to repudiate something later), but nevertheless for some applications supporting non-repudiation capabilities is very useful.
- **Cryptographic Support:** If you're using cryptography, what operations use cryptography, what algorithms and key sizes are you using, and how are you managing their keys (including distribution and destruction)?
- **User Data Protection :** This class specifies requirement for protecting user data, and is a big class in the CC with many families inside it. The basic idea is that you should specify a policy for data (access control or information flow rules), develop various means to implement the policy, possibly support off-line storage, import, and export, and provide integrity when transferring user data between TOEs. One often-forgotten issue is residual information protection - is it acceptable if an attacker can later recover ``deleted" data?
- **Identification and authentication:** Generally, you don't just want a user to report who they are (identification) - you need to verify their identity, a process called authentication. Passwords are the most common mechanism for authentication. It's often useful to limit the number of authentication attempts (if you can) and limit the feedback during authentication (e.g., displaying asterisks instead of the actual password).
- **Security Management:** Many systems will require some sort of management (e.g., to control who can do what), generally by those who are given a more trusted role (e.g., administrator). Be sure you think through what those special operations

are, and ensure that only those with the trusted roles can invoke them. You want to limit trust; ideally, even more trusted roles should be limited in what they can do.

- **Privacy:** Do you need to support anonymity, pseudonymity, unlinkability, or unobservability? If so, are there conditions where you want or don't want these (e.g., should an administrator be able to determine the real identity of someone hiding behind a pseudonym?). Note that these can seriously conflict with non-repudiation, if you want those too. If you're worried about sophisticated threats, these functions can be hard to provide.
- **Protection of the TOE Security Functions/Self-protection:** Clearly, if the TOE can be subverted, any security functions it provides aren't worthwhile, and in many cases a TOE has to provide at least some self-protection. Perhaps you should "test the underlying abstract machine" - i.e., test that the underlying components meet your assumptions, or have the product run self-tests (say during start-up, periodically, or on request). You should probably "fail secure", at least under certain conditions; determine what those conditions are.
- **Resource Utilization:** Perhaps you need to provide fault tolerance, a priority of service scheme, or support resource allocation (such as a quota system).
- **TOE Access:** There may be many issues controlling sessions. Perhaps there should be a limit on the number of concurrent sessions (if you're running a web service, would it make sense for the same user to be logged in simultaneously, or from two different machines?). Perhaps you should lock or terminate a session automatically (e.g., after a timeout), or let users initiate a session lock. You might want to include a standard warning banner. One surprisingly useful piece of information is displaying, on login, information about the last session (e.g., the date/time and location of the last login) and the date/time of the last unsuccessful attempt - this gives users information that can help them detect interlopers.

Perhaps sessions can only be established based on other criteria (e.g., perhaps you can only use the program during business hours).

- **Trusted path/channels:** A common trick used by attackers is to make the screen appear to be something it isn't, e.g., run an ordinary program that looks like a login screen or a forged web site. Thus, perhaps there needs to be a "trusted path" - a way that users can ensure that they are talking to the "real" program.

3. FUNDAMENTAL SECURITY DESIGN PRINCIPLES

The security design principles are considered while designing any security mechanism for a system. These principles are review to develop a secure system which prevents the security flaws and also prevents unwanted access to the system.

Below is the list of fundamental security design principles provided by the National Centres of Academic Excellence in Information Assurance/Cyber Defence, along with the U.S. National Security Agency and the U.S. Department of Homeland Security.

Fundamental Security Design Principles

1. Economy of Mechanism

This fundamental security principle defines that the security measures implemented in the software and the hardware must be simple and small. This would ease the testers to test the security measures thoroughly.

If the designed security mechanism is complex then it is likely that the tester would get a chance to exploit the weakness in the design.

So more the design is simple less are the opportunities for the tester to discover the flaws and more the complex is the design more are the chances to exploit flaws in the design.

2. Fail-safe Defaults

This principle says that if any user wants access to any mechanism then whether the access is permitted or denied should be based on authorization rather than elimination.

By default, all the mechanism should have a lack of access and the function of a security mechanism is to identify the condition where the access to the security mechanism should be permitted. This means by default access to all mechanism should be denied, unless any

privilege attribute is provided.

3. Complete Mediation

Some systems are designed to operate continuously such systems remember access decision. So, there must be an access control mechanism which would check every access occurring on the system.

This principle says that the system should not trust the access decisions it recovers from the system cache. This particular security design principle says that there must be a mechanism in the system that checks each access through the access control mechanism. However, this is an exhaustive approach and is rarely considered while designing a security mechanism.

4. Open Design

This security principle suggests that the security mechanism design should be open to the public. Like in the cryptographic algorithm, the encryption key is kept secret while the encryption algorithm is opened for a public investigation.

This principle is followed by the NIST (National Institute of Standards and Technology) to standardize the algorithms because it helps in worldwide adoption of NIST approved algorithms.

5. Separation of Privilege

This security principle states that whenever a user tries to gain access to a system, the access should not be granted based on a single attribute or condition.

Instead, there must be multiple situations or conditions or attribute which should be verified to grant access to the system. We also term this as a multifactor user authentication as this principle says that multiple techniques must be implemented to authenticate a user.

For example, while conducting online money transfer we require user-id, password, transaction password along with OTP.

6. Least Privilege

The least privilege security design principle states that each user should be able to access the system with the least privilege. Only those limited privileges should be assigned to

the user which are essential to perform the desired task.

An example of considering and implementing this principle is **role-based access control**.

The role-based designed security mechanism should discover and describe various roles of the users or processes.

7. Least Common Mechanism

Following the least common mechanism, a security design principle there should be minimum common functions to share between the different user. This principle reduces the count of communication paths and therefore further reduces the hardware and software implementation.

8. Psychological Acceptability

This security design principle says that the security mechanisms design to protect the system should not interfere with the working of the user every now and then.

As this would irritate the user and user may disable this security mechanism on the system. Therefore, it is suggested that the security mechanism should introduce minimum hurdles to the user of the system.

The security mechanism should not be designed such that it becomes difficult for the user to access the resources in the system.

9. Isolation

This security design principle is considered in three circumstances. The first condition, the system that has critical data, processes or resources must be isolated such that it restricts public access. It can be done in two ways.

The system with critical resources can be isolated in two ways physical and logical isolation. The physical isolation is one where the system with critical information is isolated from the system with public access information.

10. Encapsulation

This security design principle is a form of isolation which is designed on the principle of object-oriented principles. Here the processes of the protected system can only access the data object of the system and these processes can only be invoked from a domain entry point.

11. Modularity

This security designing principle says that the security mechanism must be generated as separate and protected modules and the security mechanism must be generated using the modular architecture.

This principle helps in updating the security mechanism independently without modifying the entire system.

12. Layering

Multiple security layers must be used in order to protect the opponent from accessing crucial information. Applying multiple security layers provides multiple barriers to the adversary if he tries to access the protected system.

13. Least Astonishment

This security design principle states that the user interface of the system must not amaze the user while accessing the secure system. He should be able to understand how the security mechanism is essential to protect the system.

4. ATTACK SURFACES AND ATTACK TREES

Attack Surfaces:

An attack surface consists of the reachable and exploitable vulnerabilities in a system

Examples of attack surfaces are the following:

- ✓ Open ports on outward facing Web and other servers, and code listening on those ports
- ✓ Services available on the inside of a firewall
- ✓ Code that processes incoming data, email, XML, office documents, and industry-specific custom data exchange formats
- ✓ Interfaces, SQL, and Web forms
- ✓ An employee with access to sensitive information vulnerable to a social engineering attack

Attack surfaces can be categorized as follows:

- ✓ **Network attack surface:** This category refers to vulnerabilities over an enterprise network, wide-area network, or the Internet. Included in this category are network

- protocol vulnerabilities, such as those used for a denial-of-service attack,
- ✓ disruption of communications links, and various forms of intruder attacks.
 - ✓ **Software attack surface:** This refers to vulnerabilities in application, utility, or operating system code. A particular focus in this category is Web server software.
 - ✓ **Human attack surface:** This category refers to vulnerabilities created by personnel or outsiders, such as social engineering, human error, and trusted insiders.

An attack surface analysis is a useful technique for assessing the scale and severity of threats to a system.

A systematic analysis of points of vulnerability makes developers and security analysts aware of where security mechanisms are required. Once an attack surface is defined, designers may be able to find ways to make the surface smaller, thus making the task of the adversary more difficult. The attack surface also provides guidance on setting priorities for testing, strengthening security measures, and modifying the service or application.

As illustrated in Figure 1.3, the use of layering, or defense in depth, and attack surface reduction complement each other in mitigating security risk.

Attack Trees:

An attack tree is a branching, hierarchical data structure that represents a set of potential techniques for exploiting security vulnerabilities.

The security incident that is the goal of the attack is represented as the root node of the tree, and the ways that an attacker could reach that goal are iteratively and incrementally represented as branches and subnodes of the tree.

Each subnode defines a subgoal, and each subgoal may have its own set of further subgoals, and so on. The final nodes on the paths outward from the root, that is, the leaf nodes, represent different ways to initiate an attack. Each node other than a leaf is either an AND-node or an OR-node.

To achieve the goal represented by an AND-node, the subgoals represented by all of that node's subnodes must be achieved; and for an OR-node, at least one of the subgoals must

be achieved. Branches can be labeled with values representing difficulty, cost, or other attack attributes, so that alternative attacks can be compared.

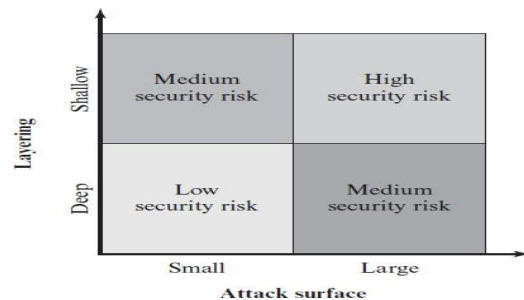


Fig 1.3 Defence in Depth and Attack Surface

The motivation for the use of attack trees is to effectively exploit the information available on attack patterns. Organizations such as CERT publish security advisories that have enabled the development of a body of knowledge about both general attack strategies and specific attack patterns. Security analysts can use the attack tree to document security attacks in a structured form that reveals key vulnerabilities. The attack tree can guide both the design of systems and applications, and the choice and strength of countermeasures. Figure 1.4 is an example of an attack tree analysis for an Internet banking authentication application. The root of the tree is the objective of the attacker, which is to compromise a user's account. The shaded boxes on the tree are the leaf nodes, which represent events that comprise the attacks. Note that in this tree, all the nodes other than leaf nodes are OR-nodes.

The analysis to generate this tree considered the three components involved in authentication:

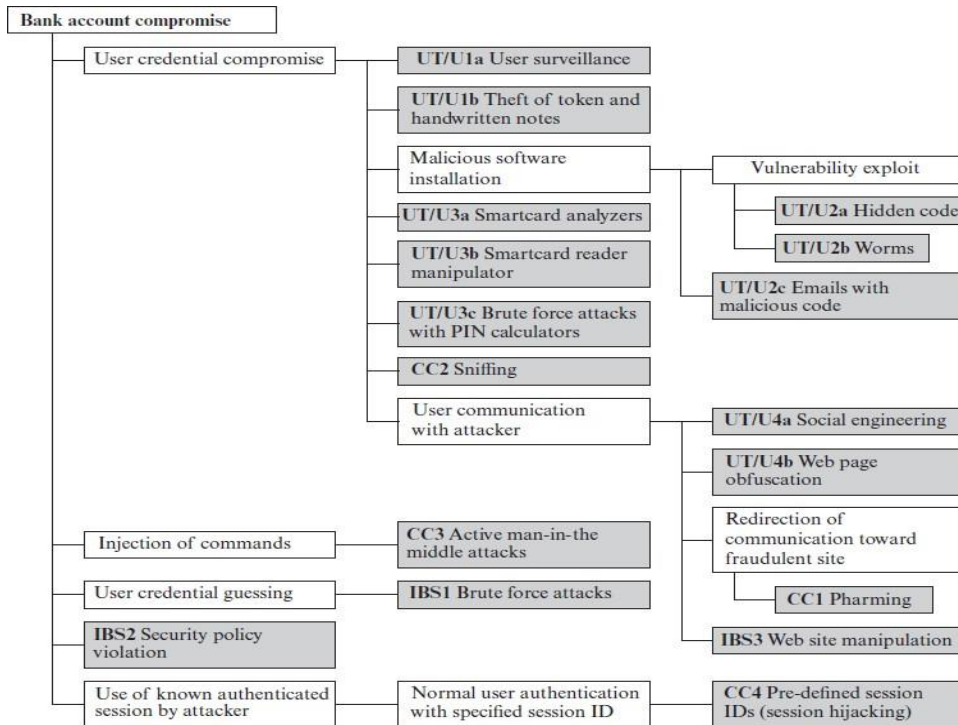


Figure 1.4 An Attack Tree for Internet Banking Authentication

- ✓ **User terminal and user (UT/U):** These attacks target the user equipment, including the tokens that may be involved, such as smartcards or other password generators, as well as the actions of the user.
- ✓ **Communications channel (CC):** This type of attack focuses on communication links.
- ✓ **Internet banking server (IBS):** These types of attacks are offline attacks against the servers that host the Internet banking application.

Five overall attack strategies can be identified, each of which exploits one or more of the three components. The five strategies are as follows:

■ **User credential compromise:** This strategy can be used against many elements of the attack surface. There are procedural attacks, such as monitoring a user's action to observe a PIN or other credential, or theft of the user's token or handwritten notes. An adversary may also compromise token information using a variety of token attack tools, such as

hacking the smartcard or using a brute force approach to guess the PIN. Another possible strategy is to embed malicious software to compromise the user's login and password. An adversary may also attempt to obtain credential information via the communication channel (sniffing). Finally, an adversary may use various means to engage in communication with the target user, as shown in Figure 1.4.

■ **Injection of commands:** In this type of attack, the attacker is able to intercept communication between the UT and the IBS. Various schemes can be used to be able to impersonate the valid user and so gain access to the banking system.

■ **User credential guessing:** It is reported in [HILT06] that brute force attacks against some banking authentication schemes are feasible by sending random usernames and passwords. The attack mechanism is based on distributed zombie personal computers, hosting automated programs for username- or password-based calculation.

■ **Security policy violation:** For example, violating the bank's security policy in combination with weak access control and logging mechanisms, an employee may cause an internal security incident and expose a customer's account.

■ **Use of known authenticated session:** This type of attack persuades or forces the user to connect to the IBS with a preset session ID. Once the user authenticates to the server, the attacker may utilize the known session ID to send packets to the IBS, spoofing the user's identity.

Figure 1.4 provides a thorough view of the different types of attacks on an Internet banking authentication application. Using this tree as a starting point, security analysts can assess the risk of each attack and, using the design principles outlined in the preceding section, design a comprehensive security facility.

5. COMPUTER SECURITY STRATEGY

- ✓ Computer security strategy formally known as Cyber Security Strategy.
- ✓ To design and implement a secure cyberspace, some stringent strategies have been put in place.

The major strategies employed to ensure cybersecurity, which include the following –

Strategy 1 – Creating a Secure Cyber Ecosystem

The cyber ecosystem involves a wide range of varied entities like devices (communication technologies and computers), individuals, governments, private organizations, etc., which interact with each other for numerous reasons.

This strategy explores the idea of having a strong and robust cyber-ecosystem where the cyber-devices can work with each other in the future to prevent cyber-attacks, reduce their effectiveness, or find solutions to recover from a cyber-attack.

Automation, Interoperability, and Authentication.

- **Automation** – It eases the implementation of advanced security measures, enhances the swiftness, and optimizes the decision-making processes.
- **Interoperability** – It toughens the collaborative actions, improves awareness, and accelerates the learning procedure. There are three types of interoperability –
 - Semantic (i.e., shared lexicon based on common understanding)
 - Technical
 - Policy – Important in assimilating different contributors into an inclusive cyber-defense structure.
- **Authentication** – It improves the identification and verification technologies that work in order to provide –
 - Security
 - Affordability
 - Ease of use and administration
 - Scalability
 - Interoperability

6.NUMBER THEORY

Number Theory plays an important role in encryption algorithm. Cryptography is the practice of hiding information, converting some secret information to not readable texts.

Important concepts in Number Theory

Prime Numbers- A positive integer p is said to be a prime if it has only two factors namely 1 and p itself.

For Example: Primes are 2, 3, 5, 7, 11, 13, 17 ...

Divisors: A positive integer a is said to divide an integer b if there exist an integer c such that $b = a \cdot c$ and written as $a \mid b$.

For Example: $2 \mid 10$ as $10 = 2 \cdot 5$ but 3 do not divide 10 as there does not exist any integer c such that $10 = 3 \cdot c$

Greatest Common Divisor: Let a and b be two positive integers then an integer d is called greatest common divisor of a and b if $d \mid a$ and $d \mid b$ i.e. d is common divisor of a and b . And if any integer c is such that $c \mid a$ and $c \mid b$ then $c \mid d$ i.e. any other common divisor of a and b will divide d it is denoted by $d = (a, b)$

For Example: $6 = (24, 30)$

Two numbers a and b are said to relatively prime or co prime if their greatest common divisor is 1 i.e. $(a, b) = 1$

For Example: 10 and 11 are co prime

Euler's ' ϕ ' Function : An arithmetic function Euler's Toitent function ' ϕ ' is defined as $\phi(n)$ = number of positive integers less than or equal to n and co prime to n i.e. $\phi(n)$ = number of positive integers ' a ' such that $1 \leq a \leq n$ and $\text{g.c.d.}(a, n) = 1$

For Example: $\phi(15) = 8$ as primes relative to 15 are given by 1, 2, 4, 7, 8, 11, 13, and 14.

And $\phi(mn) = \phi(m)\phi(n)$ where m and n are relatively prime.

Some simple properties of congruence are given below:

(1) $a \equiv b \pmod{m}$ iff $b \equiv a \pmod{m}$

(2) $a \equiv b + c \pmod{m}$ iff $a - c \equiv b \pmod{m}$

(3) $(\text{mod}) a_1 \equiv b_1 \pmod{m}$ and $(\text{mod}) a_2 \equiv b_2 \pmod{m}$ then $(\text{mod}) a_1 + a_2 \equiv b_1 + b_2 \pmod{m}$

(4) $a \equiv b \pmod{m}$ and c is any integer then $ca \equiv cb \pmod{m}$

(5) $a \pm mk \equiv a \pmod{m}$ where k is any integer

6.1 DIVISIBILITY AND THE DIVISION ALGORITHM

Divisibility

We say that a nonzero b **divides** a if $a = mb$ for some m , where a , b , and m are integers. That is, b divides a if there is no remainder on division. The notation $b \mid a$ is commonly used to mean b divides a . Also, if $b \mid a$, we say that b is a **divisor** of a .

The positive divisors of 24 are 1, 2, 3, 4, 6, 8, 12, and 24.
 $13 \nmid 182$; $-5 \nmid 30$; $17 \nmid 289$; $-3 \nmid 33$; $17 \mid 0$

Subsequently, we will need some simple properties of divisibility for integers, which are as follows:

- If $a \mid 1$, then $a = \pm 1$.
- If $a \mid b$ and $b \mid a$, then $a = \pm b$.
- Any $b \neq 0$ divides 0.
- If $a \mid b$ and $b \mid c$, then $a \mid c$:

$$11 \mid 66 \text{ and } 66 \mid 198 \Rightarrow 11 \mid 198$$

- If $b \mid g$ and $b \mid h$, then $b \mid (mg + nh)$ for arbitrary integers m and n .

To see this last point, note that

- If $b \mid g$, then g is of the form $g = b * g_1$ for some integer g_1 .
- If $b \mid h$, then h is of the form $h = b * h_1$ for some integer h_1 .

So

$mg + nh = mbg_1 + nbh_1 = b * (mg_1 + nh_1)$ and therefore b divides $mg + nh$.

$b = 7; g = 14; h = 63; m = 3; n = 2$
 $7 \mid 14$ and $7 \mid 63$.
 To show $7 \mid (3 \times 14 + 2 \times 63)$,
 we have $(3 \times 14 + 2 \times 63) = 7(3 \times 2 + 2 \times 9)$,
 and it is obvious that $7 \mid (7(3 \times 2 + 2 \times 9))$.

The Division Algorithm:

Given any positive integer n and any nonnegative integer a , if we divide a by n , we get an integer quotient q and an integer remainder r that obey the following relationship:

$$a = qn + r \quad 0 \leq r < n; \quad q = :a/n; \quad (2.1)$$

where $:x$; is the largest integer less than or equal to x . Equation (2.1) is referred to as the division algorithm.

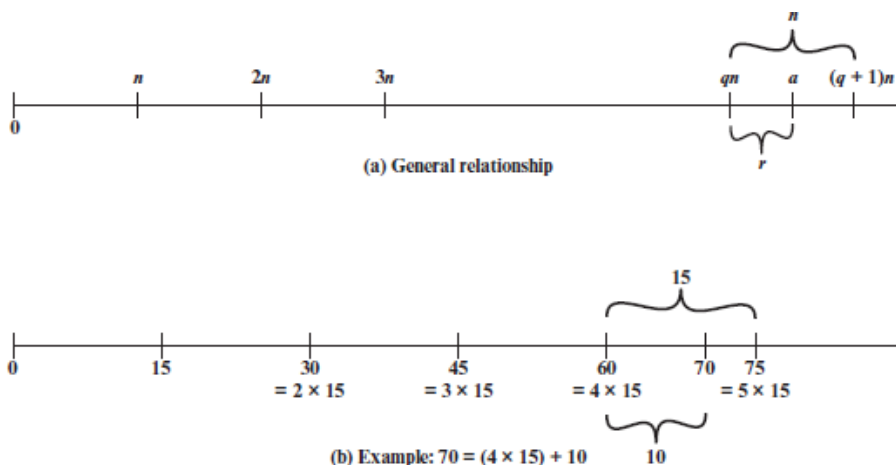


Fig 1.6 The Relationship $a = qn + r; 0 \leq r < n$

Figure 1.6a demonstrates that, given a and positive n , it is always possible to find q and r that satisfy the preceding relationship. Represent the integers on the number line; a will fall somewhere on that line (positive a is shown, a similar demonstration can be made for

negative a). Starting at 0, proceed to n , $2n$, up to qn , such that $qn \leq a$ and $(q + 1)n > a$. The distance from qn to a is r , and we have found the unique values of q and r . The remainder r is often referred to as a **residue**.

$$\begin{array}{l} a = 11; \quad n = 7; \quad 11 = 1 \times 7 + 4; \quad r = 4 \quad q = 1 \\ a = -11; \quad n = 7; \quad -11 = (-2) \times 7 + 3; \quad r = 3 \quad q = -2 \end{array}$$

6.2 THE EUCLIDEAN ALGORITHM

One of the basic techniques of number theory is the Euclidean algorithm, which is a simple procedure for determining the greatest common divisor of two positive integers. First, we need a simple definition: Two integers are **relatively prime** if and only if their only common positive integer factor is 1.

Greatest Common Divisor

Recall that nonzero b is defined to be a divisor of a if $a = mb$ for some m , where a , b , and m are integers. We will use the notation $\gcd(a, b)$ to mean the **greatest common divisor** of a and b . The greatest common divisor of a and b is the largest integer that divides both a and b . We also define $\gcd(0, 0) = 0$.

More formally, the positive integer c is said to be the greatest common divisor of a and b if

1. c is a divisor of a and of b .
2. any divisor of a and b is a divisor of c .

An equivalent definition is the following:

$$\gcd(a, b) = \max\{k, \text{ such that } k|a \text{ and } k|b\}$$

Because we require that the greatest common divisor be positive, $\gcd(a, b) = \gcd(a, -b) = \gcd(-a, b) = \gcd(-a, -b)$. In general, $\gcd(a, b) = \gcd(|a|, |b|)$.

$$\gcd(60, 24) = \gcd(60, -24) = 12$$

Also, because all nonzero integers divide 0, we have $\gcd(a, 0) = |a|$.

We stated that two integers a and b are relatively prime if and only if their only common

positive integer factor is 1. This is equivalent to saying that a and b are relatively prime if $\gcd(a, b) = 1$.

8 and 15 are relatively prime because the positive divisors of 8 are 1, 2, 4, and 8, and the positive divisors of 15 are 1, 3, 5, and 15. So 1 is the only integer on both lists.

Finding the Greatest Common Divisor

We now describe an algorithm credited to Euclid for easily finding the greatest common divisor of two integers (Figure 1.7). This algorithm has broad significance in cryptography. The explanation of the algorithm can be broken down into the following points:

1. Suppose we wish to determine the greatest common divisor d of the integers a and b ; that is determine $d = \gcd(a, b)$. Because $\gcd(|a|, |b|) = \gcd(a, b)$, there is no harm in assuming $a \geq b > 0$.
2. Dividing a by b and applying the division algorithm, we can state:

$$a = q_1b + r_1 \quad 0 \leq r_1 < b \quad (2.2)$$

3. First consider the case in which $r_1 = 0$. Therefore, b divides a and clearly no larger number divides both b and a , because that number would be larger than b . So we have $d = \gcd(a, b) = b$.
4. The other possibility from Equation (2.2) is $r_1 \neq 0$. For this case, we can state that $d \mid r_1$. This is due to the basic properties of divisibility: the relations $d \mid a$ and $d \mid b$ together imply that $d \mid (a - q_1b)$, which is the same as $d \mid r_1$.
5. Before proceeding with the Euclidian algorithm, we need to answer the question: What is the $\gcd(b, r_1)$? We know that $d \mid b$ and $d \mid r_1$. Now take any arbitrary integer c that divides both b and r_1 . Therefore, $c \mid (q_1b + r_1) = a$. Because c divides both a and b , we must have $c \leq d$, which is the greatest common divisor of a and b . Therefore $d = \gcd(b, r_1)$.

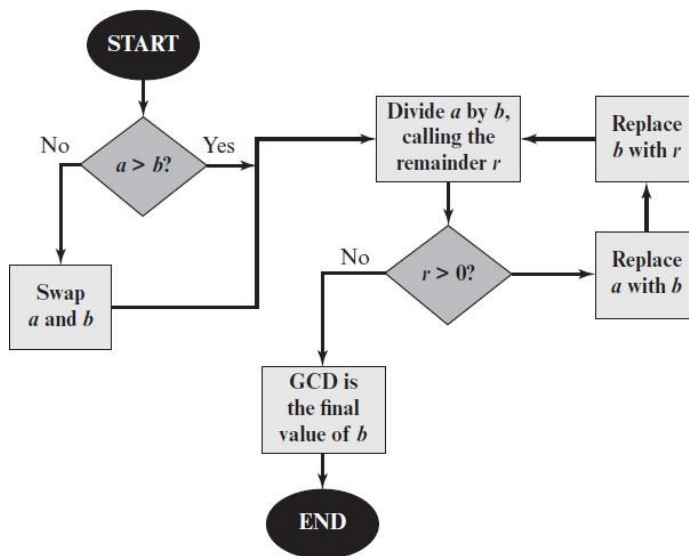


Fig 1.7 Euclidean Algorithm

Same GCD

GCD	GCD
$710 = 2 \times 310 + 90$	$310 = 3 \times 90 + 40$
$310 = 3 \times 90 + 40$	$90 = 2 \times 40 + 10$
$90 = 2 \times 40 + 10$	$40 = 4 \times 10$

Curved arrows in the original image connect the remainder of one equation to the dividend of the next, showing the iterative reduction process.

Fig 1.8 Euclidean Algorithm Example: gcd(710, 310)

6.3 MODULAR ARITHMETIC

The Modulus

If a is an integer and n is a positive integer, we define $a \bmod n$ to be the remainder when a is divided by n . The integer n is called the **modulus**. Thus, for any integer a , we can rewrite Equation (2.1) as follows:

$$a = qn + r \quad 0 \leq r < n; q = \lfloor a/n \rfloor$$

$$a = \lfloor a/n \rfloor \times n + (a \bmod n)$$

$$11 \bmod 7 = 4; \quad -11 \bmod 7 = 3$$

Two integers a and b are said to be **congruent modulo n** , if $(a \bmod n) = (b \bmod n)$. This is written as $a \equiv b \pmod{n}$.²

$$73 \equiv 4 \pmod{23}; \quad 21 \equiv -9 \pmod{10}$$

Note that if $a \equiv 0 \pmod{n}$, then $n \mid a$.

Properties of Congruences

Congruences have the following properties:

1. $a \equiv b \pmod{n}$ if $n \mid (a - b)$.
2. $a \equiv b \pmod{n}$ implies $b \equiv a \pmod{n}$.
3. $a \equiv b \pmod{n}$ and $b \equiv c \pmod{n}$ imply $a \equiv c \pmod{n}$.

To demonstrate the first point, if $n \mid (a - b)$, then $(a - b) = kn$ for some k . So we can write $a = b + kn$. Therefore, $(a \bmod n) = (\text{remainder when } b + kn \text{ is divided by } n) = (\text{remainder when } b \text{ is divided by } n) = (b \bmod n)$.

$$\begin{array}{lll} 23 \equiv 8 \pmod{5} & \text{because} & 23 - 8 = 15 = 5 \times 3 \\ -11 \equiv 5 \pmod{8} & \text{because} & -11 - 5 = -16 = 8 \times (-2) \\ 81 \equiv 0 \pmod{27} & \text{because} & 81 - 0 = 81 = 27 \times 3 \end{array}$$

The remaining points are as easily proved.

Modular Arithmetic Operations:

Note that, by definition (Figure 1.6), the $(\text{mod } n)$ operator maps all integers into the set of integers $\{0, 1, \dots, (n - 1)\}$. This suggests the question: Can we perform arithmetic operations within the confines of this set? It turns out that we can; this technique is known as **modular arithmetic**.

Modular arithmetic exhibits the following properties:

1. $[(a \text{ mod } n) + (b \text{ mod } n)] \text{ mod } n = (a + b) \text{ mod } n$
2. $[(a \text{ mod } n) - (b \text{ mod } n)] \text{ mod } n = (a - b) \text{ mod } n$
3. $[(a \text{ mod } n) \times (b \text{ mod } n)] \text{ mod } n = (a \times b) \text{ mod } n$

We demonstrate the first property. Define $(a \text{ mod } n) = ra$ and $(b \text{ mod } n) = rb$. Then we can write $a = ra + jn$ for some integer j and $b = rb + kn$ for some integer k .

Then,

$$\begin{aligned}(a + b) \text{ mod } n &= (ra + jn + rb + kn) \text{ mod } n \\&= (ra + rb + (k + j)n) \text{ mod } n \\&= (ra + rb) \text{ mod } n \\&= [(a \text{ mod } n) + (b \text{ mod } n)] \text{ mod } n\end{aligned}$$

The remaining properties are proven as easily. Here are examples of the three properties:

$$\begin{aligned}11 \text{ mod } 8 &= 3; 15 \text{ mod } 8 = 7 \\[(11 \text{ mod } 8) + (15 \text{ mod } 8)] \text{ mod } 8 &= 10 \text{ mod } 8 = 2 \\(11 + 15) \text{ mod } 8 &= 26 \text{ mod } 8 = 2 \\[(11 \text{ mod } 8) - (15 \text{ mod } 8)] \text{ mod } 8 &= -4 \text{ mod } 8 = 4 \\(11 - 15) \text{ mod } 8 &= -4 \text{ mod } 8 = 4 \\[(11 \text{ mod } 8) \times (15 \text{ mod } 8)] \text{ mod } 8 &= 21 \text{ mod } 8 = 5 \\(11 \times 15) \text{ mod } 8 &= 165 \text{ mod } 8 = 5\end{aligned}$$

Exponentiation is performed by repeated multiplication, as in ordinary arithmetic.

$$\begin{aligned}\text{To find } 11^7 \text{ mod } 13, \text{ we can proceed as follows:} \\11^2 &= 121 \equiv 4 \pmod{13} \\11^4 &= (11^2)^2 \equiv 4^2 \equiv 3 \pmod{13} \\11^7 &= 11 \times 11^2 \times 11^4 \\11^7 &\equiv 11 \times 4 \times 3 \equiv 132 \equiv 2 \pmod{13}\end{aligned}$$

Properties of Modular Arithmetic

Define the set Z_n as the set of nonnegative integers less than n : $Z_n = \{0, 1, c, (n - 1)\}$

Table: Arithmetic Modulo 8

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

(a) Addition modulo 8

×	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	0	2	4	6
3	0	3	6	1	4	7	2	5
4	0	4	0	4	0	4	0	4
5	0	5	2	7	4	1	6	3
6	0	6	4	2	0	6	4	2
7	0	7	6	5	4	3	2	1

(b) Multiplication modulo 8

w	$-w$	w^{-1}
0	0	—
1	7	1
2	6	—
3	5	3
4	4	—
5	3	5
6	2	—
7	1	7

(c) Additive and multiplicative inverse modulo 8

This is referred to as the **set of residues**, or **residue classes** (mod n). To be more precise, each integer in Z_n represents a residue class. We can label the residue classes (mod n) as $[0], [1], [2], c, [n - 1]$, where

$$[r] = \{a: a \text{ is an integer, } a \equiv r \pmod{n}\}$$

The residue classes (mod 4) are

$$[0] = \{ \dots, -16, -12, -8, -4, 0, 4, 8, 12, 16, \dots \}$$

$$[1] = \{ \dots, -15, -11, -7, -3, 1, 5, 9, 13, 17, \dots \}$$

$$[2] = \{ \dots, -14, -10, -6, -2, 2, 6, 10, 14, 18, \dots \}$$

$$[3] = \{ \dots, -13, -9, -5, -1, 3, 7, 11, 15, 19, \dots \}$$

Of all the integers in a residue class, the smallest nonnegative integer is the one used to represent the residue class. Finding the smallest nonnegative integer to which k is congruent modulo n is called **reducing k modulo n** .

If we perform modular arithmetic within Z_n , the properties shown in Table hold for integers in Z_n . We show in the next section that this implies that Z_n is a commutative ring with a multiplicative identity element.

There is one peculiarity of modular arithmetic that sets it apart from ordinary arithmetic. First, observe that (as in ordinary arithmetic) we can write the following:

$$\text{if } (a + b) \equiv (a + c) \pmod{n} \text{ then } b \equiv c \pmod{n} \quad (2.4)$$

$$(5 + 23) \equiv (5 + 7) \pmod{8}; 23 \equiv 7 \pmod{8}$$

Equation (2.4) is consistent with the existence of an additive inverse. Adding the additive inverse of a to both sides of Equation (2.4), we have

$$\begin{aligned} ((-a) + a + b) &\equiv ((-a) + a + c) \pmod{n} \\ b &\equiv c \pmod{n} \end{aligned}$$

Table: Properties of Modular Arithmetic for Integers in Z_n

Property	Expression
Commutative Laws	$(w + x) \bmod n = (x + w) \bmod n$ $(w \times x) \bmod n = (x \times w) \bmod n$
Associative Laws	$[(w + x) + y] \bmod n = [w + (x + y)] \bmod n$ $[(w \times x) \times y] \bmod n = [w \times (x \times y)] \bmod n$
Distributive Law	$[w \times (x + y)] \bmod n = [(w \times x) + (w \times y)] \bmod n$
Identities	$(0 + w) \bmod n = w \bmod n$ $(1 \times w) \bmod n = w \bmod n$
Additive Inverse ($-w$)	For each $w \in Z_n$, there exists a z such that $w + z \equiv 0 \pmod{n}$

However, the following statement is true only with the attached condition:

$$\text{if } (a \times b) \equiv (a \times c) \pmod{n} \text{ then } b \equiv c \pmod{n} \text{ if } a \text{ is relatively prime to } n \quad (2.5)$$

Recall that two integers are **relatively prime** if their only common positive integer factor is 1. Similar to the case of Equation (2.4), we can say that Equation (2.5) is consistent with the existence of a multiplicative inverse. Applying the multiplicative inverse of a to both sides of Equation (2.5), we have

$$\begin{aligned} ((a^{-1})ab) &\equiv ((a^{-1})ac)(\text{mod } n) \\ b &\equiv c(\text{mod } n) \end{aligned}$$

To see this, consider an example in which the condition of Equation (2.5) does not hold. The integers 6 and 8 are not relatively prime, since they have the common factor 2. We have the following:

$$\begin{aligned} 6 \times 3 &= 18 \equiv 2(\text{mod } 8) \\ 6 \times 7 &= 42 \equiv 2(\text{mod } 8) \end{aligned}$$

Yet $3 \not\equiv 7(\text{mod } 8)$.

Euclidean Algorithm Revisited

The Euclidean algorithm can be based on the following theorem: For any integers a, b , with $a \geq b \geq 0$,

$$\gcd(a, b) = \gcd(b, a \bmod b) \tag{2.6}$$

$$\gcd(55, 22) = \gcd(22, 55 \bmod 22) = \gcd(22, 11) = 11$$

To see that Equation (2.6) works, let $d = \gcd(a, b)$. Then, by the definition of $\gcd$, $d \mid a$ and $d \mid b$. For any positive integer b , we can express a as

$$\begin{aligned} a &= kb + r \equiv r(\text{mod } b) \\ a \bmod b &= r \end{aligned}$$

with k, r integers. Therefore, $(a \bmod b) = a - kb$ for some integer k . But because $d \mid b$, it also divides kb . We also have $d \mid a$. Therefore, $d \mid (a \bmod b)$. This shows that d is a common divisor of b and $(a \bmod b)$. Conversely, if d is a common divisor of b and $(a \bmod b)$, then $d \mid kb$ and thus $d \mid [kb + (a \bmod b)]$, which is equivalent to $d \mid a$. Thus, the set of common divisors of a and b is equal to the set of common divisors of b and $(a \bmod b)$. Therefore, the $\gcd$ of one pair is the same as the $\gcd$ of the other pair, proving the theorem.

Equation (2.6) can be used repetitively to determine the greatest common divisor.

$$\begin{aligned} \gcd(18, 12) &= \gcd(12, 6) = \gcd(6, 0) = 6 \\ \gcd(11, 10) &= \gcd(10, 1) = \gcd(1, 0) = 1 \end{aligned}$$

This is the same scheme shown in Equation (2.3), which can be rewritten in the following way.

Euclidean Algorithm	
Calculate	Which satisfies
$r_1 = a \bmod b$	$a = q_1b + r_1$
$r_2 = b \bmod r_1$	$b = q_2r_1 + r_2$
$r_3 = r_1 \bmod r_2$	$r_1 = q_3r_2 + r_3$
•	•
•	•
•	•
$r_n = r_{n-2} \bmod r_{n-1}$	$r_{n-2} = q_nr_{n-1} + r_n$
$r_{n+1} = r_{n-1} \bmod r_n = 0$	$r_{n-1} = q_{n+1}r_n + 0$ $d = \gcd(a, b) = r_n$

We can define the Euclidean algorithm concisely as the following recursive function.

```
Euclid(a,b)
  if (b=0) then return a;
  else return Euclid(b, a mod b);
```

6.4 PRIME NUMBERS

An integer $p > 1$ is a prime number if and only if its only divisors are ± 1 and $\pm p$. **Prime numbers** play a critical role in number theory. Table shows the primes less than 2000. Note the way the primes are distributed. In particular, note the number of primes in each range of 100 numbers.

Any integer $a > 1$ can be factored in a unique way as

$$a = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_i^{a_i} \tag{2.9}$$

where $p_1 < p_2 < \dots < p_t$ are prime numbers and where each a_i is a positive integer. This is known as the fundamental theorem of arithmetic; a proof can be found in any text on number theory.\

$$\begin{aligned} 91 &= 7 \times 13 \\ 3600 &= 2^4 \times 3^2 \times 5^2 \\ 11011 &= 7 \times 11^2 \times 13 \end{aligned}$$

It is useful for what follows to express this another way. If P is the set of all prime numbers, then any positive integer a can be written uniquely in the following form:

$$a = \prod_{p \in P} p^{a_p} \quad \text{where each } a_p \geq 0$$

The right-hand side is the product over all possible prime numbers p ; for any particular value of a , most of the exponents a_p will be 0.

The value of any given positive integer can be specified by simply listing all the nonzero exponents in the foregoing formulation.

The integer 12 is represented by $\{a_2 = 2, a_3 = 1\}$.
 The integer 18 is represented by $\{a_2 = 1, a_3 = 2\}$.
 The integer 91 is represented by $\{a_7 = 1, a_{13} = 1\}$.

Multiplication of two numbers is equivalent to adding the corresponding exponents.

$$\text{Given } a = \prod_{p \in P} p^{a_p}, b = \prod_{p \in P} p^{b_p}$$

Define $k = ab$. We know that the integer k can be expressed as the product of powers of primes:

$$k = \prod_{p \in P} p^{k_p}$$

It follows that $k_p = a_p + b_p$ for all $p \in P$

$$\begin{aligned}
 k &= 12 \times 18 = (2^2 \times 3) \times (2 \times 3^2) = 216 \\
 k_2 &= 2 + 1 = 3; k_3 = 1 + 2 = 3 \\
 216 &= 2^3 \times 3^3 = 8 \times 27
 \end{aligned}$$

What does it mean, in terms of the prime factors of a and b , to say that a divides b ? Any integer of the form pn can be divided only by an integer that is of a lesser or equal power of the same prime number, pj with $j \leq n$. Thus, we can say the following.

Given

$$\begin{aligned}
 a &= \prod_{p \in \mathbb{P}} p^{a_p}, b = \prod_{p \in \mathbb{P}} p^{b_p} \\
 \text{If } a \mid b, \text{ then } a_p &\leq b_p \text{ for all } p.
 \end{aligned}$$

$$\begin{aligned}
 a &= 12; b = 36; 12 \mid 36 \\
 12 &= 2^2 \times 3; 36 = 2^2 \times 3^2 \\
 a_2 &= 2 = b_2 \\
 a_3 &= 1 \leq 2 = b_3 \\
 \text{Thus, the inequality } a_p &\leq b_p \text{ is satisfied for all prime numbers.}
 \end{aligned}$$

It is easy to determine the greatest common divisor of two positive integers if we express each integer as the product of primes.

$$\begin{aligned}
 300 &= 2^2 \times 3^1 \times 5^2 \\
 18 &= 2^1 \times 3^2 \\
 \gcd(18, 300) &= 2^1 \times 3^1 \times 5^0 = 6
 \end{aligned}$$

The following relationship always holds:

$$\text{If } k = \gcd(a, b), \text{ then } k_p = \min(a_p, b_p) \text{ for all } p.$$

Determining the prime factors of a large number is no easy task, so the preceding relationship does not directly lead to a practical method of calculating the greatest common divisor.

6.5 FERMAT'S AND EULER'S THEOREMS

Two theorems that play important roles in public-key cryptography are Fermat's theorem

and Euler's theorem.

Fermat's Theorem

Fermat's theorem states the following: If p is prime and a is a positive integer not divisible by p , then

$$a^{p-1} \equiv 1 \pmod{p} \quad (2.10)$$

Proof: Consider the set of positive integers less than p : $\{1, 2, \dots, p-1\}$ and multiply each element by a , modulo p , to get the set $X = \{a \bmod p, 2a \bmod p, \dots, (p-1)a \bmod p\}$. None of the elements of X is equal to zero because p does not divide a . Furthermore, no two of the integers in X are equal. To see this, assume that $ja \equiv ka \pmod{p}$, where $1 \leq j < k \leq p-1$.

Because a is relatively prime to p , we can eliminate a from both sides of the equation [see Equation (2.3)] resulting in

$$j \equiv k \pmod{p}.$$

This last equality is impossible, because j and k are both positive integers less than p . Therefore, we know that the $(p-1)$ elements of X are all positive integers with no two elements equal. We can conclude the X consists of the set of integers $\{1, 2, \dots, p-1\}$ in some order. Multiplying the numbers in both sets (p and X) and taking the result mod p yields

$$a \times 2a \times \dots \times (p-1)a \equiv [(1 \times 2 \times \dots \times (p-1)) \pmod{p}]$$

$$a^{p-1}(p-1)! \equiv (p-1)! \pmod{p}$$

We can cancel the $(p-1)!$ term because it is relatively prime to p [see Equation (2.5)].

This yields Equation (2.10), which completes the proof.

$$a = 7, p = 19$$

$$7^2 = 49 \equiv 11 \pmod{19}$$

$$7^4 \equiv 121 \equiv 7 \pmod{19}$$

$$7^8 \equiv 49 \equiv 11 \pmod{19}$$

$$7^{16} \equiv 121 \equiv 7 \pmod{19}$$

$$a^{p-1} = 7^{18} = 7^{16} \times 7^2 \equiv 7 \times 11 \equiv 1 \pmod{19}$$

An alternative form of Fermat's theorem is also useful: If p is prime and a is a positive integer, then

$$a^p \equiv a \pmod{p} \quad (2.11)$$

Note that the first form of the theorem [Equation (2.10)] requires that a be relatively prime to p , but this form does not.

$$\begin{array}{ll} p = 5, a = 3 & a^p = 3^5 = 243 \equiv 3 \pmod{5} = a \pmod{p} \\ p = 5, a = 10 & a^p = 10^5 = 100000 \equiv 10 \pmod{5} \equiv 0 \pmod{5} = a \pmod{p} \end{array}$$

Euler's Totient Function

Before presenting Euler's theorem, we need to introduce an important quantity in number theory, referred to as **Euler's totient function**. This function, written $\phi(n)$, is defined as the number of positive integers less than n and relatively prime to n . By convention,

$$\phi(1) = 1.$$

Determine $\phi(37)$ and $\phi(35)$.

Because 37 is prime, all of the positive integers from 1 through 36 are relatively prime to 37. Thus $\phi(37) = 36$.

To determine $\phi(35)$, we list all of the positive integers less than 35 that are relatively prime to it:

1, 2, 3, 4, 6, 8, 9, 11, 12, 13, 16, 17, 18
19, 22, 23, 24, 26, 27, 29, 31, 32, 33, 34

There are 24 numbers on the list, so $\phi(35) = 24$.

Table lists the first 30 values of $\phi(n)$. The value $\phi(1)$ is without meaning but is defined to have the value 1. It should be clear that, for a prime number p ,

$$\phi(p) = p - 1$$

Now suppose that we have two prime numbers p and q with $p \neq q$. Then we can show that, for $n = pq$,

Table: Some Values of Euler's Totient Function $\phi(n)$

Euler's Theorem:

Euler's theorem states that for every a and n that are relatively prime:

$$a^{\phi(n)} \equiv 1 \pmod{n} \quad (2.12)$$

Proof: Equation (2.12) is true if n is prime, because in that case, $\phi(n) = (n - 1)$ and Fermat's theorem holds. However, it also holds for any integer n . Recall that $\phi(n)$ is the number of positive integers less than n that are relatively prime to n . Consider the set of such integers, labeled as

$$R = \{x_1, x_2, \dots, x_{\phi(n)}\}$$

That is, each element x_i of R is a unique positive integer less than n with $\gcd(x_i, n) = 1$.

Now multiply each element by a , modulo n :

$$S = \{(ax_1 \bmod n), (ax_2 \bmod n), \dots, (ax_{\phi(n)} \bmod n)\}$$

The set S is a permutation of R , by the following line of reasoning:

1. Because a is relatively prime to n and x_i is relatively prime to n , ax_i must also be relatively prime to n . Thus, all the members of S are integers that are less than n and that are relatively prime to n .
2. There are no duplicates in S . Refer to Equation (2.5). If $ax_i \bmod n = ax_j \bmod n$, then $x_i = x_j$.

Therefore,

$$\begin{aligned} \prod_{i=1}^{\phi(n)} (ax_i \bmod n) &= \prod_{i=1}^{\phi(n)} x_i \\ \prod_{i=1}^{\phi(n)} ax_i &\equiv \prod_{i=1}^{\phi(n)} x_i \pmod{n} \\ a^{\phi(n)} \times \left[\prod_{i=1}^{\phi(n)} x_i \right] &\equiv \prod_{i=1}^{\phi(n)} x_i \pmod{n} \\ a^{\phi(n)} &\equiv 1 \pmod{n} \end{aligned}$$

which completes the proof. This is the same line of reasoning applied to the proof of Fermat's theorem.

$$\begin{array}{ll} a = 3; n = 10; \phi(10) = 4; & a^{\phi(n)} = 3^4 = 81 = 1(\bmod 10) = 1(\bmod n) \\ a = 2; n = 11; \phi(11) = 10; & a^{\phi(n)} = 2^{10} = 1024 = 1(\bmod 11) = 1(\bmod n) \end{array}$$

As is the case for Fermat's theorem, an alternative form of the theorem is also useful:

$$a^{\phi(n)+1} \equiv a(\bmod n) \quad (2.13)$$

Again, similar to the case with Fermat's theorem, the first form of Euler's theorem [Equation (2.12)] requires that a be relatively prime to n , but this form does not.

6.6 THE CHINESE REMAINDER THEOREM

One of the most useful results of number theory is the **Chinese remainder theorem** (CRT). In essence, the CRT says it is possible to reconstruct integers in a certain range from their residues modulo a set of pairwise relatively prime moduli.

The 10 integers in Z_{10} , that is the integers 0 through 9, can be reconstructed from their two residues modulo 2 and 5 (the relatively prime factors of 10). Say the known residues of a decimal digit x are $r_2 = 0$ and $r_5 = 3$; that is, $x \bmod 2 = 0$ and $x \bmod 5 = 3$. Therefore, x is an even integer in Z_{10} whose remainder, on division by 5, is 3. The unique solution is $x = 8$.

The CRT can be stated in several ways. We present here a formulation that is most useful from the point of view of this text.

$$M = \prod_{i=1}^k m_i$$

where the m_i are pairwise relatively prime; that is, $\gcd(m_i, m_j) = 1$ for $1 \dots i, j \dots k$, and $i \neq j$. We can represent any integer A in Z_M by a k -tuple whose elements are in Z_{m_i} using the following correspondence:

$$A \leftrightarrow (a_1, a_2, \dots, a_k) \quad (2.15)$$

where $A \in Z_M$, $a_i \in Z_{m_i}$, and $a_i = A \bmod m_i$ for $1 \leq i \leq k$

The CRT makes two assertions.

1. The mapping of Equation (2.15) is a one-to-one correspondence (called a **bijection**) between ZM and the Cartesian product $Zm_1 * Zm_2 * \dots * Zm_k$. That is, for every integer A such that $0 \leq A \leq M$, there is a unique k -tuple $(a_1, a_2, \dots, a_k)$ with $0 \leq a_i \leq m_i$ that represents it, and for every such k -tuple $(a_1, a_2, \dots, a_k)$, there is a unique integer A in Z_M .
2. Operations performed on the elements of ZM can be equivalently performed on the corresponding k -tuples by performing the operation independently in each coordinate position in the appropriate system.

Let us demonstrate the **first assertion**. The transformation from A to $(a_1, a_2, \dots, a_k)$, is obviously unique; that is, each a_i is uniquely calculated as $a_i = A \bmod m_i$. Computing A from $(a_1, a_2, \dots, a_k)$ can be done as follows. Let

$M_i = M/m_i$ for $1 \leq i \leq k$. Note that $M_i = m_1 \times m_2 \times \dots \times m_{i-1} \times m_{i+1} \times \dots \times m_k$, so that $M_i \equiv 0 \pmod{m_j}$ for all $j \neq i$. Then let

$$c_i = M_i \times (M_i^{-1} \bmod m_i) \quad \text{for } 1 \leq i \leq k \quad (2.16)$$

By the definition of M_i , it is relatively prime to m_i and therefore has a unique multiplicative inverse mod m_i . So Equation (2.16) is well defined and produces a unique value c_i . We can now compute

$$A \equiv \left(\sum_{i=1}^k a_i c_i \right) \pmod{M} \quad (2.17)$$

To show that the value of A produced by Equation (2.17) is correct, we must show that $a_i = A \bmod m_i$ for $1 \leq i \leq k$. Note that $c_j \equiv M_j \equiv 0 \pmod{m_i}$ if $j \neq i$, and that $c_i \equiv 1 \pmod{m_i}$. It follows that $a_i = A \bmod m_i$.

The **second assertion** of the CRT, concerning arithmetic operations, follows from the rules for modular arithmetic. That is, the second assertion can be stated as follows: If

$$\begin{aligned} A &\leftrightarrow (a_1, a_2, \dots, a_k) \\ B &\leftrightarrow (b_1, b_2, \dots, b_k) \end{aligned}$$

then

$$\begin{aligned} (A + B) \bmod M &\leftrightarrow ((a_1 + b_1) \bmod m_1, \dots, (a_k + b_k) \bmod m_k) \\ (A - B) \bmod M &\leftrightarrow ((a_1 - b_1) \bmod m_1, \dots, (a_k - b_k) \bmod m_k) \\ (A \times B) \bmod M &\leftrightarrow ((a_1 \times b_1) \bmod m_1, \dots, (a_k \times b_k) \bmod m_k) \end{aligned}$$

6.7 Fast computation of powers in multiplicative functions:

In number theory, specifically in the context of modular arithmetic, primitive roots are significant elements. Let's break down what primitive roots are and their connection to the structure of the multiplicative group of integers modulo a prime p , denoted as F^*_p .

Primitive Roots:

A primitive root modulo p is an integer g such that the powers of g modulo p generate all non-zero residues modulo p . In other words, g is a generator of the multiplicative group F^*_p .

More formally, g is a primitive root modulo p if for every integer a coprime to p , there exists an integer k such that $g^k \equiv a \pmod{p}$.

Structure of F^*_p :

The multiplicative group F^*_p consists of all positive integers less than p that are coprime to p . Its order is $p-1$.

Primitive roots play a crucial role in understanding the structure of F^*_p . If a primitive root modulo p exists, then F^*_p is cyclic, meaning it can be generated by a single element. This generator, which is the primitive root, generates all other elements in the group by taking successive powers of it.

Applications:

Cryptography: Primitive roots are used in cryptographic algorithms, particularly in public-key cryptography systems like Diffie-Hellman key exchange and ElGamal encryption.

Number Theory: They have applications in various number theoretic problems, including factorization and discrete logarithm problem.

Random Number Generation: Primitive roots can be utilized for generating pseudorandom numbers in certain algorithms.

Finding Primitive Roots:

Finding primitive roots modulo a prime p is not always straightforward. However, for certain primes, primitive roots exist. The existence of primitive roots modulo p is guaranteed if p is of the form $2, 4, p^k$, or $2p^k$, where p is an odd prime and k is a positive

integer.

For primes $p=2$ or $p=4$, F^*_p is cyclic, and 1 is a primitive root.

For other primes, finding primitive roots involves some computational techniques, like trial and error or more sophisticated algorithms like the Index Calculus algorithm.

6.8 Number in other bases:

In number theory, the representation of numbers in different bases plays a crucial role in understanding various properties and patterns. Here's how numbers in different bases are relevant in number theory:

1. Prime Factorization:

Representation in different bases can sometimes reveal patterns that aid in prime factorization. For instance, the binary representation of a number may show a repetitive pattern that indicates divisibility by a certain prime number.

2. Digital Roots:

Digital roots, obtained by repeatedly summing the digits of a number until a single-digit number is obtained, may exhibit interesting properties in different bases. These properties often lead to divisibility tests and other number-theoretic insights.

3. Congruences:

Representing numbers in different bases is essential when dealing with congruences modulo n . The choice of base can simplify certain congruence relations and reveal congruence patterns that may not be apparent in base-10 representation.

4. Quadratic Residues:

Representation in different bases can be useful when studying quadratic residues modulo n . For example, the Legendre symbol, used to determine whether a number is a quadratic residue modulo a prime, can sometimes be simplified by representing numbers in bases where certain properties become more apparent.

5. Continued Fractions:

Representation in different bases is also relevant in the study of continued fractions.

Continued fractions provide an alternative representation of real numbers and are used in various areas of number theory, including Diophantine approximation and Pell's equation.

6. Recurrence Relations:

Certain recurrence relations and sequences exhibit interesting properties when represented in different bases. For example, the Fibonacci sequence in base b may have distinct patterns compared to its decimal representation.

6.8 Fast computation of powers in $\mathbb{Z}/m\mathbb{Z}$:

In number theory, the computation of powers in the ring $\mathbb{Z}/m\mathbb{Z}$ is often approached using techniques that exploit the properties of modular arithmetic. The most commonly used algorithm for fast computation of powers in this context is the exponentiation by squaring algorithm. This algorithm is essentially the same as the one described in the previous response, but it's worth emphasizing its relevance within the realm of number theory.

Exponentiation by Squaring Algorithm:

The exponentiation by squaring algorithm leverages the binary representation of the exponent to minimize the number of multiplications required to compute the power.

Initialize result: Start with $\text{result} = 1$.

Iterate through the bits of the exponent b :

For each bit of b , starting from the most significant bit (leftmost bit):

Square the base and take the result modulo m .

If the current bit is 1, multiply the result by the base and take the result modulo m .

Return the result.

Example: Euler's Totient Function ($\phi(n)$):

Euler's totient function computes the count of numbers less than n that are coprime to n .

Efficient computation of $\phi(n^k)$ for large k is crucial in various applications.

Using the property $\phi(ab) = \phi(a)\phi(b)$ for coprime a and b , we can compute $\phi(n^k)$ efficiently:

1. Prime Factorization of n : Decompose n into its prime factors: $n = p_1^{a_1} p_2^{a_2} \dots p_r^{a_r}$

2. Compute $\phi(p_i^{a_i})$ for Each Prime Power: Since $\phi(p^k) = p^k - p^{k-1}$, we can calculate $\phi(p_i^{a_i})$ using this formula.

Combine Results: Use the multiplicative property of $\phi(n)$ to combine the results obtained

for each prime power.

6.9 Fast computation of powers in group theory:

In number theory, particularly when dealing with computations involving powers in groups, certain algorithms and techniques can significantly speed up the process. Here are some methods used for fast computation of powers in number theory:

Binary Exponentiation (Exponentiation by Squaring):

As mentioned earlier, this algorithm efficiently computes a^n by recursively squaring and multiplying.

It's particularly useful in modular exponentiation, where computations are done modulo m .

Reduces the number of multiplications required from n to $O(\log n)$.

Modular Exponentiation:

In number theory, many computations involve modular arithmetic, where numbers are reduced modulo a prime or composite number.

Modular exponentiation can be efficiently computed using binary exponentiation or Montgomery exponentiation.

Pohlig-Hellman Algorithm:

Used for computing discrete logarithms in finite fields of small characteristic in number theory.

Reduces the problem to smaller subgroups to compute the discrete logarithm faster.

Index Calculus Algorithm:

Efficient for computing discrete logarithms in certain finite fields in number theory.

It's particularly useful in fields where the Number Field Sieve (NFS) is not applicable.

Shanks' Baby-Step Giant-Step Algorithm:

Used for computing discrete logarithms in cyclic groups.

Reduces the problem to $O(\sqrt{q})$ operations, where q is the order of the group.

Square-and-Multiply for Exponential Functions:

In number theory, especially in modular arithmetic, square-and-multiply algorithm is used for efficiently computing powers of numbers modulo m .

Fast Fourier Transform (FFT):

In certain finite fields, FFT can be utilized to speed up polynomial multiplication, which in turn can accelerate exponentiation.

These techniques are crucial in number theory, especially in cryptographic applications where efficient computation of powers is essential for security and performance reasons. Depending on the specific problem and the properties of the numbers involved, different methods may be more suitable or efficient.

6.10 Fast computation of powers in fields:

In number theory, fields are crucial mathematical structures, and efficient computation of powers within them is essential for various algorithms and cryptographic protocols. Here are some methods used for fast computation of powers in fields:

Binary Exponentiation (Exponentiation by Squaring):

This algorithm efficiently computes a^n by recursively squaring and multiplying.

It's particularly useful in modular exponentiation within finite fields, where computations are done modulo p , a prime number.

Reduces the number of multiplications required from n to $O(\log n)$.

Montgomery Exponentiation:

Specifically designed for modular exponentiation in finite fields.

Reduces the number of modular multiplications required, which is particularly advantageous in cryptographic applications.

Nth Power Residue Symbol:

Used in certain fields to efficiently compute the n th power residue symbol of an element.

This symbol is crucial in certain cryptographic algorithms, such as primality testing and factoring algorithms.

Nimbirovic and Reitwiesner Polynomials:

These polynomials are used for efficient computation of powers in finite fields, particularly in certain elliptic curve cryptography algorithms.

Fast Fourier Transform (FFT):

In certain finite fields, FFT can be utilized to speed up polynomial multiplication, which

in turn can accelerate exponentiation.

Cyclotomic Coset Method:

Useful for certain types of finite field arithmetic, especially when dealing with computations in cyclotomic fields.

Optimized Arithmetic in Specific Fields:

For certain types of fields, such as binary fields (Galois fields), specialized techniques and optimizations exist for efficient computation of powers.

Quadratic Residuosity and Quadratic Non-Residuosity Testing:

In certain fields, efficient algorithms exist for determining whether an element is a quadratic residue or a quadratic non-residue, which can be useful in various cryptographic applications.

These methods are essential in number theory, particularly in cryptography, where efficient computation of powers in finite fields is crucial for security and performance reasons. Depending on the specific field and the problem at hand, different methods may be more suitable or efficient.

6.11 Fast computation of powers in problems:

In number theory, fast computation of powers is crucial for various problems, ranging from primality testing to cryptographic algorithms. Here are some methods used for fast computation of powers in number theory problems:

Binary Exponentiation (Exponentiation by Squaring):

This algorithm efficiently computes a^n by recursively squaring and multiplying.

It's particularly useful in modular exponentiation, where computations are done modulo m .

Reduces the number of multiplications required from n to $O(\log n)$.

Modular Exponentiation:

Many problems in number theory involve modular arithmetic, where numbers are reduced modulo a prime or composite number.

Modular exponentiation can be efficiently computed using binary exponentiation or Montgomery exponentiation.

Discrete Logarithm Problem Algorithms:

Problems involving discrete logarithms, such as determining the exponent in modular exponentiation, are crucial in number theory and cryptography.

Algorithms like Pollard's rho algorithm, Pohlig-Hellman algorithm, and index calculus algorithms are used for solving discrete logarithm problems efficiently.

Primality Testing Algorithms:

Fast exponentiation is often used in primality testing algorithms such as Miller-Rabin primality test and Solovay-Strassen primality test.

These algorithms involve modular exponentiation and require efficient computation of powers to determine the primality of a number.

Factorization Algorithms:

Factorization algorithms like Pollard's rho algorithm, Pollard's $p-1$ algorithm, and Lenstra's elliptic curve factorization algorithm involve exponentiation operations.

Fast computation of powers is crucial for the efficient execution of these algorithms in factorizing integers.

Cryptography:

Many cryptographic algorithms rely on fast exponentiation, including RSA, Diffie-Hellman key exchange, and elliptic curve cryptography.

Efficient computation of powers is essential for secure and fast encryption and decryption processes in these algorithms.

Elliptic Curve Cryptography (ECC):

ECC involves operations on points of elliptic curves, including scalar multiplication, which requires fast exponentiation.

Techniques such as Montgomery ladder algorithm and sliding window method are used for efficient scalar multiplication in ECC.

These methods are fundamental in solving various number theory problems efficiently. Depending on the specific problem and the requirements, different algorithms and techniques may be applied for fast computation of powers.

UNIT II

ENCRYPTION TECHNIQUES AND KEY MANAGEMENT

Symmetric Encryption Principles – Data Encryption Standards – Advanced Encryption Standards – Stream Ciphers and RC4 – Cipher Block Modes Operation – Digital Signatures – Key Distributions – Public Key Cryptosystems: RSA, Elliptic Curve Cryptography – Key Exchange Algorithms: Diffie Hellmen and ELGamal Key Exchange.

2.1 SYMMETRIC ENCRYPTION PRINCIPLES

An original message is known as the **plaintext**, while the coded message is called the **ciphertext**. The process of converting from plaintext to ciphertext is known as **enciphering** or **encryption**; restoring the plaintext from the ciphertext is **deciphering** or **decryption**. The many schemes used for encryption constitute the area of study known as **cryptography**. Such a scheme is known as a **cryptographic system** or a **cipher**. Techniques used for deciphering a message without any knowledge of the enciphering details fall into the area of **cryptanalysis**.

Cryptanalysis is what the layperson calls “breaking the code.” The areas of cryptography and cryptanalysis together are called **cryptology**.

Symmetric encryption, also referred to as conventional encryption, secret - key, or single key encryption, was the only type of encryption in use before the development of public - key encryption in the late 1970s . It remains the most widely used of the two types of encryptions. A symmetric encryption scheme has five ingredients as shown in below diagram.

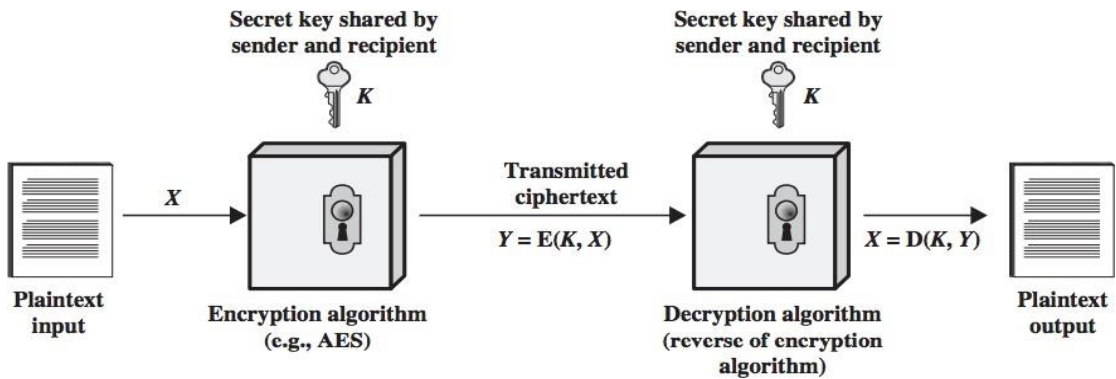


Fig 2.1 Simplified Model of Symmetric Encryption

A symmetric encryption scheme has five ingredients (Figure 2.1):

Plaintext: This is the original intelligible message or data that is fed into the algorithm as input.

■ **Encryption algorithm:** The encryption algorithm performs various substitutions and transformations on the plaintext.

■ **Secret key:** The secret key is also input to the encryption algorithm. The key is a value independent of the plaintext and of the algorithm. The algorithm will produce a different output depending on the specific key being used at the time. The exact substitutions and transformations performed by the algorithm depend on the key.

■ **Ciphertext:** This is the scrambled message produced as output. It depends on the plaintext and the secret key. For a given message, two different keys will produce two different ciphertexts. The ciphertext is an apparently random stream of data and, as it stands, is unintelligible.

■ **Decryption algorithm:** This is essentially the encryption algorithm run in reverse. It takes the ciphertext and the secret key and produces the original plaintext.

There are two requirements for secure use of conventional encryption:

Let us take a closer look at the essential elements of a symmetric encryption scheme, using Figure 2.2. A source produces a message in plaintext, $X = [X_1, X_2, \dots, X_M]$. The M elements of X are letters in some finite alphabet. Traditionally, the alphabet usually consisted of the 26 capital letters. Nowadays, the binary alphabet $\{0, 1\}$ is typically used.

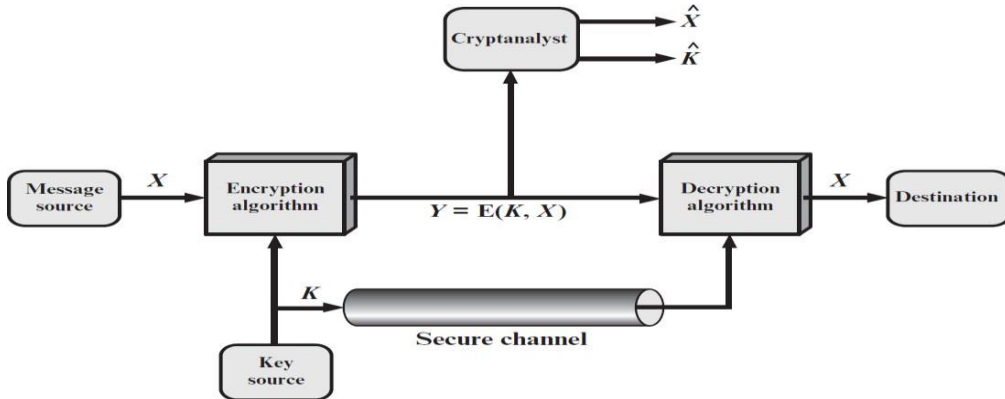


Fig 2.2 Model of Symmetric Cryptosystem

For encryption, a key of the form $K = [K1, K2, c, KJ]$ is generated. If the key is generated at the message source, then it must also be provided to the destination by means of some secure channel. Alternatively, a third party could generate the key and securely deliver it to both source and destination.

With the message X and the encryption key K as input, the encryption algorithm forms the ciphertext $Y = [Y1, Y2, c, YN]$. We can write this as

$$Y = E(K, X)$$

This notation indicates that Y is produced by using encryption algorithm E as a function of the plaintext X , with the specific function determined by the value of the key K .

The intended receiver, in possession of the key, is able to invert the transformation:

$$X = D(K, Y)$$

2.2 Data Encryption Standard (DES)

2.2.1 Stream Ciphers and Block Ciphers:

A **stream cipher** is one that encrypts a digital data stream one bit or one byte at a time. Examples of classical stream ciphers are the autokeyed Vigenère cipher and the Vernam cipher.

In the ideal case, a one-time pad version of the Vernam cipher would be used (Figure 2.3), in which the keystream (ki) is as long as the plaintext bit stream (pi). If the

cryptographic keystream is random, then this cipher is unbreakable by any means other than acquiring the keystream.

A **block cipher** is one in which a block of plaintext is treated as a whole and used to produce a ciphertext block of equal length. Typically, a block size of 64 or 128 bits is used. As with a stream cipher, the two users share a symmetric encryption key.

Using some of the modes of operation, a block cipher can be used to achieve the same effect as a stream cipher.

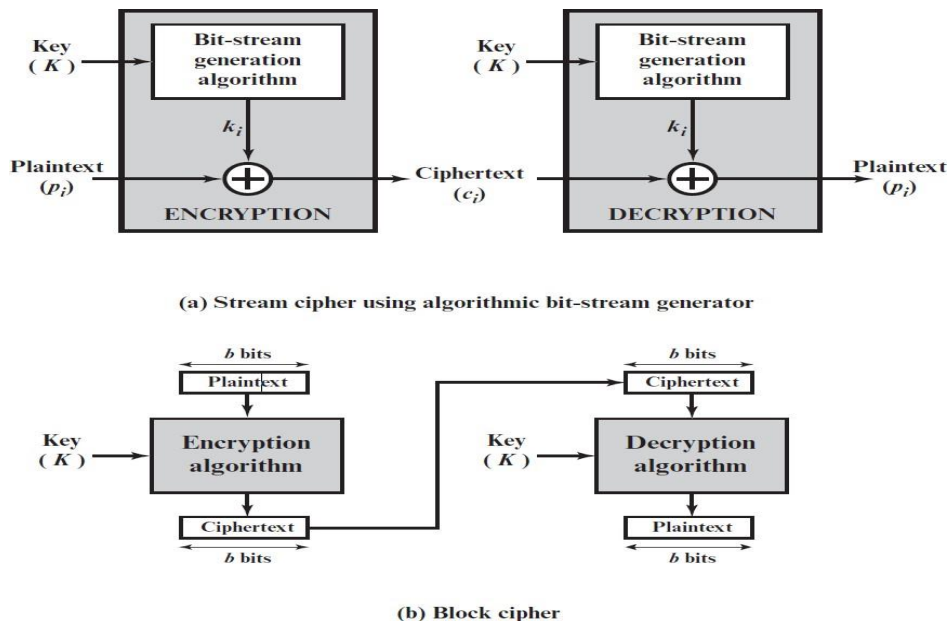


Fig 2.3 Stream Cipher and Block Cipher

2.2.2 Data Encryption Standard (DES):

DES was issued in 1977 by the National Bureau of Standards, now the National Institute of Standards and Technology (NIST), as Federal Information Processing Standard 46 (FIPS PUB 46). The algorithm itself is referred to as the Data Encryption Algorithm (DEA). For DEA, data are encrypted in 64-bit blocks using a 56-bit key. The algorithm transforms 64-bit input in a series of steps into a 64-bit output. The same steps, with the same key, are used to reverse the encryption.

DES Encryption:

The overall scheme for DES encryption is illustrated in Figure 2.4. As with any

encryption scheme, there are two inputs to the encryption function: the plaintext to be encrypted and the key. In this case, the plaintext must be 64 bits in length and the key is 56 bits in length.

Looking at the left-hand side of the figure, we can see that the processing of the plaintext proceeds in three phases. First, the 64-bit plaintext passes through an initial permutation (IP) that rearranges the bits to produce the *permuted input*.

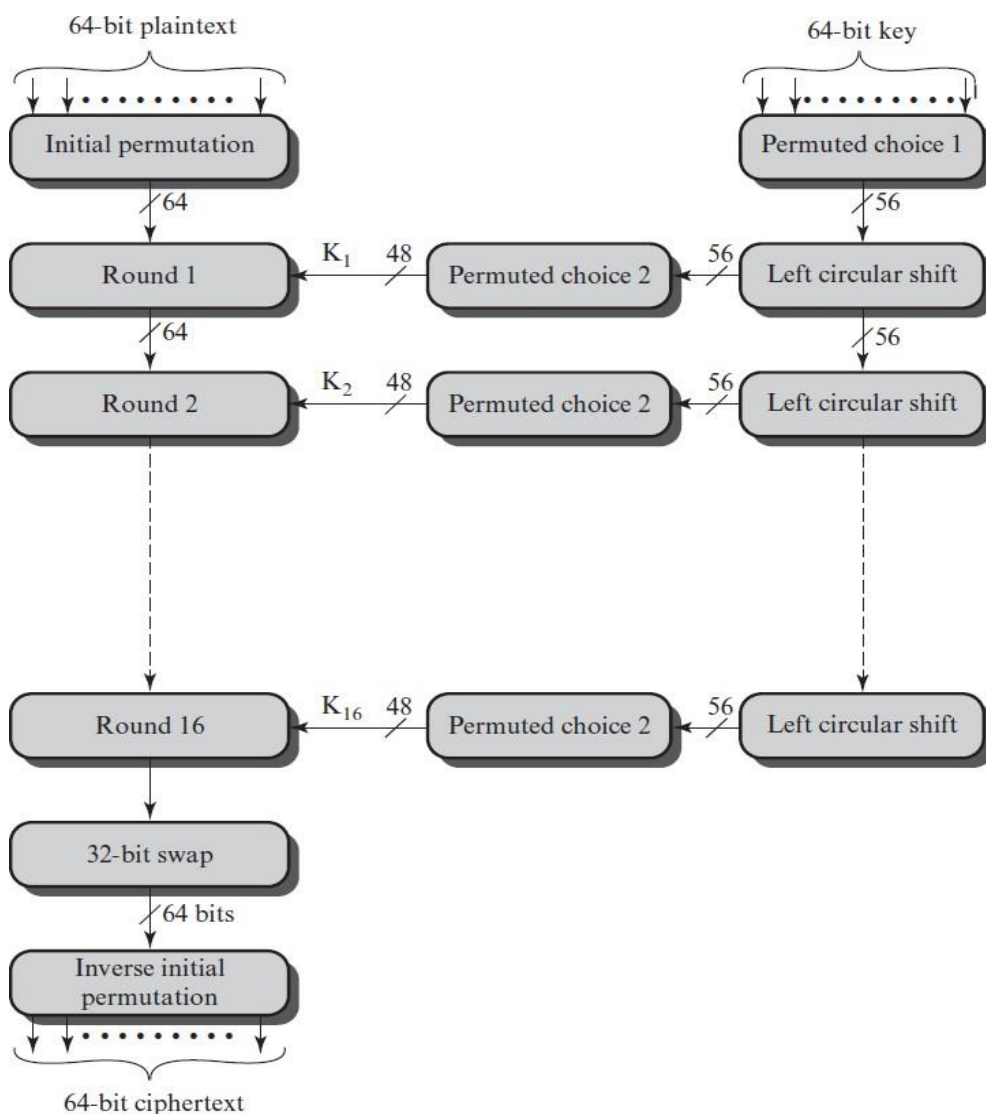


Fig 2.4 General Depiction of DES Encryption Algorithm

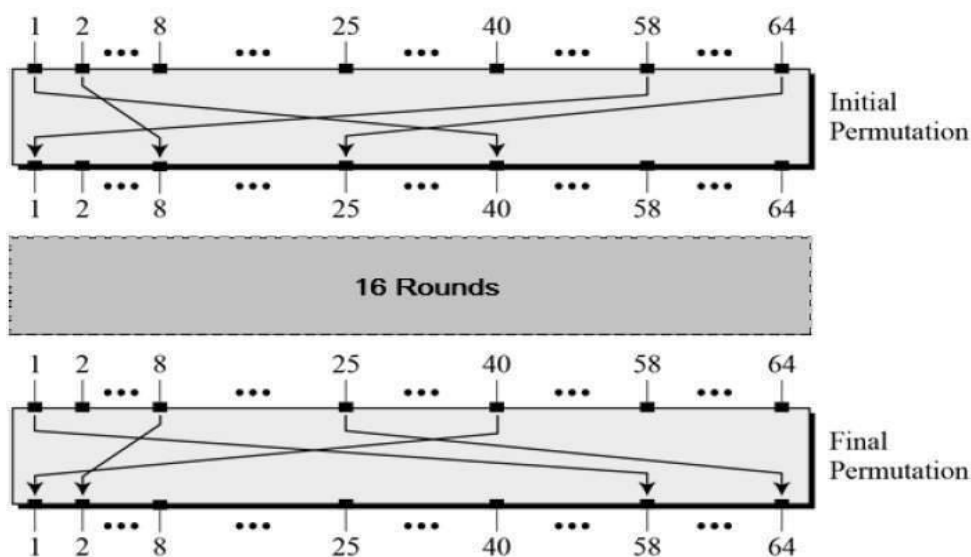
The right-hand portion of Figure 2.4 shows the way in which the 56-bit key is used. Initially, the key is passed through a permutation function. Then, for each of the sixteen rounds, a *subkey* (K_i) is produced by the combination of a left circular shift and a permutation. The permutation function is the same for each round, but a different subkey is produced because of the repeated shifts of the key bits.

DES Decryption

As with any Feistel cipher, decryption uses the same algorithm as encryption, except that the application of the subkeys is reversed. Additionally, the initial and final permutations are reversed.

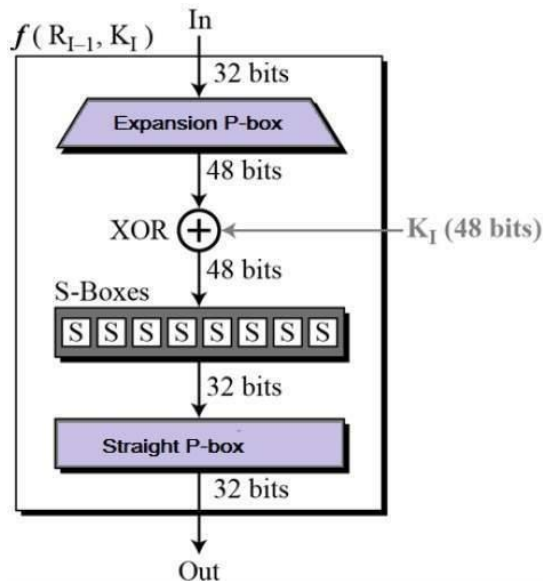
Initial Permutation and Final Permutation (Inverse Initial Permutation):

The initial and final permutations are straight Permutation boxes (P-boxes) that are inverses of each other. They have no cryptography significance in DES. The initial and final permutations are shown as follows –

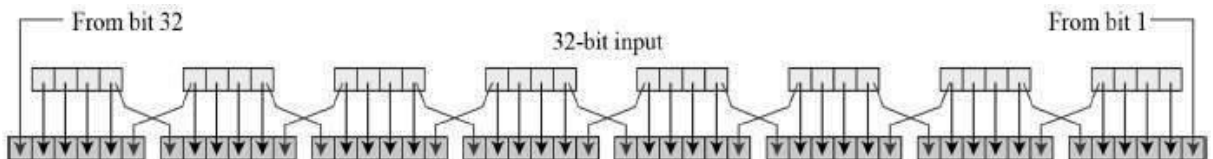


Round Function

The heart of this cipher is the DES function, f . The DES function applies a 48-bit key to the rightmost 32 bits to produce a 32-bit output.



Expansion Permutation Box – Since right input is 32-bit and round key is a 48-bit, we first need to expand right input to 48 bits. Permutation logic is graphically depicted in the following illustration –

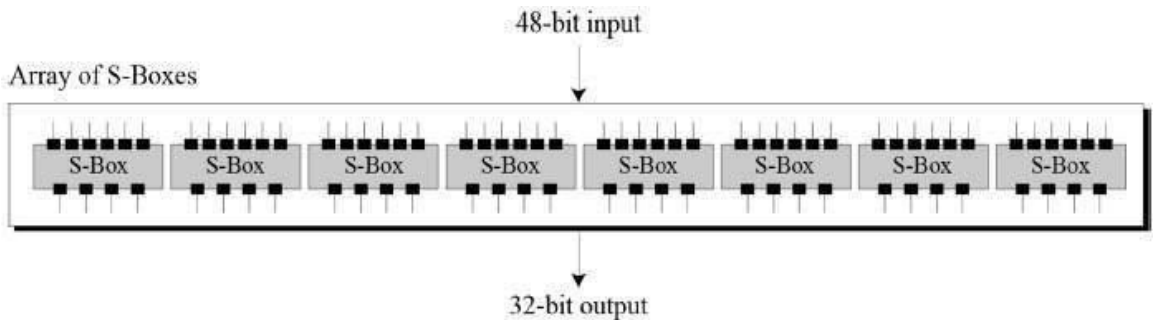


The graphically depicted permutation logic is generally described as table in DES specification illustrated as shown –

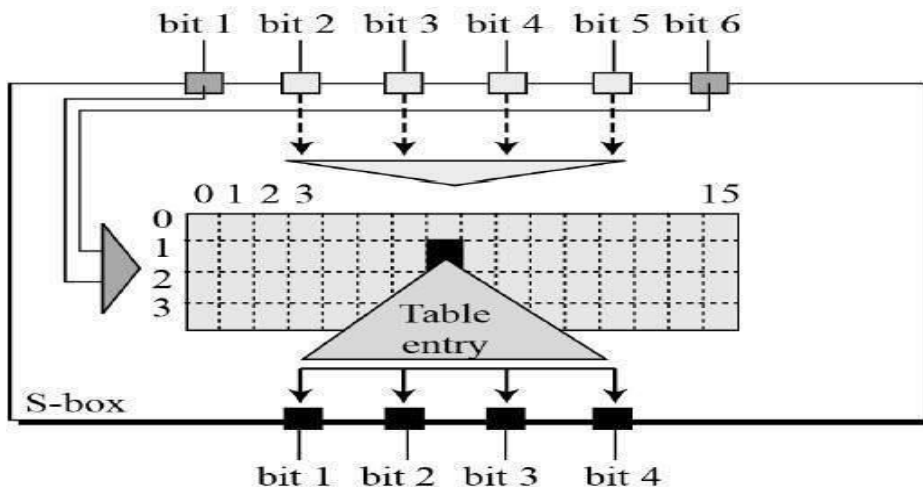
32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	31	31	32	01

XOR (Whitener). – After the expansion permutation, DES does XOR operation on the expanded right section and the round key. The round key is used only in this operation.

Substitution Boxes. – The S-boxes carry out the real mixing (confusion). DES uses 8 S-boxes, each with a 6-bit input and a 4-bit output. Refer the following illustration –



The S-box rule is illustrated below –



- There are a total of eight S-box tables. The output of all eight s-boxes is then combined in to 32 bit section.

Straight Permutation – The 32 bit output of S-boxes is then subjected to the straight permutation with rule shown in the following illustration:

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Key Generation

The round-key generator creates sixteen 48-bit keys out of a 56-bit cipher key. The process of key generation is depicted in the following illustration –

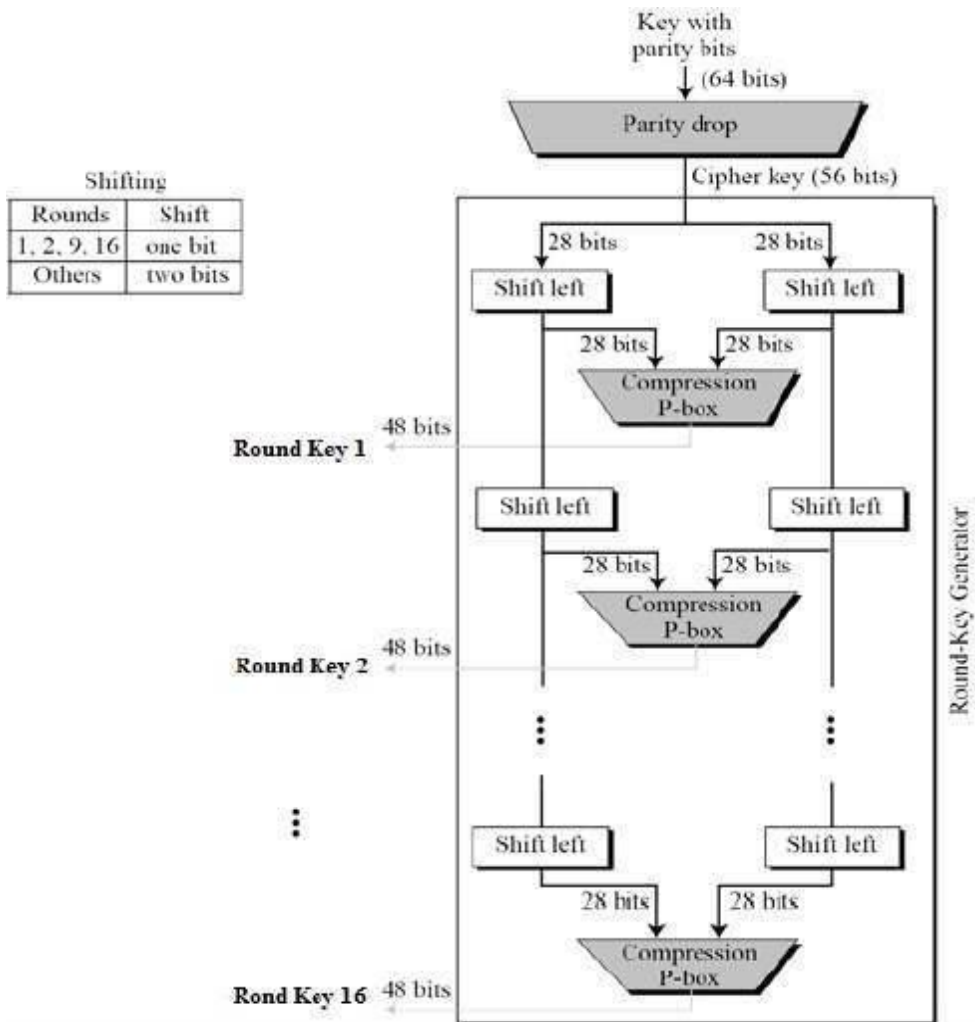


Fig 2.5 DES Key Generation

The logic for Parity drop, shifting, and Compression P-box is given in the DES description.

DES Analysis

The DES satisfies both the desired properties of block cipher. These two properties make cipher very strong.

- **Avalanche effect** – A small change in plaintext results in the very great change in the ciphertext.
- **Completeness** – Each bit of ciphertext depends on many bits of plaintext.

During the last few years, cryptanalysis have found some weaknesses in DES when key selected are weak keys. These keys shall be avoided.

DES has proved to be a very well-designed block cipher. There have been no significant cryptanalytic attacks on DES other than exhaustive key search.

2.3 Advanced Encryption Standard (AES)

The more popular and widely adopted symmetric encryption algorithm likely to be encountered nowadays is the Advanced Encryption Standard (AES). It is found at least six time faster than triple DES.

A replacement for DES was needed as its key size was too small. With increasing computing power, it was considered vulnerable against exhaustive key search attack. Triple DES was designed to overcome this drawback but it was found slow.

The features of AES are as follows –

- Symmetric key symmetric block cipher
- 128-bit data, 128/192/256-bit keys
- Stronger and faster than Triple-DES
- Provide full specification and design details
- Software implementable in C and Java

Operation of AES

AES is an iterative rather than Feistel cipher. It is based on ‘substitution–permutation network’. It comprises of a series of linked operations, some of which involve replacing inputs by specific outputs (substitutions) and others involve shuffling bits around (permutations).

Interestingly, AES performs all its computations on bytes rather than bits. Hence, AES treats the 128 bits of a plaintext block as 16 bytes. These 16 bytes are arranged in four

columns and four rows for processing as a matrix –

Unlike DES, the number of rounds in AES is variable and depends on the length of the key. AES uses 10 rounds for 128-bit keys, 12 rounds for 192-bit keys and 14 rounds for 256-bit keys. Each of these rounds uses a different 128-bit round key, which is calculated from the original AES key.

The schematic of AES structure is given in the following illustration –

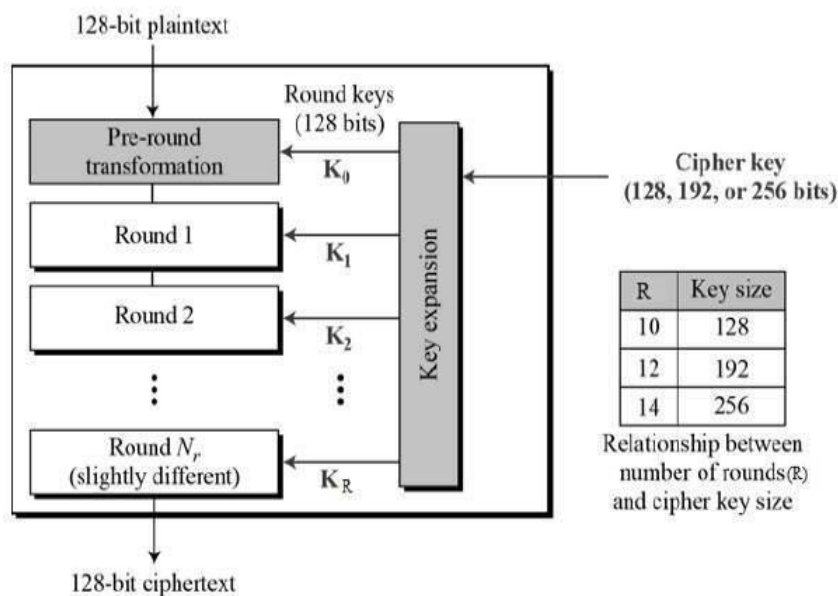
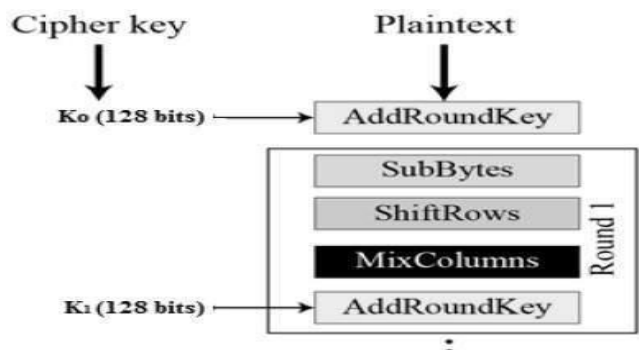


Fig 2.6 AES

Encryption Process

Here, we restrict to description of a typical round of AES encryption. Each round comprise of four sub-processes. The first-round process is depicted below –



Byte Substitution (SubBytes)

The 16 input bytes are substituted by looking up a fixed table (S-box) given in design.

The result is in a matrix of four rows and four columns.

Shiftrows

Each of the four rows of the matrix is shifted to the left. Any entries that ‘fall off’ are re-inserted on the right side of row. Shift is carried out as follows –

- First row is not shifted.
- Second row is shifted one (byte) position to the left.
- Third row is shifted two positions to the left.
- Fourth row is shifted three positions to the left.
- The result is a new matrix consisting of the same 16 bytes but shifted with respect to each other.

MixColumns

Each column of four bytes is now transformed using a special mathematical function. This function takes as input the four bytes of one column and outputs four completely new bytes, which replace the original column. The result is another new matrix consisting of 16 new bytes. It should be noted that this step is not performed in the last round.

Addroundkey

The 16 bytes of the matrix are now considered as 128 bits and are XORed to the 128 bits of the round key. If this is the last round then the output is the ciphertext. Otherwise, the resulting 128 bits are interpreted as 16 bytes and we begin another similar round.

Decryption Process

The process of decryption of an AES ciphertext is similar to the encryption process in the reverse order. Each round consists of the four processes conducted in the reverse order –

- Add round key
- Mix columns
- Shift rows

- Byte substitution

Since sub-processes in each round are in reverse manner, unlike for a Feistel Cipher, the encryption and decryption algorithms need to be separately implemented, although they are very closely related.

AES Analysis

In present day cryptography, AES is widely adopted and supported in both hardware and software. Till date, no practical cryptanalytic attacks against AES has been discovered. Additionally, AES has built-in flexibility of key length, which allows a degree of ‘future-proofing’ against progress in the ability to perform exhaustive key searches.

However, just as for DES, the AES security is assured only if it is correctly implemented and good key management is employed.

2.4 Stream Cipher and RC4

A typical stream cipher encrypts plaintext one byte at a time, although a stream cipher may be designed to operate on one bit at a time or on units larger than a byte at a time. Figure 2.6 is a representative diagram of stream cipher structure. In this structure a key is input to a pseudorandom bit generator that produces a stream of 8-bit numbers that are apparently random.

A pseudorandom stream is one that is generated by an algorithm but is unpredictable without knowledge of the input key. The output of the generator, called a keystream, is combined one byte at a time with the plaintext stream using the bitwise exclusive-OR (XOR) operation. For example, if the next byte generated by the generator is 01101100 and the next plaintext byte is 11001100, then the resulting ciphertext byte is:

$$\begin{array}{rcl}
 & 11001100 & \text{plaintext} \\
 \oplus & 01101100 & \text{key stream} \\
 \hline
 & 10100000 & \text{ciphertext}
 \end{array}$$

Decryption requires the use of the same pseudorandom sequence:

$$\begin{array}{rcl}
 & 10100000 & \text{ciphertext} \\
 \oplus & 01101100 & \text{key stream} \\
 \hline
 & 11001100 & \text{plaintext}
 \end{array}$$

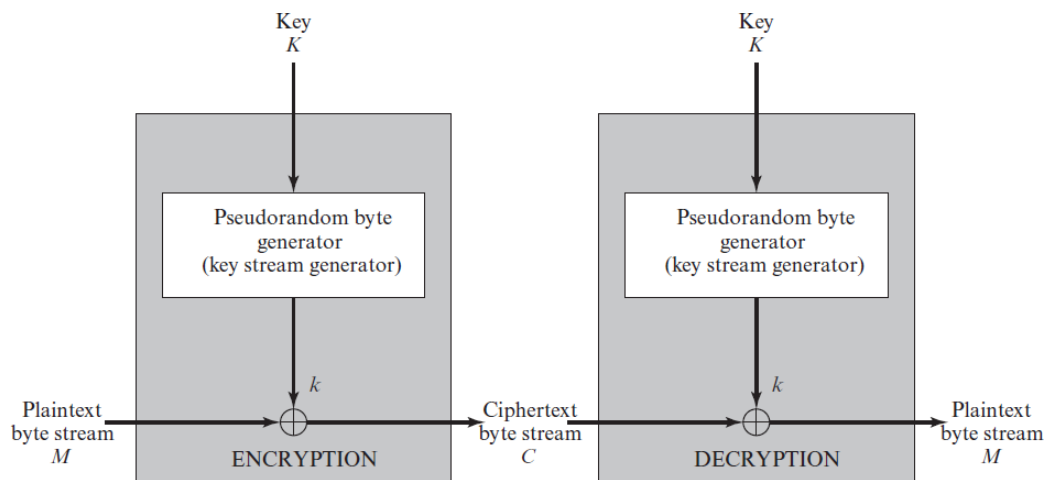


Fig 2.6 Stream Cipher Diagram

For applications that require encryption/decryption of a stream of data, such as over a data communications channel or a browser/Web link, a stream cipher might be the better alternative. For applications that deal with blocks of data, such as file transfer, email, and database, block ciphers may be more appropriate. However, either type of cipher can be used in virtually any application.

The RC4 Algorithm:

RC4 is a stream cipher designed in 1987 by Ron Rivest for RSA Security. It is a variable keysize stream cipher with byte-oriented operations. The algorithm is based on the use of a random permutation. Analysis shows that the period of the cipher is overwhelmingly likely to be greater than 10^{100} . Eight to sixteen machine operations are required per output byte, and the cipher can be expected to run very quickly in software. RC4 was kept as a trade secret by RSA Security. In September 1994, the RC4 algorithm was anonymously posted on the Internet on the Cypherpunks anonymous remailers list.

The RC4 algorithm is remarkably simply and quite easy to explain. A variable-length key of from 1 to 256 bytes (8 to 2048 bits) is used to initialize a 256-byte state vector S , with elements $S[0], S[1], \dots, S[255]$. At all times, S contains a permutation of all 8-bit numbers

from 0 through 255. For encryption and decryption, a byte k is generated from S by selecting one of the 255 entries in a systematic fashion. As each value for k is generated, the entries in S are once again permuted.

Initialization of S

To begin, the entries of S are set equal to the values from 0 through 255 in ascending order; that is; $S[0] = 0$, $S[1] = 1$, ..., $S[255] = 255$. A temporary vector, T , is also created. If the length of the key K is 256 bytes, then K is transferred to T . Otherwise, for a key of length keylen bytes, the first keylen elements of T are copied from K and then K is repeated as many times as necessary to fill out T . These preliminary operations can be summarized as follows:

```
/* Initialization */  
for  $i = 0$  to 255 do  
   $S[i] = i$ ;  
   $T[i] = K[i \bmod \text{keylen}]$ ;
```

Next we use T to produce the initial permutation of S . This involves starting with $S[0]$ and going through to $S[255]$, and, for each $S[i]$, swapping $S[i]$ with another byte in S according to a scheme dictated by $T[i]$:

```
/* Initial Permutation of S */  
 $j = 0$ ;  
for  $i = 0$  to 255 do  
   $j = (j + S[i] + T[i]) \bmod 256$ ;  
  Swap ( $S[i]$ ,  $S[j]$ );
```

Because the only operation on S is a swap, the only effect is a permutation. S still contains all the numbers from 0 through 255.

Stream Generation

Once the S vector is initialized, the input key is no longer used. Stream generation involves starting with $S[0]$ and going through to $S[255]$, and, for each $S[i]$, swapping $S[i]$ with another byte in S according to a scheme dictated by the current configuration of S . After $S[255]$ is reached, the process continues, starting over again at $S[0]$:

```

/* Stream Generation */
i, j = 0;
while (true)
    i = (i + 1) mod 256;
    j = (j + S[i]) mod 256;
    Swap (S[i], S[j]);
    t = (S[i] + S[j]) mod 256;
    k = S[t];

```

To encrypt, XOR the value k with the next byte of plaintext. To decrypt, XOR the value k with the next byte of ciphertext.

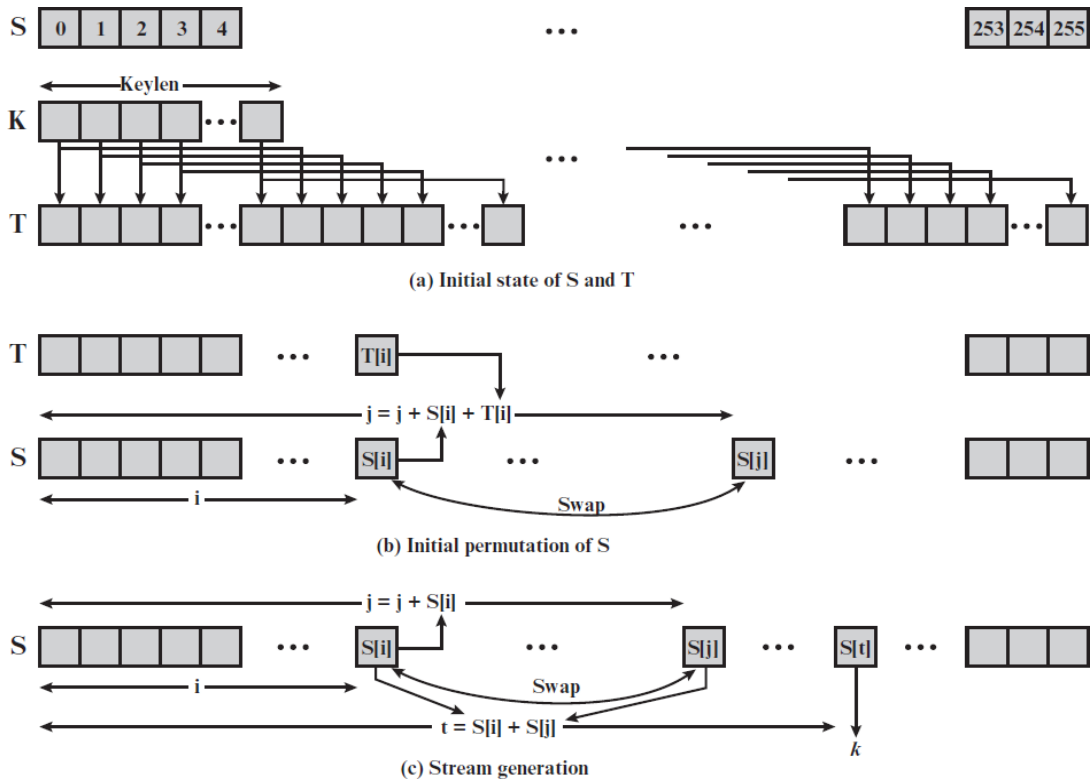


Fig 2.7 RC4

2.5 BLOCK CIPHER MODES OF OPERATION

A block cipher processes the data blocks of fixed size. Usually, the size of a message is larger than the block size. Hence, the long message is divided into a series of sequential message blocks, and the cipher operates on these blocks one at a time.

2.5.1 Electronic Code Book (ECB) Mode:

This mode is a most straightforward way of processing a series of sequentially listed message blocks.

Operation:

- The user takes the first block of plaintext and encrypts it with the key to produce the first block of ciphertext.
- He then takes the second block of plaintext and follows the same process with same key and so on so forth.

The ECB mode is deterministic, that is, if plaintext block $P_1, P_2, \dots, P_m$ are encrypted twice under the same key, the output ciphertext blocks will be the same.

In fact, for a given key technically we can create a codebook of ciphertexts for all possible plaintext blocks. Encryption would then entail only looking up for required plaintext and select the corresponding ciphertext. Thus, the operation is analogous to the assignment of code words in a codebook, and hence gets an official name – Electronic Codebook mode of operation (ECB). It is illustrated as follows –

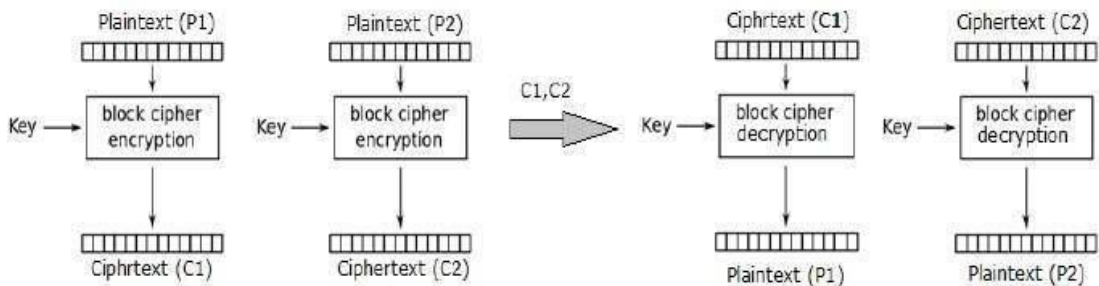


Fig 2.8 Electronic Code Book (ECB) Mode

2.5.2 Cipher Block Chaining (CBC) Mode

CBC mode of operation provides message dependence for generating ciphertext and makes the system non-deterministic.

Operation

The operation of CBC mode is depicted in the following illustration. The steps are as follows

- Load the n-bit Initialization Vector (IV) in the top register.
- XOR the n-bit plaintext block with data value in top register.
- Encrypt the result of XOR operation with underlying block cipher with key K.
- Feed ciphertext block into top register and continue the operation till all plaintext blocks are processed.
- For decryption, IV data is XORed with first ciphertext block decrypted. The first ciphertext block is also fed into to register replacing IV for decrypting next ciphertext block.

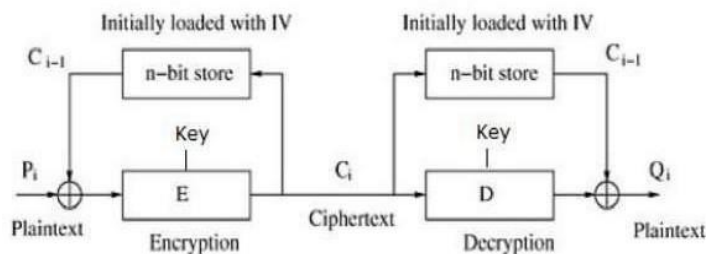


Fig 2.9 Cipher Block Chaining (CBC) Mode

2.5.3 Cipher Feedback (CFB) Mode

In this mode, each ciphertext block gets ‘fed back’ into the encryption process in order to encrypt the next plaintext block.

Operation

The operation of CFB mode is depicted in the following illustration. For example, in the present system, a message block has a size ‘s’ bit where $1 < s < n$. The CFB mode requires an initialization vector (IV) as the initial random n-bit input block. The IV need not be secret. Steps of operation are –

- Load the IV in the top register.
- Encrypt the data value in top register with underlying block cipher with key K.
- Take only ‘s’ number of most significant bits (left bits) of output of encryption process and XOR them with ‘s’ bit plaintext message block to generate ciphertext block.
- Feed ciphertext block into top register by shifting already present data to the left

and continue the operation till all plaintext blocks are processed.

- Essentially, the previous ciphertext block is encrypted with the key, and then the result is XORed to the current plaintext block.
- Similar steps are followed for decryption. Pre-decided IV is initially loaded at the start of decryption.

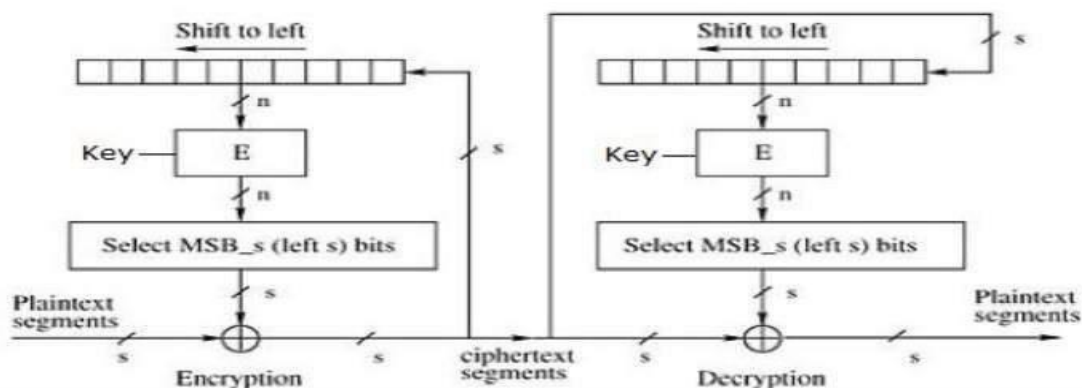


Fig 2.10 Cipher Feedback (CFB) Mode

2.5.4 Output Feedback (OFB) Mode

It involves feeding the successive output blocks from the underlying block cipher back to it. These feedback blocks provide string of bits to feed the encryption algorithm which act as the key-stream generator as in case of CFB mode.

The key stream generated is XOR-ed with the plaintext blocks. The OFB mode requires an IV as the initial random n -bit input block. The IV need not be secret.

The operation is depicted in the following illustration –

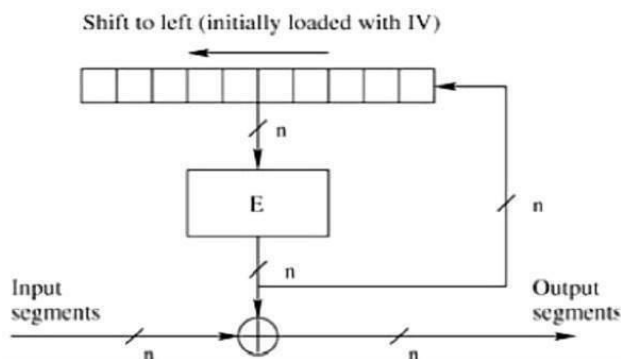


Fig 2.11 Output Feedback (OFB) Mode

2.5.5 Counter (CTR) Mode

It can be considered as a counter-based version of CFB mode without the feedback. In this mode, both the sender and receiver need to access to a reliable counter, which computes a new shared value each time a ciphertext block is exchanged. This shared counter is not necessarily a secret value, but challenge is that both sides must keep the counter synchronized.

Operation

Both encryption and decryption in CTR mode are depicted in the following illustration.

Steps in operation are –

- Load the initial counter value in the top register is the same for both the sender and the receiver. It plays the same role as the IV in CFB (and CBC) mode.
- Encrypt the contents of the counter with the key and place the result in the bottom register.
- Take the first plaintext block P1 and XOR this to the contents of the bottom register. The result of this is C1. Send C1 to the receiver and update the counter. The counter update replaces the ciphertext feedback in CFB mode.
- Continue in this manner until the last plaintext block has been encrypted.
- The decryption is the reverse process. The ciphertext block is XORed with the output of encrypted contents of counter value. After decryption of each ciphertext block counter is updated as in case of encryption.

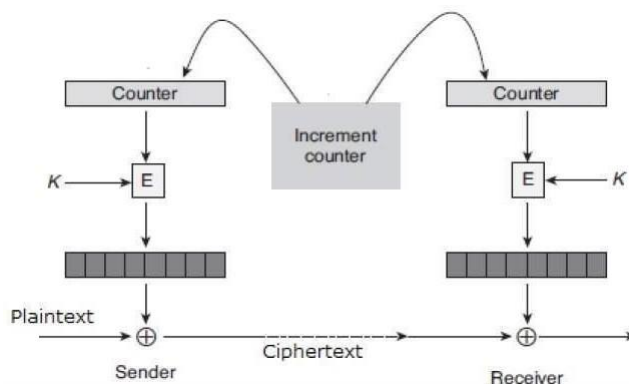


Fig 2.12 Counter (CTR) Mode

2.6 DIGITAL SIGNATURE

We have been using handwritten signatures to authenticate handwritten or typed messages in the physical world for a long time. Their main use is to bind the signatory with a particular message. Digital Signature accomplishes the same thing with digital messages. The signatory, a person or an entity, is bound to digital data. This binding is verifiable by the receiver or a third party, if necessary.

Digital Signature in **Cryptography** is a value calculated from the data along with a secret key that only the signer is aware of. The receiver needs to be assured that the message belongs to the sender. This is crucial in businesses as the chances of disputes over data exchange are high.

Digital Signature Algorithms

There are three algorithms at work in Digital Signatures. They are as follows:

Key Generation Algorithms

Key Generation Algorithms help ensure authenticity and integrity or it would be very easy to tamper with the data. They also prevent anyone from pretending to be the sender.

Signing Algorithms

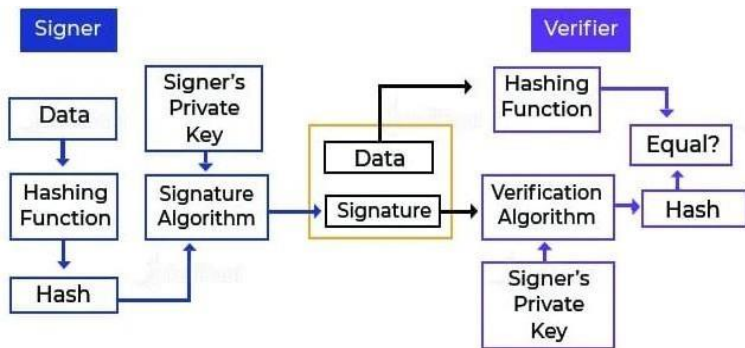
Signing Algorithms make one-way hashes of the data that has to be signed. Then they encrypt the hash value using the signature key. The encrypted hash along with the other information is the Digital Signature.

Signature Verification Algorithms

Signature Verification Algorithms help process the Digital Signature and the verification key to generate some values. The algorithm also processes the same hash function on the data received and creates a hash value

How to Create a Digital Signature?

Digital Signature is based on Public Key Cryptography. Let's look at how a Digital Signature is created:



- People who adopt the Digital Signature scheme have a public-private key pair.
- The pairs for encryption/decryption and signing/verifying are usually different. The signature key is the private key that is used for signing and the verification key is the public key.
- The signer feeds all data to the hashing function and in turn, generates a hash of data.
- The signature key and hash value are fed into the signature algorithm to help produce a Digital Signature on the given hash. Once the signature is appended to the data, both are sent forward to the verifier.
- The verifier will then feed the Digital Signature and the verification key into the verification algorithm. The verification algorithm will generate an output value.
- Verifier is also responsible for running some hashing functions on the data received so it can generate a hash value.
- Verification is processed by comparing the hash value generated by the verifier and the output of the verification algorithm. The result of this comparison helps the verifier decide if the Digital Signature is valid or not.
- Nobody else has access to the private key of the signer and the Digital Signature is created using this key, so the signer cannot reject signing the document in the future.

Importance of Digital Signature

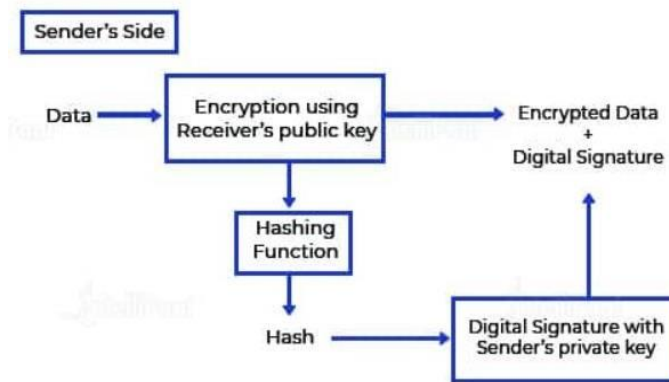
Privacy, Integrity, Non-Repudiation, and Authentication can be provided as part of a cryptosystem if public-key encryption is added to the Digital Signature Scheme.

Digital Signature Encryption

Sending an encrypted message is a safer choice than sending plaintexts. In public-key

encryption, the sender's public key is openly available and anyone can spoof their identity to send encrypted messages. The best way out of this is to use a Digital Signature along with encrypted messages.

Now, this can be done in one of two ways – sign-then-encrypt or encrypt-then-sign. The sign-then-encrypt system is prone to exploitation as the receiver could spoof the sender's identity and send it to a third party. The encrypt-then-sign system happens to be more reliable and hence, widely accepted. We can illustrate this system with the following diagram –

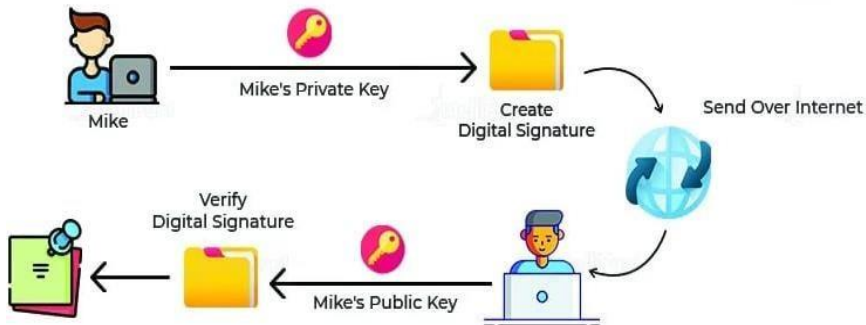


When the receiver receives the encrypted data, they first verify the signature with the help of the sender's public key. Once the signature is verified, the verifier can decrypt the data using their private key.

Digital Signature Example

To get a better understanding of the process, let's look at a Digital Signature example step-by-step.

Let's suppose you have to send a message to someone in ABC Office.



- The first step would be for you to type out the message or ready the file you want to send. Your private key would work as the stamp for this file. It could be a code or a password. Then you press send and the email reaches ABC Office via the internet.
- In the second step, the ABC Office would receive your file and verify your signature using your public key. They will then be able to access the encrypted file.
- The final step would require the ABC Office to use the private key that you've shared, to reveal whatever file you've mailed them. If the recipient doesn't have your private key, they won't be able to access the information in the document.

2.7 PUBLIC KEY CRYPTOSYSTEM

Asymmetric algorithms rely on one key for encryption and a different but related key for decryption. These algorithms have the following important characteristic.

- It is computationally infeasible to determine the decryption key given only knowledge of the cryptographic algorithm and the encryption key.

In addition, some algorithms, such as RSA, also exhibit the following characteristic.

- Either of the two related keys can be used for encryption, with the other used for decryption.

A **public-key encryption** scheme has six

- **Plaintext:** This is the readable message or data that is fed into the algorithm as input.

■ **Encryption algorithm:** The encryption algorithm performs various transformations on the plaintext.

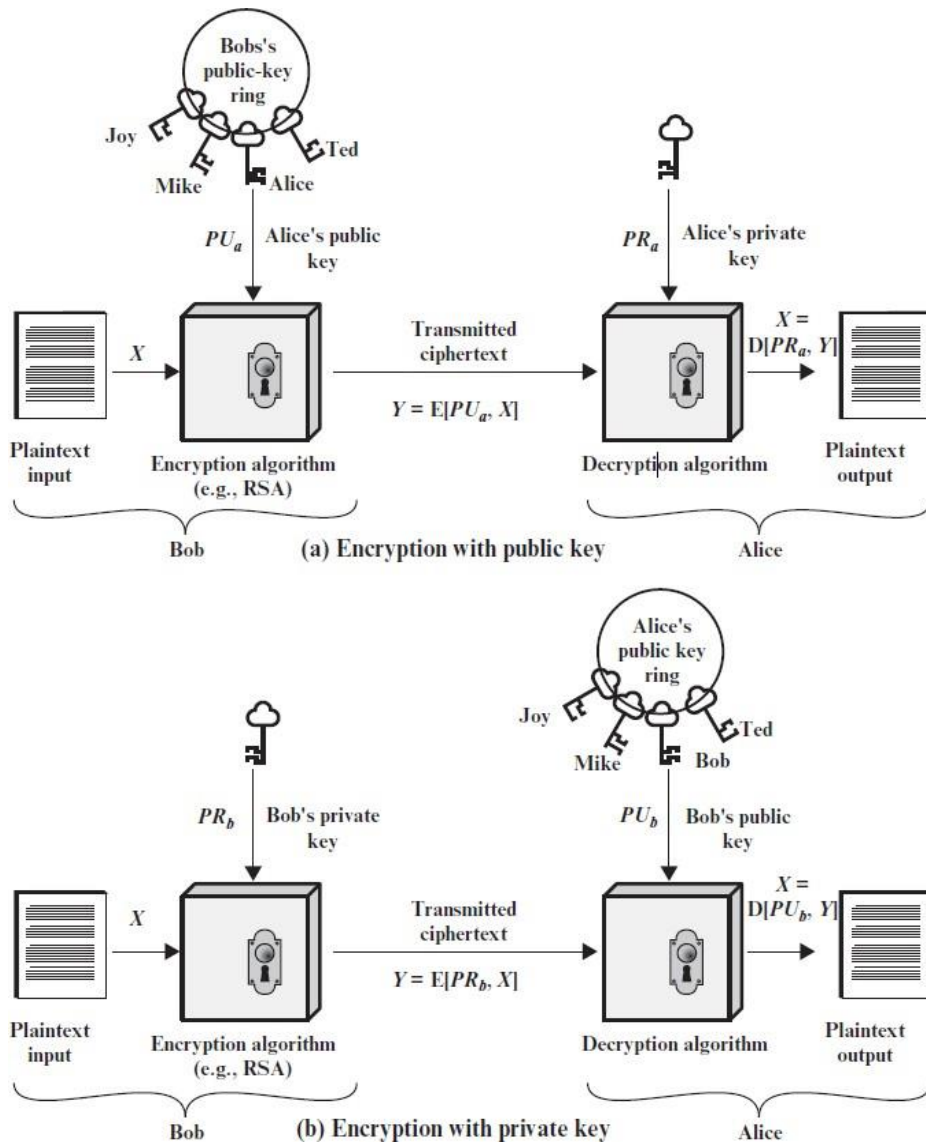


Fig 2.13 Public-Key Cryptography

Public and private keys: This is a pair of keys that have been selected so that if one is used for encryption, the other is used for decryption. The exact transformations performed by the algorithm depend on the public or private key that is provided as input.

■ **Ciphertext:** This is the encrypted message produced as output. It depends on the plaintext and the key. For a given message, two different keys will produce two different ciphertexts.

Decryption algorithm: This algorithm accepts the ciphertext and the matching key and produces the original plaintext.

Conventional Encryption	Public-Key Encryption
<i>Needed to Work:</i> 1. The same algorithm with the same key is used for encryption and decryption. 2. The sender and receiver must share the algorithm and the key. <i>Needed for Security:</i> 1. The key must be kept secret. 2. It must be impossible or at least impractical to decipher a message if the key is kept secret. 3. Knowledge of the algorithm plus samples of ciphertext must be insufficient to determine the key.	<i>Needed to Work:</i> 1. One algorithm is used for encryption and a related algorithm for decryption with a pair of keys, one for encryption and one for decryption. 2. The sender and receiver must each have one of the matched pair of keys (not the same one). <i>Needed for Security:</i> 1. One of the two keys must be kept secret. 2. It must be impossible or at least impractical to decipher a message if one of the keys is kept secret. 3. Knowledge of the algorithm plus one of the keys plus samples of ciphertext must be insufficient to determine the other key.

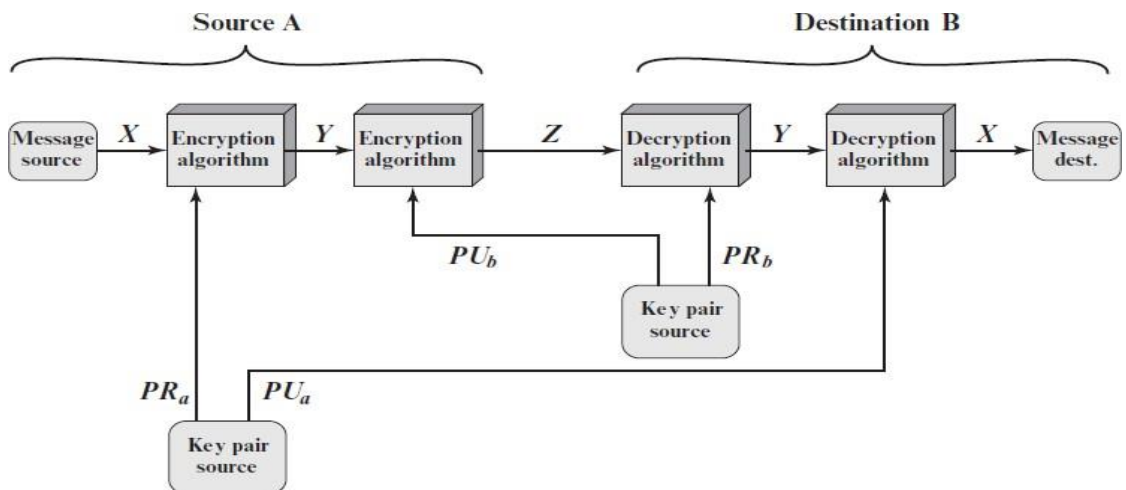


Fig 2.14 Public-Key Cryptosystem: Authentication and Secrecy

In broad terms, we can classify the use of **public-key cryptosystems** into three categories

■ **Encryption/decryption:** The sender encrypts a message with the recipient's public key, and the recipient decrypts the message with the recipient's private key.

■ **Digital signature:** The sender "signs" a message with its private key. Signing is achieved by a cryptographic algorithm applied to the message or to a small block of data that is a function of the message.

■ **Key exchange:** Two sides cooperate to exchange a session key, which is a secret key for symmetric encryption generated for use for a particular transaction (or session) and valid for a short period of time. Several different approaches are possible, involving the private key(s) of one or both parties.

Some algorithms are suitable for all three applications, whereas others can be used only for one or two of these applications.

2.8 THE RSA ALGORITHM

The RSA algorithm is a public-key signature algorithm developed by Ron Rivest, Adi Shamir, and Leonard Adleman (RSA). Their paper was first published in 1977, and the algorithm uses logarithmic functions to keep the working complex enough to withstand brute force and streamlined enough to be fast post-deployment. The image below shows it verifies the digital signatures using RSA methodology.

RSA algorithm uses the following procedure to generate public and private keys:

- Select two large prime numbers, **p** and **q**.
- Multiply these numbers to find **n = p x q**, where **n** is called the modulus for encryption and decryption.
- Choose a number **e** less than **n**, such that **n** is relatively prime to **(p - 1) x (q - 1)**. It means that **e** and **(p - 1) x (q - 1)** have no common factor except 1. Choose "e" such that $1 < e < \phi(n)$, **e** is prime to $\phi(n)$,

$$\boxed{\text{gcd}(e, \phi(n)) = 1}$$

- If $n = p \times q$, then the public key is $\langle e, n \rangle$. A plaintext message m is encrypted using public key $\langle e, n \rangle$. To find ciphertext from the plain text following formula is used to get ciphertext C .

$$C = m^e \bmod n$$

Here, m must be less than n . A larger message ($>n$) is treated as a concatenation of messages, each of which is encrypted separately.

- To determine the private key, we use the following formula to calculate the d such that:

$$D_e \bmod \{(p - 1) \times (q - 1)\} = 1 \quad \text{Or} \quad D_e \bmod \phi(n) = 1$$

- The private key is $\langle d, n \rangle$. A ciphertext message c is decrypted using private key $\langle d, n \rangle$. To calculate plain text m from the ciphertext c following formula is used to get plain text m .

$$m = c^d \bmod n$$

Example 1:

This example shows how we can encrypt plaintext 9 using the RSA public-key encryption algorithm. This example uses prime numbers 7 and 11 to generate the public and private keys.

Explanation:

Step 1: Select two large prime numbers, p , and q .

$$p = 7$$

$$q = 11$$

Step 2: Multiply these numbers to find $n = p \times q$, where n is called the modulus for encryption and decryption.

First, we calculate

$$n = p \times q$$

$$n = 7 \times 11$$

$$n = 77$$

Step 3: Choose a number **e** less than **n**, such that **n** is relatively prime to **(p - 1) x (q - 1)**. It means that **e** and **(p - 1) x (q - 1)** have no common factor except 1. Choose "e" such that $1 < e < \phi(n)$, **e** is prime to $\phi(n)$, $\gcd(e, \phi(n)) = 1$.

Second, we calculate

$$\phi(n) = (p - 1) \times (q - 1)$$

$$\phi(n) = (7 - 1) \times (11 - 1)$$

$$\phi(n) = 6 \times 10$$

$$\phi(n) = 60$$

Let us now choose relative prime **e** of 60 as 7.

Thus the public key is $\langle e, n \rangle = (7, 77)$

Step 4: A plaintext message **m** is encrypted using public key $\langle e, n \rangle$. To find ciphertext from the plain text following formula is used to get ciphertext **C**.

To find ciphertext from the plain text following formula is used to get ciphertext **C**.

$$C = m^e \bmod n$$

$$C = 9^7 \bmod 77$$

$$C = 37$$

Step 5: The private key is $\langle d, n \rangle$. To determine the private key, we use the following formula **d** such that:

$$D_e \bmod \{(p - 1) \times (q - 1)\} = 1$$

$$7d \bmod 60 = 1, \text{ which gives } d = 43$$

The private key is $\langle d, n \rangle = (43, 77)$

Step 6: A ciphertext message **c** is decrypted using private key $\langle d, n \rangle$. To calculate plain text **m** from the ciphertext **c** following formula is used to get plain text **m**.

$$m = c^d \bmod n$$

$$m = 37^{43} \bmod 77$$

$$m = 9$$

In this example, Plain text = 9 and the ciphertext = 37

2.9 DIFFIE–HELLMAN KEY EXCHANGE

The purpose of the algorithm is to enable two users to securely exchange a key that can then be used for subsequent symmetric encryption of messages. The algorithm itself is limited to the exchange of secret values.

Primitive root of a prime number p is one whose powers modulo p generate all the integers from 1 to $p - 1$. That is, if a is a primitive root of the prime number p , then the numbers.

$$a \bmod p, a^2 \bmod p, \dots, a^{p-1} \bmod p$$

are distinct and consist of the integers from 1 through $p - 1$ in some permutation.

For any integer b and a primitive root a of prime number p , we can find a unique exponent i such that

$$b \equiv a^i \pmod{p} \quad \text{where } 0 \leq i \leq (p - 1)$$

The exponent i is referred to as the **discrete logarithm** of b for the base a , mod p . We express this value as $\text{dlog}_{a,p}(b)$. See Chapter 2 for an extended discussion of discrete logarithms.

The Algorithm

Figure 2.15 summarizes the Diffie–Hellman key exchange algorithm. For this scheme, there are two publicly known numbers: a prime number q and an integer a that is a primitive root of q . Suppose the users A and B wish to create a shared key.

User A selects a random integer $X_A < q$ and computes $Y_A = a^{X_A} \bmod q$.

Similarly, user B independently selects a random integer $X_B < q$ and computes $Y_B = a^{X_B} \bmod q$. Each side keeps the X value private and makes the Y value available publicly to the other side. Thus, X_A is A's private key and Y_A is A's corresponding public key, and similarly for B. User A computes the key as $K = (Y_B)^{X_A} \bmod q$ and user B computes the key as $K = (Y_A)^{X_B} \bmod q$. These two calculations produce identical results:

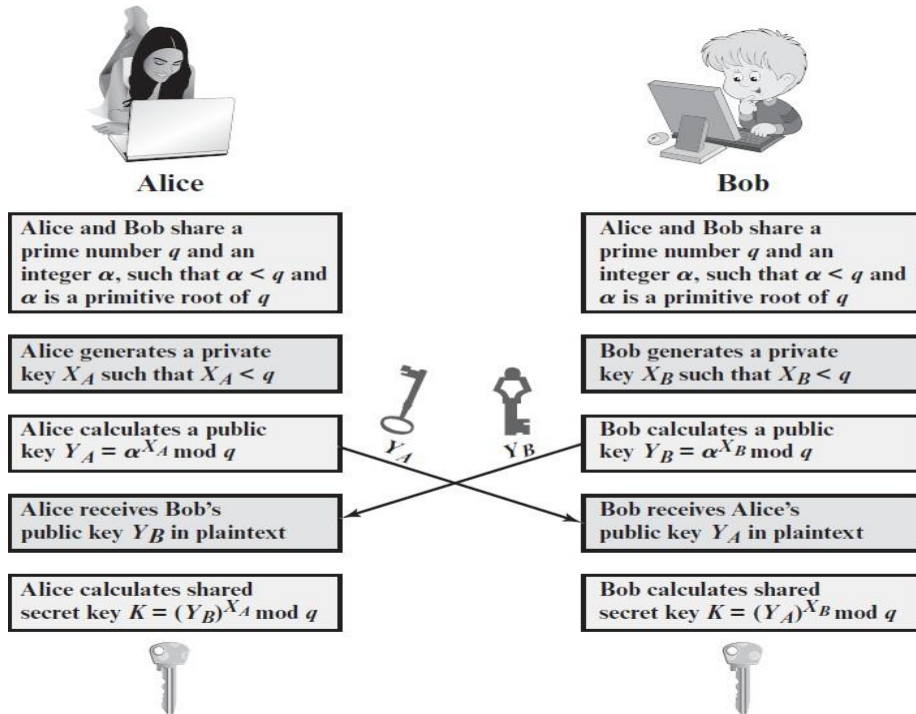


Fig 2.15 The Diffie–Hellman Key Exchange

$$\begin{aligned}
 K &= (Y_B)^{X_A} \bmod q \\
 &= (\alpha^{X_B} \bmod q)^{X_A} \bmod q \\
 &= (\alpha^{X_B})^{X_A} \bmod q \\
 &= \alpha^{X_B X_A} \bmod q \\
 &= (\alpha^{X_A})^{X_B} \bmod q \\
 &= (\alpha^{X_A} \bmod q)^{X_B} \bmod q \\
 &= (Y_A)^{X_B} \bmod q
 \end{aligned}$$

by the rules of modular arithmetic

Here is an example. Key exchange is based on the use of the prime number $q = 353$ and a primitive root of 353, in this case $a = 3$. A and B select private keys $X_A = 97$ and $X_B = 233$, respectively. Each computes its public key:

A computes $Y_A = 3^{97} \bmod 353 = 40$.

B computes $Y_B = 3^{233} \bmod 353 = 248$.

After they exchange public keys, each can compute the common secret key:

A computes $K = (Y_B)^{X_A} \bmod 353 = 248^{97} \bmod 353 = 160$.

B computes $K = (Y_A)^{X_B} \bmod 353 = 40^{233} \bmod 353 = 160$.

We assume an attacker would have available the following information:

$$q = 353; \alpha = 3; Y_A = 40; Y_B = 248$$

In this simple example, it would be possible by brute force to determine the secret key 160. In particular, an attacker E can determine the common key by discovering a solution to the equation $3^a \bmod 353 = 40$ or the equation $3^b \bmod 353 = 248$. The brute-force approach is to calculate powers of 3 modulo 353, stopping when the result equals either 40 or 248. The desired answer is reached with the exponent value of 97, which provides $3^{97} \bmod 353 = 40$.

With larger numbers, the problem becomes impractical.

2.10 ELGAMAL CRYPTOGRAPHIC SYSTEM

These steps are summarized in Figure 2.16. It corresponds to Alice generates a public/private key pair; Bob encrypts using Alice's public key; and Alice decrypts using her private key.

Let us demonstrate why the Elgamal scheme works. First, we show how K is recovered by the decryption process:

$K = (Y_A)^k \bmod q$	K is defined during the encryption process
$K = (\alpha^{X_A} \bmod q)^k \bmod q$	substitute using $Y_A = \alpha^{X_A} \bmod q$
$K = \alpha^{kX_A} \bmod q$	by the rules of modular arithmetic
$K = (C_1)^{X_A} \bmod q$	substitute using $C_1 = \alpha^k \bmod q$

Next, using K , we recover the plaintext as

$$C_2 = KM \bmod q$$
$$(C_2 K^{-1}) \bmod q = KMK^{-1} \bmod q = M \bmod q = M$$

We can restate the Elgamal process as follows, using Figure 2.16.

1. Bob generates a random integer k .
2. Bob generates a one-time key K using Alice's public-key components Y_A, q , and k .
3. Bob encrypts k using the public-key component α , yielding C_1 . C_1 provides sufficient information for Alice to recover K .
4. Bob encrypts the plaintext message M using K .
5. Alice recovers K from C_1 using her private key.
6. Alice uses K^{-1} to recover the plaintext message from C_2 .

Global Public Elements	
q	prime number
α	$\alpha < q$ and α a primitive root of q

Key Generation by Alice	
Select private X_A	$X_A < q - 1$
Calculate Y_A	$Y_A = \alpha^{X_A} \bmod q$
Public key	$\{q, \alpha, Y_A\}$
Private key	X_A

Encryption by Bob with Alice's Public Key	
Plaintext:	$M < q$
Select random integer k	$k < q$
Calculate K	$K = (Y_A)^k \bmod q$
Calculate C_1	$C_1 = \alpha^k \bmod q$
Calculate C_2	$C_2 = KM \bmod q$
Ciphertext:	(C_1, C_2)

Decryption by Alice with Alice's Private Key	
Ciphertext:	(C_1, C_2)
Calculate K	$K = (C_1)^{X_A} \bmod q$
Plaintext:	$M = (C_2 K^{-1}) \bmod q$

Fig 2.16 The Elgamal Cryptosystem

2.11 ELLIPTIC CURVE CRYPTOGRAPHY

Elliptic curve cryptography (ECC) is a public-key encryption algorithm based on the elliptic curve defined over a finite field. ECC is a modern encryption algorithm that provides greater security with shorter key lengths, allowing it to be used by devices with less computational power like smartphones to communicate securely over the internet.

How does ECC use elliptic curves?

Elliptic curves are the underlying backbone of the ECC. ECC algorithms can use different underlying elliptic curves to prioritize cryptographic strength, speed, and key length. ECC relies on some of the geometrical properties of the elliptic curve, such as horizontal symmetry, and a straight line crossing the elliptic curve will intersect it at most 3 points.

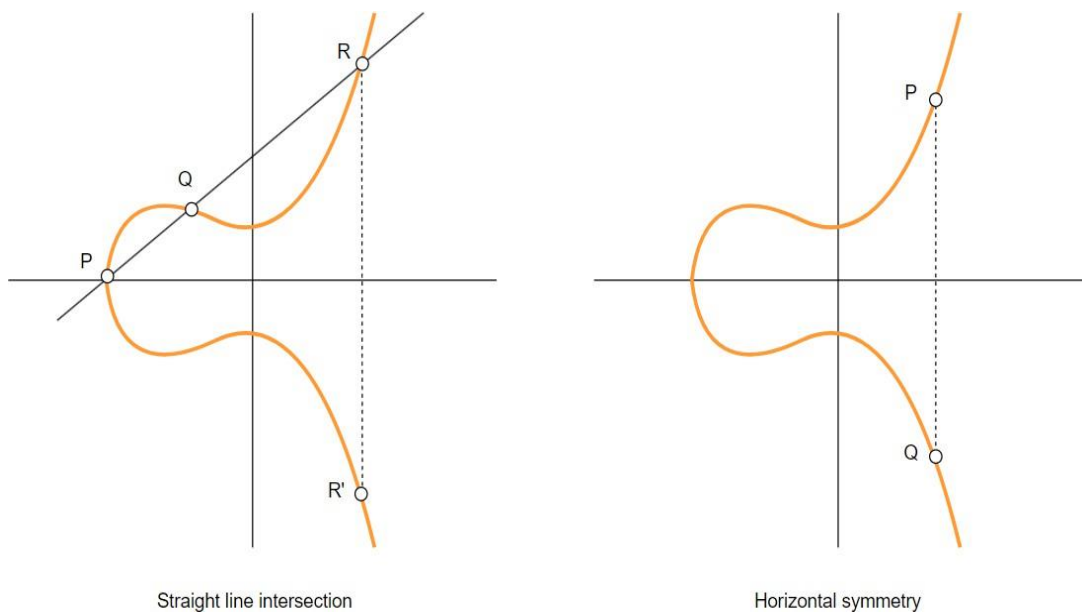
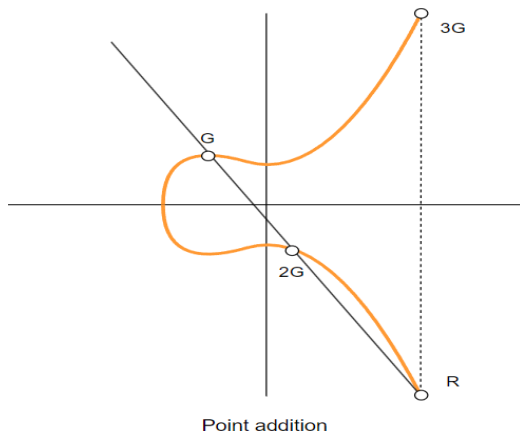
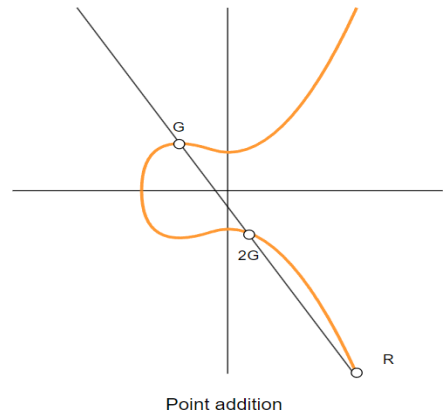
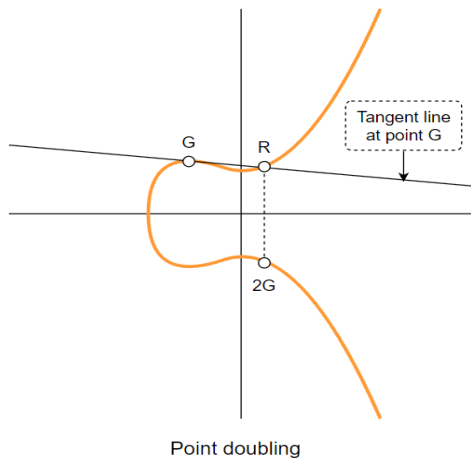


Fig 2.17 The geometric properties of an elliptic curve

The above-mentioned geometric properties play a key role in generating public and private keys used in ECC. As explained below, these properties help generate a public key from the randomly selected private key. Point addition and point doubling are applied to the points on elliptic curves.



The above illustration represents the process of point doubling and point addition as explained below.

- **Point doubling:** When only one point is provided, a straight line can not be drawn, so we draw a tangent at the point provided. The point where that tangent intersects the curve again is mirrored horizontally to get another point.
- **Point addition:** Given that two points are provided on the curve, we draw a straight line passing through those points. The point where that straight line intersects the curve for the 3rd and the final time is mirrored horizontally to get the next point.

ECC algorithms

ECC provides several different groups of algorithms based on their use cases, defined over an elliptic curve over finities fields.

- **Digital Signatures:** ECC provides algorithms for digital signatures like the elliptic curve digital signature algorithm ECDSA and EdDSA.
- **Encryption:** ECC provides algorithms for encrypting messages, namely ECIES and EEECC.
- **Key agreement:** ECC provides algorithms like ECDH and FHEMQV for key agreement.

All these algorithms rely on elliptic curves like secp256K1, curve25519, and p251 to calculate keys and ciphers. Furthermore, they rely on the **elliptic curve discrete logarithm problem (ECDLP)** to ensure the difficulty of breaking these algorithms, thus ensuring the network's security.

Advantages of ECC

The advantages of ECC are as follows:

- **Shorter key lengths:** ECC provides a greater level of security while having a shorter key size. The energy required to break a 3072-length key generated by the RSA encryption method is the same amount of energy required to break a 256-length key generated by ECC. This shows that the 256-length key generated by the ECC is as cryptographically as strong as the 3072-length one generated by the RSA encryption.
- **Less computational power:** As the key sizes are shorter, it is computationally less expensive to encrypt and decrypt the data.
- **Fast key generation:** The key generation process is relatively simple and computationally less expensive as it involves securely generating a random integer within a specified range. This makes it extremely fast compared to the RSA encryption algorithm.

- **Fast signatures:** The ECC uses the ECDSA, which is extremely fast while generating the digital signature as it involves the simple step of multiplying a point on the curve.

Disadvantages of ECC

The disadvantages of ECC are as follows:

- **Complicated:** The ECC is quite complicated to implement, making it more prone to errors, thus compromising the system's overall security if not implemented properly.
- **Random number generator:** The system's security is compromised if a broken random number generator is used at the time of private key selection.
- **Patents:** It is one of the main factors restricting the widespread use of the ECC algorithm. Certicom and National Security Agency NSA own some of the main patents.
- **Signature verification:** Although the signature generation process is fast, the verification process takes some time due to its computationally intensive calculations.
- **Backdoor:** There are growing concerns that NSA may have implemented a backdoor into the ECC algorithms allowing them to monitor the encrypted messages being sent.

UNIT III

AUTHENTICATION, INTEGRITY AND ACCESS CONTROL

Authentication: Security Hash Function - HMAC - Electronic User Authentication Principles, Password Based Authentication, Token Based and Remote Authentication; Internet Authentication Applications: Kerberos X.509 - Public Key Infrastructure; Access Control: Access Control Principles - Subjects, Objects, and Access Rights - Discretionary Access Control - Example: UNIX File Access Control - Role Based Access Control - Attribute-Based Access Control - Identity, Credential, and Access Management - Trust Frameworks.

3.1. SECURITY HASH FUNCTIONS

- ✓ A **hash function** H accepts a variable-length block of data M as input and produces a fixed-size hash value $h = H(M)$.
- ✓ A “good” hash function has the property that the results of applying the function to a large set of inputs will produce outputs that are evenly distributed and apparently random. In general terms, the principal object of a hash function is data integrity. A change to any bit or bits in M results, with high probability, in a change to the hash value.
- ✓ The kind of hash function needed for security applications is referred to as a **cryptographic hash function**.
- ✓ A cryptographic hash function is an algorithm for which it is computationally infeasible (because no attack is significantly more efficient than brute force) to find either a data object that maps to a pre-specified hash result (the one-way property) or two data objects that map to the same hash result (the collision-free property).

- ✓ Because of these characteristics, hash functions are often used to determine whether or not data has changed.

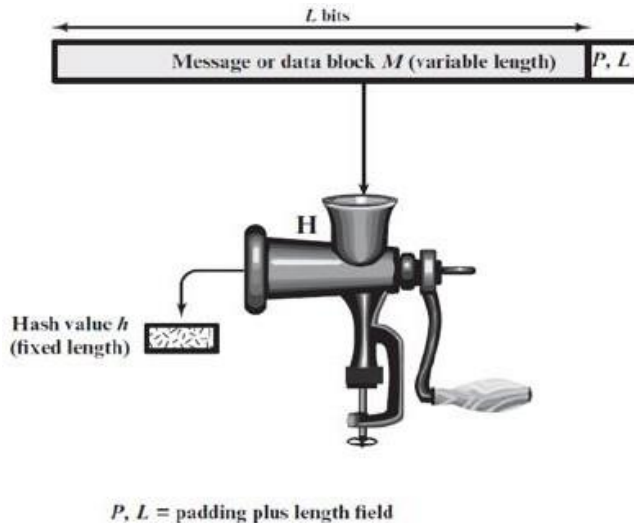


Fig 3.1 Cryptographic Hash function = $H(M)$

The above figure depicts the general operation of a cryptographic hash function.

- ✓ Typically, the input is padded out to an integer multiple of some fixed length (e.g., 1024 bits), and the padding includes the value of the length of the original message in bits.
- ✓ The length field is a security measure to increase the difficulty for an attacker to produce an alternative message with the same hash value.

3.2 Applications of Security Hash Functions:

The most versatile cryptographic algorithm is the cryptographic hash function. It is used in a wide variety of security applications and Internet protocols. The following are various applications where it is employed.

Message Authentication:

- ✓ Message authentication is a mechanism or service used to verify the integrity of a message.
- ✓ Message authentication assures that data received are exactly as sent (i.e., there is no modification, insertion, deletion, or replay).
- ✓ When a hash function is used to provide message authentication, the hash function value is often referred to as a **message digest**.

The essence of the use of a hash function for message integrity is as follows.

- ✓ The sender computes a hash value as a function of the bits in the message and transmits both the hash value and the message.
- ✓ The receiver performs the same hash calculation on the message bits and compares this value with the incoming hash value.
- ✓ If there is a mismatch, the receiver knows that the message (or possibly the hash value) has been altered (Figure a).
- ✓ The hash value must be transmitted in a secure fashion. That is, the hash value must be protected so that if an adversary alters or replaces the message, it is not feasible for adversary to also alter the hash value to fool the receiver. This type of attack is shown in Figure b.

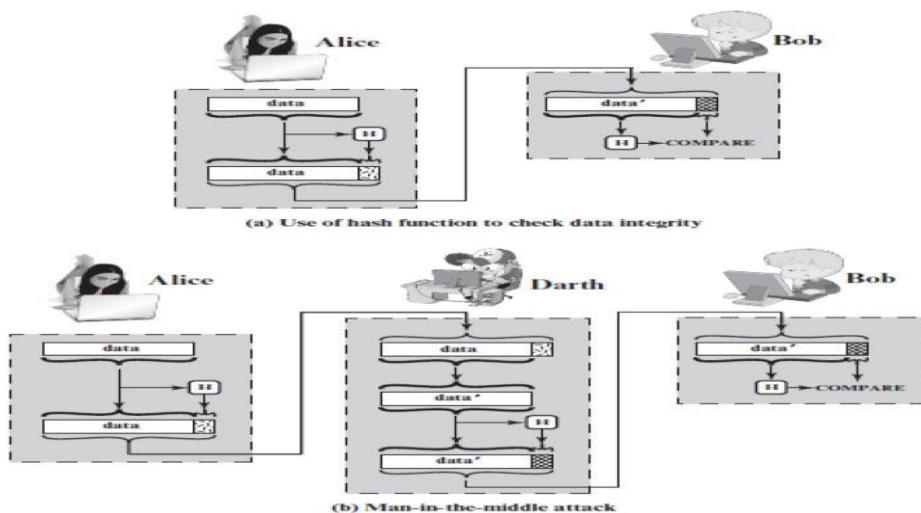
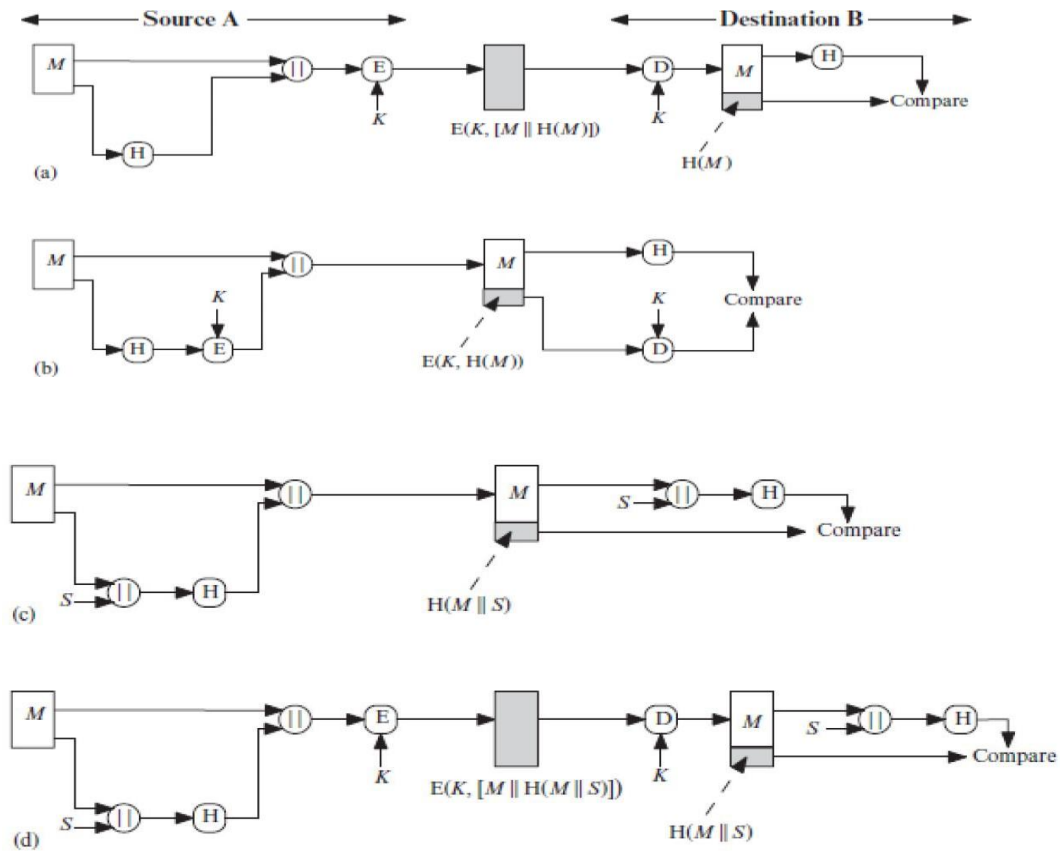


Fig 3.2 Attack against Hash Function



More commonly, message authentication is achieved using a **message authentication code (MAC)**, also known as a **keyed hash function**.

- ✓ Typically, MACs are used between two parties that share a secret key to authenticate information exchanged between those parties.
- ✓ A MAC function takes as input a secret key and a data block and produces a hash value, referred to as the MAC, which is associated with the protected message.
- ✓ If the integrity of the message needs to be checked, the MAC function can be applied to the message and the result compared with the associated MAC value.
- ✓ An attacker who alters the message will be unable to alter the associated MAC value without knowledge of the secret key.

3.3 MESSAGE AUTHENTICATION CODE

An alternative authentication technique involves the use of a secret key to generate a small fixed-size block of data, known as a **cryptographic checksum** or MAC, that is appended to the message.

- ✓ This technique assumes that two communicating parties, say A and B, share a common secret key K .
- ✓ When A has a message to send to B, it calculates the MAC as a function of the message and the key:

$$\text{MAC} = C(K, M)$$

where

M = input message

C = MAC function

K = shared secret key

MAC = message authentication code

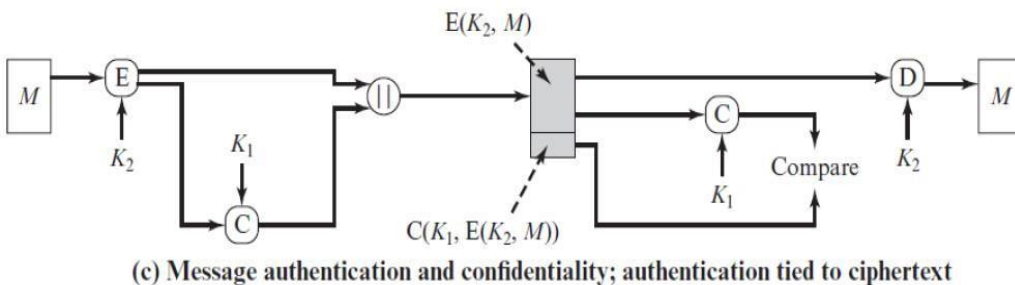
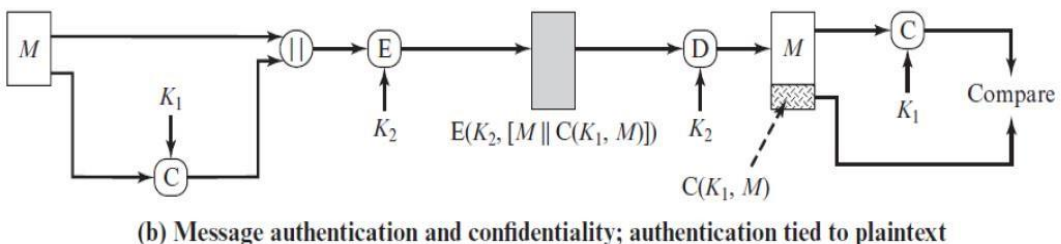
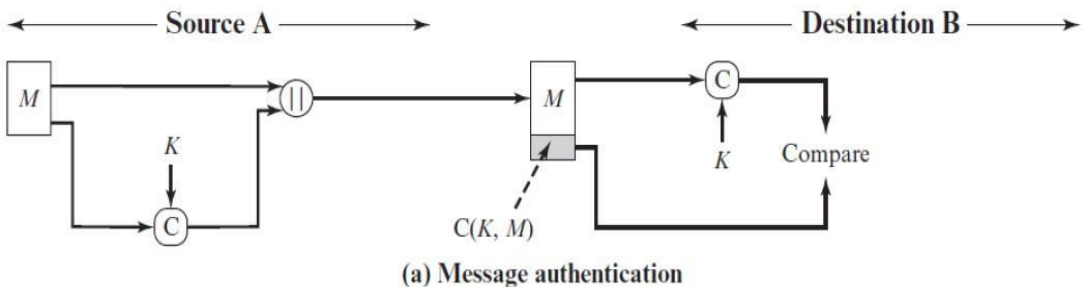
The message plus MAC are transmitted to the intended recipient.

- ✓ The recipient performs the same calculation on the received message, using the same secret key, to generate a new MAC.
- ✓ The received MAC is compared to the calculated MAC (Figure a). If we assume that only the receiver and the sender know the identity of the secret key, and if the received MAC matches the calculated MAC, then

1. The receiver is assured that the message has not been altered. If an attacker alters the message but does not alter the MAC, then the receiver's calculation of the MAC will differ from the received MAC. Because the attacker is assumed not to know the secret key, the attacker cannot alter the MAC to correspond to the alterations in the message.
2. The receiver is assured that the message is from the alleged sender. Because no one else knows the secret key, no one else could prepare a message with a proper MAC.
3. If the message includes a sequence number (such as is used with HDLC, X.25, and TCP), then the receiver can be assured of the proper sequence because an attacker cannot successfully alter the sequence number.

A MAC function is similar to encryption. One difference is that the MAC algorithm need not be reversible, as it must be for decryption.

- ✓ In general, the MAC function is a many-to-one function.
- ✓ The domain of the function consists of messages of some arbitrary length, whereas the range consists of all possible MACs and all possible keys.
- ✓ If an n -bit MAC is used, then there are 2^n possible MACs, whereas there are N possible messages with $N \gg 2^n$. Furthermore, with a k -bit key, there are 2^k possible keys.
- ✓ For example, suppose that we are using 100-bit messages and a 10-bit MAC. Then, there are a total of 2100 different messages but only 210 different MACs. So, on average, each MAC value is generated by a total of $2100/210 = 290$ different messages. If a 5-bit key is used, then there are $25 = 32$ different mappings from the set of messages to the set of MAC values.



HMAC Algorithm

H = embedded hash function (e.g., MD5, SHA-1, RIPEMD-160)

IV = initial value input to hash function

M = message input to HMAC (including the padding specified in the embedded hash function)

Y_i = i th block of M , $0 \leq i \leq (L - 1)$

L = number of blocks in M

b = number of bits in a block

n = length of hash code produced by embedded hash function

K = secret key; recommended length is $\geq n$; if key length is greater than b , the key is input to the hash function to produce an n -bit key

K^+ = K padded with zeros on the left so that the result is b bits in length

$ipad$ = 00110110 (36 in hexadecimal) repeated $b/8$ times

$opad$ = 01011100 (5C in hexadecimal) repeated $b/8$ times

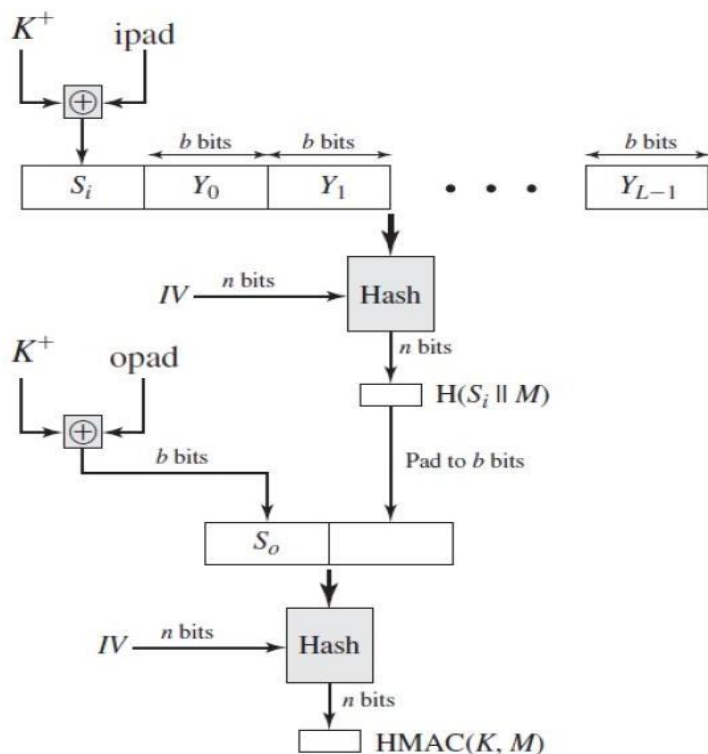


Fig 3.3 HMAC Structure

HMAC can be expressed as: $\text{HMAC}(K, M) = H[(K+ \oplus \text{opad}) \parallel H[(K+ \oplus \text{ipad}) \parallel M]]$

The algorithm is as follows:

1. Append zeros to the left end of K to create a b-bit string K+ (e.g., if K is of length 160 bits and , then will be appended with 44 zeroes).
2. XOR (bitwise exclusive-OR) with ipad to produce the b-bit block Si.
3. Append M to Si.
4. Apply H to the stream generated in step 3.
5. XOR K+ with opad to produce the b-bit block So
6. Append the hash result from step 4 to So.
7. Apply H to the stream generated in step 6 and output the result.

3.4 USER AUTHENTICATION

User authentication is the basis for most types of access control and for user accountability. RFC 4949 defines user authentication as follows:

The process of verifying an identity claimed by or for a system entity.

An authentication process consists of two steps:

- **Identification step:** Presenting an identifier to the security system. (Identifiers should be assigned carefully, because authenticated identities are the basis for other security services, such as access control service.)
- **Verification step:** Presenting or generating authentication information that corroborates the binding between the entity and the identifier.

3.4.1 Electronic User Authentication Principles

NIST SP 800-63-2 (*Electronic Authentication Guideline*, August 2013) defines electronic user authentication as the process of establishing confidence in user identities that are presented electronically to an information system. Systems can use the authenticated identity to determine if the authenticated individual is authorized to perform particular functions, such as database transactions or access to system resources. In many cases, the authentication and transaction or other authorized function take place across an open network such as the Internet. Equally authentication and subsequent authorization can take place locally, such as across a local

area network.

A Model for Electronic User Authentication

SP 800-63-2 defines a general model for user authentication that involves a number of entities and procedures. We discuss this model with reference to Figure 4.4.

The initial requirement for performing user authentication is that the user must be registered with the system. The following is a typical sequence for registration. An applicant applies to a **registration authority (RA)** to become a **subscriber** of a **credential service provider (CSP)**. In this model, the RA is a trusted entity that establishes and vouches for the identity of an applicant to a CSP. The CSP then engages in an exchange with the subscriber. Depending on the details of the overall authentication system, the CSP issues some sort of electronic credential to the subscriber. The **credential** is a data structure that authoritatively binds an identity and additional attributes to a token possessed by a subscriber, and can be verified when presented to the verifier in an authentication transaction. The token could be an encryption key or an encrypted password that identifies the subscriber. The token may be issued

by the CSP, generated directly by the subscriber, or provided by a third party. The token and credential may be used in subsequent authentication events.

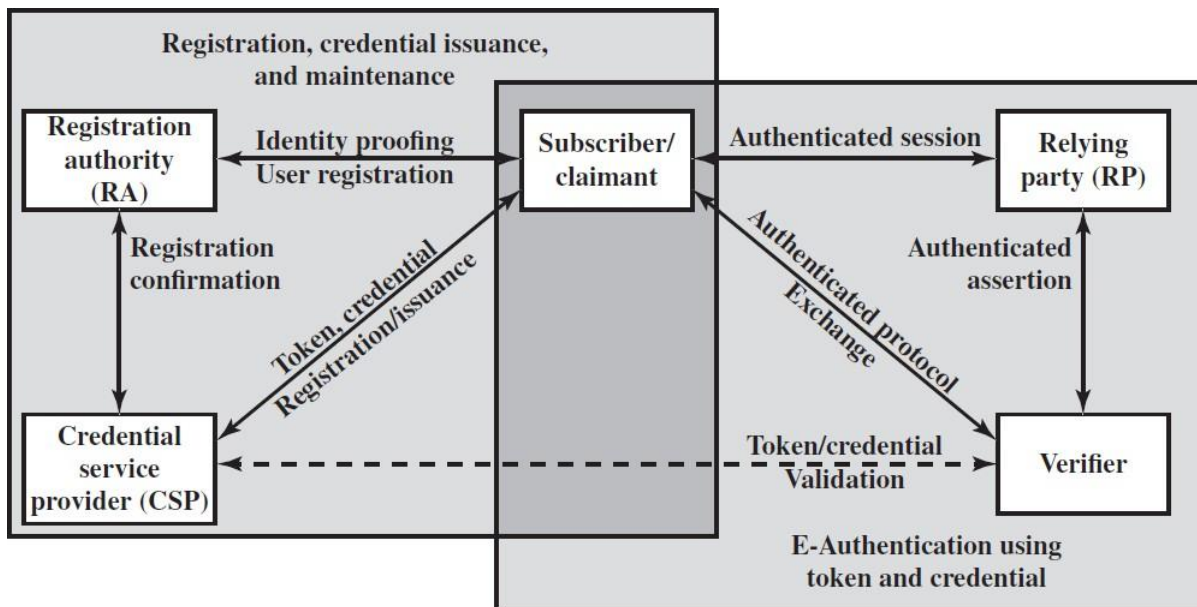


Fig 3.4.1 The NIST SP 800-63-2 E-Authentication Architectural Model

3.4.2 Password-Based Authentication

A widely used line of defense against intruders is the password system. Virtually all multiuser systems, network-based servers, Web-based e-commerce sites, and other similar services require that a user provide not only a name or identifier (ID) but also a password. The system compares the password to a previously stored password for that user ID, maintained in a system password file. The password serves to authenticate the ID of the individual logging on to the system. In turn, the ID provides security in the following ways:

- 3.4.2.1 The ID determines whether the user is authorized to gain access to a system. In some systems, only those who already have an ID filed on the system are allowed to gain access.
- 3.4.2.2 The ID determines the privileges accorded to the user. A few users may have supervisory or “superuser” status that enables them to read files and perform functions that are especially protected by the operating system. Some systems have guest or anonymous accounts, and users of these accounts have more limited privileges than others.
- 3.4.2.3 The ID is used in what is referred to as discretionary access control. For example, by listing the IDs of the other users, a user may grant permission to them to read files owned by that user.

The Use of Hashed Passwords

A widely used password security technique is the use of hashed passwords and a salt value. This scheme is found on virtually all UNIX variants as well as on a number of other operating systems. The following procedure is employed (Figure 4.5a).

When a user attempts to log on to a UNIX system, the user provides an ID and a password (Figure 4.5b). The operating system uses the ID to index into the password file and retrieve the plaintext salt and the encrypted password. The salt and user-supplied password are used as input to the encryption routine. If the result matches the stored value, the password is accepted.

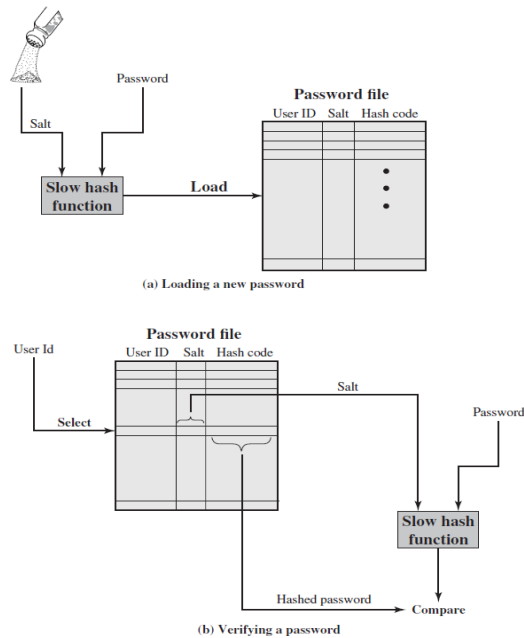


Fig 3.4.2 UNIX Password Scheme

The salt serves three purposes:

- It prevents duplicate passwords from being visible in the password file. Even if two users choose the same password, those passwords will be assigned different salt values. Hence, the hashed passwords of the two users will differ.
- It greatly increases the difficulty of offline dictionary attacks. For a salt of length b bits, the number of possible passwords is increased by a factor of 2^b , increasing the difficulty of guessing a password in a dictionary attack.
- It becomes nearly impossible to find out whether a person with passwords on two or more systems has used the same password on all of them.

Password File Access Control

One way to thwart a password attack is to deny the opponent access to the password file. If the hashed password portion of the file is accessible only by a privileged user, then the opponent cannot read it without already knowing the password of a privileged user. Often, the hashed passwords are kept in a separate file from the user IDs, referred to as a shadow password file. Special attention is paid to making the shadow password file protected from unauthorized access. Although password file protection is certainly worthwhile,

there remain vulnerabilities:

- Many systems, including most UNIX systems, are susceptible to unanticipated break-ins. A hacker may be able to exploit a software vulnerability in the operating system to bypass the access control system long enough to extract the password file. Alternatively, the hacker may find a weakness in the file system or database management system that allows access to the file.
- An accident of protection might render the password file readable, thus compromising all the accounts.
- Some of the users have accounts on other machines in other protection domains, and they use the same password. Thus, if the passwords could be read by anyone on one machine, a machine in another location might be compromised.
- A lack of or weakness in physical security may provide opportunities for a hacker. Sometimes there is a backup to the password file on an emergency repair disk or archival disk. Access to this backup enables the attacker to read the password file. Alternatively, a user may boot from a disk running another operating system such as Linux and access the file from this OS.
- Instead of capturing the system password file, another approach to collecting user IDs and passwords is through sniffing network traffic.
- Thus, a password protection policy must complement access control measures with techniques to force users to select passwords that are difficult to guess.

Password Selection Strategies

When not constrained, many users choose a password that is too short or too easy to guess. At the other extreme, if users are assigned passwords consisting of eight randomly selected printable characters, password cracking is effectively impossible. But it would be almost as impossible for most users to remember their passwords.

Fortunately, even if we limit the password universe to strings of characters that are reasonably memorable, the size of the universe is still too large to permit practical cracking. Our goal, then, is to eliminate guessable passwords while allowing the user to select a password that is memorable.

Four basic techniques are in use:

- User education
- Computer-generated passwords
- Reactive password checking
- Complex password policy

3.4.3 Token-Based Authentication

Objects that a user possesses for the purpose of user authentication are called tokens. In this section, we examine two types of tokens that are widely used; these are cards that have the appearance and size of bank cards (see Table).

Memory Cards

Memory cards can store but not process data. The most common such card is the bankcard with a magnetic stripe on the back. A magnetic stripe can store only a simple security code, which can be read (and unfortunately reprogrammed) by an inexpensive card reader. There are also memory cards that include an internal electronic memory.

Table: Types of Cards Used as Tokens

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart Contact Contactless	Electronic memory and processor inside Electrical contacts exposed on surface Radio antenna embedded inside	Biometric ID card

Memory cards can be used alone for physical access, such as a hotel room. For authentication, a user provides both the memory card and some form of password or personal identification number (PIN). A typical application is an automatic teller machine (ATM). The memory card, when combined with a PIN or password, provides significantly greater security than a password alone. An adversary must gain physical possession of the card (or be able to duplicate it) plus must gain knowledge of the PIN. Among the potential drawbacks are the following [NIST95]:

- **Requires special reader:** This increases the cost of using the token and creates the requirement to maintain the security of the reader's hardware and software.

- **Token loss:** A lost token temporarily prevents its owner from gaining system access. Thus there is an administrative cost in replacing the lost token. In addition, if the token is found, stolen, or forged, then an adversary now need only determine the PIN to gain unauthorized access.
- **User dissatisfaction:** Although users may have no difficulty in accepting the use of a memory card for ATM access, its use for computer access may be deemed inconvenient.

Electronic Identity Cards

An application of increasing importance is the use of a smart card as a national identity card for citizens. A national electronic identity (eID) card can serve the same purposes as other national ID cards, and similar cards such as a driver's license, for access to government and commercial services. In addition, an eID card can provide stronger proof of identity and be used in a wider variety of applications. In effect, an eID card is a smart card that has been verified by the national government as valid and authentic.

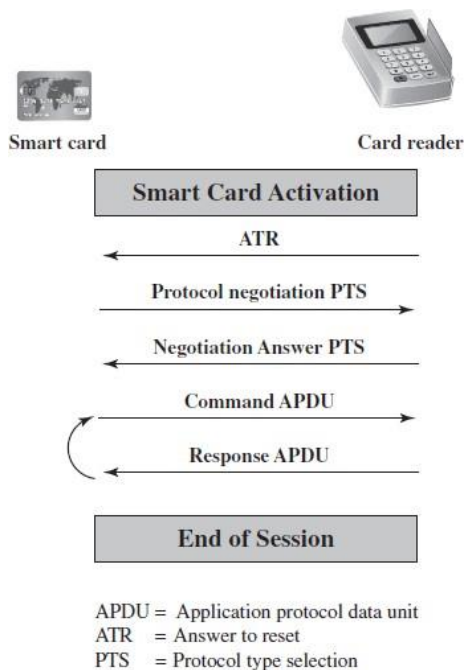


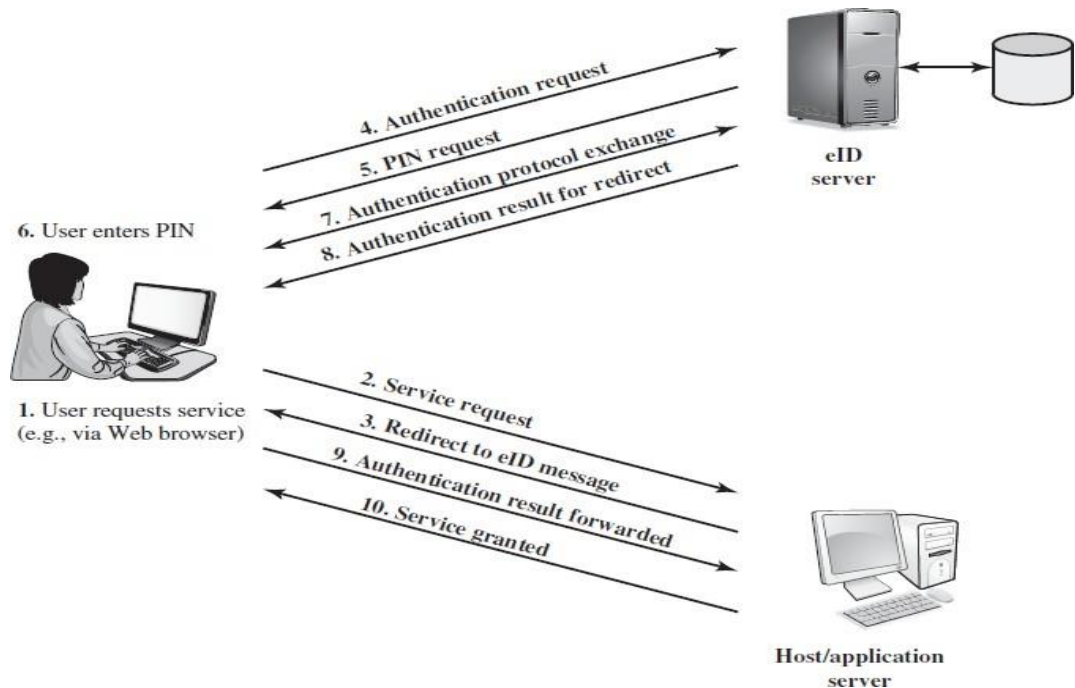
Fig 4.6 Smart Card/Reader Exchange

One of the most recent and most advanced eID deployments is the German eID card *neuer Personalausweis* [POLL12]. The card has human-readable data printed on its surface, including the following:

- **Personal data:** Such as name, date of birth, and address; this is the type of printed information found on passports and driver's licenses.
- **Document number:** An alphanumerical nine-character unique identifier of each card.
- **Card access number (CAN):** A six-digit decimal random number printed on the face of the card. This is used as a password, as explained subsequently.
- **Machine readable zone (MRZ):** Three lines of human- and machine-readable text on the back of the card. This may also be used as a password.

Eid Functions The card has the following three separate electronic functions, each with its own protected dataset (Table):

- **ePass:** This function is reserved for government use and stores a digital representation of the cardholder's identity. This function is similar to, and may be used for, an electronic passport. Other government services may also use ePass. The ePass function must be implemented on the card.
- **eID:** This function is for general-purpose use in a variety of government and commercial applications. The eID function stores an identity record that authorized service can access with cardholder permission. Citizens choose whether they want this function activated.
- **eSign:** This optional function stores a private key and a certificate verifying the key; it is used for generating a digital signature. A private sector trust center issues the certificate.



User Authentication with eID

Password Authenticated Connection Establishment (PACE) Password Authenticated Connection Establishment (PACE) ensures that the contactless RF chip in the eID card cannot be read without explicit access control. For online applications, access to the card is established by the user entering the 6-digit PIN, which should only be known to the holder of the card. For offline applications, either the MRZ printed on the back of the card or the six-digit card access number (CAN) printed on the front is used.

3.5 INTERNET AUTHENTICATION APPLICATIONS

3.5.1 KERBEROS

There are a number of approaches that organizations can use to secure networked servers and hosts. Systems that use one-time passwords thwart any attempt to guess or capture a user's password. These systems require special equipment such as smart cards or synchronized password generators to operate and have been slow to gain acceptance for general networking use. Another approach is the use of biometric systems. These are automated methods of verifying or recognizing identity on the basis of some physiological characteristic, such as a fingerprint or iris pattern, or a behavioral

characteristic, such as handwriting or keystroke patterns.

The Kerberos Protocol

The basic idea is simple. In an unprotected network environment, any client can apply to any server for service. The obvious security risk is that of impersonation.

An opponent can pretend to be another client and obtain unauthorized privileges on server machines. To counter this threat, servers must be able to confirm the identities of clients who request service. Each server can be required to undertake this task for each client/server interaction, but in an open environment, this places a substantial burden on each server. An alternative is to use an authentication server (AS) that knows the passwords of all clients and stores these in a centralized database.

Then the user can log onto the AS for identity verification. Once the AS has verified the user's identity, it can pass this information on to an application server, which will then accept service requests from the client. The trick is how to do all this in a secure way. It simply will not do to have the client send the user's password to the AS over the network: An opponent could observe the password on the network and later reuse it. It also will not do for Kerberos to send a plain message to a server validating a client: An opponent could impersonate the AS and send a false validation.

The way around this problem is to use encryption and a set of messages that accomplish the task (Figure 3.5.1). In the case of Kerberos, the Data Encryption Standard (DES) is the encryption algorithm that is used.

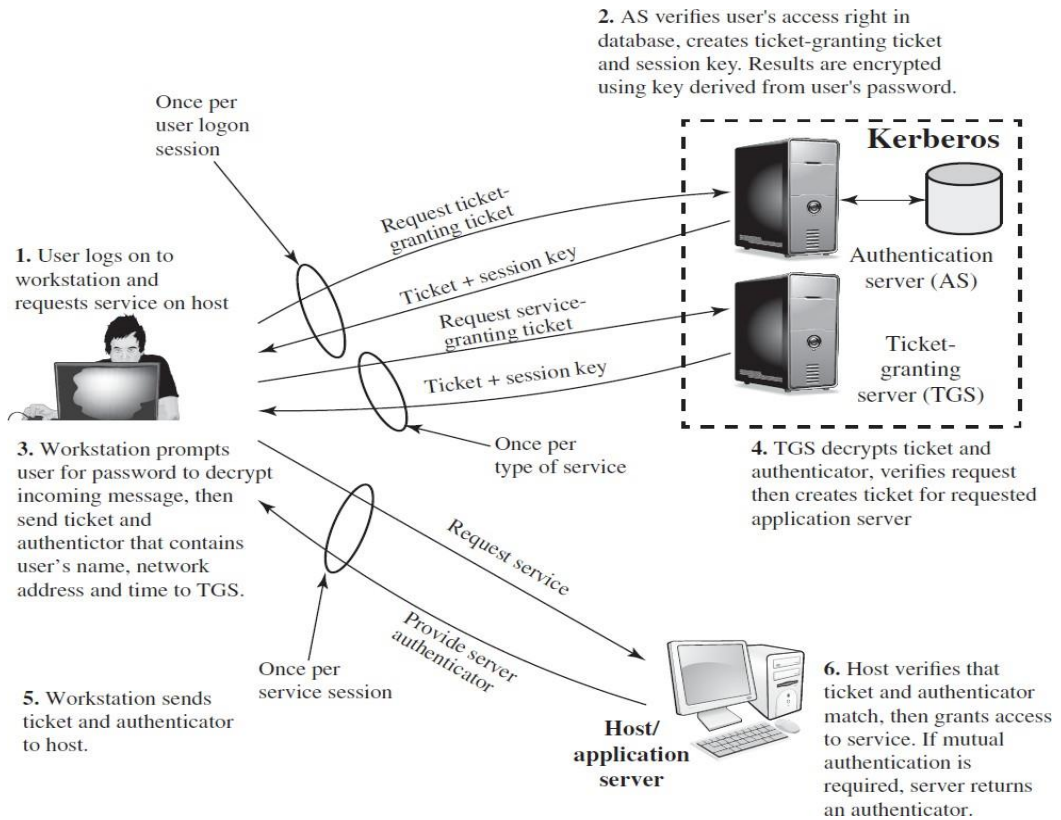


Fig 3.5.1.1 Overview of Kerberos

The AS shares a unique secret key with each server. These keys have been distributed physically or in some other secure manner. This will enable the AS to send messages to application servers in a secure fashion. To begin, the user logs on to a workstation and requests access to a particular server. The client process representing the user sends a message to the AS that includes the user's ID and a request for what is known as a ticket-granting ticket (TGT). The AS checks its database to find the password of this user. Then the AS responds with a TGT and a one-time encryption key, known as a session key, both

encrypted using the user's password as the encryption key. When this message arrives back at the client, the client prompts the user for his or her password, generates the key, and attempts to decrypt the incoming message. If the correct password has been supplied, the ticket and session key are successfully recovered.

Notice what has happened. The AS has been able to verify the user's identity since this user knows the correct password, but it has been done in such a way that the password is never passed over the network. In addition, the AS has passed information to the client that will be used later on to apply to a server for service, and that information is secure since it is encrypted with the user's password.

The ticket constitutes a set of credentials that can be used by the client to apply for service. The ticket indicates that the AS has accepted this client and its user. The ticket contains the user's ID, the server's ID, a timestamp, a lifetime after which the ticket is invalid, and a copy of the same session key sent in the outer message to the client. The entire ticket is encrypted using a secret DES key shared by the AS and the server. Thus, no one can tamper with the ticket.

Now, Kerberos could have been set up so that the AS would send back a ticket granting access to a particular application server. This would require the client to request a new ticket from the AS for each service that the user wants to use during a logon session, which would in turn require that the AS query the user for his or her password for each service request or else to store the password in memory for the duration of the logon session. The first course is inconvenient for the user and the second course is a security risk. Therefore, the AS supplies a ticket good not for a specific application service but for a special ticket-granting server (TGS). The AS gives the client a ticket that can be used to get more tickets!

The idea is that this ticket can be used by the client to request multiple service-granting tickets. So the ticket-granting ticket is to be reusable. However, we do not wish an opponent to be able to capture the ticket and use it. Consider the following scenario: An

opponent captures the ticket and waits until the user has logged off the workstation. Then the opponent either gains access to that workstation or configures his workstation with the same network address as that of the victim. Then the opponent would be able to reuse the ticket to spoof the TGS. To counter this, the ticket includes a timestamp, indicating the date and time at which the ticket was issued, and a lifetime, indicating the length of time for which the ticket is valid (e.g., 8 hours). Thus, the client now has a reusable ticket and need not bother the user for a password for each new service request. Finally, note that the ticket-granting ticket is encrypted with a secret key known only to the AS and the TGS. This prevents alteration of the ticket. The ticket is reencrypted with a key based on the user's password. This assures that the ticket can be recovered only by the correct user, providing the authentication.

Let us see how this works. The user has requested access to server V. The client process representing the user (C) has obtained a ticket-granting ticket and a temporary session key. The client then sends a message to the TGS requesting a ticket for user X that will grant service to server V. The message includes the ID of server V and the ticket-granting ticket. The TGS decrypts the incoming ticket (remember, the ticket is encrypted by a key known only to the AS and the TGS) and verifies the success of the decryption by the presence of its own ID. It checks to make sure that the lifetime has not expired. Then it compares the user ID and network address with the incoming information to authenticate the user.

At this point, the TGS is almost ready to grant a service-granting ticket to the client. But there is one more threat to overcome. The heart of the problem is the lifetime associated with the ticket-granting ticket. If this lifetime is very short (e.g., minutes), then the user will be repeatedly asked for a password.

To get around this problem, the AS has provided both the client and the TGS with a secret session key that they now share. The session key, recall, was in the message from the AS to the client, encrypted with the user's password. It was also buried in the ticket-granting ticket, encrypted with the key shared by the AS and TGS. In the message to the TGS requesting a service-granting ticket, the client includes an authenticator encrypted with

the session key, which contains the ID and address of the user and a timestamp.

The next two steps in the protocol repeat the last two. The TGS sends a service granting ticket and a new session key to the client. The entire message is encrypted with the old session key, so that only the client can recover the message. The ticket is encrypted with a secret key shared only by the TGS and server V. The client now has a reusable service-granting ticket for V. Each time user X wishes to use service V, the client can then send this ticket plus an authenticator to server V. The authenticator is encrypted with the new session key.

If mutual authentication is required, the server can reply with the value of the timestamp from the authenticator, incremented by 1, and encrypted in the session key. The client can decrypt this message to recover the incremented timestamp.

Because the message was encrypted by the session key, the client is assured that it could have been created only by V. The contents of the message assures C that this is not a replay of an old reply.

Finally, at the conclusion of this process, the client and server share a secret key. This key can be used to encrypt future messages between the two or to exchange a new session key for that purpose.

Kerberos Realms and Multiple Kerberi

A full-service Kerberos environment consisting of a Kerberos server, a number of clients, and a number of application servers, requires the following:

1. The Kerberos server must have the user ID and password of all participating users in its database. All users are registered with the Kerberos server.
2. The Kerberos server must share a secret key with each server. All servers are registered with the Kerberos server.

Such an environment is referred to as a realm. Networks of clients and servers under different administrative organizations generally constitute different realms (Figure 3.5.1.2). That is, it generally is not practical, or does not conform to administrative policy, to have users and servers in one administrative domain registered with a Kerberos server elsewhere. However, users in one realm may need access to servers in

other realms, and some servers may be willing to provide service to users from other realms, provided that those users are authenticated.

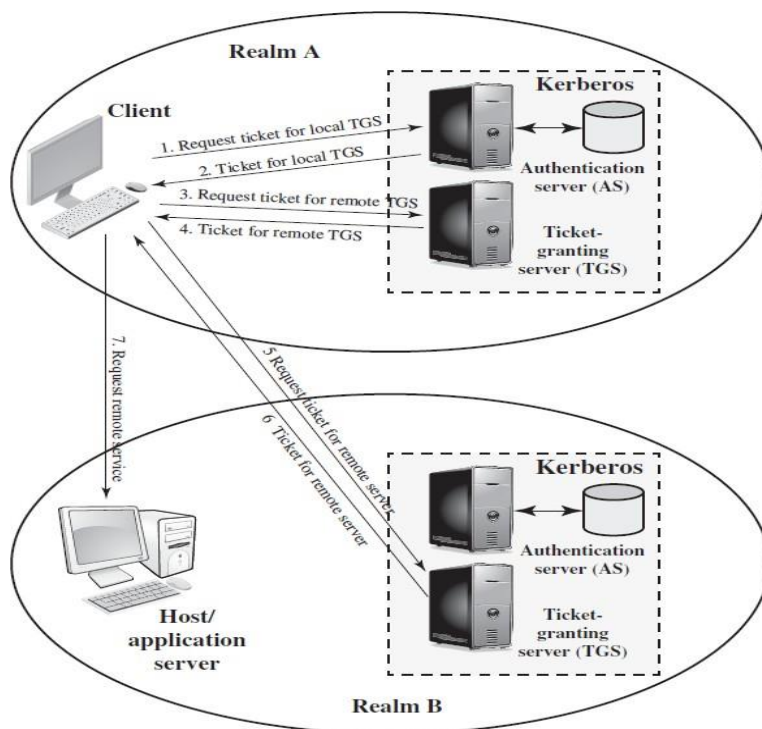


Fig 3.5.1.2 Request for Service in Another Realm

3.5.2 X.509

Typically, the third party is a certificate authority (CA) that is trusted by the user community, such as a government agency, financial institution, telecommunications company, or other trusted peak organization. A user can present his or her public key to

the authority in a secure manner and obtain a certificate. The user can then publish the certificate, or send it to others. Anyone needing this user's public key can obtain the certificate and verify that it is valid by way of the attached trusted signature, provided they can verify the CA's public key. Figure 3.5.2 illustrates this process.

The X.509 ITU-T standard, also specified in RFC 5280, is the most widely accepted format for public-key certificates. X.509 certificates are used in most network security applications, including IP security (IPSEC), secure sockets layer (SSL), secure electronic transactions (SET), and S/MIME, as well as in eBusiness applications.

An X.509 certificate includes the elements shown in Figure 4.10a. Key elements include the key owning Subject's X.500 name and public-key information, the Period of validity dates, the CA's Issuer name, and their Signature that binds all this information together. Current X.509 certificates use the version 3 format that includes a general extension mechanism to provide more flexibility and to convey information needed in special circumstances.

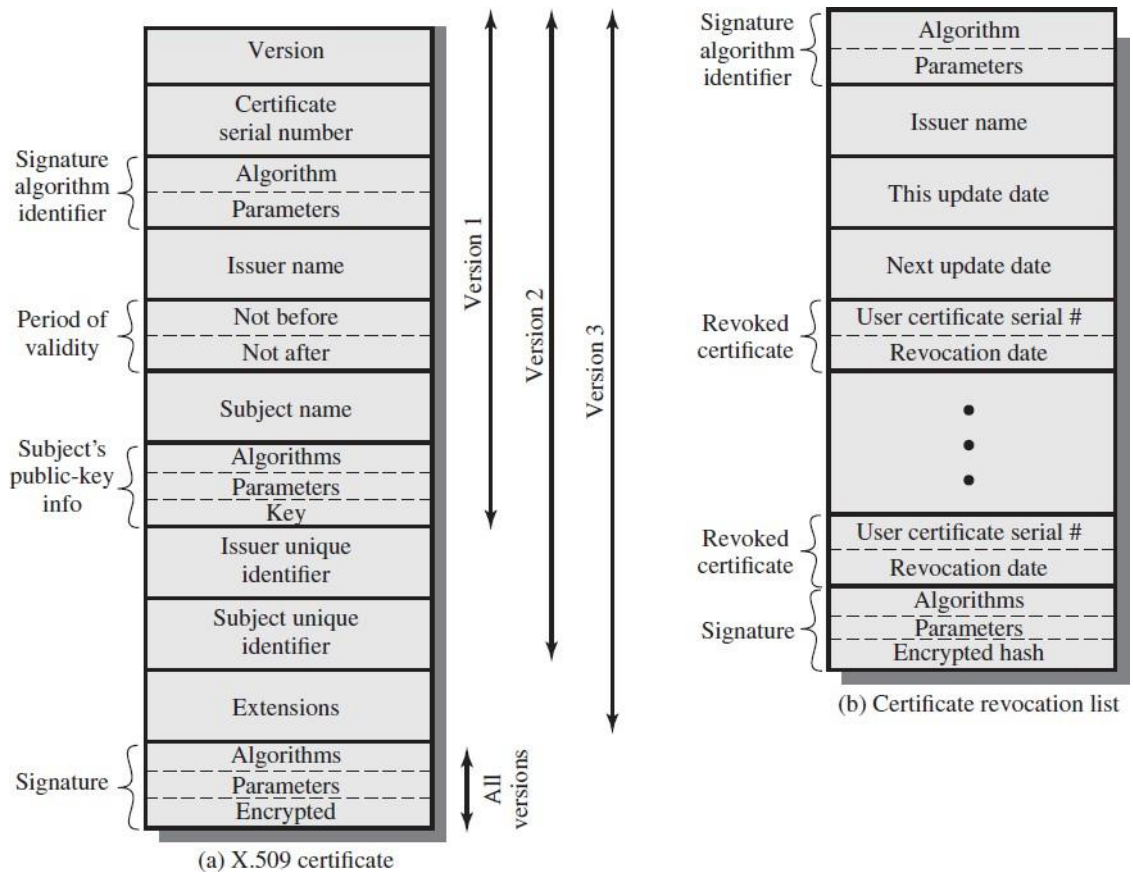


Fig 3.5.2 X.509 Formats

One important extension, in the “Basic Constraints” set, specifies whether the certificate is that of a CA or not. A CA certificate is used only to sign other certificates. Otherwise the certificate belongs to an “end-user” (or “end-entity”), and may be used for verifying server or client identities, signing or encrypting email or other content, signing executable code, or other uses in applications such as those we listed above. The usage of any certificate’s key can be restricted by including the “Key Usage” and “Extended Key Usage” extensions that specify a set of approved uses. “End-user” certificates are not permitted to sign other certificates, apart from the special case of proxy-certificates.

3.5.3 PUBLIC-KEY INFRASTRUCTURE

RFC 4949 (Internet Security Glossary) defines public-key infrastructure (PKI) as the set of hardware, software, people, policies, and procedures needed to create, manage, store, distribute, and revoke digital certificates based on asymmetric cryptography. The

principal objective for developing a PKI is to enable secure, convenient, and efficient acquisition of public keys.

In order to verify a certificate, you need to know the public key of the signing CA. This could, in turn, be provided in another certificate, signed by a parent CA, with the CA's organized in a hierarchy. Eventually, however, you must reach the top of the hierarchy, and have a copy of the public key for that root CA. The X.509 standard describes a PKI model that originally assumed there would be a single internationally specified hierarchy of government regulated CAs. This did not happen. Instead, current X.509 PKI implementations came with a large list of CAs and their public keys, known as a "trust store." These CAs usually either directly sign "end-user" certificates or sign a small number of Intermediate-Cas that in turn sign "end-user" certificates. Thus all the hierarchies are very small, and all are equally trusted. Users and servers that want an automatically verified certificate must acquire it from one of these CAs. Alternatively, they can use either a "self-signed" certificate or a certificate signed by some other CA. However, in both these cases, such certificates will initially be recognized as "untrusted" and the user presented with stark warnings about accepting such certificates, even if they are actually legitimate.

Public Key Infrastructure X.509 (PKIX)

The Internet Engineering Task Force (IETF) Public Key Infrastructure X.509 (PKIX) working group has been the driving force behind setting up a formal (and generic) model based on X.509 that is suitable for deploying a certificate-based architecture on the Internet. This section briefly describes the PKIX model. For more detail, see [STAL14a]. Figure 4.11 shows the interrelationship among the key elements of the PKIX model. These elements include the End entity (e.g., user or server) that the certificate is for and the Certificate authority that issues the certificates. The CA's management functions may be further divided to include the Registration authority (RA) that handles end entity registration and the CRL issuer and Repository that manage CRLs.

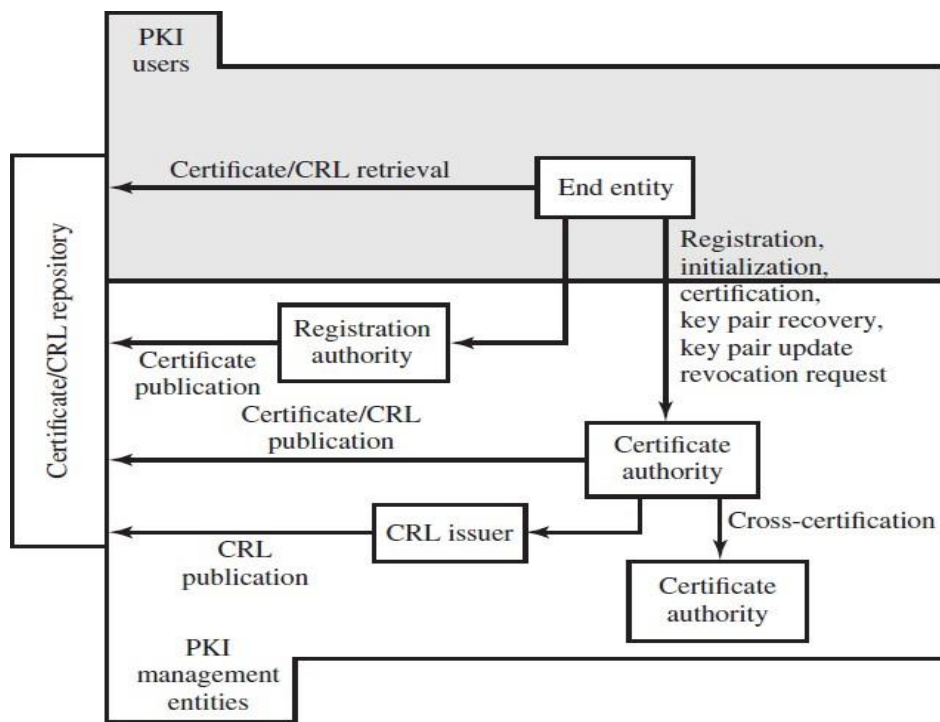


Fig 3.5.3 PKIX Architectural Model

PKIX identifies a number of management functions that potentially need to be supported by management protocols. These are indicated in Figure 3.5.3 and include user Registration, Initialization of key material, Certification in which a CA issues a certificate, Key pair recovery and update, Revocation request for a certificate, and Cross certification between CAs.

3.6 ACCESS CONTROL

Two definitions of access control are useful in understanding its scope.

1. NIST IR 7298, Glossary of Key Information Security Terms, defines access control as the process of granting or denying specific requests to: (1) obtain and use information and related information processing services; and (2) enter specific physical facilities.
2. RFC 4949, Internet Security Glossary, defines access control as a process by which use of system resources is regulated according to a security policy and is permitted only by authorized entities (users, programs, processes, or other systems) according to that policy.

3.6.1 ACCESS CONTROL PRINCIPLES

Access Control Context

In addition to access control, this context involves the following entities and functions:

- **Authentication:** Verification that the credentials of a user or other system entity are valid.
- **Authorization:** The granting of a right or permission to a system entity to access a system resource. This function determines who is trusted for a given purpose.
- **Audit:** An independent review and examination of system records and activities in order to test for adequacy of system controls, to ensure compliance with established policy and operational procedures, to detect breaches in security, and to recommend any indicated changes in control, policy and procedures.

An access control mechanism mediates between a user (or a process executing on behalf of a user) and system resources, such as applications, operating systems, firewalls, routers, files, and databases. The system must first authenticate an entity seeking access. Typically, the authentication function determines whether the user is permitted to access the system at all. Then the access control function determines if the specific requested access by this user is permitted. A security administrator maintains an authorization database that specifies what type of access to which resources is allowed for this user. The access control function consults this database to determine whether to grant access. An auditing function monitors and keeps a record of user accesses to system resources. In the simple model of Figure 3.6.1, the access control function is shown as a single logical module. In practice, a number of components may cooperatively share the access control function. All operating systems have at least a rudimentary, and in many cases a quite robust, access control component. Add-on security packages can supplement the native access control capabilities of the operating system. Particular applications or utilities, such as a database management system, also incorporate access control functions. External devices, such as firewalls, can also provide access control services.

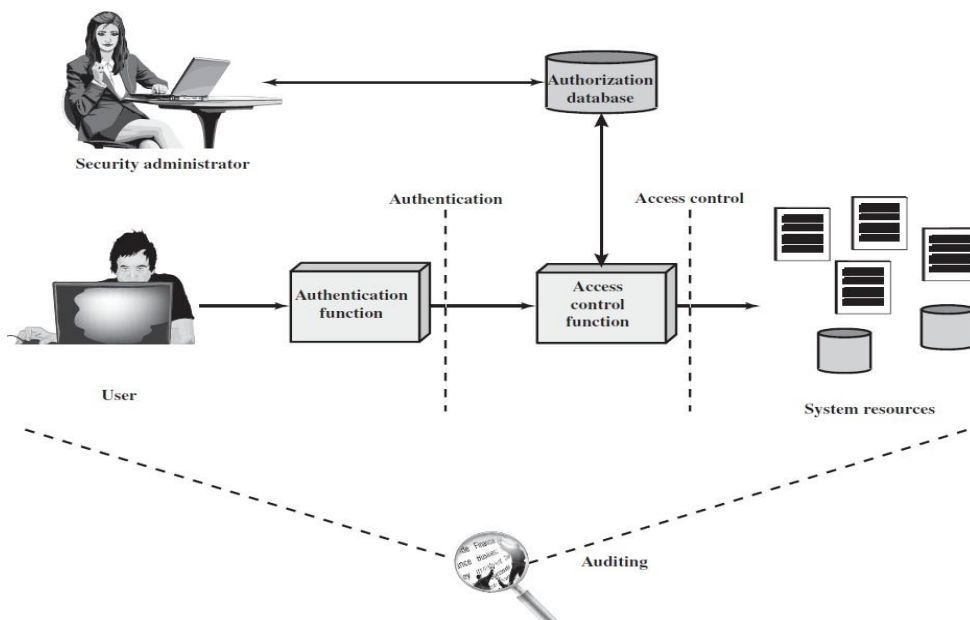


Fig 3.6.1 Relationship Among Access Control and Other Security Functions

Access Control Policies

An access control policy, which can be embodied in an authorization database, dictates what types of access are permitted, under what circumstances, and by whom. Access control policies are generally grouped into the following categories:

- **Discretionary access control (DAC):** Controls access based on the identity of the requestor and on access rules (authorizations) stating what requestors are (or are not) allowed to do. This policy is termed discretionary because an entity might have access rights that permit the entity, by its own volition, to enable another entity to access some resource.
- **Mandatory access control (MAC):** Controls access based on comparing security labels (which indicate how sensitive or critical system resources are) with security clearances (which indicate system entities are eligible to access certain resources). This policy is termed mandatory because an entity that has clearance to access a resource may not, just by its own volition, enable another entity to access that resource.
- **Role-based access control (RBAC):** Controls access based on the roles that users have within the system and on rules stating what accesses are allowed to users in given roles.

- **Attribute-based access control (ABAC):** Controls access based on attributes of the user, the resource to be accessed, and current environmental conditions.

3.6.2 SUBJECTS, OBJECTS, AND ACCESS RIGHTS

The basic elements of access control are: subject, object, and access right.

A subject is an entity capable of accessing objects. Generally, the concept of subject equates with that of process. Any user or application actually gains access to an object by means of a process that represents that user or application. The process takes on the attributes of the user, such as access rights.

A subject is typically held accountable for the actions they have initiated, and an audit trail may be used to record the association of a subject with security relevant actions performed on an object by the subject.

Basic access control systems typically define three classes of subject, with different access rights for each class:

- **Owner:** This may be the creator of a resource, such as a file. For system resources, ownership may belong to a system administrator. For project resources, a project administrator or leader may be assigned ownership.
- **Group:** In addition to the privileges assigned to an owner, a named group of users may also be granted access rights, such that membership in the group is sufficient to exercise these access rights. In most schemes, a user may belong to multiple groups.
- **World:** The least amount of access is granted to users who are able to access the system but are not included in the categories owner and group for this resource.

An **object** is a resource to which access is controlled. In general, an object is an entity used to contain and/or receive information. Examples include records, blocks, pages, segments, files, portions of files, directories, directory trees, mailboxes, messages, and programs. Some access control systems also encompass, bits, bytes, words, processors, communication ports, clocks, and network nodes.

The number and types of objects to be protected by an access control system depends on the environment in which access control operates and the desired tradeoff between

security on the one hand and complexity, processing burden, and ease of use on the other hand.

An **access right** describes the way in which a subject may access an object. Access rights could include the following:

- **Read:** User may view information in a system resource (e.g., a file, selected records in a file, selected fields within a record, or some combination). Read access includes the ability to copy or print.
- **Write:** User may add, modify, or delete data in system resource (e.g., files, records, programs). Write access includes read access.
- **Execute:** User may execute specified programs.
- **Delete:** User may delete certain system resources, such as files or records.
- **Create:** User may create new files, records, or fields.
- **Search:** User may list the files in a directory or otherwise search the directory.

3.6.3 DISCRETIONARY ACCESS CONTROL

In practice, an access matrix is usually sparse and is implemented by decomposition in one of two ways. The matrix may be decomposed by columns, yielding access control lists (ACLs); see Figure 3.6.3b. For each object, an ACL lists users and their permitted access rights. The ACL may contain a default, or public, entry. This allows users that are not explicitly listed as having special rights to have a default set of rights. The default set of rights should always follow the rule of least privilege or read-only access, whichever is applicable. Elements of the list may include individual users as well as groups of users. When it is desired to determine which subjects have which access rights to a particular resource, ACLs are convenient, because each ACL provides the information for a given resource. However, this data structure is not convenient for determining the access rights available to a specific user.

Decomposition by rows yields capability tickets (Figure 3.6.3c). A capability ticket specifies authorized objects and operations for a particular user. Each user has a number of tickets and may be authorized to loan or give them to others. Because tickets may be dispersed around the system, they present a greater security problem than access control

lists. The integrity of the ticket must be protected, and guaranteed (usually by the operating system). In particular, the ticket must be unforgeable. One way to accomplish this is to have the operating system hold all tickets on behalf of users. These tickets would have to be held in a region of memory inaccessible to users. Another alternative is to include an unforgeable token in the capability. This could be a large random password, or a cryptographic message authentication code. This value is verified by the relevant resource whenever access is requested. This form of capability ticket is appropriate for use in a distributed environment, when the security of its contents cannot be guaranteed.

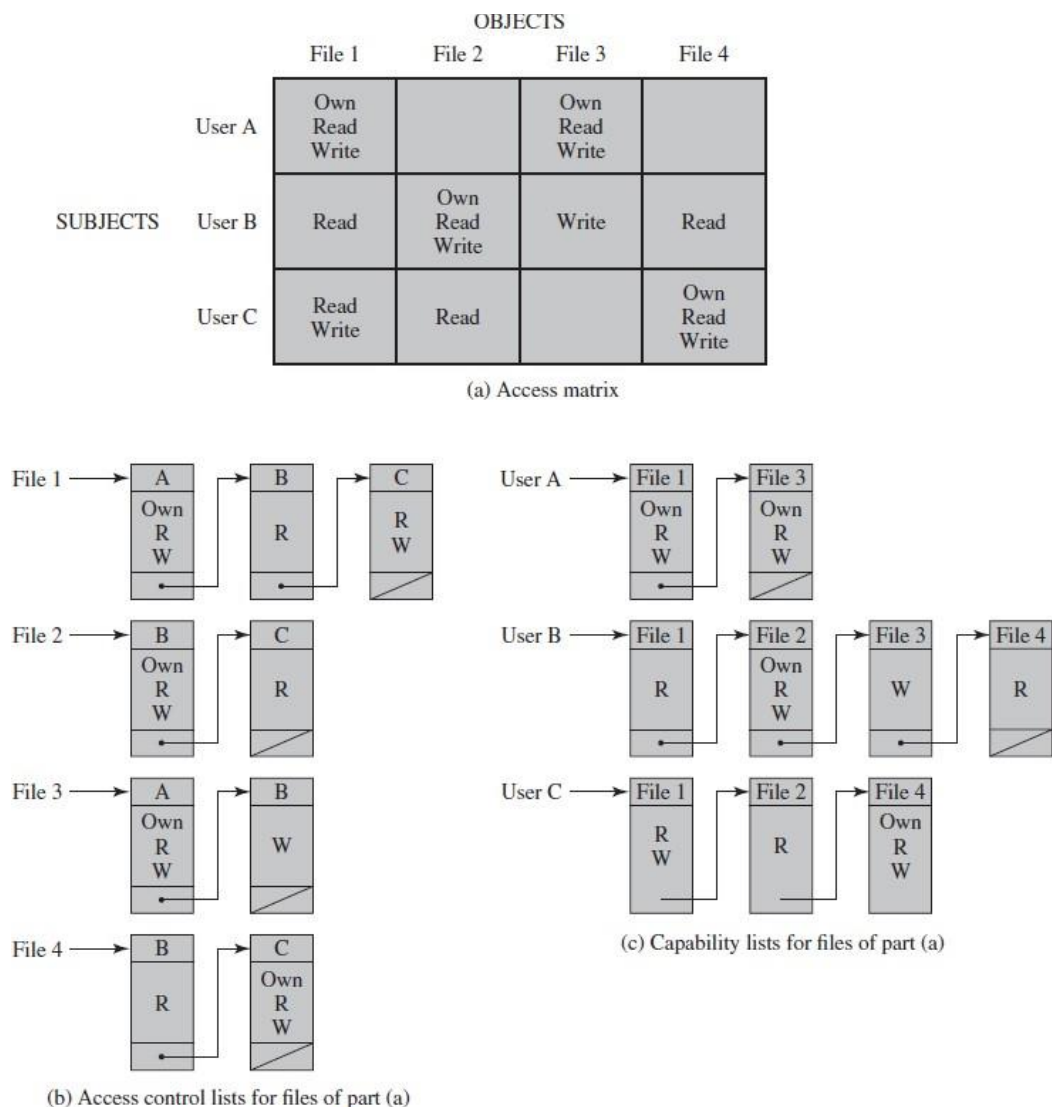


Fig 3.6.3 Example of Access Control Structures

The convenient and inconvenient aspects of capability tickets are the opposite of those for ACLs. It is easy to determine the set of access rights that a given user has, but more difficult to determine the list of users with specific access rights for a specific resource. [SAND94] proposes a data structure that is not sparse, like the access matrix, but is more convenient than either ACLs or capability lists (Table). An authorization table contains one row for one access right of one subject to one resource.

Sorting or accessing the table by subject is equivalent to a capability list. Sorting or accessing the table by object is equivalent to an ACL. A relational database can easily implement an authorization table of this type.

Table: Authorization Table for Files

Subject	Access Mode	Object
A	Own	File 1
A	Read	File 1
A	Write	File 1
A	Own	File 3
A	Read	File 3
A	Write	File 3
B	Read	File 1
B	Own	File 2
B	Read	File 2
B	Write	File 2
B	Write	File 3
B	Read	File 4
C	Read	File 1
C	Write	File 1
C	Read	File 2
C	Own	File 4
C	Read	File 4
C	Write	File 4

An Access Control Model

To represent the protection state, we extend the universe of objects in the access control matrix to include the following:

- **Processes:** Access rights include the ability to delete a process, stop (block), and wake up a process.
- **Devices:** Access rights include the ability to read/write the device, to control its operation (e.g., a disk seek), and to block/unblock the device for use.
- **Memory locations or regions:** Access rights include the ability to read/write certain regions of memory that are protected such that the default is to disallow access.
- **Subjects:** Access rights with respect to a subject have to do with the ability to grant or delete access rights of that subject to other objects, as explained subsequently.

		OBJECTS								
		Subjects			Files		Processes		Disk drives	
		S ₁	S ₂	S ₃	F ₁	F ₂	P ₁	P ₂	D ₁	D ₂
SUBJECTS	S ₁	control	owner	owner control	read*	read owner	wakeup	wakeup	seek	owner
	S ₂		control		write*	execute			owner	seek*
	S ₃			control		write	stop			

* = copy flag set

Figure 3.6.4 Extended Access Control Matrix

For an access control matrix A , each entry $A[S, X]$ contains strings, called access attributes, that specify the access rights of subject S to object X . For example, S_1 may read file F_1 , because 'read' appears in $A[S_1, F_1]$.

From a logical or functional point of view, a separate access control module is associated with each type of object (Figure 4.15). The module evaluates each request by a subject to access an object to determine if the access right exists. An access attempt triggers the following steps:

1. A subject S_0 issues a request of type α for object X .
2. The request causes the system (the operating system or an access control interface module of some sort) to generate a message of the form (S_0, α, X) to the controller for X .
3. The controller interrogates the access matrix A to determine if α is in $A[S_0, X]$.

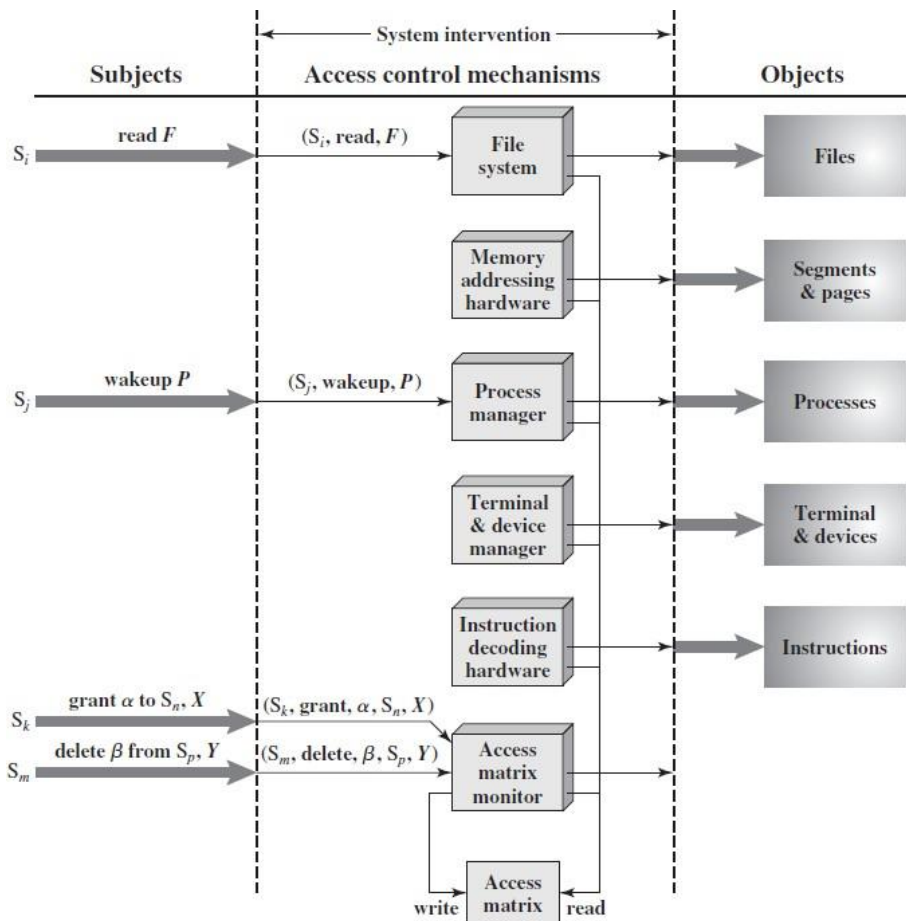


Fig 3.6.5 An Organization of the Access Control Function

The first three rules deal with transferring, granting, and deleting access rights. Suppose that the entry α^* exists in $A[S_0, X]$. This means that S_0 has access right α to subject X and, because of the presence of the copy flag, can transfer this right, with or without copy flag, to another subject. Rule R1 expresses this capability. A subject would transfer the access right without the copy flag if there were a concern that the new subject would maliciously transfer the right to another subject that should not have that access right. For example, S_1 may place 'read' or 'read*' in any matrix entry in the F1 column. Rule R2 states that if S_0 is designated as the owner of object X , then S_0 can grant an access right to that object for any other subject. Rule R2 states that S_0 can add any access right to $A[S, X]$ for any S , if S_0 has 'owner' access to X . Rule R3 permits S_0 to delete any access right from any matrix entry in a row for which S_0 controls the subject and for any matrix entry in a column for which S_0 owns the object. Rule R4 permits a subject to read that portion of the matrix that it owns or controls.

The remaining rules in Table govern the creation and deletion of subjects and objects. Rule R5 states that any subject can create a new object, which it owns, and can then grant and delete access to the object. Under Rule R6, the owner of an object can destroy the object, resulting in the deletion of the corresponding column of the access matrix. Rule R7 enables any subject to create a new subject; the creator owns the new subject and the new subject has control access to itself. Rule R8 permits the owner of a subject to delete the row and column (if there are subject columns) of the access matrix designated by that subject.

The set of rules in Table 3.6 is an example of the rule set that could be defined for an access control system. The following are examples of additional or alternative rules that could be included. A transfer-only right could be defined, which results in the transferred right being added to the target subject and deleted from the transferring subject. The number of owners of an object or a subject could be limited to one by not allowing the copy flag to accompany the owner right. The ability of one subject to create another subject and to have 'owner' access right to that subject can be used to define a hierarchy of subjects.

Rule	Command (by S_0)	Authorization	Operation
R1	transfer $\left\{ \begin{smallmatrix} \alpha^* \\ \alpha \end{smallmatrix} \right\}$ to S, X	' α^* ' in $A[S_0, X]$	store $\left\{ \begin{smallmatrix} \alpha^* \\ \alpha \end{smallmatrix} \right\}$ in $A[S, X]$
R2	grant $\left\{ \begin{smallmatrix} \alpha^* \\ \alpha \end{smallmatrix} \right\}$ to S, X	'owner' in $A[S_0, X]$	store $\left\{ \begin{smallmatrix} \alpha^* \\ \alpha \end{smallmatrix} \right\}$ in $A[S, X]$
R3	delete α from S, X	'control' in $A[S_0, S]$ or 'owner' in $A[S_0, X]$	delete α from $A[S, X]$
R4	$w \leftarrow$ read S, X	'control' in $A[S_0, S]$ or 'owner' in $A[S_0, X]$	copy $A[S, X]$ into w
R5	create object X	None	add column for X to A ; store 'owner' in $A[S_0, X]$
R6	destroy object X	'owner' in $A[S_0, X]$	delete column for X from A
R7	create subject S	none	add row for S to A ; execute create object S ; store 'control' in $A[S, S]$
R8	destroy subject S	'owner' in $A[S_0, S]$	delete row for S from A ; execute destroy object S

Example: UNIX File Access Control

All types of UNIX files are administered by the operating system by means of inodes. An inode (index node) is a control structure that contains the key information needed by the operating system for a particular file. Several file names may be associated with a single inode, but an active inode is associated with exactly one file, and each file is controlled by exactly one inode. The attributes of the file as well as its permissions and other control information are stored in the inode. On the disk, there is an inode table, or inode list, that contains the inodes of all the files in the file system. When a file is opened, its inode is brought into main memory and stored in a memory-resident inode table.

Directories are structured in a hierarchical tree. Each directory can contain files and/or other directories. A directory that is inside another directory is referred to as a

subdirectory. A directory is simply a file that contains a list of file names plus pointers to associated inodes. Thus, associated with each directory is its own inode.

Traditional UNIX File Access Control

Most UNIX systems depend on, or at least are based on, the file access control scheme introduced with the early versions of UNIX. Each UNIX user is assigned a unique user identification number (user ID). A user is also a member of a primary group, and possibly a number of other groups, each identified by a group ID. When a file is created, it is designated as owned by a particular user and marked with that user's ID. It also belongs to a specific group, which initially is either its creator's primary group, or the group of its parent directory if that directory has SetGID permission set. Associated with each file is a set of 12 protection bits. The owner ID, group ID, and protection bits are part of the file's inode.

Nine of the protection bits specify read, write, and execute permission for the owner of the file, other members of the group to which this file belongs, and all other users. These form a hierarchy of owner, group, and all others, with the highest relevant set of permissions being used. An example in which the file owner has read and write access; all other members of the file's group have read access; and users outside the group have no access rights to the file. When applied to a directory, the read and write bits grant the right to list and to create/rename/delete files in the directory. The execute bit grants the right to descend into the directory or search it for a filename.

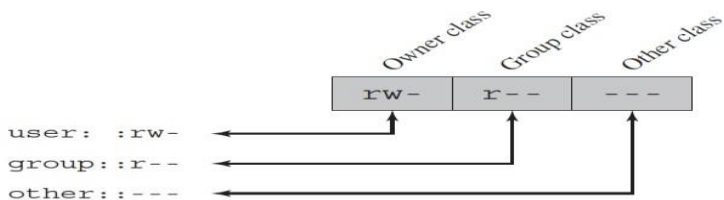
The remaining three bits define special additional behavior for files or directories. Two of these are the "set user ID" (SetUID) and "set group ID" (SetGID) permissions. If these are set on an executable file, the operating system functions as follows. When a user (with execute privileges for this file) executes the file, the system temporarily allocates the rights of the user's ID of the file creator, or the file's group, respectively, to those of the user executing the file. These are known as the "effective user ID" and "effective group ID" and are used in addition to the "real user ID" and "real group ID" of the executing

user when making access control Decisions for this program.

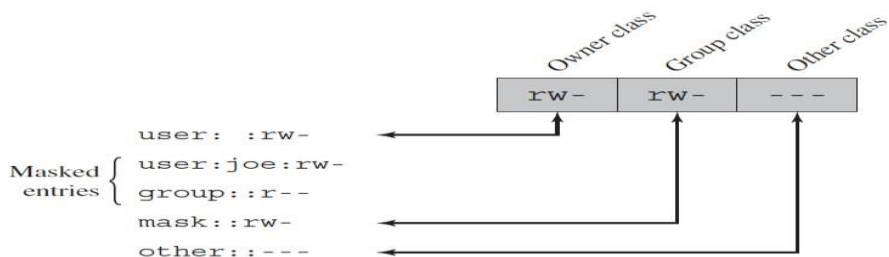
This change is only effective while the program is being executed. This feature enables the creation and use of privileged programs that may use files normally inaccessible to other users. It enables users to access certain files in a controlled fashion. Alternatively, when applied to a directory, the SetGID permission indicates that newly created files will inherit the group of this directory. The SetUID permission is ignored.

The final permission bit is the “sticky” bit. When set on a file, this originally indicated that the system should retain the file contents in memory following execution. This is no longer used. When applied to a directory, though, it specifies that only the owner of any file in the directory can rename, move, or delete that file. This is useful for managing files in shared temporary directories.

One particular user ID is designated as “superuser.” The superuser is exempt from the usual file access control constraints and has systemwide access. Any program that is owned by, and SetUID to, the “superuser” potentially grants unrestricted access to the system to any user executing that program. Hence great care is needed when writing such programs.



(a) Traditional UNIX approach (minimal access control list)



(b) Extended access control list

UNIX File Access Control

This access scheme is adequate when file access requirements align with users and a modest number of groups of users. For example, suppose a user wants to give read access for file X to users A and B and read access for file Y to users B and C. We would need at least two user groups, and user B would need to belong to both groups in order to access the two files. However, if there are a large number of different groupings of users requiring a range of access rights to different files, then a very large number of groups may be needed to provide this. This rapidly becomes unwieldy and difficult to manage, even if possible at all. One way to overcome this problem is to use access control lists, which are provided in most modern UNIX systems.

A final point to note is that the traditional UNIX file access control scheme implements a simple protection domain structure. A domain is associated with the user, and switching the domain corresponds to changing the user ID temporarily.

Access Control Lists in UNIX

Many modern UNIX and UNIX-based operating systems support access control lists, including FreeBSD, OpenBSD, Linux, and Solaris. In this section, we describe FreeBSD, but other implementations have essentially the same features and interface.

The feature is referred to as extended access control list, while the traditional UNIX approach is referred to as minimal access control list.

FreeBSD allows the administrator to assign a list of UNIX user IDs and groups to a file by using the `setfacl` command. Any number of users and groups can be associated with a file, each with three protection bits (read, write, execute), offering a flexible mechanism for assigning access rights. A file need not have an ACL but may be protected solely by the traditional UNIX file access mechanism.

FreeBSD files include an additional protection bit that indicates whether the file has an extended ACL. FreeBSD and most UNIX implementations that support extended ACLs use the following strategy

1. The owner class and other class entries in the 9-bit permission field have the

same meaning as in the minimal ACL case.

2. The group class entry specifies the permissions for the owner group for this file. These permissions represent the maximum permissions that can be assigned to named users or named groups, other than the owning user. In this latter role, the group class entry functions as a mask.

3. Additional named users and named groups may be associated with the file, each with a 3-bit permission field. The permissions listed for a named user or named group are compared to the mask field. Any permission for the named user or named group that is not present in the mask field is disallowed.

When a process requests access to a file system object, two steps are performed. Step 1 selects the ACL entry that most closely matches the requesting process. The ACL entries are looked at in the following order: owner, named users, (owning or named) groups, others. Only a single entry determines access. Step 2 checks if the matching entry contains sufficient permissions. A process can be a member in more than one group; so more than one group entry can match. If any of these matching group entries contain the requested permissions, one that contains the requested permissions is picked (the result is the same no matter which entry is picked). If none of the matching group entries contains the requested permissions, access will be denied no matter which entry is picked.

3.7 ROLE-BASED ACCESS CONTROL

Traditional DAC systems define the access rights of individual users and groups of users. In contrast, RBAC is based on the roles that users assume in a system rather than the user's identity. Typically, RBAC models define a role as a job function within an organization. RBAC systems assign access rights to roles instead of individual users. In turn, users are assigned to different roles, either statically or dynamically, according to their responsibilities.

RBAC now enjoys widespread commercial use and remains an area of active research. The National Institute of Standards and Technology (NIST) has issued a standard,

Security Requirements for Cryptographic Modules (FIPS PUB 140-3, September 2009), that requires support for access control and administration through roles.

The relationship of users to roles is many to many, as is the relationship of roles to resources, or system objects (Figure 3.7). The set of users changes, in some environments frequently, and the assignment of a user to one or more roles may also be dynamic. The set of roles in the system in most environments is relatively static, with only occasional additions or deletions. Each role will have specific access rights to one or more resources. The set of resources and the specific access rights associated with a particular role are also likely to change infrequently.

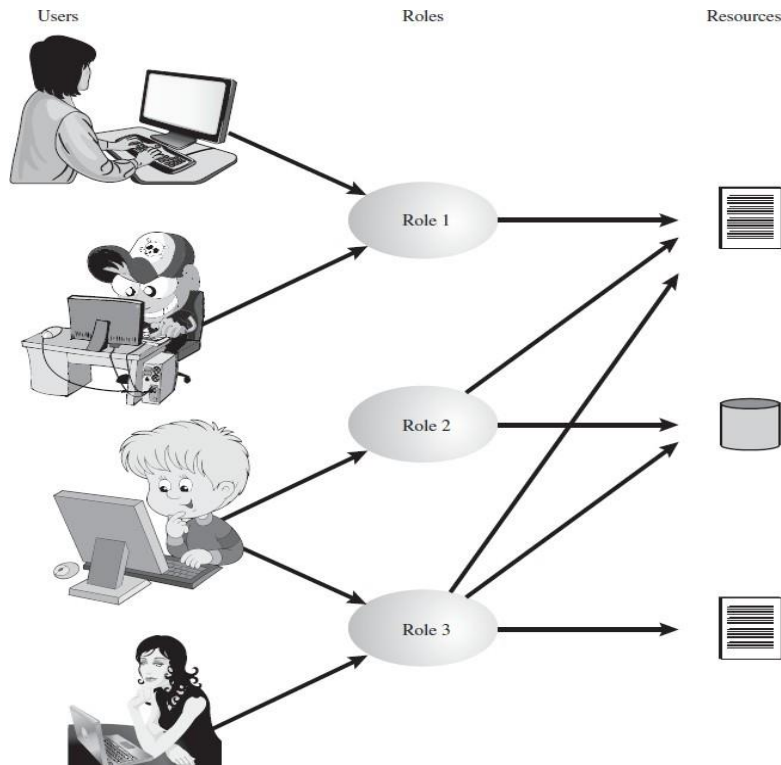


Fig 3.7 Users, Roles, and Resources

We can use the access matrix representation to depict the key elements of an RBAC system in simple terms, as shown in Figure 3.8. The upper matrix relates individual users to roles. Typically there are many more users than roles. Each matrix entry is either blank

or marked, the latter indicating that this user is assigned to this role. Note that a single user may be assigned multiple roles (more than one mark in a row) and that multiple users may be assigned to a single role (more than one mark in a column). The lower matrix has the same structure as the DAC access control matrix, with roles as subjects. Typically, there are few roles and many objects, or resources.

In this matrix the entries are the specific access rights enjoyed by the roles. Note that a role can be treated as an object, allowing the definition of role hierarchies.

ROLES

	R_1	R_2	$\dots$	R_n
U_1	$\times$			
U_2	$\times$			
U_3		$\times$		$\times$
U_4				$\times$
U_5				$\times$
U_6				$\times$
$\vdots$				
U_m	$\times$			

OBJECTS

	R_1	R_2	R_n	F_1	F_2	P_1	P_2	D_1	D_2
R_1	control	owner	owner control	read *	read owner	wakeup	wakeup	seek	owner
R_2		control		write *	execute			owner	seek *
$\vdots$									
R_n			control		write	stop			

Fig 3.8 Access Control Matrix Representation of RBAC

RBAC Reference Models

A variety of functions and services can be included under the general RBAC approach. To clarify the various aspects of RBAC, it is useful to define a set of abstract models of RBAC functionality.

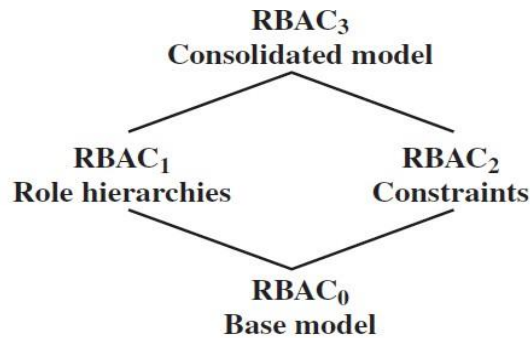
[SAND96] defines a family of reference models that has served as the basis for ongoing standardization efforts

Base Model—RBAC0 Figure 3.9b, without the role hierarchy and constraints, contains the four types of entities in an RBAC0 system:

- **User:** An individual that has access to this computer system. Each individual has an associated user ID.
- **Role:** A named job function within the organization that controls this computer system. Typically, associated with each role is a description of the authority and responsibility conferred on this role, and on any user who assumes this role.
- **Permission:** An approval of a particular mode of access to one or more objects. Equivalent terms are access right, privilege, and authorization.
- **Session:** A mapping between a user and an activated subset of the set of roles to which the user is assigned.

The arrowed lines in Figure 3.9b indicate relationships, or mappings, with a single arrowhead indicating one and a double arrowhead indicating many. Thus, there is a many-to-many relationship between users and roles: One user may have multiple roles, and multiple users may be assigned to a single role. Similarly, there is a many-to-many relationship between roles and permissions. A session is used to define a temporary one-

to-many relationship between a user and one or more of the roles to which the user has been assigned. The user establishes a session with only the roles needed for a particular task; this is an example of the concept of least privilege.



(a) Relationship among RBAC models

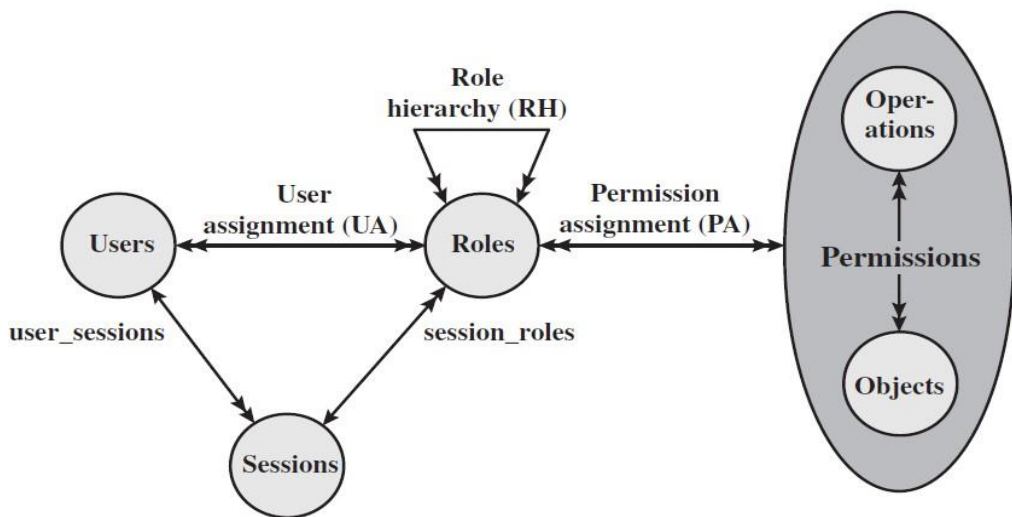


Fig 3.9 A Family of Role-Based Access Control Models RBAC0 is

the minimum requirement for an RBAC system. RBAC1 adds role hierarchies and RBAC2 adds constraints. RBAC3 includes RBAC1 and RBAC2

The many-to-many relationships between users and roles and between roles and permissions provide a flexibility and granularity of assignment not found in conventional DAC schemes. Without this flexibility and granularity, there is a greater risk that a user

may be granted more access to resources than is needed because of the limited control over the types of access that can be allowed. The NIST RBAC document gives the following examples: Users may need to list directories and modify existing files without creating new files, or they may need to append records to a file without modifying existing records.

Table: Scope RBAC Models

Models	Hierarchies	Constraints
RBAC ₀	No	No
RBAC ₁	Yes	No
RBAC ₂	No	Yes
RBAC ₃	Yes	Yes

Role Hierarchies—RBAC1 Role hierarchies provide a means of reflecting the hierarchical structure of roles in an organization. Typically, job functions with greater responsibility have greater authority to access resources. A subordinate job function may have a subset of the access rights of the superior job function. Role hierarchies make use of the concept of inheritance to enable one role to implicitly include access rights associated with a subordinate role.

Figure 3.10 is an example of a diagram of a role hierarchy. By convention, subordinate roles are lower in the diagram. A line between two roles implies that the upper role includes all of the access rights of the lower role, as well as other access rights not available to the lower role. One role can inherit access rights from multiple subordinate roles. For example, in Figure 3.10, the Project Lead role includes all of the access rights of the Production Engineer role and of the Quality Engineer role. More than one role can inherit from the same subordinate role. For example, both the Production Engineer role and the Quality Engineer role include all of the access rights of the Engineer role. Additional access rights are also assigned to the Production Engineer Role and a different set of additional access rights are assigned to the Quality

Engineer role. Thus, these two roles have overlapping access rights, namely the access rights they share with the Engineer role.

Constraints—RBAC2 Constraints provide a means of adapting RBAC to the specifics of administrative and security policies in an organization. A constraint is a defined relationship among roles or a condition related to roles. [SAND96] lists the following types of constraints: mutually exclusive roles, cardinality, and prerequisite roles.

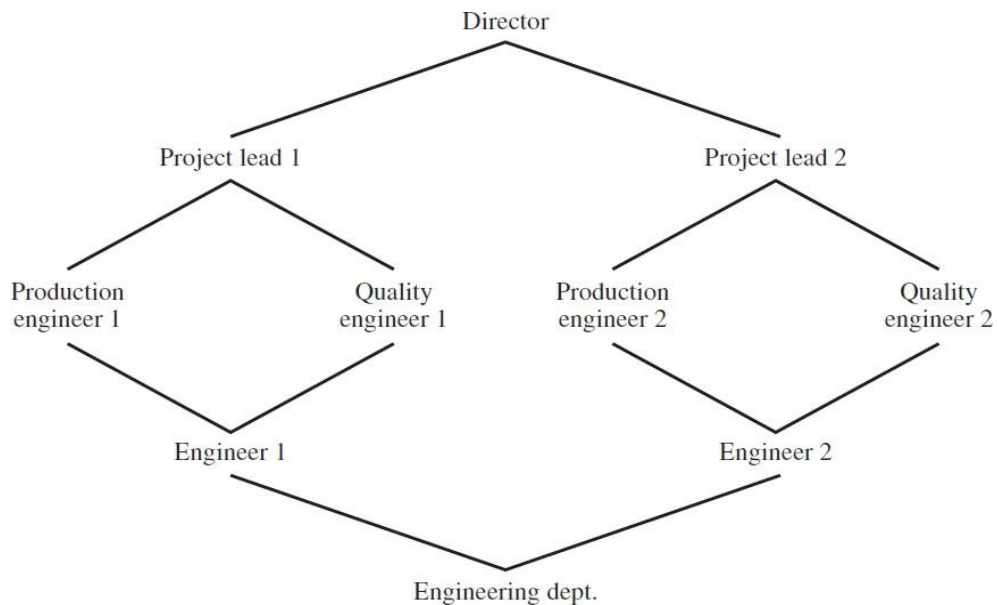


Fig 3.10 Example of Role Hierarchy

Mutually exclusive roles are roles such that a user can be assigned to only one role in the set. This limitation could be a static one, or it could be dynamic, in the sense that a user could be assigned only one of the roles in the set for a session. The mutually exclusive constraint supports a separation of duties and capabilities within an organization. This separation can be reinforced or enhanced by use of mutually exclusive permission assignments. With this additional constraint, a mutually exclusive set of roles has the following properties:

1. A user can only be assigned to one role in the set (either during a session or statically).

2. Any permission (access right) can be granted to only one role in the set.

Thus, the set of mutually exclusive roles have non-overlapping permissions. If two users are assigned to different roles in the set, then the users have non-overlapping permissions while assuming those roles. The purpose of mutually exclusive roles is to increase the difficulty of collusion among individuals of different skills or divergent job functions to thwart security policies.

Cardinality refers to setting a maximum number with respect to roles. One such constraint is to set a maximum number of users that can be assigned to a given role. For example, a project leader role or a department head role might be limited to a single user. The system could also impose a constraint on the number of roles that a user is assigned to, or the number of roles a user can activate for a single session. Another form of constraint is to set a maximum number of roles that can be granted a particular permission; this might be a desirable risk mitigation technique for a sensitive or powerful permission.

3.8 ATTRIBUTE-BASED ACCESS CONTROL

A relatively recent development in access control technology is the attribute-based access

control (ABAC) model. An ABAC model can define authorizations that express conditions on properties of both the resource and the subject. For example, consider a configuration in which each resource has an attribute that identifies the subject that created the resource. Then, a single access rule can specify the ownership privilege for all the creators of every resource. The strength of the ABAC approach is its flexibility and expressive power. [PLAT13] points out that the main obstacle to its adoption in real systems has been concern about the performance impact of evaluating predicates on both resource and user properties for each access.

There are three key elements to an ABAC model:

Attributes

Attributes are characteristics that define specific aspects of the subject, object, environment conditions, and/or requested operations that are predefined and preassigned by an authority. Attributes contain information that indicates the class of information given by the attribute, a name, and a value (e.g., Class=HospitalRecordsAccess, Name=PatientInformationAccess, Value=MFBusinessHoursOnly).

The following are the three types of attributes in the ABAC model:

- **Subject attributes:** A subject is an active entity (e.g., a user, an application, a process, or a device) that causes information to flow among objects or changes the system state. Each subject has associated attributes that define the identity and characteristics of the subject. Such attributes may include the subject's identifier, name, organization, job title, and so on. A subject's role can also be viewed as an attribute.
- **Object attributes:** An object, also referred to as a resource, is a passive (in the context of the given request) information system-related entity (e.g., devices, files, records, tables, processes, programs, networks, domains) containing or receiving information. As with subjects, objects have attributes that can be leveraged to make access control decisions. A Microsoft Word document, for example, may have attributes such as title, subject, date, and author. Object attributes can often be extracted from the metadata of the object. In particular, a variety of Web service metadata attributes may be relevant for access control purposes, such as ownership, service taxonomy, or even Quality of Service (QoS)

attributes.

- **Environment attributes:** These attributes have so far been largely ignored in most access control policies. They describe the operational, technical, and even situational environment or context in which the information access occurs. For example, attributes, such as current date and time, the current virus/hacker activities, and the network's security level (e.g., Internet vs. intranet), are not associated with a particular subject nor a resource, but may nonetheless be relevant in applying an access control policy.

ABAC Logical Architecture

Figure illustrates in a logical architecture the essential components of an ABAC system. An access by a subject to an object proceeds according to the following steps:

1. A subject requests access to an object. This request is routed to an access control mechanism.
2. The access control mechanism is governed by a set of rules (2a) that are defined by a preconfigured access control policy. Based on these rules, the access control mechanism assesses the attributes of the subject (2b), object (2c), and current environmental conditions (2d) to determine authorization.
3. The access control mechanism grants the subject access to the object if access is authorized and denies access if it is not authorized.

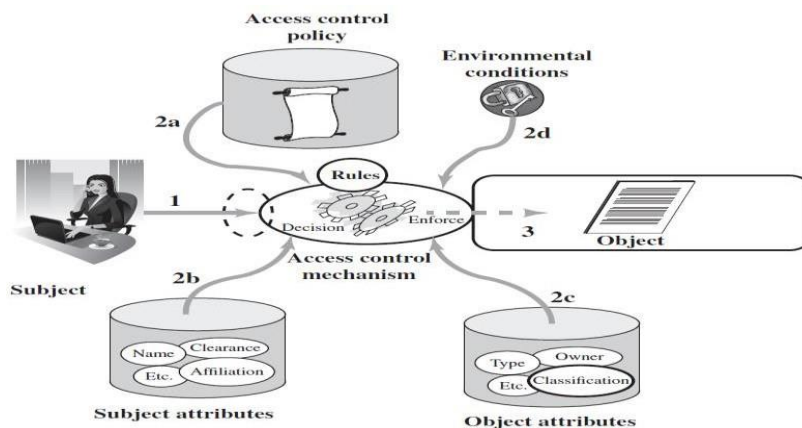
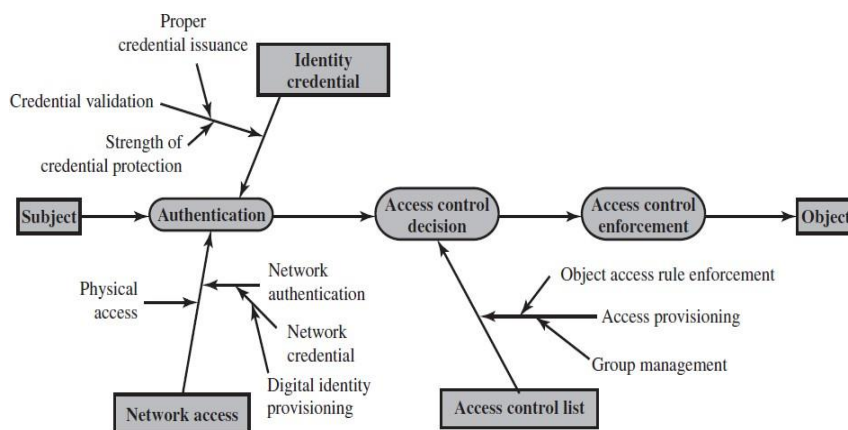


Fig Simple ABAC Scenario

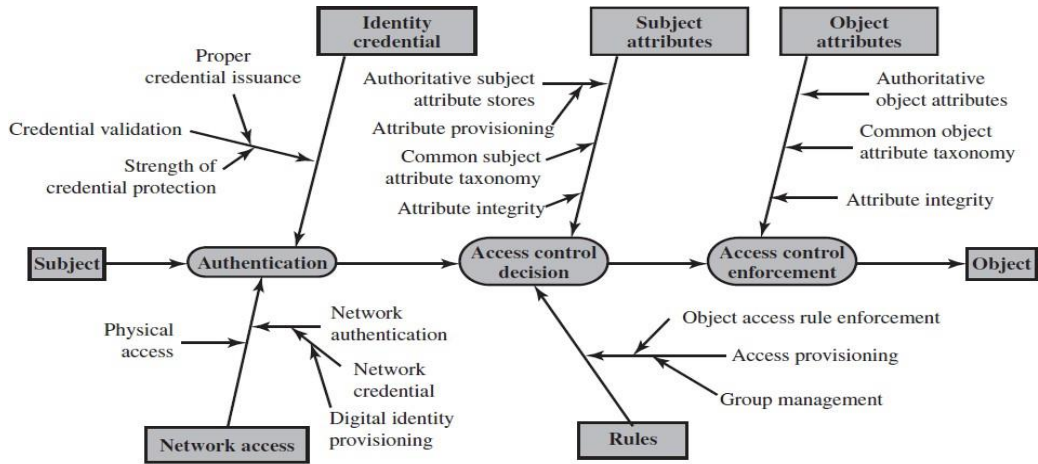
It is clear from the logical architecture that there are four independent sources of information used for the access control decision. The system designer can decide which attributes are important for access control with respect to subjects, objects, and environmental conditions. The system designer or other authority can then define access control policies, in the form of rules, for any desired combination of attributes of subject, object, and environmental conditions. It should be evident that this approach is very powerful and flexible. However, the cost, both in terms of the complexity of the design and implementation and in terms of the performance impact, is likely to exceed that of other access control approaches. This is a tradeoff that the system authority must make. Figure , taken from NIST SP 800-162 [HU13], provides a useful way of grasping the scope of an ABAC model compared to a DAC model using access control lists (ACLs).

ABAC Policies

A policy is a set of rules and relationships that govern allowable behavior within an organization, based on the privileges of subjects and how resources or objects are to be protected under which environment conditions. In turn, privileges represent the authorized behavior of a subject; they are defined by an authority and embodied in a policy. Other terms that are commonly used instead of privileges are rights, authorizations, and entitlements. Policy is typically written from the perspective of the object that needs protecting and the privileges available to subjects.



(a) ACL Trust Chain



(b) ABAC Trust Chain

ACL and ABAC Trust Relationships

We now define an ABAC policy model, based on the model presented in [YUAN05]. The following conventions are used:

1. S, O, and E are subjects, objects, and environments, respectively;
2. SA_k ($1 \leq k \leq K$), OA_m ($1 \leq m \leq M$), and EA_n ($1 \leq n \leq N$) are the pre-defined Attributes for subjects, objects, and environments, respectively;
3. $ATTR(s)$, $ATTR(o)$, and $ATTR(e)$ are attribute assignment relations for subject s, object o, and environment e, respectively:

$$ATTR(s) \subseteq SA_1 \times SA_2 \times \dots \times SA_K$$

$$ATTR(r) \subseteq OA_1 \times OA_2 \times \dots \times OA_M$$

$$ATTR(o) \subseteq EA_1 \times EA_2 \times \dots \times EA_N$$

We also use the function notation for the value assignment of individual attributes. For example:

`Role(s) = "Service Consumer"`

`ServiceOwner(o) = "XYZ, Inc."`

`CurrentDate(e) = "01-23-2005"`

4. In the most general form, a Policy Rule, which decides on whether a subject s can access an object o in a particular environment e , is a Boolean function of the attributes of s , o , and e :

Rule: $\text{can_access}(s, o, e) \leftarrow f(\text{ATTR}(s), \text{ATTR}(o), \text{ATTR}(e))$

Given all the attribute assignments of s , o , and e , if the function's evaluation is true, then the access to the resource is granted; otherwise the access is denied.

5. A policy rule base or policy store may consist of a number of policy rules, covering many subjects and objects within a security domain. The access control decision process in essence amounts to the evaluation of applicable policy rules in the policy store.

Now consider the example of an online entertainment store that streams movies to users for a flat monthly fee. We will use this example to contrast RBAC and ABAC approaches. The store must enforce the following access control policy based on the user's age and the movie's content rating:

Movie Rating	Users Allowed Access
R	Age 17 and older
PG-13	Age 13 and older
G	Everyone

In an RBAC model, every user would be assigned one of three roles: Adult, Juvenile, or Child, possibly during registration. There would be three permissions created: Can view R-rated movies, Can view PG-13-rated movies, and Can view G-rated movies. The Adult role gets assigned with all three permissions; the Juvenile role gets Can view PG-13-rated movies and Can view G-rated movies permissions, and the Child role gets the Can view G-rated movies permission only. Both the user-to-role assignment and the permission-to-role assignment are manual administrative tasks.

The ABAC approach to this application does not need to explicitly define roles. Instead, whether a user u can access or view a movie m (in a security environment e which is ignored here) would be resolved by evaluating a policy rule such as the following:

```

R1:can_access(u, m, e) ←
  (Age(u) ≥ 17 ∧ Rating(m) ∈ {R, PG-13, G}) ∨
  (Age(u) ≥ 13 ∧ Age(u) < 17 ∧ Rating(m) ∈ {PG-13, G}) ∨
  (Age(u) < 13 ∧ Rating(m) ∈ {G})

```

where Age and Rating are the subject attribute and the object attribute, respectively. The advantage of the ABAC model shown here is that it eliminates the definition and management of static roles, hence eliminating the need for the administrative tasks for user-to-role assignment and permission-to-role assignment.

The advantage of ABAC is more clearly seen when we impose finer-grained policies. For example, suppose movies are classified as either New Release or Old Release, based on release date compared to the current date, and users are classified as Premium User and Regular User, based on the fee they pay. We would like to enforce a policy that only premium users can view new movies. For the RBAC model, we would have to double the number of roles, to distinguish each user by age and fee, and we would have to double the number of separate permissions as well.

In general, if there are K subject attributes and M object attributes, and if for each attribute, $\text{Range}()$ denotes the range of possible values it can take, then the respective number of roles and permissions required for an RBAC model are:

$$\prod_{k=1}^K \text{Range}(SA_k) \quad \text{and} \quad \prod_{m=1}^M \text{Range}(SA_m)$$

Thus we can see that as the number of attributes increases to accommodate finer-grained policies, the number of roles and permissions grows exponentially. In contrast, the ABAC model deals with additional attributes in an efficient way. For this example, the policy R1 defined previously still applies. We need two new rules:

```

R2:can_access(u, m, e) ←
  (MembershipType(u) = Premium)
  (MembershipType(u) = Regular ∧ MovieType(m) = OldRelease)
R3:can_access(u, m, e) ← R3 ∧ R4

```

With the ABAC model, it is also easy to add environmental attributes. Suppose we wish to add a new policy rule that is expressed in words as follows: Regular users are allowed

to view new releases in promotional periods. This would be difficult to express in an RBAC model. In an ABAC model, we only need add a conjunctive (AND) rule that checks to see the environmental attribute today's date falls in a promotional period.

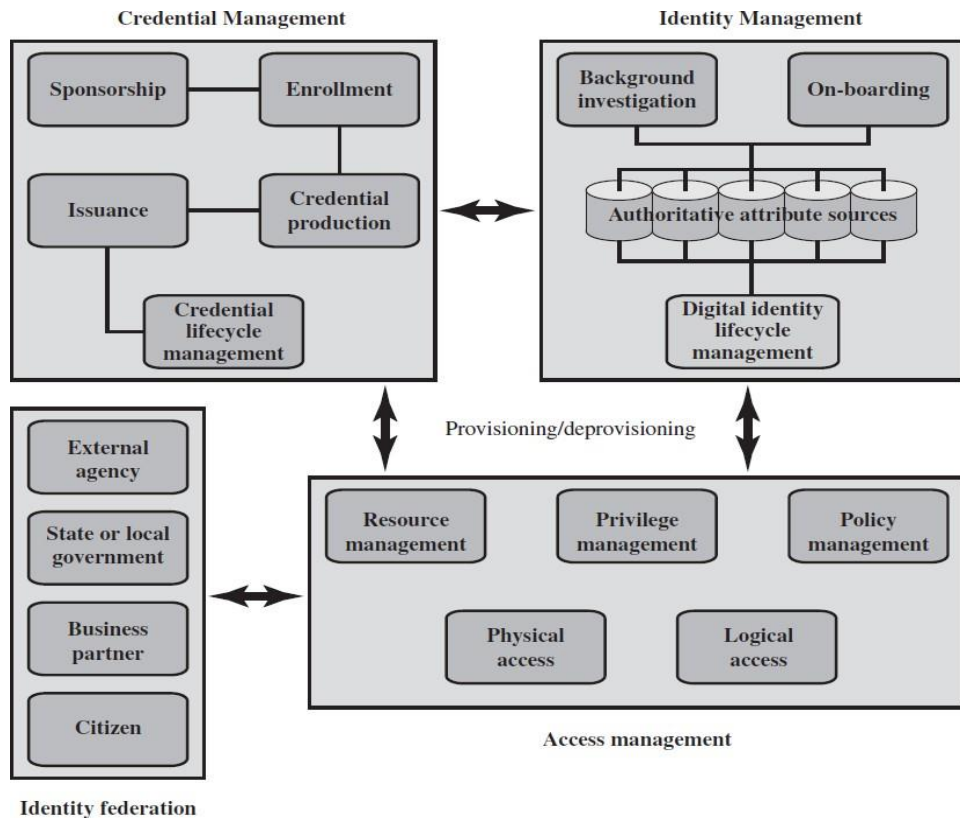
3.9 IDENTITY, CREDENTIAL, AND ACCESS MANAGEMENT

This section provides an overview of the concept of identity, credential, and access management (ICAM).

ICAM is a comprehensive approach to managing and implementing digital identities (and associated attributes), credentials, and access control. ICAM has been developed by the U.S. government, but is applicable not only to government agencies, but also may be deployed by enterprises looking for a unified approach to access control. ICAM is designed to

- Create trusted digital identity representations of individuals and what the ICAM documents refer to as nonperson entities (NPEs). The latter include processes, applications, and automated devices seeking access to a resource.
- Bind those identities to credentials that may serve as a proxy for the individual or NPE in access transactions. A credential is an object or data structure that authoritatively binds an identity (and optionally, additional attributes) to a token possessed and controlled by a subscriber.
- Use the credentials to provide authorized access to an agency's resources.

An overview of the logical components of an ICAM architecture. We examine each of the main components in the following subsections.



Identity, Credential, and Access Management (ICAM)

Identity Management

Identity management is concerned with assigning attributes to a digital identity and connecting that digital identity to an individual or NPE. The goal is to establish a trustworthy digital identity that is independent of a specific application or context. The traditional, and still most common approach, to access control for applications and programs is to create a digital representation of an identity for the specific use of the application or program. As a result, maintenance and protection of the identity itself is treated as secondary to the mission associated with the application. Further, there is considerable overlap in effort in establishing these application-specific identities.

Unlike accounts used to logon to networks, systems, or applications, enterprise identity records are not tied to job title, job duties, location, or whether access is needed to a

specific system. Those items may become attributes tied to an enterprise identity record, and may also become part of what uniquely identifies an individual in a specific application. Access control decisions will be based on the context and relevant attributes of a user—not solely their identity. The concept of an enterprise identity is that individuals will have a single digital representation of themselves that can be leveraged across departments and agencies for multiple purposes, including access control.

A final element of identity management is lifecycle management, which includes the following:

- Mechanisms, policies, and procedures for protecting personal identity information
- Controlling access to identity data
- Techniques for sharing authoritative identity data with applications that need it
- Revocation of an enterprise identity

Credential Management

As mentioned, a credential is an object or data structure that authoritatively binds an identity (and optionally, additional attributes) to a token possessed and controlled by a subscriber. Examples of credentials are smart cards, private/public cryptographic keys, and digital certificates. Credential management is the management of the life cycle of the credential. Credential management encompasses the following five logical components:

1. An authorized individual sponsors an individual or entity for a credential to establish the need for the credential. For example, a department supervisor sponsors a department employee.
2. The sponsored individual enrolls for the credential, a process which typically consists of identity proofing and the capture of biographic and biometric data. This step may also involve incorporating authoritative attribute data, maintained by the identity management component.
3. A credential is produced. Depending on the credential type, production may involve encryption, the use of a digital signature, the production of a smartcard, or other functions.
4. The credential is issued to the individual or NPE. 5. Finally, a credential must be

maintained over its life cycle, which might include revocation, reissuance/replacement, reenrollment, expiration, personal identification number (PIN) reset, suspension, or reinstatement.

Access Management

The access management component deals with the management and control of the ways entities are granted access to resources. It covers both logical and physical access, and may be internal to a system or an external element. The purpose of access management is to ensure that the proper identity verification is made when an individual attempts to access security sensitive buildings, computer systems, or data. The access control function makes use of credentials presented by those requesting access and the digital identity of the requestor. Three support elements are needed for an enterprise-wide access control facility:

- **Resource management:** This element is concerned with defining rules for a resource that requires access control. The rules would include credential requirements and what user attributes, resource attributes, and environmental conditions are required for access of a given resource for a given function.
- **Privilege management:** This element is concerned with establishing and maintaining the entitlement or privilege attributes that comprise an individual's access profile. These attributes represent features of an individual that can be used as the basis for determining access decisions to both physical and logical resources. Privileges are considered attributes that can be linked to a digital identity.
- **Policy management:** This element governs what is allowable and unallowable in an access transaction. That is, given the identity and attributes of the requestor, the attributes of the resource or object, and environmental conditions, a policy specifies what actions this user can perform on this object.

3.10 TRUST FRAMEWORKS

The interrelated concepts of trust, identity, and attributes have become core concerns of Internet businesses, network service providers, and large enterprises. These concerns can clearly be seen in the e-commerce setting. For efficiency, privacy, and legal simplicity,

parties to transactions generally apply the need-to-know principle: What do you need to know about someone in order to deal with them? The answer varies from case to case, and includes such attributes as professional registration or license number, organization and department, staff ID, security clearance, customer reference number, credit card number, unique health identifier, allergies, blood type, social security number, address, citizenship status, social networking handle, pseudonym, and so on. The attributes of an individual that must be known and verified to permit a transaction depend on context.

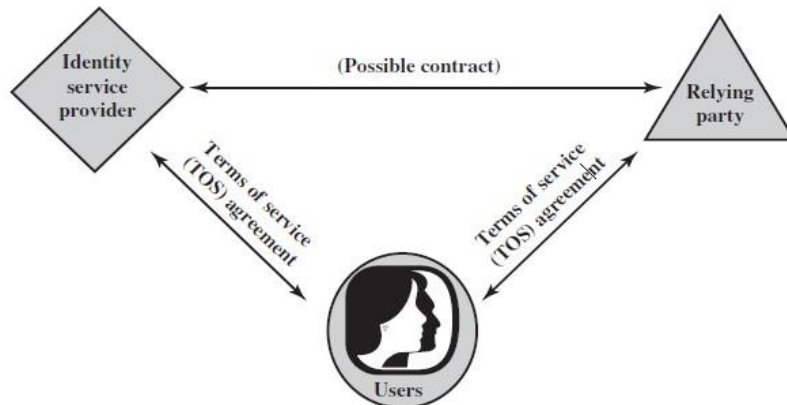
Traditional Identity Exchange Approach

Online or network transactions involving parties from different organizations, or between an organization and an individual user such as an online customer, generally require the sharing of identity information. This information may include a host of associated attributes in addition to a simple name or numerical identifier. Both the party disclosing the information and the party receiving the information need to have a level of trust about security and privacy issues related to that information.

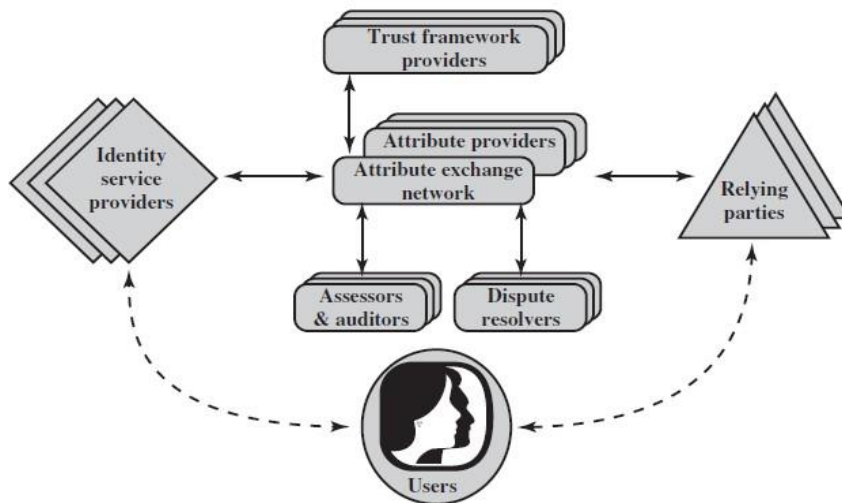
Figure a shows the traditional technique for the exchange of identity information. This involves users developing arrangements with an identity service provider to procure digital identity and credentials, and arrangements with parties that provide end-user services and applications and that are willing to rely on the identity and credential information generated by the identity service provider.

The arrangement of Figure (a) must meet a number of requirements. The relying party requires that the user has been authenticated to some degree of assurance, that the attributes imputed to the user by the identity service provider are accurate, and that the identity service provider is authoritative for those attributes. The identity service provider requires assurance that it has accurate information about the user and that, if it shares information, the relying party will use it in accordance with contractual terms and conditions and the law. The user requires assurance that the identity service provider and

relying party can be entrusted with sensitive information and that they will abide by the user's preferences and respect the user's privacy. Most importantly, all the parties want to know if the practices described by the other parties are actually those implemented by the parties, and how reliable those parties are.



(a) Traditional triangle of parties involved in an exchange of identity information



(b) Identity attribute exchange elements

Fig Identity Information Exchange Approaches

Open Identity Trust Framework

Without some universal standard and framework, the arrangement of Figure 4.24a must be replicated in multiple contexts. A far preferable approach is to develop an open,

standardized approach to trustworthy identity and attribute exchange. In the remainder of this section, we examine such an approach that is gaining increasing acceptance.

Unfortunately, this topic is burdened with numerous acronyms, so it is best to begin with a definition of the most important of these:

- **OpenID:** This is an open standard that allows users to be authenticated by certain cooperating sites (known as Relying Parties) using a third party service, eliminating the need for Webmasters to provide their own ad hoc systems and allowing users to consolidate their digital identities. Users may create accounts with their preferred OpenID identity providers, and then use those accounts as the basis for signing on to any website which accepts OpenID authentication.
- **OIDF:** The OpenID Foundation is an international nonprofit organization of individuals and companies committed to enabling, promoting, and protecting OpenID technologies. OIDF assists the community by providing needed infrastructure and help in promoting and supporting expanded adoption of OpenID.
- **ICF:** The Information Card Foundation is a nonprofit community of companies and individuals working together to evolve the Information Card ecosystem. Information Cards are personal digital identities that people can use online, and the key component of identity metasecosystems. Visually, each Information Card has a card-shaped picture and a card name associated with it that enable people to organize their digital identities and to easily select one they want to use for any given interaction.
- **OITF:** The Open Identity Trust Framework is a standardized, open specification of a trust framework for identity and attribute exchange, developed jointly by OIDF and ICF.
- **OIX:** The Open Identity Exchange Corporation is an independent, neutral, international provider of certification trust frameworks conforming to the Open Identity Trust Frameworks model.
- **AXN:** An Attribute Exchange Network (AXN) is an online Internet-scale gateway for identity service providers and relying parties to efficiently access user asserted,

permissioned, and verified online identity attributes in high volumes at affordable costs. System managers need to be able to trust that the attributes associated with a subject or an object are authoritative and are exchanged securely. One approach to providing that trust within an organization is the ICAM model, specifically the ICAM components (Figure). Combined with an identity federation functionality that is shared with other organizations, attributes can be exchanged in a trustworthy fashion, supporting secure access control.

In digital identity systems, a trust framework functions as a certification program. It enables a party who accepts a digital identity credential (called the relying party) to trust the identity, security, and privacy policies of the party who issues the credential (called the identity service provider) and vice versa. More formally, OIX defines a trust framework as a set of verifiable commitments from each of the various parties in a transaction to their counter parties. These commitments include (1) controls (including regulatory and contractual obligations) to help ensure commitments are delivered and (2) remedies for failure to meet such commitments. A trust framework is developed by a community whose members have similar goals and perspectives. It defines the rights and responsibilities of that community's participants; specifies the policies and standards specific to the community; and defines the community-specific processes and procedures that provide assurance. Different trust frameworks can exist, and sets of participants can tailor trust frameworks to meet their particular needs.

Figure (b) shows the elements involved in the OITF. Within any given organization or agency, the following roles are part of the overall framework:

- **Relying parties (RPs):** Also called service providers, these are entities delivering services to specific users. RPs must have confidence in the identities and/or attributes of their intended users, and must rely upon the various credentials presented to evince those attributes and identities.
- **Subjects:** These are users of an RP's services, including customers, employees, trading

partners, and subscribers.

- **Attribute providers (APs):** APs are entities acknowledged by the community of interest as being able to verify given attributes as presented by subjects and which are equipped through the AXN to create conformant attribute credentials according to the rules and agreements of the AXN. Some APs will be sources of authority for certain information; more commonly APs will be brokers of derived attributes.

- **Identity providers (IDPs):** These are entities able to authenticate user credentials and to vouch for the names (or pseudonyms or handles) of subjects, and which are equipped through the AXN or some other compatible Identity and Access Management (IDAM) system to create digital identities that may be used to index user attributes.

There are also the following important support elements as part on an AXN:

- **Assessors:** Assessors evaluate identity service providers and RPs and certify that they are capable of following the OITF provider's blueprint.

- **Auditors:** These entities may be called on to check that parties' practices have been in line with what was agreed for the OITF.

- **Dispute resolvers:** These entities provide arbitration and dispute resolution under OIX guidelines.

- **Trust framework providers:** A trust framework provider is an organization that translates the requirements of policymakers into an own blueprint for a trust framework that it then proceeds to build, doing so in a way that is consistent with the minimum requirements set out in the OITF specification. In almost all cases, there will be a reasonably obvious candidate organization to take on this role, for each industry sector or large organization that decides it is appropriate to interoperate with an AXN.

The solid arrowed lines in Figure 4.24b indicate agreements with the trust framework provider for implementing technical, operations, and legal requirements. The dashed arrowed lines indicate other agreements potentially affected by these requirements. In general terms, the model illustrated in Figure 4.24b would operate in the following way.

Responsible persons within participating organizations determine the technical, operational, and legal requirements for exchanges of identity information that fall under their authority. They then select OITF providers to implement these requirements. These OITF providers translate the requirements into a blueprint for a trust framework that may include additional conditions of the OITF provider. The OITF provider vets identity service providers and RPs and contracts with them to follow its trust framework requirements when conducting exchanges of identity information. The contracts carry provisions relating to dispute resolvers and auditors for contract interpretation and enforcement.

UNIT IV

CYBER SECURITY

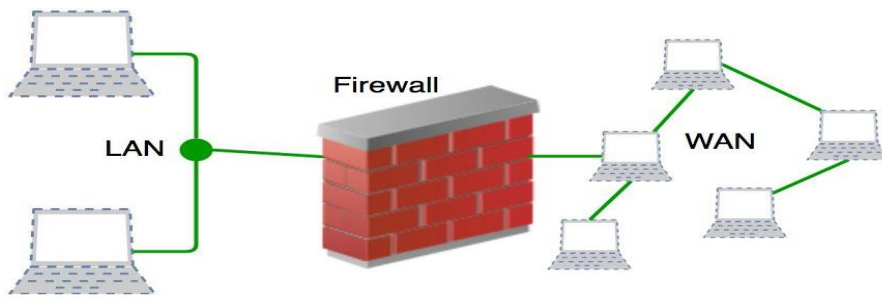
System Security: Firewall, Viruses, Worms, Ransomware, Keylogger, Greyware, IDS, DDoS Network Security: SSL - TLS - HTTPS -IP Security; OS Security: Introduction to Operating System Security - System Security Planning - Operating Systems Hardening - Application Security - Security Maintenance - Linux/Unix Security - Windows Security - Virtualization Security; Wireless Security: Risks and Threats of Wireless - Wireless LAN Security - Wireless Security Policy - Wireless Security Architectures - Wireless security Tools.

System Security: Firewall:

A firewall is a network security device, either hardware or software-based, which monitors all incoming and outgoing traffic and based on a defined set of security rules accepts, rejects, or drops that specific traffic.

- **Accept:** *allow the traffic*
- **Reject:** *block the traffic but reply with an “unreachable error”*
- **Drop :** *block the traffic with no reply*

A firewall is a type of network security device that filters incoming and outgoing network traffic with security policies that have previously been set up inside an organization. A firewall is essentially the wall that separates a private internal network from the open Internet at its very basic level.



Working of Firewall

Firewall match the network traffic against the rule set defined in its table. Once the rule is matched, associate action is applied to the network traffic. For example, Rules are defined as any employee from Human Resources department cannot access the data from code server and at the same time another rule is defined like system administrator can access the data from both Human Resource and technical department. Rules can be defined on the firewall based on the necessity and security policies of the organization. From the perspective of a server, network traffic can be either outgoing or incoming. Firewall maintains a distinct set of rules for both the cases. Mostly the outgoing traffic, originated from the server itself, allowed to pass. Still, setting a rule on outgoing traffic is always better in order to achieve more security and prevent unwanted communication. Incoming traffic is treated differently. Most traffic which reaches on the firewall is one of these three major Transport Layer protocols- TCP, UDP or ICMP. All these types have a source address and destination address. Also, TCP and UDP have port numbers. ICMP uses *type code* instead of port number which identifies purpose of that packet.

Types of Firewall

Firewalls can be categorized based on their generation.

1. Packet Filtering Firewall

Packet filtering firewall is used to control network access by monitoring outgoing and incoming packets and allowing them to pass or stop based on source and destination IP address, protocols, and ports. It analyses traffic at the transport protocol layer (but mainly uses first 3 layers). Packet firewalls treat each packet in isolation. They have no ability to tell whether a packet is part of an existing stream of traffic. Only It can allow or deny the packets based on unique packet headers. Packet filtering firewall maintains a filtering table that decides whether the packet will be forwarded or discarded. From the given filtering table, the packets will be filtered according to the following rules:

	Source IP	Dest. IP	Source Port	Dest. Port	Action
1	192.168.21.0	--	--	--	deny
2	--	--	--	23	deny
3	--	192.168.21.3	--	--	deny
4	--	192.168.21.0	--	>1023	Allow

Sample Packet Filter Firewall Rule

- Incoming packets from network 192.168.21.0 are blocked.
- Incoming packets destined for the internal TELNET server (port 23) are blocked.
- Incoming packets destined for host 192.168.21.3 are blocked.
- All well-known services to the network 192.168.21.0 are allowed.

2. Stateful Inspection Firewall

Stateful firewalls (performs Stateful Packet Inspection) are able to determine the connection state of packet, unlike Packet filtering firewall, which makes it more efficient. It keeps track of the state of networks connection travelling across it, such as TCP streams. So the filtering decisions would not only be based on defined rules, but also on packet's history in the state table.

3. Software Firewall

A software firewall is any firewall that is set up locally or on a cloud server. When it comes to controlling the inflow and outflow of data packets and limiting the number of networks that can be linked to a single device, they may be the most advantageous. But the problem with software firewall is they are time-consuming.

4. Hardware Firewall

They also go by the name "firewalls based on physical appliances." It guarantees that the malicious data is halted before it reaches the network endpoint that is in danger.

5. Application Layer Firewall

Application layer firewall can inspect and filter the packets on any OSI layer, up to the application layer. It has the ability to block specific content, also recognize when certain application and protocols (like HTTP, FTP) are being misused. In other words, Application layer firewalls are hosts that run proxy servers. A proxy firewall prevents the direct connection between either side of the firewall, each packet has to pass through the proxy.

6. Next Generation Firewalls (NGFW)

NGFW consists of Deep Packet Inspection, Application Inspection, SSL/SSH inspection and many functionalities to protect the network from these modern threats.

7. Proxy Service Firewall

This kind of firewall filters communications at the application layer, and protects the network. A proxy firewall acts as a gateway between two networks for a particular application.

8. Circuit Level Gateway Firewall

This works as the Sessions layer of the OSI Model's . This allows for the simultaneous setup of two Transmission Control Protocol (TCP) connections. It can effortlessly allow data packets to flow without using quite a lot of computing power. These firewalls are ineffective because they do not inspect data packets; if malware is found in a data packet, they will permit it to pass provided that TCP connections are established properly.

Functions of Firewall

- Every piece of data that enters or leaves a computer network must go via the firewall.
- If the data packets are safely routed via the firewall, all of the important data remains intact.
- A firewall logs each data packet that passes through it, enabling the user to keep track of all network activities.
- Since the data is stored safely inside the data packets, it cannot be altered.
- Every attempt for access to our operating system is examined by our firewall, which also blocks traffic from unidentified or undesired sources.

Advantages of using Firewall

- Protection from unauthorized access:
- Prevention of malware and other threats:
- Control of network access:
- Monitoring of network activity:
- Regulation compliance:
- Network segmentation:

Disadvantages of using Firewall

- Complexity:
- Limited Visibility:

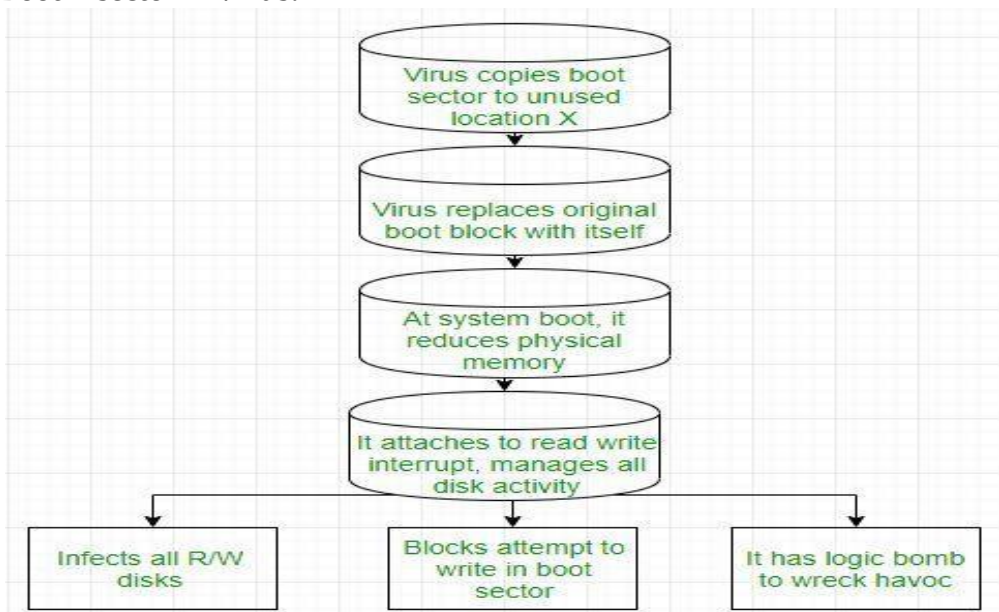
- False sense of security:
- Limited adaptability:
- Performance impact:
- Limited scalability:
- Limited VPN support:
- Cost:

Viruses:

A virus is a fragment of code embedded in a legitimate program. Viruses are self-replicating and are designed to infect other programs. They can wreak havoc in a system by modifying or destroying files causing system crashes and program malfunctions. On reaching the target machine a virus dropper(usually a trojan horse) inserts the virus into the system.

Various types of viruses:

- **File Virus:**
- **Boot sector Virus:**



- Macro Virus:
- Source code Virus:
- Polymorphic Virus:
- Encrypted Virus:

- Stealth Virus:
- Tunneling Virus:
- Multipartite Virus:
- Armored Virus:

- **Browser Hijacker:**

As the name suggests this virus is coded to target the user's browser and can alter the browser settings. It is also called the browser redirect virus because it redirects your browser to other malicious sites that can harm your computer system.

- **Memory Resident Virus:**

Resident viruses installation store for your RAM and meddle together along with your device operations. They behave in a very secret and dishonest way that they can even connect themselves for the anti-virus software program files.

- **Direct Action Virus:**

The main perspective of this virus is to replicate and take action when it is executed. When a particular condition is met the virus will get into action and infect files in the directory that are specified in the AUTOEXEC.BAT file path.

- **Overwrite virus:**

This type of virus deletes the information contained in the file that it infects, rendering them partially or totally useless once they have been infected.

- **Directory Virus:**

This virus is also called File System Virus or Cluster Virus. It infects the directory of the computer by modifying the path that is indicating the location of a file.

- **Companion Virus:**

This kind of virus usually use the similar file name and create a different extension of it. For example, if there's a file "Hello.exe", the virus will create another file named "Hello.com" and will hide in the new file

- **FAT Virus:**

The **File Allocation Table** is the part of the disk used to store all information about the location of files, available space, unusable space etc.

This virus affects the FAT section and may damage crucial information.

Worms:

A worm is a harmful software (virus) that repeats itself as it moves from computer to computer, leaving copies of itself in each computer's memory. A worm finds a computer's vulnerability and spreads like an illness throughout its associated network, constantly looking for new holes. Worms, like viruses, are spread by email attachments from seemingly trustworthy senders. Worms then propagate through a user's email account and address book to contacts.

Because the worm or its programmer can use the infected system's computing capacity, they are frequently integrated into a botnet. Cyber thieves then employ these, for example, in DDoS assaults or crypto mining.

What are the types of computer worms?

Malicious computer worms come in a variety of forms –

Email worms

To spread, email worms create and send outbound messages to all addresses in a user's contact list. When the recipient opens the mail, it contains a malicious executable file that infects the new system.

Successful email worms typically use social engineering and phishing approaches to persuade users to open the linked file.

File-sharing worms

File-sharing worms are malicious programs that hide as media files.

Stuxnet, one of the most well-known computer worms of all time, comprises two parts: a worm that spreads malware via USB devices infected with the host file and malware that targets supervisory control and data acquisition systems. Industrial contexts, such as power utilities, water supply services, and sewage plants are frequently targeted by file-sharing worms.

Cryptoworms

Cryptoworms encrypt data on the victim's computer system. This worm can be used in ransomware attacks, in which the attackers contact the victim and seek payment in exchange for a key to decrypt their files.

Internet worms

Some computer worms are designed to attack prominent websites that have weak security. They can infect a computer viewing the website if they can infect the site.

Internet worms then propagate to other devices connected to the infected PC via internet and private network connections.

Worms that spread via instant messaging

Instant messaging worms, like email worms, are disguised as attachments or links, which the worm uses to spread throughout the infected user's contact list. The only difference is that it comes as an instant message on a chat site rather than an email.

If the worm hasn't had time to replicate itself on the machine, it may usually be stopped by resetting the user's chat service account password.

Virus vs. computer worm

Because they behave similarly, some people mistakenly believe that a computer worm and a computer virus are the same things. They might even call it a "worm computer virus" or "worm virus malware." The truth is that the two threats are similar yet distinct.

The major distinction between a virus and a worm is that viruses require human action to activate, whereas worms replicate only in the presence of a host system. In other words, unless you run a virus, your computer will not be harmed. A virus on a flash drive connected to your computer, for example, will not harm your system unless you activate it. A worm, as previously stated, does not require a host system or user input to spread.

How to prevent worms?

Malicious software can take many forms, including computer worms. Take these measures to help protect your computer against worms and other internet threats.

- Because software vulnerabilities are a common source of infection for computer worms, make sure your computer's operating system and programs are up to date. Because updates frequently include patches for security problems, install them as soon as they become available.
- Another popular method for hackers to transmit worms is through phishing (and other types of malware). When opening unwanted emails, be particularly cautious, especially those from unknown senders that include attachments or questionable URLs.
- Make sure you have a good internet security software solution to help you block these dangers. Anti-phishing technologies, as well as defenses against viruses, spyware, ransomware, and other online threats, should be included in a solid solution.

Ransomware:

Ransomware is a malicious software which infects many computers and displays messages demanding you pay a fee to get your system working again. The malware class is a criminal moneymaking scheme that is set up by an email message, instant messages or misleading links on the website. It locks a computer screen or encrypts predefined files with a password.

History of Ransomware

In 2005, the first case was reported in Russia. Scams have spread around the world, with new types still targeting victims. In September **2013**, CryptoLocker appeared and targeted versions of Windows. Ransomware successfully infected thousands of personal computers and business systems. The victims opened emails from **FedEx, UPS, DHS** and many companies implementing the customer support services.

Examples of Ransomware

Scareware is also a type of Ransomware. It uses deceit tactics or intimidation to pay victims. A message suddenly claims that your computer has various problems, and online payment is necessary to fix it!

Nevertheless, another type of Ransomware appears to be from a local law enforcement office by opening a page to a law enforcement agency and a computer user claiming he was caught doing illegal activities online.

The files are locked into **hard-to-crack, encrypted files**, making it difficult for users to pay to recover.

Attackers ask for **\$ 100 to \$ 200**. Other attacks are much wanted, especially if the attacker is aware that mortgage data can cause financial losses to the company to a large extent. The cybercriminals who set up these scams can earn significant sums.

There is no guarantee that computer users will fully recover the system. While some hackers instruct victims to pay through bitcoins, money packs, or other online methods, attackers may demand credit card data, adding to the level of other financial losses.

Types of Ransomware

There are 3 types of Ransomware, which are given below:

Scareware

Scareware is not as scary. It includes security software and technical support

scams. In Scareware, we receive a **pop-up** message claiming that the malware was discovered, and the only way to get rid of the virus is to pay online. If you do nothing, you continuously receive **pop-ups**, but the files are almost safe.

If you do not have the company's software on your computer, it will not monitor you for ransomware infection.

Screen locker

Upgrade the Terror Alert Orange for these people. When lock-screen Ransomware is found on our computer, it means that you are out of your PC. If they suspect you of theft, child pornography or other cybercrimes, they will go through the appropriate legal channels.

Encrypt Ransomware

It is dangerous stuff. These are the people who are snatching our files and encrypt them, demanding payment for decryption and redistribution. This type of Ransomware is so terrible because once cybercriminals pile up our files; no security software or system can restore or return it until you pay the ransom. And if you pay, there is no guarantee that cybercriminals will give those files back to you.

Keylogger:

Key loggers also known as keystroke loggers, may be defined as the recording of the key pressed on a system and saved it to a file, and the that file is accessed by the person using this malware. Key logger can be software or can be hardware. **Working:** Mainly key-loggers are used to steal password or confidential details such as bank information etc. First key-logger was invented in 1970's and was a hardware key logger and first software key-logger was developed in 1983. **1. Software key-loggers :** Software key-loggers are the computer programs which are developed to steal password from the victims computer. However key loggers are used in IT organizations to troubleshoot technical problems with computers and business networks. Also Microsoft windows 10 also has key-logger installed in it.

1. **JavaScript based key logger** – It is a malicious script which is installed into a web page, and listens for key to press such as oneKeyUp(). These scripts can be sent by various methods, like sharing through social media, sending as a mail file, or RAT file.
2. **Form Based Key loggers** – These are key-loggers which activates when a person

fills a form online and when click the button submit all the data or the words written is sent via file on a computer. Some key-loggers works as a API in running application it looks like a simple application and whenever a key is pressed it records it.

2. Hardware Key-loggers : These are not dependent on any software as these are hardware key-loggers. keyboard hardware is a circuit which is attached in a keyboard itself that whenever the key of that keyboard pressed it gets recorded.

1. **USB keylogger** – There are USB connector key-loggers which has to be connected to a computer and steals the data. Also some circuits are built into a keyboard so no external wire i used or shows on the keyboard.
2. **Smartphone sensors** – Some cool android tricks are also used as key loggers such as android accelerometer sensor which when placed near to the keyboard can sense the vibrations and the graph then used to convert it to sentences, this technique accuracy is about 80%. Now a day's crackers are using keystroke logging Trojan, it is a malware which is sent to a victims computer to steal the data and login details.

So key-loggers are the software malware or a hardware which is used to steal , or snatch our login details, credentials , bank information and many more. Some keylogger application used in 2020 are:

1. Kidlogger
2. Best Free Keylogger
3. Windows Keylogger
4. Refog Personal Monitor
5. All In One Keylogger

Prevention from key-loggers : These are following below

1. **Anti-Key-logger** – As the name suggest these are the software which are anti / against key loggers and main task is to detect key-logger from a computer system.
2. **Anti-Virus** – Many anti-virus software also detect key loggers and delete them from the computer system. These are software anti-software so these cannot get rid from the hardware key-loggers.
3. **Automatic form filler** – This technique can be used by the user to not fill forms on regular bases instead use automatic form filler which will give a shield against

key-loggers as keys will not be pressed.

4. **One-Time-Passwords** – Using OTP's as password may be safe as every time we login we have to use a new password.
5. **Patterns or mouse-recognition** – On android devices used pattern as a password of applications and on PC use mouse recognition, mouse program uses mouse gestures instead of stylus.
6. **Voice to Text Converter** – This software helps to prevent Keylogging which targets a specific part of our keyboard.

Greyware:

As its name suggests, Grayware is a software type that comes between malicious(black) and non-malicious(white) software categories. Grayware programs can be used for both legal and illegal activities. For example, Spyware and Keyloggers can be categorized under Grayware. They can be legally used by organizations to track the activities of their employees. Similarly, they can also be used by hackers to illegally spy on users.

Grayware are unwanted applications that might not be threatening like other malware like Trojans or Ransomware but still can cause a lot of trouble to the system and frustration to the users. Grayware can also make way for the entry of more malicious programs.

Common types of Grayware

To better understand the Grayware let us know about its common types. You might be aware of these terms.

Adware

Adware is the advertising-based software that is designed to generate and display ads on the user's browser and application screens. They are not malicious by nature but can cause frustration to users by continuous ad bombardment. Along with that, they can also open the gate for other malware by spreading malicious ads.

Spyware

Spyware is a type of malicious program specifically prepared to illegally spy on the victim's system and collect & share its activities. It generally does not damage the system but can completely ruin the privacy of its victim.

Madware

Madware is short for Mobile Adware. The irrelevant and unintended ads that you see on

your smartphones are caused by Malware. Some Malware can also act like Spyware on mobile phones. Malware is generally found in Android devices.

Keyloggers

Keylogger is a malware type that can record and log your keystrokes and mouse clicks. It can collect the private information of the user such as banking details, social security numbers, etc. This information can then pass to the cybercriminals for conducting illicit activities. Keyloggers are also used by organizations for monitoring the activities of their workers.

How to remove Grayware from your device?

Though they are not devastating like other malware, Grayware still needs to be removed from the system so that your privacy is protected and the system runs smoothly. Here is how you can do that.

- Open the task manager of your system and look for the application you don't remember installing. Also, monitor which process or program is unnecessarily consuming the system resources.
- Uninstall those applications from the Control Panel or system settings. Also, completely delete the folders where those applications were installed.
- Empty the recycle bin and delete the temporary files of your system.
- If the above steps are too complicated or tedious for you, just use any robust antimalware. Antimalware is specially designed for finding and deleting such malicious programs.

IDS:

A system called an intrusion detection system (IDS) observes network traffic for malicious transactions and sends immediate alerts when it is observed. It is software that checks a network or system for malicious activities or policy violations. Each illegal activity or violation is often recorded either centrally using an SIEM system or notified to an administration. IDS monitors a network or system for malicious activity and protects a computer network from unauthorized access from users, including perhaps insiders. The intrusion detector learning task is to build a predictive model (i.e. a classifier) capable of distinguishing between 'bad connections' (intrusion/attacks) and 'good (normal) connections'.

Working of Intrusion Detection System(IDS)

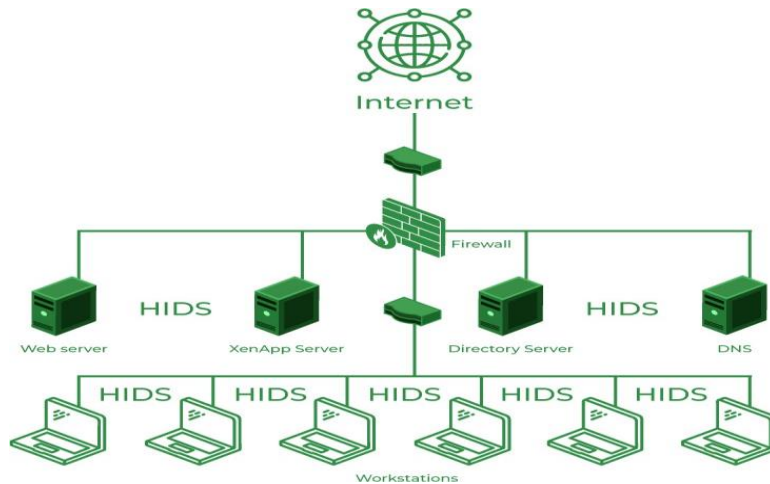
- An IDS (Intrusion Detection System) monitors the traffic on a computer network to detect any suspicious activity.

- It analyzes the data flowing through the network to look for patterns and signs of abnormal behavior.
- The IDS compares the network activity to a set of predefined rules and patterns to identify any activity that might indicate an attack or intrusion.
- If the IDS detects something that matches one of these rules or patterns, it sends an alert to the system administrator.
- The system administrator can then investigate the alert and take action to prevent any damage or further intrusion.

Classification of Intrusion Detection System(IDS)

Intrusion Detection System are classified into 5 types:

- **Network Intrusion Detection System (NIDS):** Network intrusion detection systems (NIDS) are set up at a planned point within the network to examine traffic from all devices on the network. It performs an observation of passing traffic on the entire subnet and matches the traffic that is passed on the subnets to the collection of known attacks. Once an attack is identified or abnormal behavior is observed, the alert can be sent to the administrator. An example of a NIDS is installing it on the subnet where firewalls are located in order to see if someone is trying to crack the firewall.
- **Host Intrusion Detection System (HIDS):** Host intrusion detection systems (HIDS) run on independent hosts or devices on the network. A HIDS monitors the incoming and outgoing packets from the device only and will alert the administrator if suspicious or malicious activity is detected. It takes a snapshot of existing system files and compares it with the previous snapshot. If the analytical system files were edited or deleted, an alert is sent to the administrator to investigate. An example of HIDS usage can be seen on mission-critical machines, which are not expected to change their layout.



- Protocol-based Intrusion Detection System (PIDS):** Protocol-based intrusion detection system (PIDS) comprises a system or agent that would consistently reside at the front end of a server, controlling and interpreting the protocol between a user/device and the server. It is trying to secure the web server by regularly monitoring the HTTPS protocol stream and accepting the related HTTP protocol. As HTTPS is unencrypted and before instantly entering its web presentation layer then this system would need to reside in this interface, between to use the HTTPS.
- Application Protocol-based Intrusion Detection System (APIDS):** An application Protocol-based Intrusion Detection System (APIDS) is a system or agent that generally resides within a group of servers. It identifies the intrusions by monitoring and interpreting the communication on application-specific protocols. For example, this would monitor the SQL protocol explicitly to the middleware as it transacts with the database in the web server.
- Hybrid Intrusion Detection System:** Hybrid intrusion detection system is made by the combination of two or more approaches to the intrusion detection system. In the hybrid intrusion detection system, the host agent or system data is combined with network information to develop a complete view of the network system. The hybrid intrusion detection system is more effective in comparison to the other intrusion detection system. Prelude is an example of Hybrid IDS.

Intrusion Detection System Evasion Techniques

- **Fragmentation:** Dividing the packet into smaller packet called fragment and the process is known as fragmentation. This makes it impossible to identify an intrusion because there can't be a malware signature.
- **Packet Encoding:** Encoding packets using methods like Base64 or hexadecimal can hide malicious content from signature-based IDS.
- **Traffic Obfuscation:** By making message more complicated to interpret, obfuscation can be utilised to hide an attack and avoid detection.
- **Encryption:** Several security features, such as data integrity, confidentiality, and at a privacy, are provided by encryption. Unfortunately, security features are used by malware developers to hide attacks and avoid detection.

Benefits of IDS

- **Detects malicious activity:** IDS can detect any suspicious activities and alert the system administrator before any significant damage is done.
- **Improves network performance:** IDS can identify any performance issues on the network, which can be addressed to improve network performance.
- **Compliance requirements:** IDS can help in meeting compliance requirements by monitoring network activity and generating reports.
- **Provides insights:** IDS generates valuable insights into network traffic, which can be used to identify any weaknesses and improve network security.

Detection Method of IDS

- **Signature-based Method:** Signature-based IDS detects the attacks on the basis of the specific patterns such as the number of bytes or a number of 1s or the number of 0s in the network traffic. It also detects on the basis of the already known malicious instruction sequence that is used by the malware. The detected patterns in the IDS are known as signatures. Signature-based IDS can easily detect the attacks whose pattern (signature) already exists in the system but it is quite difficult to detect new malware attacks as their pattern (signature) is not known.
- **Anomaly-based Method:** Anomaly-based IDS was introduced to detect unknown malware attacks as new malware is developed rapidly. In anomaly-based IDS there is the use of machine learning to create a trustful activity model

and anything coming is compared with that model and it is declared suspicious if it is not found in the model. The machine learning-based method has a better-generalized property in comparison to signature-based IDS as these models can be trained according to the applications and hardware configurations.

DDoS Network Security:

Secure Sockets Layer:

SSL Architecture

SSL is designed to make use of TCP to provide a reliable end-to-end secure service. SSL is not a single protocol but rather two layers of protocols, as illustrated in Figure. The SSL Record Protocol provides basic security services to various higher layer protocols. In particular, the Hypertext Transfer Protocol (HTTP), which provides the transfer service for Web client/server interaction, can operate on top of SSL. Three higher-layer protocols are defined as part of SSL: the Handshake Protocol, The Change Cipher Spec Protocol, and the Alert Protocol. These SSL-specific protocols are used in the management of SSL exchanges and are examined later in this section. Two important SSL concepts are the SSL session and the SSL connection, which are defined in the specification as follows.

- Connection: A connection is a transport (in the OSI layering model definition) that provides a suitable type of service. For SSL, such connections are peer-to-peer relationships. The connections are transient. Every connection is associated with one session.
- Session: An SSL session is an association between a client and a server. Sessions are created by the Handshake Protocol. Sessions define a set of cryptographic

SSL Handshake Protocol	SSL Change Cipher Spec Protocol	SSL Alert Protocol	HTTP
SSL Record Protocol			
TCP			
IP			

Figure : SSL Protocol Stack

security parameters which can be shared among multiple connections. Sessions are used to avoid the expensive negotiation of new security parameters for each connection.

Between any pair of parties (applications such as HTTP on client and server), there may be multiple secure connections. In theory, there may also be multiple simultaneous sessions between parties, but this feature is not used in practice.

There are a number of states associated with each session. Once a session is established, there is a current operating state for both read and write (i.e., receive and send). In addition, during the Handshake Protocol, pending read and write states are created. Upon successful conclusion of the Handshake Protocol, the pending states become the current states.

A session state is defined by the following parameters.

- **Session identifier:** An arbitrary byte sequence chosen by the server to identify an active or resumable session state.
- **Peer certificate:** An X509.v3 certificate of the peer. This element of the state may be null.
- **Compression method:** The algorithm used to compress data prior to encryption.
- **Cipher spec:** Specifies the bulk data encryption algorithm (such as null, AES, etc.) and a hash algorithm (such as MD5 or SHA-1) used for MAC calculation. It also defines cryptographic attributes such as the `hash_size`.
- **Master secret:** 48-byte secret shared between the client and server.
- **Is resumable:** A flag indicating whether the session can be used to initiate new connections. A connection state is defined by the following parameters.
- **Server and client random:** Byte sequences that are chosen by the server and client for each connection.
- **Server write MAC secret:** The secret key used in MAC operations on data sent by the server.
- **Client write MAC secret:** The secret key used in MAC operations on data sent by the client.
- **Server write key:** The secret encryption key for data encrypted by the server and decrypted by the client.
- **Client write key:** The symmetric encryption key for data encrypted by the client and decrypted by the server.
- **Initialization vectors:** When a block cipher in CBC mode is used, an initialization vector (IV) is maintained for each key. This field is first initialized

by the SSL Handshake Protocol.

- **Sequence numbers:** Each party maintains separate sequence numbers for transmitted and received messages for each connection. When a party sends or receives a change cipher spec message, the appropriate sequence number is set to zero. Sequence numbers may not exceed $2^{64}-1$.

SSL Record Protocol

The SSL Record Protocol provides two services for SSL connections:

- **Confidentiality:** The Handshake Protocol defines a shared secret key that is used for conventional encryption of SSL payloads.
- **Message Integrity:** The Handshake Protocol also defines a shared secret key that is used to form a message authentication code (MAC).

Figure indicates the overall operation of the SSL Record Protocol. The Record Protocol takes an application message to be transmitted, fragments the data into manageable blocks, optionally compresses the data, applies a MAC, encrypts, adds a header, and transmits the resulting unit in a TCP segment. Received data are decrypted, verified, decompressed, and reassembled before being delivered to higher-level users. The first step is fragmentation. Each upper-layer message is fragmented into blocks of 214 bytes (16384 bytes) or less. Next, compression is optionally applied. Compression must be lossless and may not increase the content length by more than 1024 bytes.¹In SSLv3 (as well as the current version of TLS), no compression algorithm is specified, so the default compression algorithm is null. The next step in processing is to compute a message authentication code over the compressed data.

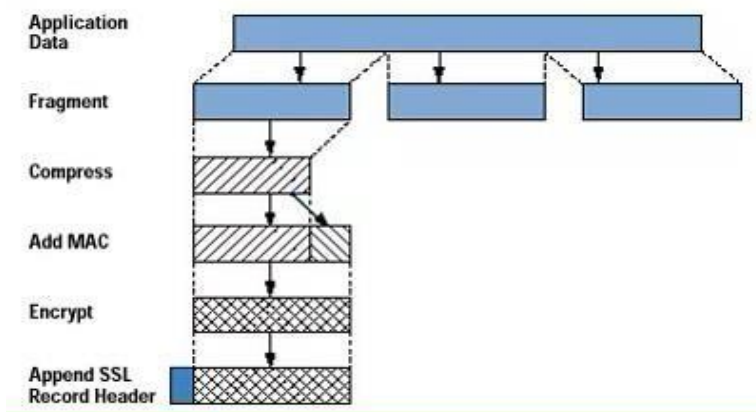


Figure : SSL Record Protocol Operation

Transport Layer Security:

TLS is an IETF standardization initiative whose goal is to produce an Internet standard version of SSL. TLS is defined as a Proposed Internet Standard in RFC 5246. RFC 5246 is very similar to SSLv3. In this section, we highlight the differences.

Version Number

The TLS Record Format is the same as that of the SSL Record Format (Figure 16.4), and the fields in the header have the same meanings. The one difference is in version values. For the current version of TLS, the major version is 3 and the minor version is 3.

Message Authentication Code

There are two differences between the SSLv3 and TLS MAC schemes: the actual algorithm and the scope of the MAC calculation. TLS makes use of the HMAC algorithm defined in RFC 2104. Recall from Chapter 12 that HMAC is defined as

$$\text{HMACK}(M) = H[(K + \text{opad}) || H[(K + \text{ipad}) || M]]$$

where

H = embedded hash function (for TLS, either MD5 or SHA-1)

M = message input to HMAC

K^+ = secret key padded with zeros on the left so that the result is equal to the block length of

the hash code (for MD5 and SHA-1, block length = 512 bits)

ipad = 00110110 (36 in hexadecimal) repeated 64 times (512 bits)

opad = 01011100 (5C in hexadecimal) repeated 64 times (512 bits)

SSLv3 uses the same algorithm, except that the padding bytes are concatenated with the secret key rather than being XORed with the secret key padded to the block length. The level of security should be about the same in both cases.

For TLS, the MAC calculation encompasses the fields indicated in the following expression:

$$\begin{aligned} &\text{MAC}(\text{MAC_write_secret}, \text{seq_num} || \text{TLSCompressed.type} || \\ &\quad \text{TLSCompressed.version} || \text{TLSCompressed.length} \\ &|| \\ &\quad \text{TLSCompressed.fragment}) \end{aligned}$$

The MAC calculation covers all of the fields covered by the SSLv3 calculation, plus the field TLSCompressed. version, which is the version of the protocol being employed.

Pseudorandom Function

TLS makes use of a pseudorandom function referred to as PRF to expand secrets into blocks of data for purposes of key generation or validation. The objective is to make use of a relatively small shared secret value but to generate longer blocks of data in a way that is secure from the kinds of attacks made on hash functions and MACs. The PRF is based on the data expansion function (Figure 16.7) given as

$$\begin{aligned} P_hash(secret, seed) = & HMAC_hash(secret, A(1) \parallel seed) \parallel \\ & HMAC_hash(secret, A(2) \parallel seed) \parallel \\ & HMAC_hash(secret, A(3) \parallel seed) \parallel \dots \end{aligned}$$

where $A()$ is defined as

$$\begin{aligned} A(0) &= seed \\ A(i) &= HMAC_hash(secret, A(i-1)) \end{aligned}$$

The data expansion function makes use of the HMAC algorithm with either MD5 or SHA-1 as the underlying hash function. As can be seen, P_hash can be iterated as many times as necessary to produce the required quantity of data. For example, if P_SHA-1 was used to generate 64 bytes of data, it would have to be iterated four times, producing 80 bytes of data of which the last 16 would be discarded. In this case, P_MD5 would also have to be iterated four times, producing exactly 64 bytes of data. Note that each iteration involves two executions of HMAC—each of which in turn involves two executions of the underlying hash algorithm.

To make PRF as secure as possible, it uses two hash algorithms in a way that should guarantee its security if either algorithm remains secure. PRF is defined as

$$PRF(secret, label, seed) = P_hash(S1, label \parallel seed)$$

PRF takes as input a secret value, an identifying label, and a seed value and produces an output of arbitrary length.

Alert Codes

TLS supports all of the alert codes defined in SSLv3 with the exception of

no_certificate. A number of additional codes are defined in TLS; of these, the following are always fatal.

- record_overflow: A TLS record was received with a payload (ciphertext) whose length exceeds 2¹⁴+2048 or greater than 2¹⁴+1024 bytes. bytes, or the ciphertext decrypted to a length
- unknown_ca: A valid certificate chain or partial chain was received, but the certificate was not accepted because the CA certificate could not be located or could not be matched with a known, trusted CA.
- access_denied: A valid certificate was received, but when access control was applied, the sender decided not to proceed with the negotiation.
- decode_error: A message could not be decoded, because either a field was out of its specified range or the length of the message was incorrect.
- protocol_version: The protocol version the client attempted to negotiate is recognized but not supported.
- insufficient_security: Returned instead of handshake_failure when a negotiation has failed specifically because the server requires ciphers more secure than those supported by the client.
- unsupported_extension: Sent by clients that receive an extended server hello containing an extension not in the corresponding client hello.
- internal_error: An internal error unrelated to the peer or the correctness of the protocol makes it impossible to continue.
- decrypt_error: A handshake cryptographic operation failed, including being unable to verify a signature, decrypt a key exchange, or validate a finished message.

HTTPS:

HTTPS (HTTP over SSL) refers to the combination of HTTP and SSL to implement secure communication between a Web browser and a Web server. The HTTPS capability is built into all modern Web browsers. Its use depends on the Web server supporting HTTPS communication. For example, search engines do not support HTTPS.

The principal difference seen by a user of a Web browser is that URL (uniform resource locator) addresses begin with https:// rather than http://. A normal HTTP connection uses port 80. If HTTPS is specified, port 443 is used, which invokes SSL.

When HTTPS is used, the following elements of the communication are encrypted:

- URL of the requested document
- Contents of the document
- Contents of browser forms (filled in by browser user)
- Cookies sent from browser to server and from server to browser
- Contents of HTTP header

HTTPS is documented in RFC 2818, HTTP over TLS. There is no fundamental change in using HTTP over either SSL or TLS, and both implementations are referred to as HTTPS.

Connection Initiation

For HTTPS, the agent acting as the HTTP client also acts as the TLS client. The client initiates a connection to the server on the appropriate port and then sends the TLS Client Hello to begin the TLS handshake. When the TLS handshake has finished, the client may then initiate the first HTTP request. All HTTP data is to be sent as TLS application data. Normal HTTP behavior, including retained connections, should be followed.

We need to be clear that there are three levels of awareness of a connection in HTTPS. At the HTTP level, an HTTP client requests a connection to an HTTP server by sending a connection request to the next lowest layer. Typically, the next lowest layer is TCP, but it also may be TLS/SSL. At the level of TLS, a session is established between a TLS client and a TLS server. This session can support one or more connections at any time. As we have seen, a TLS request to establish a connection begins with the establishment of a TCP connection between the TCP entity on the client side and the TCP entity on the server side.

Connection Closure

An HTTP client or server can indicate the closing of a connection by including the following line in an HTTP record: Connection: close. This indicates that the connection will be closed after this record is delivered.

The closure of an HTTPS connection requires that TLS close the connection with the peer TLS entity on the remote side, which will involve closing the underlying TCP connection. At the TLS level, the proper way to close a connection is for each side to use the TLS alert protocol to send a close_notify alert. TLS implementations must initiate an

exchange of closure alerts before closing a connection. A TLS implementation may, after sending a closure alert, close the connection without waiting for the peer to send its closure alert, generating an “incomplete close”. Note that an implementation that does this may choose to reuse the session. This should only be done when the application knows (typically through detecting HTTP message boundaries) that it has received all the message data that it cares about.

HTTP clients also must be able to cope with a situation in which the underlying TCP connection is terminated without a prior `close_notify` alert and without a `Connection: close` indicator. Such a situation could be due to a programming error on the server or a communication error that causes the TCP connection to drop. However, the unannounced TCP closure could be evidence of some sort of attack. So the HTTPS client should issue some sort of security warning when this occurs.

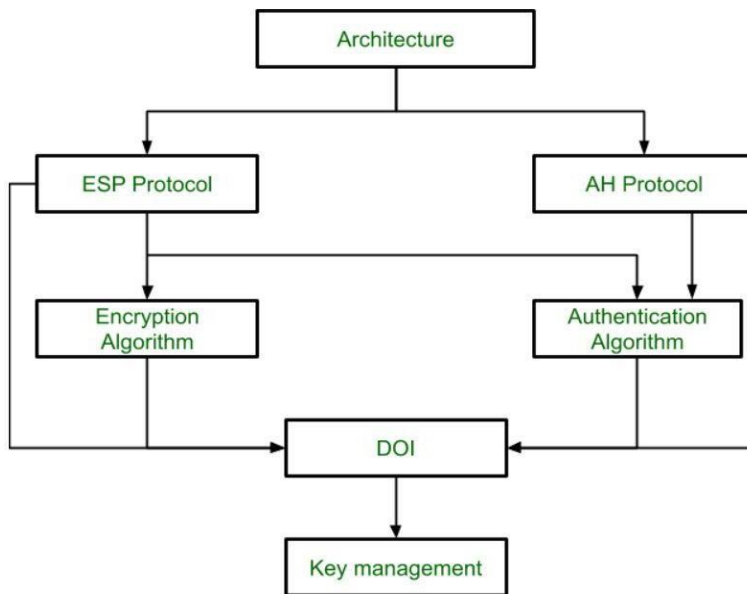
IP Security:

Internet Protocol Security defined only two protocols for securing the IP packets which were Authentication Header(AH) and Encapsulating Security Payload(ESP). The former protocol i.e. AH provides data integrity and non-replay services, and the latter protocol i.e. ESP encrypts and authenticates data.

The Internet Protocol Security suite also includes Internet Key Exchange (IKE), which is basically used widely to generate shared security keys with the purpose of establishing a security association (SA). Security Associations are majorly needed for the purpose of the encryption process as well as for the decryption process to negotiate a security level between two entities. A special router or firewall is required that works between two networks which helps to handle the security association negotiation process.

Architecture of IPsec:

Read the article architecture of Internet Protocol Security to get the complete details about this.



Architecture of IPsec

Protocols behind IPsec:

There are majorly four protocols behind IPsec which are as follows:

1. Internet Protocol Authentication Header (IP AH): Internet Protocol Authentication Header basically includes functionalities like data integrity and transport protection services. The authentication Header was designed for the purpose of adding authentication data. It also provides the feature of data integrity, authentication, and anti-replay and one of its drawbacks are that it does not provide encryption. The anti-replay protection protects against unauthorized transmission of packets. One more disadvantage is that it does not protect the confidentiality of data at all.

2. Internet Protocol Encapsulating Security Payload (IP ESP): Internet Protocol Encapsulating Security Payload was majorly specified in RFC 4303, ESP provides fabulous features like authentication, integrity, and confidentiality with the help of

encryption of IP packets. It also helps to provide data integrity, encryption, and authentication. Authentication for the payload is one of its important features of it.

3. Internet Key Exchange (IKE): Internet Key Exchange is a special protocol that helps to enable two systems or devices to establish a secure and strong communication channel over a nonreliable network also. This protocol achieves this using a series of key exchanges to create a secure and strong tunnel between a client and a server with the help of which they can send encrypted traffic easily and securely. The security of the tunnel is based on the Diffie-Hellman key exchange method, which is one of the widely used techniques used for security.

4. Internet Security Association and Key Management Protocol (ISAKMP): Internet Security Association and Key Management Protocol are simply specified as one of the parts of IKE protocol. It is a framework that is majorly used for key establishment, authentication, and negotiation of a security association for a secure exchange of packets over an Internet Protocol layer. In other words, we can say that this protocol defines the security parameters for how two systems can communicate with each other. Each security association defines a connection in one direction, from one host to another. The security association includes all attributes which are required for a connection, including the cryptographic algorithm, the IPsec mode, the encryption key, and any other parameters which are related to data transmission which are required to establish a secure connection.

Advantages of IPsec:

1. IPsec provides network-layer security as it works on the network layer and provides transparency to applications.
2. It provides confidentiality during any kind of data exchange.
3. As it is implemented on the network layer, IPsec has zero dependability on applications.

Disadvantages of IPsec:

1. IPsec has a wide access range, In IPsec networks giving access to a single device can give access privilege to other devices too.
2. In many of the cases, it brings a couple of incompatibility issues with different software.
3. In many cases, IPsec leads to high CPU usage.

OS Security: Introduction to Operating System Security

Computer client and server systems are central components of the IT infrastructure for most organizations, may hold critical data and applications, and are a necessary tool for the function of an organization. Accordingly, we need to be aware of the expected presence of vulnerabilities in operating systems and applications as distributed, and the existence of worms scanning for such vulnerabilities at high rates. Thus, it is quite possible for a system to be compromised during the installation process, before it can install the latest patches or implement other hardening measures. Hence, building and deploying a system should be a planned process designed to counter such a threat, and to maintain security during its operational lifetime.

States that this process must:

- Assess risks and plan the system deployment.
- Secure the underlying operating system and then the key applications.
- Ensure any critical content is secured.
- Ensure appropriate network protection mechanisms are used.
- Ensure appropriate processes are used to maintain security.

System Security Planning

It provides a list of items that should be considered during the system security planning process. While its focus is on secure server deployment, much of the list applies equally well to client system design.

This list includes consideration of:

- The purpose of the system, the type of information stored, the applications and services provided, and their security requirements.
- The categories of users of the system, the privileges they have, and the types of information they can access.
- How the users are authenticated.
- How access to the information stored on the system is managed.
- What access the system has to information stored on other hosts, such as file or database servers, and how this is managed.

- Who will administer the system, and how they will manage the system (via local or remote access).
- Any additional security measures required on the system, including the use of host firewalls, anti-virus or other malware protection mechanisms, and logging.

Operating Systems Hardening

The first critical step in securing a system is to secure the base operating system upon which all other applications and services rely. A good security foundation needs a properly installed, patched, and configured operating system. Unfortunately, the default configuration for many operating systems often maximizes ease of use and functionality, rather than security. Further, since every organization has its own security needs, the appropriate security profile, and hence configuration, will also differ.

While the details of how to secure each specific operating system differ, the broad approach is similar. Appropriate security configuration guides and checklists exist for most common operating systems, and these should be consulted, though always informed by the specific needs of each organization and their systems. In some cases, automated tools may be available to further assist in securing the system configuration.

It suggests the following basic steps that should be used to secure an operating system:

- Install and patch the operating system.
- Harden and configure the operating system to adequately address the identified security needs of the system by:
 - Removing unnecessary services, applications, and protocols.
 - Configuring users, groups, and permissions.
 - Configuring resource controls.
- Install and configure additional security controls, such as anti-virus, host-based firewalls, and intrusion detection systems (IDS), if needed.
- Test the security of the basic operating system to ensure that the steps taken adequately address its security needs.

Operating System Installation: Initial Setup and Patching

- Remove Unnecessary Services, Application, and Protocols
- Configure Users, Groups, and Authentication
- Configure Resource Controls
- Install Additional Security Controls
- Test the System Security

Application Security

Once the base operating system is installed and appropriately secured, the required services and applications must next be installed and configured. The steps for this very much mirror the list already given in the previous section. The concern, as with the base operating system, is to only install software on the system that is required to meet its desired functionality, in order to reduce the number of places vulnerabilities may be found. Software that provides remote access or service is of particular concern, since an attacker may be able to exploit this to gain remote access to the system. Hence any such software needs to be carefully selected and configured, and updated to the most recent version available.

Each selected service or application must be installed, and then patched to the most recent supported secure version appropriate for the system. This may be from additional packages provided with the operating system distribution, or from a separate third-party package. As with the base operating system, utilizing an isolated, secure build network is preferred.

Application Configuration

Any application specific configuration is then performed. This may include creating and specifying appropriate data storage areas for the application, and making appropriate changes to the application or service default configuration details.

Some applications or services may include default data, scripts, or user accounts. These should be reviewed, and only retained if required, and suitably secured. A well known example of this is found with Web servers, which often include a number of example scripts, quite a few of which are known to be insecure. These should not be used as supplied, but should be removed unless needed and secured.

As part of the configuration process, careful consideration should be given to the access rights granted to the application. Again, this is of particular concern with remotely accessed services, such as Web and file transfer services. The server application should not be granted the right to modify files, unless that function is specifically required. A very common configuration fault seen with Web and file transfer servers is for all the files supplied by the service to be owned by the same “user” account that the server executes as. The consequence is that any attacker able to exploit some vulnerability in either the server software or a script executed by the server may be able to modify any of

these files. The large number of “Web defacement” attacks is clear evidence of this type of insecure configuration. Much of the risk from this form of attack is reduced by ensuring that most of the files can only be read, but not written, by the server. Only those files that need to be modified, to store uploaded form data for example, or logging details, should be writeable by the server. Instead the files should mostly be owned and modified by the users on the system who are responsible for maintaining the information.

Encryption Technology

Encryption is a key enabling technology that may be used to secure data both in transit and when stored, as we discuss in Chapter 2 and in Parts Four and Five. If such technologies are required for the system, then they must be configured, and appropriate cryptographic keys created, signed, and secured. If secure network services are provided, most likely using either TLS or IPsec, then suitable public and private keys must be generated for each of them. Then X.509 certificates are created and signed by a suitable certificate authority, linking each service identity with the public key in use. If secure remote access is provided using Secure Shell (SSH), then appropriate server, and possibly client keys, must be created.

Cryptographic file systems are another use of encryption. If desired, then these must be created and secured with suitable keys.

Security Maintenance

Once the system is appropriately built, secured, and deployed, the process of maintaining security is continuous. This results from the constantly changing environment, the discovery of new vulnerabilities, and hence exposure to new threats. It suggests that this process of security maintenance includes the following additional steps:

- Monitoring and analyzing logging information
- Performing regular backups
- Recovering from security compromises
- Regularly testing system security
- Using appropriate software maintenance processes to patch and update all critical software, and to monitor and revise configuration as needed

We have already noted the need to configure automatic patching and update where possible, or to have a process to manually test and install patches on configuration-controlled systems, and that the system should be regularly tested using checklist or

automated tools where possible. We now consider the critical logging and backup procedures.

Logging

Logging can generate significant volumes of information. It is important that sufficient space is allocated for them. A suitable automatic log rotation and archive system should also be configured to assist in managing the overall size of the logging information.

Data Backup and Archive

Performing regular backups of data on a system is another critical control that assists with maintaining the integrity of the system and user data. There are many reasons why data can be lost from a system, including hardware or software failures, or accidental or deliberate corruption. There may also be legal or operational requirements for the retention of data. Backup is the process of making copies of data at regular intervals, allowing the recovery of lost or corrupted data over relatively short time periods of a few hours to some weeks. Archive is the process of retaining copies of data over extended periods of time, being months or years, in order to meet legal and operational requirements to access past data. These processes are often linked and managed together, although they do address distinct needs.

Linux/Unix Security

Having discussed the process of enhancing security in operating systems through careful installation, configuration, and management, we now consider some specific aspects of this process as it relates to Unix and Linux systems.

Patch Management

Ensuring that system and application code is kept up to date with security patches is a widely recognized and critical control for maintaining security. Modern Unix and Linux distributions typically include tools for automatically downloading and installing software updates, including security updates, which can minimize the time a system is vulnerable to known vulnerabilities for which patches exist. For example, Red Hat, Fedora, and CentOS include `up2date` or `yum`; SuSE includes `yast`; and Debian uses `apt-get`, though you must run it as a cron job for automatic updates. It is important to configure whichever update tool is provided on the distribution in use, to install at least critical

security patches in a timely manner.

Application and Service Configuration

Configuration of applications and services on Unix and Linux systems is most commonly implemented using separate text files for each application and service. System-wide configuration details are generally located either in the `/etc` directory or in the installation tree for a specific application. Where appropriate, individual user configurations that can override the system defaults are located in hidden “dot” files in each user’s home directory. The name, format, and usage of these files are very much dependent on the particular system version and applications in use. Hence the systems administrators responsible for the secure configuration of such a system must be suitably trained and familiar with them.

The most important changes needed to improve system security are to disable services, especially remotely accessible services, and applications, that are not required, and to then ensure that applications and services that are needed are appropriately configured, following the relevant security guidance for each.

Users, Groups, and Permissions

Unix and Linux systems implement discretionary access control to all file system resources. These include not only files and directories but devices, processes, memory, and indeed most system resources. Access is specified as granting read, write, and execute permissions to each of owner, group, and others, for each resource, as shown in Figure. These are set using the `chmod` command. Some systems also support extended file attributes with access control lists that provide more flexibility, by specifying these permissions for each entry in a list of users and groups. These extended access rights are typically set and displayed using the `getfacl` and `setfacl` commands. These commands can also be used to specify set user or set group permissions on the resource

Information on user accounts and group membership are traditionally stored in the `/etc/passwd` and `/etc/group` files, though modern systems also have the ability to import these details from external repositories queried using LDAP or NIS for example. These sources of information, and indeed of any associated authentication credentials, are specified in the PAM (pluggable authentication module) configuration for the system, often using text files in the `/etc/pam.d` directory.

It is widely accepted that the number and size of setuid root programs in particular should be minimized. They cannot be eliminated, as superuser privileges are required to access some resources on the system. The programs that manage user login, and allow network services to bind to privileged ports, are examples. However, other programs, that were once setuid root for programmer convenience, can function as well if made setgid to a suitable privileged group that has the necessary access to some resource. Programs to display system state, or deliver mail, have been modified in this way. System hardening guides may recommend further changes and indeed the removal of some such programs that are not required on a particular system.

Remote Access Controls

Given that remote exploits are of concern, it is important to limit access to only those services required. This function may be provided by a perimeter firewall. However, host-based firewall or network access control mechanisms may provide additional defences. Unix and Linux systems support several alternatives for this.

The TCP Wrappers library and tcpd daemon provide one mechanism that network servers may use. Lightly loaded services may be “wrapped” using tcpd, which listens for connection requests on their behalf. It checks that any request is permitted by configured policy before accepting it and invoking the server program to handle it. Requests that are rejected are logged. More complex and heavily loaded servers incorporate this functionality into their own connection management code, using the TCP Wrappers library, and the same policy configuration files. These files are /etc/ hosts.allow and /etc/hosts.deny, which should be set as policy requires.

There are several host firewall programs that may be used. Linux systems primarily use the iptables program to configure the netfilter kernel module. This provides comprehensive, though complex, stateful packet filtering, monitoring, and modification capabilities. BSD-based systems (including MacOSX) now use the pf program with similar capabilities. Most systems provide an administrative utility to generate common configurations and to select which services will be permitted to access the system. These should be used unless there are non-standard requirements, given the skill and knowledge

needed to run these programs to edit their configuration files.

Logging and Log Rotation

Most applications can be configured to log with levels of detail ranging from “debugging” (maximum detail) to “none.” Some middle setting is usually the best choice, but you should not assume that the default setting is necessarily appropriate.

In addition, many applications allow you to specify either a dedicated file to write application event data to, or a syslog facility to use when writing log data to `/dev/log`. If you wish to handle system logs in a consistent, centralized manner, it is usually preferable for applications to send their log data to `/dev/log`. Note, however, that `logrotate` can be configured to rotate any logs on the system, whether written by `syslogd`, `Syslog-NG`, or individual applications.

Security Testing

The system hardening guides such as those provided by the “NSA—Security Configuration Guides” include security checklists for a number of Unix and Linux distributions that may be followed.

There are also a number of commercial and open-source tools available to perform system security scanning and vulnerability testing. One of the best known is “Nessus.” This was originally an open-source tool, which was commercialized in 2005, though some limited free-use versions are available. “Tripwire” is a well-known file integrity checking tool that maintains a database of cryptographic hashes of monitored files, and scans to detect any changes, whether as a result of malicious attack, or simply accidental or incorrectly managed update. This again was originally an open-source tool, which now has both commercial and free variants available. The “Nmap” network scanner is another well-known and deployed assessment tool that focuses on identifying and profiling hosts on the target network, and the network services they offer.

Windows Security

We now consider some specific issues with the secure installation, configuration, and management of Microsoft Windows systems. These systems have for many years formed a significant portion of all “general purpose” system installations. Hence, they have been specifically targeted by attackers, and consequently security countermeasures are needed to deal with these challenges. The process of providing appropriate levels of security still follows the general outline we describe in this chapter.

Again, there are a large range of resources available to assist administrators of these systems, including reports such as [SYMA07], online resources such as the “Microsoft Security Tools & Checklists,” and specific system hardening guides such as those provided by the “NSA—Security Configuration Guides.”

Patch Management

The “Windows Update” service and the “Windows Server Update Services” assist with the regular maintenance of Microsoft software, and should be configured and used. Many other third-party applications also provide automatic update support, and these should be enabled for selected applications.

Users Administration and Access Controls

Users and groups in Windows systems are defined with a Security ID (SID). This information may be stored and used locally, on a single system, in the Security Account Manager (SAM). It may also be centrally managed for a group of systems belonging to a domain, with the information supplied by a central Active Directory (AD) system using the LDAP protocol. Most organizations with multiple systems will manage them using domains. These systems can also enforce common policy on users on any system in the domain.

Windows systems implement discretionary access controls to system resources such as files, shared memory, and named pipes. The access control list has a number of entries that may grant or deny access rights to a specific SID, which may be for an individual user or for some group of users. Windows Vista and later systems also include mandatory integrity controls. These label all objects, such as processes and files, and all users, as being of low, medium, high, or system integrity level. Then whenever data is written to an object, the system first ensures that the subject’s integrity is equal or higher than the object’s level. This implements a form of the Biba Integrity model that specifically targets the issue of untrusted remote code executing in, for example Windows Internet Explorer, trying to modify local resources.

Windows systems also define privileges, which are system wide and granted to user accounts. Examples of privileges include the ability to backup the computer (which requires overriding the normal access controls to obtain a complete backup), or the ability

to change the system time. Some privileges are considered dangerous, as an attacker may use them to damage the system. Hence they must be granted with care. Others are regarded as benign, and may be granted to many or all user accounts.

As with any system, hardening the system configuration can include further limiting the rights and privileges granted to users and groups on the system. As the access control list gives deny entries greater precedence, you can set an explicit deny permission to prevent unauthorized access to some resource, even if the user is a member of a group that otherwise grants access.

When accessing files on a shared resource, a combination of share and NTFS permissions may be used to provide additional security and granularity. For example, you can grant full control to a share, but read-only access to the files within it. If access-based enumeration is enabled on shared resources, it can automatically hide any objects that a user is not permitted to read. This is useful with shared folders containing many users' home directories, for example.

You should also ensure users with administrative rights only use them when required, and otherwise access the system as a normal user. The User Account Control (UAC) provided in Vista and later systems assists with this requirement. These systems also provide Low Privilege Service Accounts that may be used for long-lived service processes, such as file, print, and DNS services that do not require elevated privileges.

Application and Service Configuration

Unlike Unix and Linux systems, much of the configuration information in Windows systems is centralized in the Registry, which forms a database of keys and values that may be queried and interpreted by applications on these systems.

Changes to these values can be made within specific applications, setting preferences in the application that are then saved in the registry using the appropriate keys and values. This approach hides the detailed representation from the administrator. Alternatively, the registry keys can be directly modified using the "Registry Editor." This approach is more useful for making bulk changes, such as those recommended in hardening guides. These changes may also be recorded in a central repository, and pushed out whenever a user logs in to a system within a network domain.

Virtualization Security

Virtualization refers to a technology that provides an abstraction of the computing resources used by some software, which thus runs in a simulated environment called a virtual machine (VM). There are many types of virtualization; however, in this section we are most interested in full virtualization. This allows multiple full operating system instances to execute on virtual hardware, supported by a hypervisor that manages access to the actual physical hardware resources. Benefits arising from using virtualization include better efficiency in the use of the physical system resources than is typically seen using a single operating system instance. This is particularly evident in the provision of virtualized server systems. Virtualization can also provide support for multiple distinct operating systems and associated applications on the one physical system. This is more commonly seen on client systems.

There are a number of additional security concerns raised in virtualized systems, as a consequence both of the multiple operating systems executing side by side and of the presence of the virtualized environment and hypervisor as a layer below the operating system kernels and the security services they provide. Presents a survey of some of the security issues arising from such a use of virtualization.

Virtualization Alternatives

There are many forms of creating a simulated, virtualized environment. These include application virtualization, as provided by the Java Virtual Machine environment. This allows applications written for one environment, to execute on some other operating system. It also includes full virtualization, in which multiple full operating system instances execute in parallel. Each of these guest operating systems, along with their own set of applications, executes in its own VM on virtual hardware. These guest OSs are managed by a hypervisor, or virtual machine monitor (VMM), that coordinates access between each of the guests and the actual physical hardware resources, such as CPU, memory, disk, network, and other attached devices. The hypervisor provides a similar hardware interface as that seen by operating systems directly executing on the actual hardware. As a consequence, little if any modification is needed to the guest OSs and their applications. Recent generations of CPUs provide special instructions that improve the efficiency of hypervisor operation.

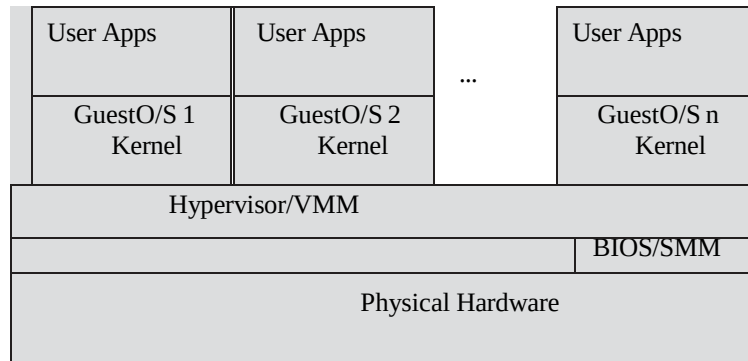


Figure 1: Native Virtualization Security Layers

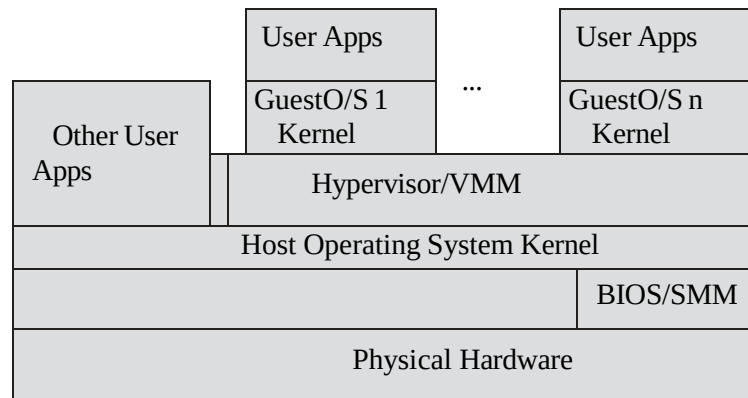


Figure 2: Hosted Virtualization Security Layers

They are arguably also more secure, as they have fewer additional layers than the alternative hosted approach. Hosted virtualization systems are more common in clients, where they run along side other applications on the host OS, and are used to support applications for alternate operating system versions or types. As this approach adds additional layers with the host OS under, and other host applications beside, the hypervisor, this may result in increased security concerns.

Virtualization Security Issues

Number of security concerns that result from the use of virtualized systems, including:

- Guest OS isolation, ensuring that programs executing within a guest OS may only access and use the resources allocated to it, and not covertly interact with programs or data either in other guest OSs or in the hypervisor.
- Guest OS monitoring by the hypervisor, which has privileged access to the programs and data in each guest OS, and must be trusted as secure from subversion and compromised use of this access.
- Virtualized environment security, particularly image and snapshot management,

which attackers may attempt to view or modify.

Securing Virtualization Systems

Provides guidance for providing appropriate security in virtualized systems, and states that organizations using virtualization should:

- Carefully plan the security of the virtualized system
- Secure all elements of a full virtualization solution, including the hypervisor, guest OSs, and virtualized infrastructure, and maintain their security
- Ensure that the hypervisor is properly secured
- Restrict and protect administrator access to the virtualization solution

Wireless Security: Risks and Threats of Wireless:

What are the risks to your wireless network?

Whether it's a home or business network, the risks to an unsecured wireless network are the same. Some of the risks include:

Piggybacking

If you fail to secure your wireless network, anyone with a wireless-enabled computer in range of your access point can use your connection. The typical indoor broadcast range of an access point is 150–300 feet. Outdoors, this range may extend as far as 1,000 feet. So, if your neighborhood is closely settled, or if you live in an apartment or condominium, failure to secure your wireless network could open your internet connection to many unintended users. These users may be able to conduct illegal activity, monitor and capture your web traffic, or steal personal files.

Wardriving

Wardriving is a specific kind of piggybacking. The broadcast range of a wireless access point can make internet connections available outside your home, even as far away as your street. Savvy computer users know this, and some have made a hobby out of driving through cities and neighborhoods with a wireless-equipped computer—sometimes with a powerful antenna—searching for unsecured wireless networks. This practice is known as “wardriving.”

Evil Twin Attacks

In an evil twin attack, an adversary gathers information about a public network access point, then sets up their system to impersonate it. The adversary uses a broadcast signal stronger than the one generated by the legitimate access point; then, unsuspecting users

connect using the stronger signal. Because the victim is connecting to the internet through the attacker's system, it's easy for the attacker to use specialized tools to read any data the victim sends over the internet. This data may include credit card numbers, username and password combinations, and other personal information. Always confirm the name and password of a public Wi-Fi hotspot prior to use. This will ensure you are connecting to a trusted access point.

Wireless Sniffing

Many public access points are not secured and the traffic they carry is not encrypted. This can put your sensitive communications or transactions at risk. Because your connection is being transmitted "in the clear," malicious actors could use sniffing tools to obtain sensitive information such as passwords or credit card numbers. Ensure that all the access points you connect to use at least WPA2 encryption.

Unauthorized Computer Access

An unsecured public wireless network combined with unsecured file sharing could allow a malicious user to access any directories and files you have unintentionally made available for sharing. Ensure that when you connect your devices to public networks, you deny sharing files and folders. Only allow sharing on recognized home networks and only while it is necessary to share items. When not needed, ensure that file sharing is disabled. This will help prevent an unknown attacker from accessing your device's files.

Shoulder Surfing

In public areas malicious actors can simply glance over your shoulder as you type. By simply watching you, they can steal sensitive or personal information. Screen protectors that prevent shoulder-surfers from seeing your device screen can be purchased for little money. For smaller devices, such as phones, be cognizant of your surroundings while viewing sensitive information or entering passwords.

Theft of Mobile Devices

Not all attackers rely on gaining access to your data via wireless means. By physically stealing your device, attackers could have unrestricted access to all of its data, as well as any connected cloud accounts. Taking measures to protect your devices from loss or theft is important, but should the worst happen, a little preparation may protect the data inside. Most mobile devices, including laptop computers, now have the ability to fully encrypt their stored data—making devices useless to attackers who cannot provide the proper

password or personal identification number (PIN). In addition to encrypting device content, it is also advisable to configure your device's applications to request login information before allowing access to any cloud-based information. Last, individually encrypt or password-protect files that contain personal or sensitive information. This will afford yet another layer of protection in the event an attacker is able to gain access to your device.

What can you do to minimize the risks to your wireless network?

- **Change default passwords.** Most network devices, including wireless access points, are pre-configured with default administrator passwords to simplify setup. These default passwords are easily available to obtain online, and so provide only marginal protection. Changing default passwords makes it harder for attackers to access a device. Use and periodic changing of complex passwords is your first line of defense in protecting your device.
- **Restrict access.** Only allow authorized users to access your network. Each piece of hardware connected to a network has a media access control (MAC) address. You can restrict access to your network by filtering these MAC addresses. Consult your user documentation for specific information about enabling these features. You can also utilize the “guest” account, which is a widely used feature on many wireless routers. This feature allows you to grant wireless access to guests on a separate wireless channel with a separate password, while maintaining the privacy of your primary credentials.
- **Encrypt the data on your network.** Encrypting your wireless data prevents anyone who might be able to access your network from viewing it. There are several encryption protocols available to provide this protection. Wi-Fi Protected Access (WPA), WPA2, and WPA3 encrypt information being transmitted between wireless routers and wireless devices. WPA3 is currently the strongest encryption. WPA and WPA2 are still available; however, it is advisable to use equipment that specifically supports WPA3, as using the other protocols could leave your network open to exploitation.
- **Protect your Service Set Identifier (SSID).** To prevent outsiders from easily accessing your network, avoid publicizing your SSID. All Wi-Fi routers allow users to protect their device's SSID, which makes it more difficult for attackers to

find a network. At the very least, change your SSID to something unique. Leaving it as the manufacturer's default could allow a potential attacker to identify the type of router and possibly exploit any known vulnerabilities.

- **Install a firewall.** Consider installing a firewall directly on your wireless devices (a host-based firewall), as well as on your home network (a router- or modem-based firewall). Attackers who can directly tap into your wireless network may be able to circumvent your network firewall—a host-based firewall will add a layer of protection to the data on your computer.
- **Maintain antivirus software.** Install antivirus software and keep your virus definitions up to date. Many antivirus programs also have additional features that detect or protect against spyware and adware.
- **Use file sharing with caution.** File sharing between devices should be disabled when not needed. You should always choose to only allow file sharing over home or work networks, never on public networks. You may want to consider creating a dedicated directory for file sharing and restrict access to all other directories. In addition, you should password protect anything you share. Never open an entire hard drive for file sharing.
- **Keep your access point software patched and up to date.** The manufacturer of your wireless access point will periodically release updates to and patches for a device's software and firmware. Be sure to check the manufacturer's website regularly for any updates or patches for your device.
- **Check your internet provider's or router manufacturer's wireless security options.** Your internet service provider and router manufacturer may provide information or resources to assist in securing your wireless network. Check the customer support area of their websites for specific suggestions or instructions.

Wireless LAN Security:

There are two characteristics of a wired LAN that are not inherent in a wireless LAN.

1. In order to transmit over a wired LAN, a station must be physically connected to the LAN. On the other hand, with a wireless LAN, any station within radio range of the other devices on the LAN can transmit. In a sense, there is a form of

authentication with a wired LAN in that it requires some positive and presumably observable action to connect a station to a wired LAN.

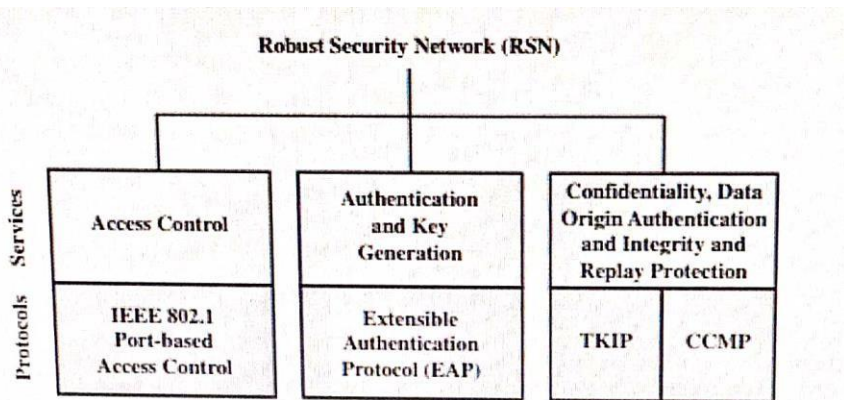
2. Similarly, in order to receive a transmission from a station that is part of a wired LAN, the receiving station also must be attached to the wired LAN. On the other hand, with a wireless LAN, any station within radio range can receive. Thus, a wired LAN provides a degree of privacy, limiting reception of data to stations connected to the LAN. These differences between wired and wireless LANs suggest the increased need for robust security services and mechanisms for wireless LANs. The original 802.11 specification included a set of security features for privacy and authentication that were quite weak. For privacy, 802.11 defined the Wired Equivalent Privacy (WEP) algorithm. The privacy portion of the 802.11 standard contained major weaknesses. Subsequent to the development of WEP, the 802.11i task group has developed a set of capabilities to address the WLAN security issues. In order to accelerate the introduction of strong security into WLANs, the Wi-Fi Alliance promulgated Wi-Fi Protected Access (WPA) as a Wi-Fi standard. WPA is a set of security mechanisms that eliminates most 802.11 security issues and was based on the current state of the 802.11i standard. The final form of the 802.11i standard is referred to as Robust Security Network (RSN). The Wi-Fi Alliance certifies vendors in compliance with the full 802.11i specification under the WPA2 program.

IEEE 802.11i Services

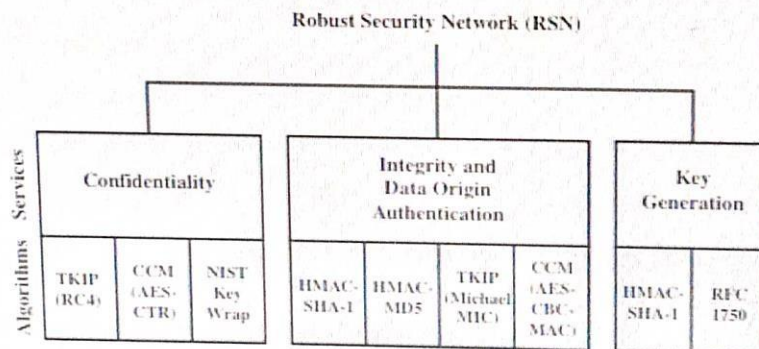
The 802.11i RSN security specification defines the following services.

- Authentication: A protocol is used to define an exchange between a user and an AS that provides mutual authentication and generates temporary keys to be used between the client and the AP over the wireless link.
- Access control: This function enforces the use of the authentication function, routes the messages properly, and facilitates key exchange. It can work with a variety of authentication protocols.
- Privacy with message integrity: MAC-level data (e.g., an LLC PDU) are encrypted along with a message integrity code that ensures that the data have not

been altered. Figure indicates the security protocols used to support these services, while Figure lists the cryptographic algorithms used for these services.



(a) Services and protocols



(b) Cryptographic Algorithms

IEEE 802.11i Phases of Operation

The operation of an IEEE 802.11i RSN can be broken down into five distinct phases of operation. The exact nature of the phases will depend on the configuration and the end points of the communication. Possibilities include:

1. Two wireless stations in the same BSS communicating via the access point (AP) for that BSS.
2. Two wireless stations (STAs) in the same ad hoc IBSS communicating directly with each other.
3. Two wireless stations in different BSSs communicating via their respective APs across a distribution system.

4. A wireless station communicating with an end station on a wired network via its AP and the distribution system.

Wireless Security Policy:

With a wireless network, you must consider security policies that will protect resources from unauthorized people. Let's take a look at what you should include in a wireless network security policy for an enterprise. Consider the following recommendations:

Activate 802.11 encryption to make data unintelligible to unauthorized users. WEP has weaknesses, making it inadequate for protecting networks containing information extremely valuable to others. There are some good hackers out there who can crack into a WEP-protected network using freely-available tools. The problem is that 802.11 doesn't support the dynamic exchange of WEP keys, leaving the same key in use for weeks, months, and years. For encryption on enterprise networks, aim higher and choose WPA, which is now part of the 802.11i standard. Just keep in mind that WPA (and WEP) only encrypts data traversing the wireless link between the client device and the access point. That may be good enough if your wired network is physically secured from hackers. If not, such as when users are accessing important information from Wi-Fi hotspots, you'll need more protection.

Utilize IPSec-based Virtual Private Network (VPN) technology for end-to-end security. If users need access to sensitive applications from Wi-Fi hotspots, definitely utilize a VPN system to provide sufficient end-to-end encryption and access control. Some companies require VPNs for all wireless client devices, even when they're connecting from inside the secured walls of the enterprise. A "full-throttle" VPN solution such as this offers good security, but it becomes costly and difficult to manage when there are hundreds of wireless users (mainly due to the need for VPN servers). As a result, consider implementing 802.11 encryption when users are operating inside the enterprise and VPNs for the likely fewer users who need access from hotspots.

Utilize 802.1x-based authentication to control access to your network. There are several flavors of 802.1x port-based authentication systems. Choose one that meets the security requirements for your company. For example, EAP-TLS may be a wise choice if you have Microsoft servers.

Establish the wireless network on a separate VLAN. A firewall can then help keep hackers located on the VLAN associated with the wireless network from having easy access to corporate servers located on different, more secured VLANs (i.e., not accessible from the wireless network). In this manner, the wireless network is similar to a public network, except you can apply encryption and authentication mechanisms to the wireless users.

Ensure firmware is up-to-date in client cards and access points. Vendors often implement patches to firmware that fix security issues. On an ongoing basis, make it a habit to check that all wireless devices have the most recent firmware releases.

Ensure only authorized people can reset the access points. Some access points will revert back to factory default settings (i.e., no security at all) when someone pushes the reset button on the access point. We've done this when performing penetration testing during security assessments to prove that this makes the access point a fragile entry point for a hacker to extend their reach into the network. As a result, provide adequate physical security for the access point hardware. For example, don't place an access point within easy reach. Instead, mount the access points out of view above ceiling tiles. Some access points don't have reset buttons and allow you to reset the access point via an RS-232 cable through a console connection. To minimize risks of someone resetting the access point in this manner, be sure to disable the console port when initially configuring the access point.

Disable access points during non-usage periods. If possible, shut down the access points when users don't need them. This limits the window of opportunity for a hacker to use an access point to their advantage as a weak interface to the rest of the network. To accomplish this, you can simply pull the power plug on each access point; however, you can also deploy power-over-Ethernet (PoE) equipment that provides this feature in a more practical manner via centralized operational support tools.

Assign "strong" passwords to access points. Don't use default passwords for access points because they are also well known, making it easy for someone to change configuration parameters on the access point to their advantage. Be sure to alter these passwords periodically. Ensure passwords are encrypted before being sent over the

network.

Don't broadcast SSIDs. If this feature is available, you can avoid having user devices automatically sniff the SSID in use by the access point. Most current computer operating systems and monitoring tools will automatically sniff the 802.11 beacon frames to obtain the SSID. With SSID broadcasting turned off, the access point will not include the SSID in the beacon frame, making most SSID sniffing tools useless. This isn't a foolproof method of hiding the SSID, however, because someone can still monitor 802.11 association frames (which always carry the SSID, even if SSID broadcasting is turned off) with a packet tracer. At least shutting off the broadcast mechanism will limit access.

Reduce propagation of radio waves outside the facility. Through the use of directional antennas and RF shielding, you can direct the propagation of radio waves inside the facility and reduce the "spillage" outside the perimeter. This not only optimizes coverage, it also minimizes the ability for a hacker located outside the controlled portion of the company to eavesdrop on user signal transmissions and interface with the corporate network through an access point. This also reduces the ability for someone to jam the wireless LAN - a form of denial-of-service attack - from outside the perimeter of the facility. In addition, consider setting access points near the edge of the building to lower transmit power to reduce range outside the facility. This testing should be part of the wireless site survey.

Implement personal firewalls. If a hacker is able to associate with an access point, which is extremely probable if there is no encryption or authentication configured, the hacker can easily access (via the Windows operating system) files on other users' devices that are associated with an access point on the same wireless network. As a result, it's crucial that all users disable file sharing for all folders and utilize personal firewalls. These firewalls are part of various operating systems, such as Windows XP and Vista, and 3rd party applications as well.

Control the deployment of wireless LANs. Ensure that all employees and organizations within the company coordinate the installation of wireless LANs with the appropriate information systems group. Forbid the use of unauthorized access points. Mandate the use of approved vendor products that you've had a chance to verify appropriate security

safeguards. Maintain a list of authorized radio NIC and access point MAC addresses that you can use as the basis for identifying rogue access points.

Wireless Security Architectures:

Wireless networks are becoming very common in the enterprise, with many devices utilizing it as a primary communication method. Although these networks provide convenience and enhanced productivity, they also provide new opportunities for abuse and breaches of security. This is especially true when wireless devices are in proximity to people or computers that run on the traditional wired network. As such, organizations must be equipped with robust security controls to protect their wireless environment from potential threats and intruders. In order for effective enterprise defense strategies to be put in place, it's important that we have a solid understanding of how wireless technologies interact with the wired network environment. In this article, we'll discuss the similarities and differences between the two network architectures and break down the wireless security concerns that must be addressed in order to protect your wired network resources.

Wireless Technology:

Wireless technology has created a new influx of networks within our organizations. This has introduced many new challenges as well as opportunities for our users. For example, Users may want to use their laptop while traveling or working on a sales presentation in a conference room, users now have access to distribute computing solutions (such as Thin Clients), and users are becoming more mobile by using more wireless devices such as PDAs, mobile phones, smartphones and wireless headsets instead of traditional computing devices such as desktops and laptops.

Wireless technology has also provided new security concerns that must be addressed in the organization's wireless network architecture. One of the key concerns is the problem of electromagnetic interference (EMI). EMI is a concern when two or more wireless devices are operating in proximity to one another. If a new wireless device is operating nearby an existing one, this may cause degradation of performance and possibly even damage to the existing device. The most common form of EMI can be attributed to the use of wireless devices in enterprise environments. An EMI issue on the wireless side is

often the cause of poor performance and increased latency in the wired network. This can also be a major concern with virtual private networks, as high-frequency radiation waves (termed “white space radiation”) can exist in this environment.

Wireless Network Security:

It’s important to understand how all wireless devices operate in order to design a secure and effective architecture for them. Wireless security is just one of these concerns. Some of the concerns and techniques for mitigating wireless networks.

- **Encryption:** A wireless protocol that uses encryption to secure data and messages over the air.
- **Rogue Access Points (AP):** An AP is a wireless access point that can be used by unauthorized users to gain access to your networks.
- **WLAN Spoofing:** Using falsified data in order to impersonate a legitimate access point (AP).

Attacks on Wireless Networks:

There are two main categories of attacks that can affect wireless networks: Distributed Denial-of-Service (DDoS) and Distributed Hash-table (Hashing). These attacks can take place on the 2.4 GHz and 5 GHz ranges of the spectrum, depending on the type of encryption used. In order to protect your organization’s wireless network resources, you should be familiar with these common attacks as well as their potential impacts on your network. Where possible, we’ll discuss how to mitigate them with Enterprise Wireless Security Design and Implementation.

Distributed Denial-of-Service:

Briefly, DDoS is an attack in which a number of computers are connected to a centralized source using peer-to-peer networking, which allows them to send packets at extremely high rates. These packets are targeted at a single victim, which will eventually result in the victim’s inability to respond to requests. DDoS attacks can be launched by using multiple wireless devices with IEEE 802.1Q encryption. This type of encryption provides fast data rates and is used in exchange for its vulnerability to the following attack:

- The attacker connects to each wireless device individually and sends it a spoofed IP packet with its source address set to that of the target (victim). The AP receives

this message and responds in kind, which results in a flood of traffic being sent directly to the victim.

- In order to mitigate DDoS attacks, you should implement a centralized wireless security solution to monitor and control.

Distributed Hash-table (Hashing):

Hashing is an attack in which the attacker computes a hash value based on the data being transmitted. If the hash value matches that of a preconfigured table, then a denial of service may occur. This type of attack comes in two varieties:

- Distributed Collision-Based Attacks (DCBA)
- Distributed Hashing-based Attack (DHBA)

Unlike the case with DCBA, where collisions are expected between APs due to their limited number of slots in IEEE 802.

Wireless security Tools:

WEP, WPA, WPA2, and the latest WPA3 are the four types of wireless network security protocols, each with increasing levels of security. While WPA2, which uses AES encryption, is commonly used, WPA3 provides additional security features such as stronger encryption and attack defense.

Wired Equivalent Privacy (WEP)

WEP developed in 1997, was designed to secure wireless networks using encryption and access restriction. However, its reliance on the insecure RC4 encryption and shared key authentication made networks vulnerable to attack. While WEP initially provided encryption similar to wired networks, its flaws were widely exploited by hackers, making it obsolete.

The protocol's discontinuation created more robust alternatives, such as WPA (Wi-Fi Protected Access). Despite its flaws, WEP's simplicity and widespread adoption originally drew attention, but its inherent vulnerabilities eventually overshadowed its benefits, emphasizing the significance of constantly updating wireless security standards.

Wi-Fi Protected Access (WPA)

WPA, launched in 2003, emerged as an effective successor to WEP, addressing its flaws. WPA uses the temporal key integrity protocol (TKIP) encryption to improve key

management and integrity checks. It has two modes: WPA-Personal for home networks and WPA-Enterprise for enterprises that use RADIUS servers.

WPA's 128-bit encryption provides enhanced protection over WEP's weaker encryption standards; however, it's still comparably weaker than WPA2 resulting in potential flaws and compatibility difficulties. Furthermore, adopting WPA may necessitate hardware modifications, providing a problem for users with older equipment.

Wi-Fi Protected Access II (WPA2)

WPA2, released in 2004, is the most popular wireless security standard that uses the AES encryption technique to provide strong security. Its advantages over WPA include better administration and lower vulnerability to assaults. WPA2 is widely adopted as the industry standard, ensuring device interoperability.

However, vulnerabilities such as the key reinstallation attack (KRACK) constitute a security risk. While appropriate for most home networks, difficulties arise in enterprise settings where sophisticated attacks are more widespread. Furthermore, older gear without WPA2 compatibility may require upgrades. Despite these issues, WPA2 remains critical to wireless network security, but with ongoing attempts to address growing threats and weaknesses.

Wi-Fi Protected Access III (WPA3)

WPA3, launched in 2018, provides greater encryption, protection against dictionary brute force attacks, and simpler device configuration via Wi-Fi Easy Connect. Despite these improvements, widespread acceptance is sluggish. WPA3 comes in three types: WPA3-Personal for home use, WPA3-Enterprise for organizational settings, and Wi-Fi Enhanced Open for non-password-protected networks.

While it enhances overall network security, drawbacks include deployment complexity, low user adoption, and compatibility issues with older devices and equipment. Despite its benefits, full-scale deployment of WPA3 has yet to occur, signaling a slow shift from older security protocols to this more modern standard.

UNIT V

SECURITY APPLICATIONS

IOT security: Introduction - Architectures - Security challenges - Security requirements - Trust, Data confidentiality, and privacy in IOT - Security in future IOT systems; Cloud Security: Security requirements - Security patterns and Architectural elements - Cloud Security Architecture - Security Management in the Cloud - Availability Management - SaaS Availability Management - PaaS Availability Management - IaaS Availability Management - Access control - Security Vulnerability, Patch and Configuration Management.

5.1 IOT security

Introduction

IoT security (internet of things security) is the technology segment focused on safeguarding connected devices and networks in IoT. IoT involves adding internet connectivity to a system of interrelated computing devices, mechanical and digital machines, objects, animals and people. Each *thing* has a unique identifier and the ability to automatically transfer data over a network. However, enabling devices to connect to the internet opens them up to serious vulnerabilities if they aren't properly protected.

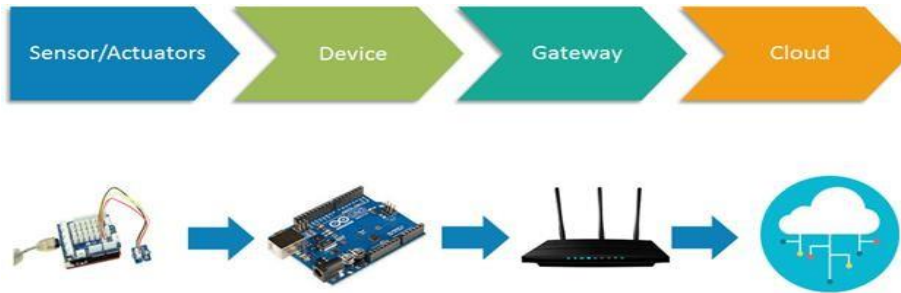
The term *IoT* is extremely broad, and as this technology continues to evolve, the term only becomes broader. From watches to thermostats to video game consoles, nearly every technological device can interact with the internet, or other devices, in some capacity. IoT security is even broader than IoT, resulting in a variety of methodologies falling under that umbrella. Application programming interface (API) security, public key infrastructure (PKI) authentication and network security are just a few of the methods IT can use to combat the growing threat of cybercrime and cyberterrorism rooted in vulnerable IoT devices.

5.1.1 IoT Architecture

There is not such a unique or standard consensus on the Internet of Things (IoT) architecture which is universally defined. The IoT architecture differs from their functional area and their solutions. However, the IoT architecture technology mainly consists of four major components:

Components of IoT Architecture

- Sensors/Devices
- Gateways and Networks
- Cloud/Management Service Layer
- Application Layer



Stages of IoT Solutions Architecture

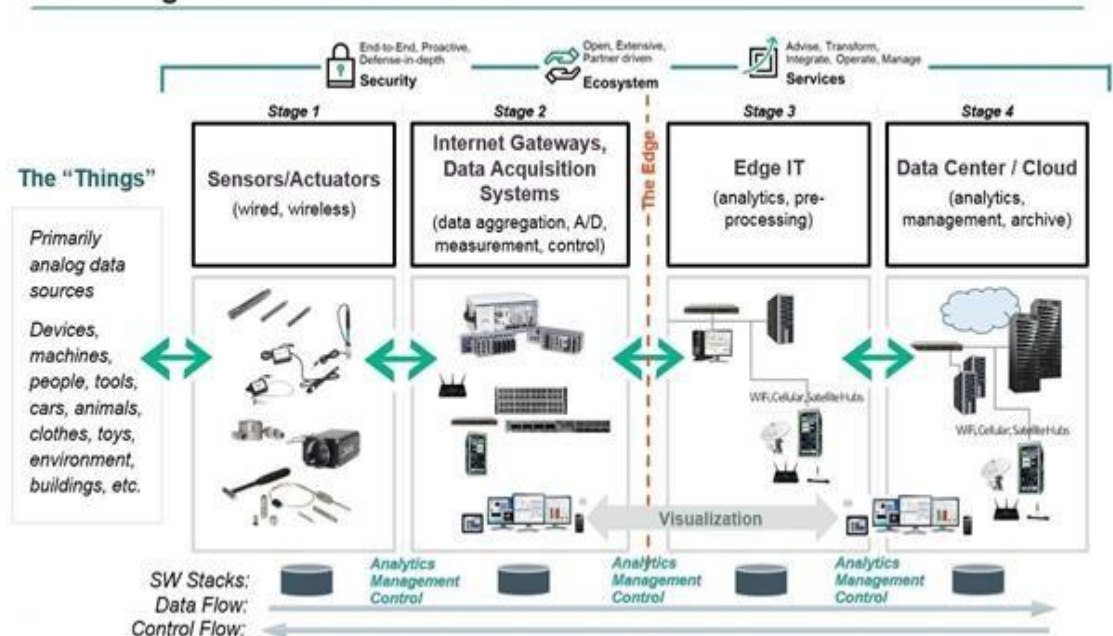
There are several layers of IoT built upon the capability and performance of IoT elements that provides the optimal solution to the business enterprises and end-users. The IoT architecture is a fundamental way to design the various elements of IoT, so that it can deliver services over the networks and serve the needs for the future.

Following are the primary stages (layers) of IoT that provides the solution for IoT architecture.

1. **Sensors/Actuators:** Sensors or Actuators are the devices that are able to emit, accept and process data over the network. These sensors or actuators may be connected either through wired or wireless. This contains GPS, Electrochemical, Gyroscope, RFID, etc. Most of the sensors need connectivity through sensors gateways. The connection of sensors or actuators can be through a Local Area Network (LAN) or Personal Area Network.

2. **Gateways and Data Acquisition:** As the large numbers of data are produced by this sensors and actuators need the high-speed Gateways and Networks to transfer the data. This network can be of type Local Area Network (LAN such as WiFi, Ethernet, etc.), Wide Area Network (WAN such as GSM, 5G, etc.).
3. **Edge IT:** Edge in the IoT Architecture is the hardware and software gateways that analyze and pre-process the data before transferring it to the cloud. If the data read from the sensors and gateways are not changed from its previous reading value then it does not transfer over the cloud, this saves the data used.
4. **Data center/ Cloud:** The Data Center or Cloud comes under the Management Services which process the information through analytics, management of device and security controls. Beside this security controls and device management the cloud transfer the data to the end users application such as Retail, Healthcare, Emergency, Environment, and Energy, etc.

The 4 Stage IoT Solutions Architecture



5.2 IoT security issues and challenges

The more ways there are for devices to connect to each other, the more opportunities there are for threat actors to intercept them. Hypertext Transfer Protocol and APIs are just two

of the channels that IoT devices rely on that hackers can intercept. The IoT umbrella doesn't strictly include internet-based devices either. Appliances that use Bluetooth technology also count as IoT devices and, therefore, require IoT security. The following IoT security challenges continue to threaten the financial safety of both individuals and organizations:

- **Remote exposure.**
- **Lack of industry foresight.**
- **Resource constraints.**
- **Weak default passwords.**
- **Multiple connected devices.**
- **Lack of encryption.**

5 IoT privacy and security best practices



- 1 Encrypt all data being transferred or stored.
- 2 Use strong passwords and two-factor authentication to prevent unwanted access.
- 3 Employ access controls so only authorized users can access IoT devices.
- 4 Separate the IoT portion of the network to prevent hackers from using it to access a core business network.
- 5 Update firmware consistently to patch vulnerabilities.

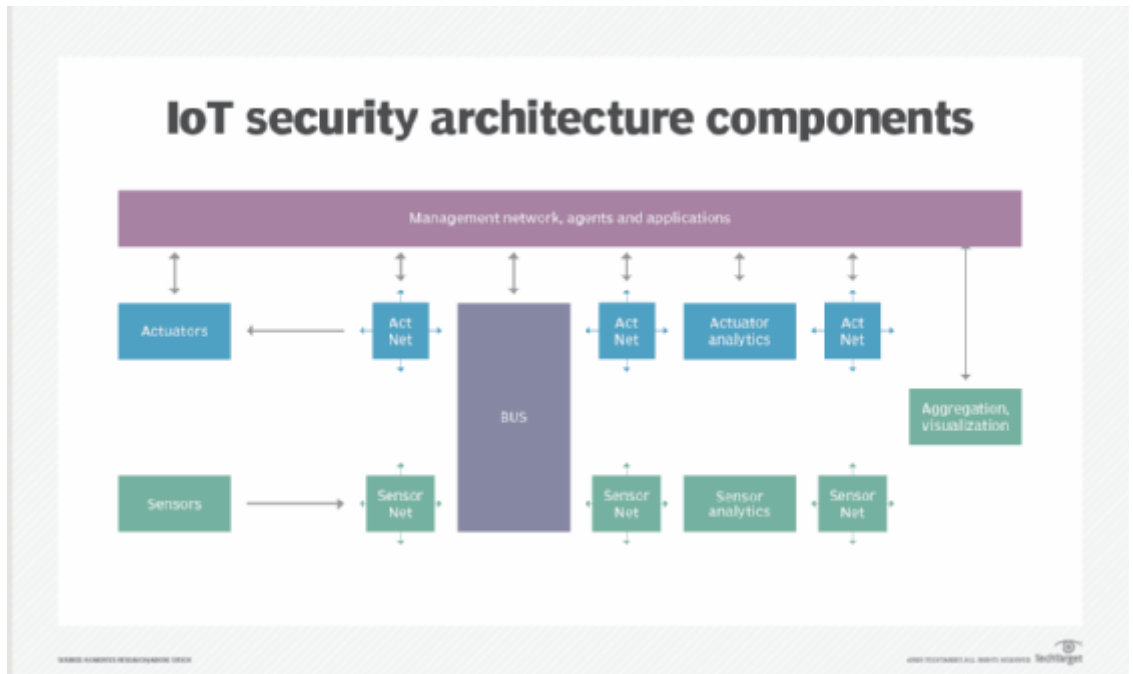
Illustration: iStockphoto.com/Andriy Hrytsiv, iStockphoto.com/Andriy Hrytsiv, iStockphoto.com/Andriy Hrytsiv

© 2019 TechTarget. All rights reserved. TechTarget

How to protect IoT systems and devices

Enterprises can use the following tools and technologies to improve their data protection protocols and security posture:

1. **Introduce IoT security during the design phase.** Of the IoT security risks and issues discussed, most can be overcome with better preparation, particularly during the research and development process at the start of any consumer-, enterprise- or industrial-based IoT (IIoT) device development. Enabling security by default is critical, along with providing the most recent operating systems and using secure hardware.
IoT developers should be mindful of cybersecurity vulnerabilities throughout each stage of development -- not just the design phase. The car key hack, for instance, can be mitigated by the driver placing their fob in a metal box or away from the windows and hallways in their home.
2. **PKI and digital certificates.** PKI can secure client-server connections between multiple networked devices. Using a two-key asymmetric cryptosystem, PKI can facilitate the encryption and decryption of private messages and interactions using digital certificates. These systems help to protect the clear text information input by users into websites to complete private transactions. E-commerce wouldn't be able to operate without the security of PKI.
3. **Network security.** Networks provide a huge opportunity for threat actors to remotely control IoT devices. Because networks involve both digital and physical components, on-premises IoT security should address both types of access points. Protecting an IoT network includes ensuring port security, disabling port forwarding and never opening ports when not needed; using antimalware, firewalls, intrusion detection systems and intrusion prevention systems; blocking unauthorized IP addresses; and ensuring systems are patched and up to date .



Protecting the network is a key responsibility of IoT security.

4. **API security.** APIs are the backbone of most sophisticated websites. They enable travel agencies, for example, to aggregate flight information from multiple airlines into one location. Unfortunately, hackers can compromise these channels of communication, making API security necessary for protecting the integrity of data being sent from IoT devices to back-end systems and ensuring only authorized devices, developers and apps communicate with APIs. T-Mobile's 2018 data breach exposed the consequences of poor API security. Due to a leaky API, the mobile giant exposed the personal data of more than 2 million customers, including billing ZIP codes, phone numbers and account numbers.

IoT attacks and security varies

IoT security methods vary depending on the specific IoT application and its place in the IoT ecosystem. For example, IoT manufacturers -- from product makers to semiconductor companies -- should concentrate on building security into their

devices from the start, making hardware tamperproof, building secure hardware, ensuring secure upgrades, providing firmware updates and patches, and performing dynamic testing.

Developers of IoT devices should focus on secure software development and secure integration. For those deploying IoT systems, hardware security and authentication are critical measures. Likewise, for operators, keeping systems up to date, mitigating malware, auditing, protecting infrastructure and safeguarding credentials are key. With any IoT deployment, it's critical to weigh the cost of security against the risks prior to installation, however.

5.3 Security requirements and Trust, Data confidentiality, and privacy in IOT

IoT Security Requirements: Authentication, Confidentiality and Access Control

- IoT enables a constant transfer and sharing of data among things and users.
- In such a sharing environment, authentication, authorization, access control and non-repudiation are important to ensure secure Communication.

Authentication and Confidentiality

- Presented intelligent Service Security Application Protocol. It combines cross-platform communications with encryption, signature, and authentication, to improve IoT apps development capabilities.
- The authors introduced the first fully implemented two-way authentication security scheme for IoT, the Datagram Transport Layer Security (DTLS) protocol, based on RSA and designed for IPv6 over Low power Wireless Personal Area Networks (6LoWPANs), placed between transport and app player. It provides message integrity, confidentiality, and authenticity.
- Classified the Key Management System (KMS) protocols in weaknesses of four major categories: key pool framework, mathematical framework, negotiation framework, and public key framework. The combinatorics-based KMS protocols suffer both connectivity and scalability, authentication.

5.4 Cloud Security:

Cloud computing refers to the on demand delivery of computing services such as applications, computing resources, storage, database, networking resources etc. through internet and on a pay as per use basis. At the present time the demand for cloud computing services are increasing with respect to that demand for cloud computing

skills is also increasing. It provides three main types of service models i.e. SaaS (Software as a Service), PaaS (Platform as a Service) and IaaS (Infrastructure as a Service). With this as starting from small to large organizations have started using cloud services so depending upon their requirement they go for the different types of cloud like Public cloud, Private cloud, Hybrid cloud, Community cloud.

Security In Cloud Computing :

Cloud computing which is one of the most demanding technology of the current time, starting from small to large organizations have started using cloud computing services. Where there are different types of cloud deployment models are available and cloud services are provided as per requirement like that internally and externally security is maintained to keep the cloud system safe. Cloud computing security or cloud security is an important concern which refers to the act of protecting cloud environments, data, information and applications against unauthorized access, DDOS attacks, malwares, hackers and other similar attacks. Community Cloud : These allow to a limited set of organizations or employees to access a shared cloud computing service environment.

Planning of security in Cloud Computing :

As security is a major concern in cloud implementation, so an organization have to plan for security based on some factors like below represents the three main factors on which planning of cloud security depends.

- Resources that can be moved to the cloud and test its sensitivity risk are picked.
- The type of cloud is to be considered.
- The risk in the deployment of the cloud depends on the types of cloud and service models.

Types of Cloud Computing Security Controls :

There are 4 types of cloud computing security controls i.e.

1. **Deterrent Controls** : Deterrent controls are designed to block nefarious attacks on a cloud system. These come in handy when there are insider attackers.
2. **Preventive Controls** : Preventive controls make the system resilient to attacks by eliminating vulnerabilities in it.

3. **Detective Controls** : It identifies and reacts to security threats and control. Some examples of detective control software are Intrusion detection software and network security monitoring tools.
4. **Corrective Controls** : In the event of a security attack these controls are activated. They limit the damage caused by the attack.

Importance of cloud security :

For the organizations making their transition to cloud, cloud security is an essential factor while choosing a cloud provider. The attacks are getting stronger day by day and so the security needs to keep up with it. For this purpose it is essential to pick a cloud provider who offers the best security and is customized with the organization's infrastructure. Cloud security has a lot of benefits –

- **Centralized security** : Centralized security results in centralizing protection. As managing all the devices and endpoints is not an easy task cloud security helps in doing so. This results in enhancing traffic analysis and web filtering which means less policy and software updates.
- **Reduced costs** : Investing in cloud computing and cloud security results in less expenditure in hardware and also less manpower in administration
- **Reduced Administration** : It makes it easier to administer the organization and does not have manual security configuration and constant security updates.
- **Reliability** : These are very reliable and the cloud can be accessed from anywhere with any device with proper authorization.

When we are thinking about cloud security it includes various types of security like access control for authorized access, network segmentation for maintaining isolated data, encryption for encoded data transfer, vulnerability check for patching vulnerable areas, security monitoring for keeping eye on various security attacks and disaster recovery for backup and recovery during data loss.

There are different types of security techniques which are implemented to make the cloud computing system more secure such as SSL (Secure Socket Layer) Encryption, Multi Tenancy based Access Control, Intrusion Detection System,

firewalls, penetration testing, tokenization, VPN (Virtual Private Networks), and avoiding public internet connections and many more techniques.

But the thing is not so simple how we think, even implementation of number of security techniques there is always security issues are involved for the cloud system. As cloud system is managed and accessed over internet so a lot of challenges arises during maintaining a secure cloud. Some cloud security challenges are

- Control over cloud data
- Misconfiguration
- Ever changing workload
- Access Management
- Disaster recovery

5.5 Cloud Security Requirements

This chapter incorporates the best practices discussed in the *Cloud Security Alliance (CSA) Security Guidance for Critical Areas of Focus in Cloud Computing*, Version 3.0, and the security control areas defined in the CSA Cloud Controls Matrix (CCM) for managing security risks associated with cloud computing. The security focus of this chapter is on Private and Private Community clouds, which CMS expects to host operational environments that have Low or Moderate system categorizations.

The CSA CCM contains a comprehensive list of control areas for consideration, mapped to the relevant NIST SP 800-53 controls. CMS has already provided guidance for most control areas in the *CMS ARS*. This topic sets forth CMS standards and guidance for implementing a subset of CCM controls areas that have special relevance in cloud environments. These areas are crucial to the security assessment of CMS cloud systems.

Cloud Security Objectives

Regardless of environment, protecting sensitive data requires implementing and enforcing security controls to reduce the risk to the CIA of that data. Because of the technology for implementing clouds, the use of shared physical equipment, lack of visibility into the security management of the system, and reliance on the maturity of

vendor processes and procedures, the risk in implementing a system in the vendor-operated federal cloud is likely higher than in an agency's dedicated computer center.

Similar to traditional computing environments, cloud computing implementations are subject to local physical threats as well as external threats. These threat sources include accidents, natural disasters, external loss of service, hostile governments, criminal organizations, and terrorist groups. Additional threat sources are intentional or unintentional introduction of vulnerabilities through internal or external authorized or unauthorized human and system access, including but not limited to, employees, contractors, and intruders. The characteristics of cloud computing, including multi-tenancy and the variety of service and deployment models, underscore the need to consider data and systems protection within the context of logical as well as physical boundaries.

Cloud computing implementations include the following major security objectives:

- Preventing unauthorized access to cloud computing infrastructure resources.
- Managing hypervisor threat vectors.
- Minimizing shared network access.
- Managing privileged user access.
- Ensuring that appropriate security safeguards are deployed at the CSP.
- Defining trust boundaries between CSPs and the CMS consumers.

Encryption and Key Management

Many CSPs offer encryption services to their customers. As CMS business owners require this service, the following requirements must be met:

- Where the CSP owns the encryption key(s), the encryption key must be different from CSP keys used to manage other CSP customers.
- The CSP CMS keys must be escrowed with CMS.
- CSPs should be able to demonstrate to CMS how the keys are managed and protected.
- The CSP must ensure that the assignment of a key manager adheres to segregation of duties (per the CMS ARS) and provides an option for CMS to perform its own

key management.

Resource Traceability and Records Management

In a traditional physical data center deployment, system managers know precisely where data resides by physical server and storage device. In a cloud, data and applications will be allocated across several hardware units, some of which may be geographically disparate.

A CSP may move data securely between its various data center environments providing the CSP complies with CMS's contractual agreement and their compliance has been documented a priori during the security authorization process. In addition, the CSP must have logging enabled to record where data resources are currently located and where they have been deployed. Depending on the data's sensitivity (e.g., PII, PHI, and FTI) and records management requirements, CMS data stewards may have to account for where the data currently resides and where it has been to demonstrate the proper application of data life-cycle management.

In addition to the requirements placed on CSPs, business applications have federal records management requirements for data life cycles as well as data retention and records destruction.

Data Availability

Data management policy may require alternate solutions for managing data and keeping sensitive data outside of the cloud environment. Data management may require traditional hosting solutions with controls in place for following the chain of data authority and integrity. A review and understanding of the controls provided in the cloud environment for managing data must be clear and well documented.

Moving data into or out of the cloud requires careful planning and execution. Although data moving into or out of a cloud can use conventional methods for small data repositories, such as through files or programmatically, large amounts of data may require physical media that would need to be passed through the CSP to load into the cloud or be removed from the cloud. CSPs offer this type of service, which requires advanced coordination and planning.

Availability of Data

Archived data must meet National Archives and Records Administration (NARA) requirements and be stored in a format capable of meeting e-discovery needs. (Please refer to CMS Cloud Computing Standards for details.)

CSPs must meet the data needs for security forensic analysis by following *CMS ARS* Audit and Accountability (AU) requirements.

Data Protection

CMS must require the CSP to safeguard any confidential information (such as PII, PHI, and FTI) in compliance with Agency standards and to comply with all applicable regulatory reporting requirements.

In a multi-tenant environment, confidential data must be protected using a combination of access control, contractual liability, and encryption.

Solutions are needed for protecting data under the following circumstances. Depending on the type of cloud service, the solution can be supplied either by the CSP or by CMS:

- Backup to tape or disk (IaaS or PaaS)
- Archived data (PaaS)
- Removal of disk for repair (IaaS)
- Protection of data between and in disaster recovery sites (IaaS)
- Data extracts sent to partners (PaaS, SaaS, or CMS)
- Shared / consolidated storage used by multiple organizations (IaaS or PaaS)
- Protection from insider theft (service provider employees) (IaaS, PaaS, or SaaS)

Hypervisor Modification Detection

The following requirements are for hypervisor oversight:

- Any changes to the networking configuration of the hypervisor must automatically generate an alert.
- Real-time security-relevant events on the hypervisor must be audited.
- Hypervisor audit logs must be audited weekly for indications of inappropriate or unusual activity.
- Unauthorized modifications to the hypervisor must be reported to the CMS security

team within 60 minutes (as required by OMB Memorandum M-07-16) if they have not been identified as a false-positive result.

Network Infrastructure Modification Detection

The following requirements are for network devices not under the direct control of CMS:

- Security-relevant events of the underlying infrastructure and network IDS must be audited in real time for indications of inappropriate or unusual activity.
- Audit logs of network infrastructure devices, such as network IDS monitors, alert logs, etc., must be reviewed weekly for indications of inappropriate or unusual activity.
- Unauthorized modifications to network infrastructure must be reported to the CMS security team within 60 minutes (as required by OMB Memorandum M-07-16) if they have not been identified as a false-positive result.

Administrative Network Port Scans

The following requirements are for network devices not under the direct control of CMS:

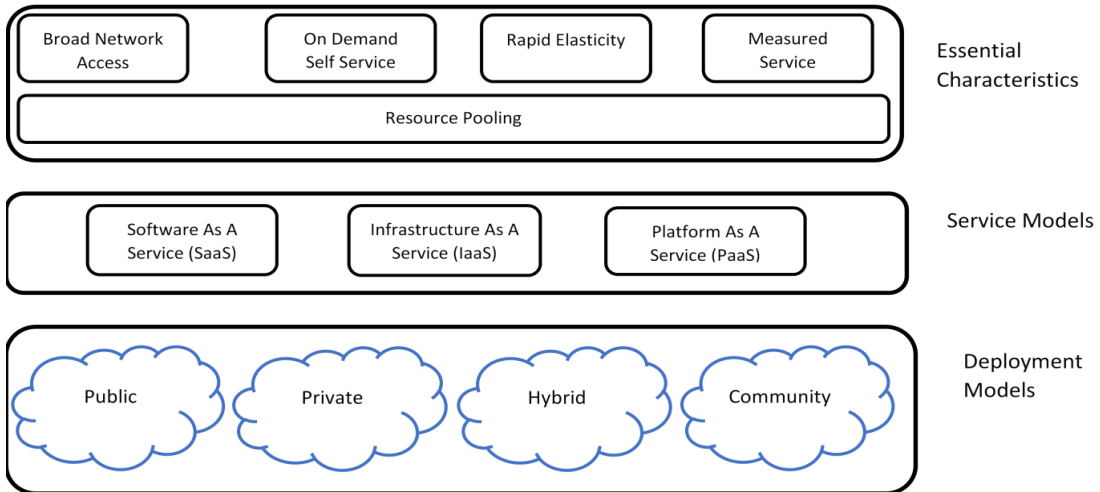
- Cloud management infrastructure devices must be scanned daily to identify any undocumented or unauthorized ports.
- Unauthorized ports and protocols must be reported to the CMS security team within 60 minutes (as required by OMB Memorandum M-07-16) if they have not been identified as false-positive results.

Management Detective Controls

Management oversight by the CSPs provides the government with insight into how its cloud is managed and confidence that it is managed securely. The detection management controls permit the government to identify patterns or weaknesses that could compromise the security of its cloud.

Definitions

This chapter relies on the NIST SP 800-145, *The NIST Definition of Cloud Computing*, September 2011, as the authoritative source for cloud terminology. Visual Model of NIST Working Definition of Cloud Computing shows a visual model of the NIST Cloud model.



Visual Model of of Cloud Computing

Term	Definition
Private Cloud	The cloud infrastructure is operated solely for an organization. It may be managed by the organization or a third party and may exist on premises or off premises.
Community Cloud	The cloud infrastructure is shared by several organizations and supports a specific community that has shared concerns (e.g., mission, security requirements, policy, and compliance considerations). It may be managed by the organizations or a third party and may exist on premises or off premises.
Public Cloud	The cloud infrastructure is made available to the general public or a large industry group and is owned by an organization selling cloud services.
Hybrid Cloud	The cloud infrastructure is a composition of two or more clouds (private, community, or public) that remain unique entities but are bound together by standardized or proprietary technology that enables data and application portability (e.g., for load balancing between clouds).

Infrastructure as a Service (IaaS)	The capability provided to the consumer is to provision processing, storage, networks, and other fundamental computing resources where the consumer is able to deploy and run arbitrary software, which can include operating systems and applications. The consumer does not manage or control the underlying cloud infrastructure but has control over operating systems, storage, deployed applications, and possibly limited control of select networking components (e.g., host firewalls).
Platform as a Service (PaaS)	The capability provided to the consumer is to deploy onto the cloud infrastructure consumer-created or acquired applications created using

Term	Definition
	programming languages and tools supported by the provider. The consumer does not manage or control the underlying cloud infrastructure, including network, servers, operating systems, or storage, but has control over the deployed applications and possibly application hosting environment configurations.
Software as a Service (SaaS)	The capability provided to the consumer is to use the provider's applications running on a cloud infrastructure. The applications are accessible from various client devices through a thin client interface such as a Web browser (e.g., Web-based email). The consumer does not manage or control the underlying cloud infrastructure, including network, servers, operating systems, storage, or even individual application capabilities, with the possible exception of limited user-specific application configuration settings.

Other terms commonly used when addressing cloud and virtualized environments include:

- **Virtual Machine** – An instance of an operating system variant (e.g., Linux) running under the control of a hypervisor.
- **Bursting** – The ability to increase VM resources, such as processor, memory, or

storage capability, provided as a service by the CSP.

- **Elasticity** – The ability to add or subtract VMs for use by a business application, implemented through configuration of the application platform (PaaS).

5.6 Design patterns for security

The following table summarizes cloud design patterns that support the goals of security.

Expand table

Pattern	Summary
Ambassador	Encapsulates and manages network communications by offloading cross-cutting tasks that are related to network communication. The resulting helper services initiate communication on behalf of the client. This mediation point provides an opportunity to augment security on network communications.
Backends Frontends	for Individualizes the service layer of a workload by creating separate services that are exclusive to a specific frontend interface. Because of this separation, the security and authorization in the service layer that support one client can be tailored to the functionality provided by that client, potentially reducing the surface area of an API and lateral movement among different backends that might expose different capabilities.
Bulkhead	Introduces intentional and complete segmentation between components to isolate the blast radius of malfunctions. You can also use this strategy to contain security incidents to the compromised bulkhead.

5.7 Architecture of Cloud Computing

Cloud Computing , which is one of the demanding technology of the current time and which is giving a new shape to every organization by providing on demand virtualized services/resources. Starting from small to medium and medium to large, every

organization use cloud computing services for storing information and accessing it from anywhere and any time only with the help of internet. In this article, we will know more about the internal architecture of cloud computing.

Transparency, scalability, security and intelligent monitoring are some of the most important constraints which every cloud infrastructure should experience. Current research on other important constraints is helping cloud computing system to come up with new features and strategies with a great capability of providing more advanced cloud solutions.

Cloud Computing Architecture :

The cloud architecture is divided into 2 parts i.e.

1. Frontend
2. Backend

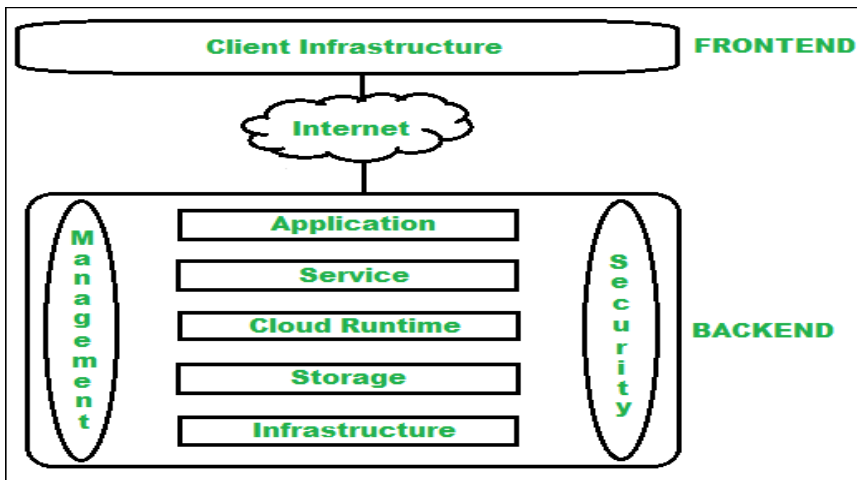
The below figure represents an internal architectural view of cloud computing.

5.8 Availability Management in Cloud Computing

Cloud Services are not immune to outages (failure/interruption) and the severity and the scope of impact on the customer can vary based on the situation. As it will depend on the criticality of the cloud application and its relationship to internal business processes.

1. **Impact on business:** In the case of business-critical applications where businesses rely on the continuous availability of service, even a few minutes of service failure can have a serious impact on the organization's productivity, revenue, customer satisfaction, and service-level compliance.
 2. **Impact on customers:** During a cloud service disruption, affected customers will not be able to access the cloud service and in some cases may suffer degraded performance or user experience. For Example:- when a storage service is disrupted, it will affect the availability and performance of a computing service that depends on the storage service.
- **Infrastructure as a Service Health Monitoring:** The following options are available to Infrastructure as a Service customer for managing the health of their service:

- Service health dashboard published by the Cloud Service Providers.
- Cloud Service Providers customer mailing list that notifies customers of occurring and recently occurred outages.
- Third-party-based service monitoring tools that periodically check the health of your Infrastructure as a Service virtual server.



Architecture of Cloud Computing

Architecture of cloud computing is the combination of both SOA (Service Oriented Architecture) and EDA (Event Driven Architecture). Client infrastructure, application, service, runtime cloud, storage, infrastructure, management and security all these are the components of cloud computing architecture.

1. Frontend:

Frontend of the cloud architecture refers to the client side of cloud computing system. Means it contains all the user interfaces and applications which are used by the client to access the cloud computing services/resources. For example, use of a web browser to access the cloud platform.

- **Client Infrastructure** – Client Infrastructure is a part of the frontend component. It contains the applications and user interfaces which are required to access the cloud platform.
- In other words, it provides a GUI(Graphical User Interface) to interact with the cloud.

2. **Backend:**

Backend refers to the cloud itself which is used by the service provider. It contains the resources as well as manages the resources and provides security mechanisms. Along with this, it includes huge storage, virtual applications, virtual machines, traffic control mechanisms, deployment models, etc.

1. **Application** – Application in backend refers to a software or platform to which client accesses. Means it provides the service in backend as per the client requirement.
2. **Service** – Service in backend refers to the major three types of cloud based services like SaaS, PaaS and IaaS. Also manages which type of service the user accesses.
3. **Runtime Cloud** – Runtime cloud in backend provides the execution and Runtime platform/environment to the Virtual machine.
4. **Storage** – Storage in backend provides flexible and scalable storage service and management of stored data.
5. **Infrastructure** – Cloud Infrastructure in backend refers to the hardware and software components of cloud like it includes servers, storage, network devices, virtualization software etc.
6. **Management** – Management in backend refers to management of backend components like application, service, runtime cloud, storage, infrastructure, and other security mechanisms etc.
7. **Security** – Security in backend refers to implementation of different security mechanisms in the backend for secure cloud resources, systems, files, and infrastructure to end-users.
8. **Internet** – Internet connection acts as the medium or a bridge between frontend and backend and establishes the interaction and communication between frontend and backend.
9. **Database** – Database in backend refers to provide database for storing structured data, such as SQL and NOSQL databases. Example of Databases services include Amazon RDS, Microsoft Azure SQL database and Google CCloud SQL.
10. **Networking** – Networking in backend services that provide networking

infrastructure for application in the cloud, such as load balancing, DNS and virtual private networks.

11. Analytics – Analytics in backend service that provides analytics capabilities for data in the cloud, such as warehousing, business intelligence and machine learning.

Benefits of Cloud Computing Architecture :

- Makes overall cloud computing system simpler.
- Improves data processing requirements.
- Helps in providing high security.
- Makes it more modularized.
- Results in better disaster recovery.
- Gives good user accessibility.
- Reduces IT operating costs.
- Provides high level reliability.
- Scalability.

5.9 Cloud Management in Cloud Computing

Cloud computing management is maintaining and controlling the cloud services and resources be it public, private or hybrid. Some of its aspects include load balancing, performance, storage, backups, capacity, deployment etc. To do so a cloud managing personnel needs full access to all the functionality of resources in the cloud. Different software products and technologies are combined to provide a cohesive cloud management strategy and process.

As we know Private cloud infrastructure is operated only for a single organization, so that can be managed by the organization or by a third party. Public cloud services are delivered over a network that is open and available for public use. In this model, the IT infrastructure is owned by a private company and members of the public can purchase or lease data storage or computing capacity as needed. Hybrid cloud environments are a combination of public and private cloud services from different providers. Most organizations store data on private cloud servers for privacy concerns, while leveraging public cloud applications at a lower price point for less sensitive information. The

combination of both the public and private cloud are known as Hybrid cloud servers.

5.10 Availability Management in Cloud Computing

Cloud Services are not immune to outages (failure/interruption) and the severity and the scope of impact on the customer can vary based on the situation. As it will depend on the criticality of the cloud application and its relationship to internal business processes.

1. **Impact on business:** In the case of business-critical applications where businesses rely on the continuous availability of service, even a few minutes of service failure can have a serious impact on the organization's productivity, revenue, customer satisfaction, and service-level compliance.
2. **Impact on customers:** During a cloud service disruption, affected customers will not be able to access the cloud service and in some cases may suffer degraded performance or user experience. For Example:- when a storage service is disrupted, it will affect the availability and performance of a computing service that depends on the storage service.

System as a Service Health Monitoring:

The following options are available to customers to stay informed on the health of their service:

- Service dashboards should be published by the Cloud Service Providers So that they can publish the current state of services and can also inform the outage or any kind of maintenance of the cloud.
- Customer should check their mailing list as the service provider might have notified them about recently occurring outages.
- Use third-party tools to check the health of the application.

Platform as a Services Customer's Responsibilities:

The following considerations are for Platform as a Services Customers:

- **PaaS platform service levels:** Customers should read and understand the terms and conditions of the Cloud Service Provider's Service Level Agreements.
- **Third-party web services provider service levels:** When your Platform as a Services application depends on a third-party service it is critical to understand the

Service Level Agreements of that service. Network connectivity parameters with third-party service providers. Example: Bandwidth and latency factors.

- **Platform as a Service Health Monitoring:** The following options are available to customers to monitor the health of their service:
 - Service health dashboard published by the Cloud Service Provider.
 - Cloud Service Providers customer mailing list that notifies customers of occurring and recently occurred outages
 - Use third-party tools to check the health of the application
- **Infrastructure as a Service Health Monitoring:** The following options are available to Infrastructure as a Service customer for managing the health of their service:
 - Service health dashboard published by the Cloud Service Providers.
 - Cloud Service Providers customer mailing list that notifies customers of occurring and recently occurred outages.
 - Third-party-based service monitoring tools that periodically check the health of your Infrastructure as a Service virtual server.

5.11 Software as a Service | SaaS

SaaS is also known as "On-Demand Software." It is a software distribution model in which services are hosted by a cloud service provider. These services are available to end-users over the internet, so the end-users do not need to install any software on their devices to access these services. Characteristics of SaaS:

- **Web-based Delivery:** SaaS apps can be accessed from anywhere with an internet connection because they are supplied over the internet, often through a web

browser. Users no longer need to install and maintain software programs on their local machines as a result.

- **Multiple Users** or "tenants" can access SaaS applications from a single instance of the program thanks to the concept of multi-tenancy. As a result, the provider can serve several clients with the same application without administering unique program instances for every client.
- **Automatic Updates:** SaaS providers are in charge of keeping the software up to date and making sure that everyone has access to the newest features and security patches. Users are no longer required to manually install updates or fixes as a result.
- **Scalable:** SaaS systems are scalable, which can readily grow or shrink in response to user demand. This frees up enterprises from worrying about infrastructure or licensing fees and lets them add or remove users as needed.
- **Pricing on a Subscription Basis:** SaaS programs are frequently sold using a subscription-based pricing model, in which customers pay a monthly or yearly price to access the program. As a result, companies won't need to invest significantly in software licenses upfront.
- **Data Security**, including data encryption, access restrictions, and backups, is the responsibility of SaaS providers. Users no longer need to handle their own data security because of this.

Services Provided by SaaS:

Business Services - SaaS Provider provides various business services to start up the business. The SaaS business services include ERP (Enterprise Resource Planning), CRM (Customer Relationship Management), billing, and sales.

Backward Skip 10sPlay VideoForward Skip 10s

Document Management - SaaS document management is a software application offered by a third party (SaaS provider) to create, manage, and track electronic documents.

Examples: Slack, Samepage, Box, and Zoho Forms.

Social Networks - As we all know, social networking sites are used by the general public, so social networking service providers use SaaS for their convenience and handle the general public's information.

Mail Services - To handle the unpredictable number of users and load on e-mail services, many e-mail providers offer their services using SaaS.

Collaboration Tools: SaaS companies provide collaboration solutions that let teams collaborate effectively no matter where they are physically located. Platforms for project management, apps for team communication, and file-sharing services are some of these resources. Examples include Slack, Microsoft Office 365, and Google Workspace (formerly G Suite).

Human Resources Management: SaaS-based HR management systems give companies tools to simplify key HR procedures, such as employee onboarding, payroll administration, timekeeping, performance reviews, and employee self-service portals.

Workday, BambooHR, and ADP Workforce Now, as examples.

Customer Support and Help Desk: SaaS platforms provide customer support and help desk solutions that enable firms to manage customer inquiries, track support tickets, and promptly address customer issues.

For instance, Salesforce Service Cloud, Freshdesk, and Zendesk.

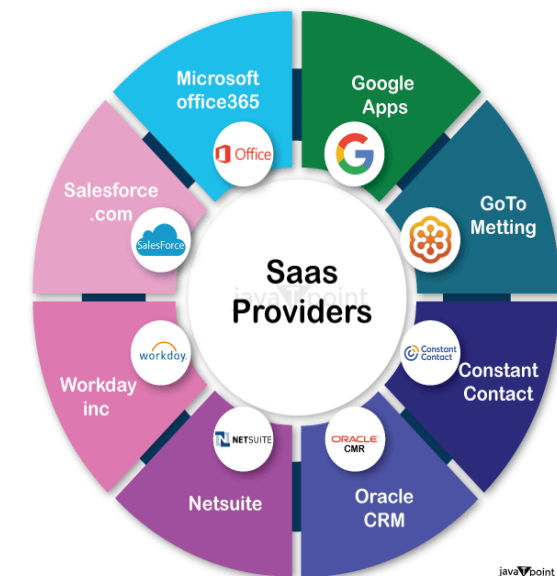
Marketing and Sales Automation: To increase productivity and boost income, firms can automate marketing campaigns, lead generation, customer relationship management, and sales activities using SaaS marketing and sales automation technologies.

Examples include Marketo, Pardot, and HubSpot.

E-commerce Platforms: SaaS-based e-commerce platforms make it simpler for businesses to launch and run online storefronts, maintain product catalogs, handle payments, and keep track of orders.

ADVERTISEMENT

Popular SaaS Providers



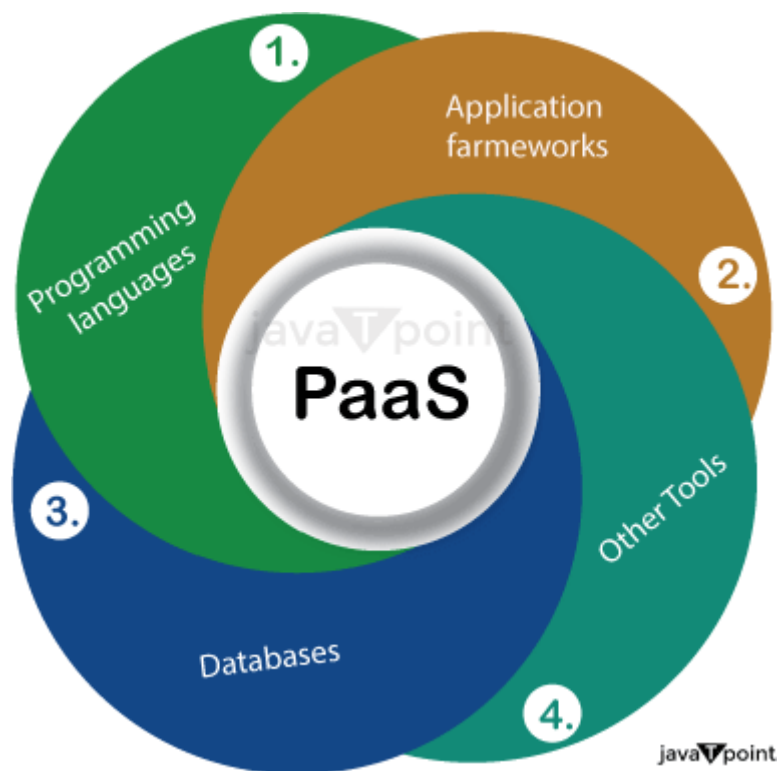
The below table shows some popular SaaS providers and services that are provided by them -

Provider	Services
Salseforce.com	On-demand CRM solutions
Microsoft Office 365	Online office suite
Google Apps	Gmail, Google Calendar, Docs, and sites
NetSuite	ERP, accounting, order management, CRM, Professionals Services Auto (PSA), and e-commerce applications.
GoToMeeting	Online meeting and video-conferencing software
Constant Contact	E-mail marketing, online survey, and event marketing
Oracle CRM	CRM applications

Workday, Inc	Human capital management, payroll, and financial management.
--------------	--

5.12 Platform as a Service | PaaS

Platform as a Service (PaaS) provides a runtime environment. It allows programmers to easily create, test, run, and deploy web applications. You can purchase these applications from a cloud service provider on a pay-as-per-use basis and access them using an Internet connection. In PaaS, back-end scalability is managed by the cloud service provider, so end-users do not need to worry about managing the infrastructure.



PaaS includes infrastructure (servers, storage, and networking) and platform (middleware, development tools, database management systems, business intelligence, and more) to support the web application life cycle.

Examples: Google App Engine, Force.com, Joyent, Azure.

Some of the Services Provided by PaaS are:

Programming Languages: A variety of programming languages are supported by PaaS providers, allowing developers to choose their favorite language to create apps.

Languages including Java, Python, Ruby,.NET, PHP, and Node.js are frequently supported.

Backward Skip 10sPlay VideoForward Skip 10s

Application Frameworks: Pre-configured application frameworks are offered by PaaS platforms, which streamline the development process. These frameworks include features like libraries, APIs, and tools for quick development, laying the groundwork for creating scalable and reliable applications. Popular application frameworks include Laravel, Django, Ruby on Rails, and Spring Framework.

Databases: Managed database services are provided by PaaS providers, making it simple for developers to store and retrieve data. These services support relational databases (like MySQL, PostgreSQL, and Microsoft SQL Server) and NoSQL databases (like MongoDB, Cassandra, and Redis). For its database services, PaaS platforms often offer automated backups, scalability, and monitoring tools.

Additional Tools and Services: PaaS providers provide a range of extra tools and services to aid in the lifecycle of application development and deployment. These may consist of the following:

- **Development Tools:** to speed up the development process, these include integrated development environments (IDEs), version control systems, build and deployment tools, and debugging tools.
- **Collaboration and Communication:** PaaS platforms frequently come with capabilities for team collaboration, including chat services, shared repositories, and project management software.
- **Analytics and Monitoring:** PaaS providers may give tools for tracking application performance, examining user behavior data, and producing insights to improve application behavior and address problems.
- **Security and Identity Management:** PaaS systems come with built-in security features like access control, encryption, and mechanisms for authentication and authorization to protect the privacy of applications and data.
- **Scalability and load balancing:** PaaS services frequently offer automatic scaling

capabilities that let applications allocate more resources as needed to manage a spike in traffic or demand. To improve performance and availability, load balancing features divide incoming requests among various instances of the application.

Advantages of PaaS

There are the following advantages of PaaS -

1) Simplified Development

PaaS allows developers to focus on development and innovation without worrying about infrastructure management.

2) Lower risk

No need for up-front investment in hardware and software. Developers only need a PC and an internet connection to start building applications.

3) Prebuilt business functionality

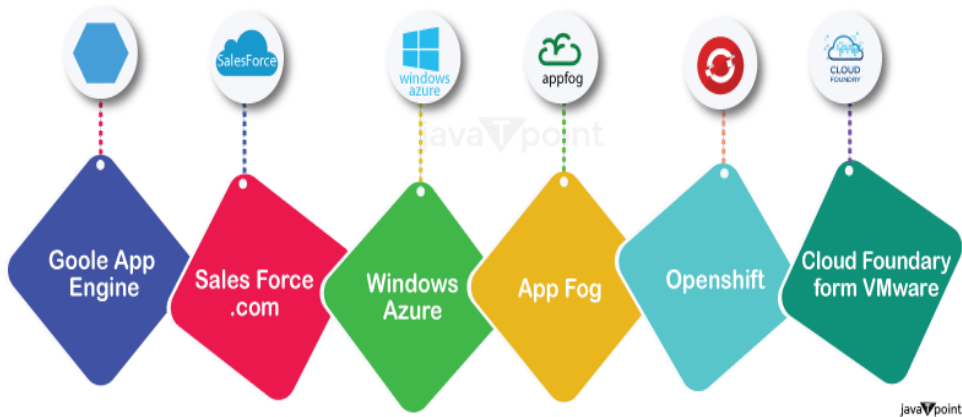
Some PaaS vendors also provide already defined business functionality so that users can avoid building everything from very scratch and hence can directly start the projects only.

4) Instant community

PaaS vendors frequently provide online communities where the developer can get ideas, share experiences, and seek advice from others.

5) Scalability

Pass Providers



javaVpoint

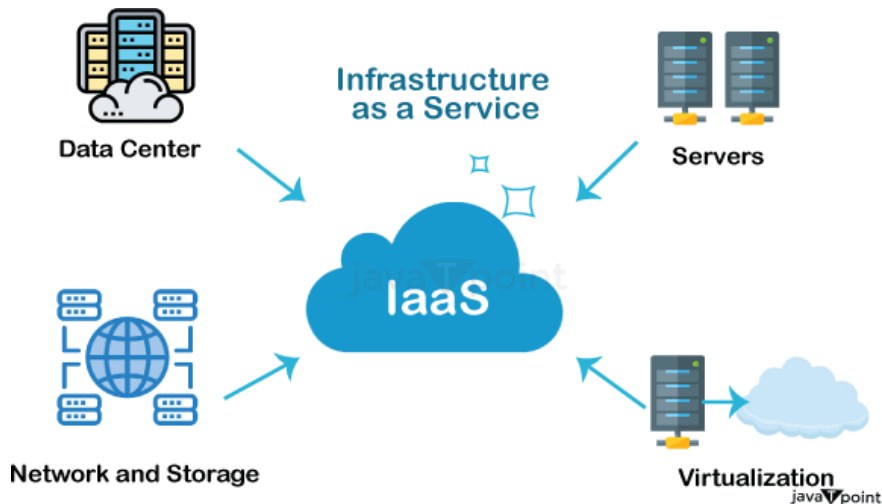
Providers	Services
Google App Engine (GAE)	App Identity, URL Fetch, Cloud storage client library, Logservice
Salesforce.com	Faster implementation, Rapid scalability, CRM Services, Sales cloud, Mobile connectivity, Chatter.
Windows Azure	Compute, security, IoT, Data Storage.
AppFog	Justcloud.com, SkyDrive, GoogleDocs
Openshift	RedHat, Microsoft Azure.
Cloud Foundry from VMware	Data, Messaging, and other services.

5.13 Infrastructure as a Service | IaaS

IaaS is also known as Hardware as a Service (HaaS). It is one of the layers of the cloud computing platform. It allows customers to outsource their IT infrastructures, such as servers, networking, processing, storage, virtual machines, and other resources. Customers access these resources on the Internet using a pay-as-per-use model.

In traditional hosting services, IT infrastructure was rented out for a specific period of time, with pre-determined hardware configuration. The client paid for the configuration and time, regardless of the actual use. With the help of the IaaS cloud computing platform layer, clients can dynamically scale the configuration to meet changing requirements and are billed only for the services actually used.

The IaaS cloud computing platform layer eliminates the need for every organization to maintain its IT infrastructure.



IaaS is offered in three models: public, private, and hybrid cloud. The private cloud implies that the infrastructure resides at the customer's premise. In the case of the public

cloud, it is located at the cloud computing platform vendor's data center, and the hybrid cloud is a combination of the two in which the customer selects the best of both public cloud and private cloud.

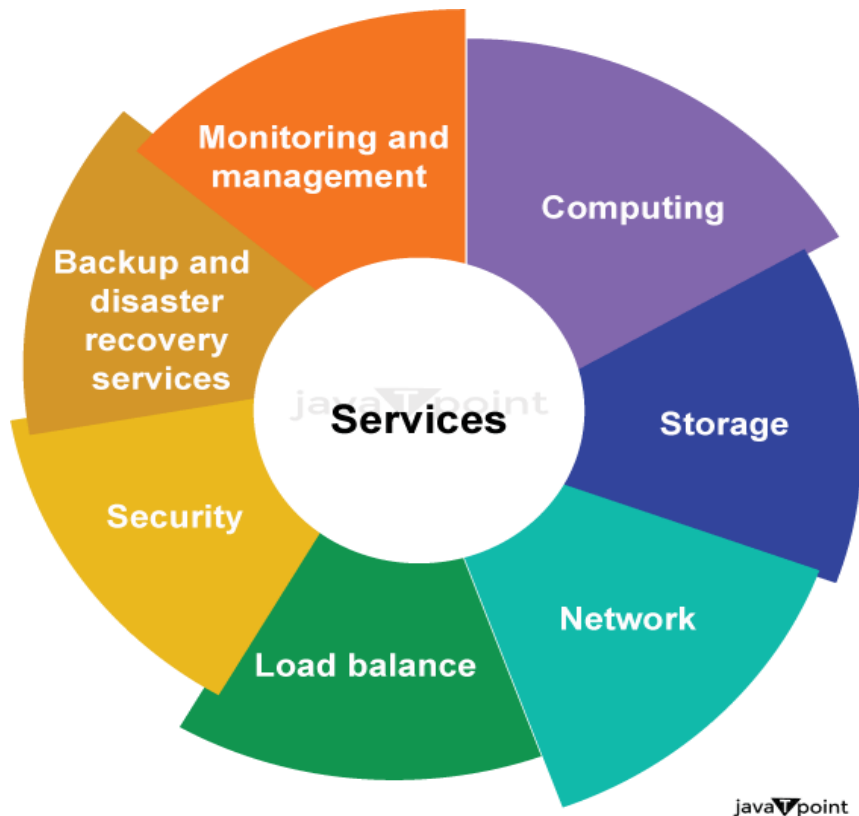
Some of the Primary Characteristics of IaaS are:

- **Scalability:** IaaS enables users to adjust computing capacity according to their demands without requiring long lead times or up-front hardware purchases.
- **Virtualization:** IaaS uses virtualization technology to generate virtualized instances that can be managed and delivered on-demand by abstracting physical computer resources.
- **Resource Pooling:** This feature enables users to share computer resources, such as networking and storage, among a number of users, maximizing resource utilization and cutting costs.
- **Elasticity:** IaaS allows users to dynamically modify their computing resources in response to shifting demand, ensuring optimum performance and financial viability.
- **Self-Service:** IaaS offers consumers "self-service" portals that let them independently deploy, administer, and monitor their computing resources without the assistance of IT employees.
- **Availability:** To ensure the high availability and reliability of services, IaaS providers often run redundant and geographically dispersed data centers.
- **Security:** To safeguard their infrastructure and client data, IaaS companies adopt security measures, including data encryption, firewalls, access controls, and threat detection.
- **Customization:** IaaS enables users to alter the operating systems, application stacks, and security settings of their virtualized instances to suit their unique requirements.

IaaS, or infrastructure as a service, is a cloud computing model that offers users virtualized computer resources on a pay-per-use basis.

Users can scale their resources up or down in accordance with their demands while taking advantage of high availability, security, and customization possibilities.

IaaS provider provides the following services -



Computing: To provision virtual machines (VMs) for end users, IaaS providers offer virtual central processing units (CPUs) and virtual main memory. As a result, users may run their workloads and apps on the provider's infrastructure without having to worry about managing the underlying hardware.

Storage: Back-end storage services are provided by IaaS providers, enabling users to store and access their files and data. This offers scalable and trustworthy storage solutions for a variety of use cases and can include block storage, object storage, or file storage alternatives.

Network: IaaS providers provide networking tools, including routers, switches, and bridges for the VMs through Network as a Service (NaaS). This enables connectivity and communication between VMs and other resources while also allowing customers to create and maintain their network architecture within the IaaS environment.

Load balancers: Infrastructure-layer load balancing services are provided by IaaS providers. Incoming network traffic is split up among many virtual machines (VMs) or resources by load balancers, resulting in effective resource management and excellent application and service availability.

Security: Security features and services are frequently offered by IaaS providers as part of their offering. To safeguard data and resources housed on the IaaS platform, this can include network security, firewall configurations, access controls, encryption, and other security measures.

Backup and disaster recovery services are provided by some IaaS providers, enabling customers to create backup copies of their data and software and put recovery plans in place in the event of data loss or system problems. This promotes business continuity and data security.

Monitoring and Management: IaaS suppliers provide tools and services for monitoring and controlling the resources and infrastructure. This can involve managing VMs,

storage, and network configurations using management panels or APIs, as well as measuring resource utilization, automating scaling, and monitoring performance.

It's vital to remember that depending on the provider and their offerings, the precise services offered by IaaS providers may change. The list above illustrates some typical IaaS providers' common services.

Virtualized Computing Resources:

- Cloud computing's Infrastructure as a Service (IaaS) model must include virtualized computer resources. IaaS enables users to rent computer infrastructure from cloud service providers over the internet, including virtual machines (VMs), virtual networks, and storage.
- In IaaS, virtual machines (VMs) are a crucial type of virtualized computing resource. Multiple operating systems and applications can operate on a single physical host machine thanks to virtual machines (VMs), which are software simulations of real hardware. Customers can select the VM that best matches their needs from a variety of VM types that IaaS providers normally offer, each with a different CPU, memory, and storage configuration.
- Virtual Networks: Another virtualized computing resource in IaaS is virtual networks. Customers can design and maintain network topologies in the cloud, including subnets, IP addresses, and routing tables, using virtual networks. Virtual networks offer clients' applications and data a secure, decoupled environment and make it simple to integrate them with on-premises networks.
- A crucial virtualized computing resource in IaaS is storage. IaaS providers frequently offer various storage options, including block, object, and file storage, each with its own performance, pricing, and cost-effectiveness features. Because storage resources are highly scalable, clients can alter their storage capacity as needed without having to change their actual hardware.

- In comparison to conventional on-premises hardware architecture, virtualized computing resources have better scalability, flexibility, and cost-effectiveness. Without making expensive hardware investments or taking care of their own data centers, customers may rent the computing capabilities they require on demand and only pay for what they use.

Top IaaS Providers who are providing IaaS cloud computing platform



Tata Communications InstaCompute InstaCompute is Tata Communications' IaaS offering. InstaCompute data centers are located in Hyderabad and Singapore, with operations in both countries.

IaaS Vendor		IaaS Solution	Details
Amazon	Web Services	Elastic, Elastic Compute Cloud (EC2) MapReduce, Route 53, Virtual Private Cloud, etc.	The cloud computing platform pioneer Amazon offers auto-scaling, cloud monitoring, and load-balancing features as part of its portfolio.

Netmagic Solutions	Netmagic Cloud IaaS	Netmagic runs from data centers in Mumbai, Chennai, and Bangalore and a virtual data center in the United States. Plans are underway to extend services to West Asia.
Rackspace	Cloud servers, cloud files, cloud sites, etc.	The cloud computing platform vendor focuses primarily on enterprise-level hosting services.
Reliance Communications	Reliance Internet Data Center	RIDC supports both traditional hosting and cloud services, with data centers in Mumbai, Bangalore, Hyderabad, and Chennai. The cloud services offered by RIDC include IaaS and SaaS.
Sify Technologies	Sify IaaS	Sify's cloud computing platform is powered by HP's converged infrastructure. The vendor offers all three types of cloud services: IaaS, PaaS, and SaaS.

1.14 Access Lists (ACL)

ACLs are a set of rules for regulating network traffic and minimising network attacks. Using a set of rules specified for the network's incoming or outgoing traffic, ACLs are used to filter traffic.

Features of Access List -

ADVERTISEMENT

- The defined set of rules is matched line by line, starting with the first line, then the second, then the third, and so on.
- Only up until the rule matches are the packets matched. No further comparisons are made after a rule matches, and that rule is then applied.
- Every ACL has an implicit denial at the end, meaning that the packet will be discarded if neither a condition nor a rule is met.

The access-list should then be applied to the interface's inbound or outbound traffic once it has been constructed:

- **Inbound access lists** - When an access list is applied to an interface's inbound packets, the inbound packets are first processed in accordance with the access list before being forwarded to the outbound interface.
- **Outbound access lists** - When an access list is applied to an interface's outbound packet header, the outbound interface will first route the packet before processing it.

Types of Access List -

There are two main types of access-list, and they are as follows:

- **Standard Access-list** : These Access-lists are created solely using the source IP address. These ACLs either allow or disallow the entire set of protocols. TCP, UDP, HTTPS, and other types of IP traffic are not differentiated. The router will recognise it as a standard ACL and the specified address as the source IP address if you use the numbers 1-99 or 1300-1999.

- **Long Access List :** The ACL that makes use of source IP, destination IP, source port, and destination port is this one. We can also specify which IP traffic should be permitted or blocked using these ACLs. These employ the 100-199 and 2000-2699 ranges. The process on the client machine sends the clock server a request at time T 0 for the clock time (time at the server).

Additionally, there are two types of access-lists:

- **Numbered access lists** are access lists that, once created, cannot be specifically deleted; for example, if we want to remove any rules from an access list, this is not allowed in the case of a numbered access list. The access list as a whole will be deleted if we attempt to remove a rule from it. Both standard and extended access lists can use the numbered access-list.
- **Named access list:** In this type of access list, a name is given to the access list in order to identify it. Unlike numbered access lists, named access lists can be deleted. These can be used with both standards and extended access lists, just like numbered access lists.

Guidelines for Access List -

- In general, the standard Access-list is used close to the destination (but not always).
- Typically, the extended Access-list is used close to the source (but not always).
- We are only allowed to assign one ACL per interface, per protocol, and per direction, meaning that each interface can only have one inbound and one outbound ACL.
- If we are using a numbered Access-list, we are unable to remove a rule from it. The entire ACL will be removed if we attempt to remove a rule. A specific rule can be deleted if named access lists are being used.
- Before implementing the access lists, carefully analyse the entire scenario because every new rule that is added will be placed at the bottom of the access list.

- Every access list has an implicit denial at the end, so we need to have at least a permit statement there; otherwise, all traffic will be blocked.
- Extended access lists cannot share the same name as standard access lists.

Benefits of Access List

- Boost network efficiency.
- Security is provided because the administrator can customise the access list to meet specific requirements and prevent malicious packets from entering the network.
- provides control over the traffic by allowing or disallowing it based on the requirements of the network.

1.15 Role of Vulnerability, Patch and Configuration Management in Cloud Security

Pre-requisite:- Cloud Computing and Cloud Based Services

The ability of malware or cracker to remotely find and change the vulnerabilities of infrastructure components is a major threat to cloud services. In public platform as a service and public infrastructure as a delivery model the vulnerability and configuration management are at greater risk because the customer is responsible for managing all that. In a cloud computing environment, the customer is responsible for sharing the lowest or highest common denominator of security with all tenants in a multi-tenant virtual environment.

Hence, The customer should understand his responsibility for security management, also the cloud service platforms should be more transparent with the customer about operations in their cloud security and help customers in understanding the functions of security management.

The infrastructure and configuration of virtual private clouds are managed by the cloud service providers, so they should provide end-to-end security.

Security Vulnerability Management:

To reduce risk from published technical vulnerabilities.

1. It is an essential threat management element in protecting network devices, hosts, and applications against vulnerability attacks.
2. Many organizations have a vulnerability management process that includes the scanning of systems connected to their network on a routine basis so that they can analyze the risk of vulnerabilities of the organization.

Hence, The vulnerability management of the cloud infrastructure (network, hosts, applications, and storage devices) is the responsibility of both the customers and the cloud service providers.

Security Configuration Management:

1. It is another essential threat management element in protecting network devices and hosts from unknown users that might exploit any weakness in configuration.
2. Security management configuration is a subset of IT configuration management and it is closely related to vulnerability management programs.
3. There should be a means for protection against the configuration of hosts, networks, and applications as in some cases they may monitor or have access to critical systems and configuration of database files including Operating system configuration, firewall policies, etc.
4. The System as a Service(SaaS), and Platform as a Service(PaaS) providers are responsible for the configuration management of their platform's customers.

Security Patch Management:

1. It plays a vital role in threat management as it protects the network from unauthorized users that may exploit a known vulnerability.
2. It makes the organization less severe risk of insider or outsider threats. Hence, System

as a Service(SaaS) providers should routinely assess new vulnerabilities and patch them in the firmware and software on all systems.

3. The Scope of patch management is in the order of SaaS, PaaS, and IaaS from low to high relevance which means the customers of the SaaS environment do not have patch management duties, but the PaaS customers are responsible for patching applications on the PaaS platform. While the IaaS customers are responsible for managing patches of the operating system, applications, and database installed and operated on the IaaS platform.